Anatomia di Tor

Il protocollo dell'anonimato, byte per byte

Alessandro Greco

Prima edizione — versione 1.0.0

Anatomia di Tor — Il protocollo dell'anonimato, byte per byte

Copyright © 2026 Alessandro Greco.

Quest'opera è libera: il testo, le figure, i sorgenti LaTeX e il codice dei laboratori si possono redistribuire e modificare secondo i termini della GNU General Public License pubblicata dalla Free Software Foundation, versione 3 o (a scelta) qualsiasi versione successiva (GPL-3.0-or-later). L'opera è distribuita nella speranza che sia utile, ma *senza alcuna garanzia*, nemmeno quella implicita di commerciabilità o di idoneità a uno scopo particolare; i dettagli sono nella licenza, `https://www.gnu.org/licenses/gpl-3.0.html`.

La forma sorgente modificabile (sorgenti LaTeX, figure TikZ, codice e rete di laboratorio) è pubblicata in `https://github.com/aleff-github/onion-routing-anatomy`.

Il libro è distribuito gratuitamente; i diritti di utilizzo e di redistribuzione sono quelli stabiliti dalla licenza indicata sopra.

«Tor» e il logo di Tor sono marchi del Tor Project, Inc. Questo libro è un'opera indipendente e non è approvata né sostenuta dal Tor Project.

I nomi di prodotti e servizi citati appartengono ai rispettivi titolari e sono usati a solo scopo descrittivo.

Prima edizione, versione 1.0.0. Formato A4.

Indice

Prefazione **xi**

I Storia e fondamenti **1**

1 Anonimato, sorveglianza e metadati **3**
1.1 Contenuto e metadati . . . 3
1.1.1 Che cosa rivelano i metadati . . . 3
1.2 L'analisi del traffico . . . 4
1.3 Un vocabolario preciso . . . 4
1.4 Chi ha bisogno di anonimato . . . 5
1.4.1 L'anonimato ama la compagnia . . . 6

2 Le radici: mix network, remailer, Crowds, Freedom **7**
2.1 La mix di Chaum (1981) . . . 7
2.2 Alta e bassa latenza . . . 8
2.3 I remailer . . . 8
2.4 Verso la bassa latenza . . . 9

3 Dall'onion routing del NRL al Tor Project **11**
3.1 Al Naval Research Laboratory . . . 11
3.2 Tre generazioni . . . 11
3.3 La rete Tor . . . 12
3.4 Un'organizzazione e una comunità . . . 13

4 Il modello di minaccia di Tor **15**
4.1 Gli obiettivi . . . 15
4.2 I non obiettivi . . . 16
4.3 L'avversario . . . 16
4.3.1 Conferma e analisi . . . 16
4.4 Che cosa Tor protegge, e che cosa no . . . 17

II Il protocollo a basso livello **19**

5 Crittografia essenziale per Tor **21**
5.1 Cinque compiti . . . 21
5.2 Cifrari . . . 22
5.2.1 AES in modalità contatore . . . 22
5.2.2 TLS . . . 22
5.3 Funzioni di hash . . . 22

5.4 MAC e derivazione di chiavi . . . 23
5.4.1 HMAC . . . 23
5.4.2 Funzioni di derivazione . . . 23
5.5 Scambio di chiavi: X25519 . . . 24
5.6 Firme: Ed25519 e RSA . . . 24
5.7 Crittografia recente . . . 24

6 Architettura della rete **27**
6.1 I ruoli . . . 27
6.2 I livelli del protocollo . . . 28
6.3 Una pagina web, dall'inizio alla fine . . . 29
6.4 Implementazioni e versioni . . . 29
6.4.1 Due implementazioni . . . 29
6.4.2 Sottoprotocolli . . . 30

7 Canali e link protocol **33**
7.1 Che cos'è un canale . . . 33
7.2 L'identità di un relay . . . 34
7.3 L'handshake in sintesi . . . 34
7.4 Il livello TLS . . . 36
7.5 CERTS: dimostrare l'identità . . . 37
7.5.1 La cella . . . 37
7.5.2 Il certificato Ed25519 di Tor . . . 37
7.5.3 La verifica . . . 38
7.5.4 La catena RSA legacy . . . 38
7.5.5 Che cosa invia davvero un relay . . . 39
7.6 AUTH_CHALLENGE e AUTHENTICATE . . . 39
7.6.1 Chi si autentica . . . 39
7.6.2 La sfida . . . 39
7.6.3 La risposta . . . 40
7.7 NETINFO: ora e indirizzi . . . 40
7.8 Dopo l'handshake . . . 41
7.9 Laboratorio: aprire un canale verificato . . . 42

8 Le celle **47**
8.1 Perché le celle . . . 47
8.2 Dal flusso TLS alle celle . . . 48
8.3 Il formato delle celle . . . 48
8.3.1 Costanti . . . 48
8.3.2 Celle a lunghezza fissa . . . 49
8.3.3 Celle a lunghezza variabile . . . 49
8.3.4 Come distinguere i due formati . . . 49
8.3.5 Il caso speciale di VERSIONS . . . 50
8.4 Il CircID . . . 50
8.4.1 Chi sceglie il CircID . . . 51
8.5 I comandi di cella . . . 51
8.5.1 DESTROY . . . 52
8.5.2 RELAY_EARLY . . . 53
8.6 Dentro le celle RELAY . . . 53
8.6.1 Recognized e Digest: un filtro a due stadi . . . 54
8.6.2 Il padding del messaggio relay . . . 54

8.6.3 I comandi relay . . . 54
8.7 Verso un nuovo formato: proposta 340 e CGO . . . 56
8.8 Anatomia di una sessione . . . 56
8.9 Laboratorio: un codificatore di celle . . . 58

9 Costruzione dei circuiti: TAP, CREATE_FAST, ntor, ntor-v3 63
9.1 Un circuito, un salto alla volta . . . 63
9.2 Le celle e i messaggi . . . 64
9.2.1 CREATE2 e CREATED2 . . . 64
9.2.2 EXTEND2 ed EXTENDED2 . . . 65
9.3 Che cosa deve fare un handshake . . . 65
9.4 TAP: il primo handshake . . . 66
9.5 CREATE_FAST: quando la chiave pubblica non serve . . . 66
9.6 ntor . . . 66
9.6.1 Gli ingredienti . . . 67
9.6.2 Il protocollo . . . 67
9.7 ntor-v3 . . . 67
9.7.1 Che cosa cambia . . . 68
9.7.2 Il protocollo . . . 68
9.7.3 Le estensioni . . . 68
9.7.4 Quale handshake usa tor oggi . . . 68
9.8 Dalle chiavi negoziate alle chiavi del circuito . . . 69
9.9 Laboratorio: creare circuiti veri . . . 69

10 La cipolla: cifratura dei relay cell e CGO 75
10.1 Lo schema tradizionale . . . 75
10.1.1 In avanti: dal client verso il circuito . . . 75
10.1.2 All'indietro: dal circuito verso il client . . . 76
10.1.3 Il circuito come «tubo che perde» . . . 76
10.2 Riconoscere una cella . . . 76
10.3 Le debolezze di tor1 . . . 77
10.3.1 Malleabilità . . . 77
10.3.2 L'attacco di tagging . . . 77
10.3.3 Altri limiti . . . 78
10.4 Counter Galois Onion . . . 78
10.4.1 Le proprietà . . . 78
10.4.2 Come funziona, a grandi linee . . . 78
10.4.3 CGO in pratica . . . 79
10.5 Laboratorio: un circuito di tre salti . . . 79

11 Stream, SENDME e controllo della congestione 83
11.1 Gli stream . . . 83
11.1.1 Aprire uno stream: BEGIN e CONNECTED . . . 83
11.1.2 Trasferire dati: DATA . . . 85
11.1.3 Chiudere: END . . . 85
11.1.4 Il DNS all'exit . . . 85
11.1.5 Stream verso le directory: BEGIN_DIR . . . 85
11.2 Il controllo di flusso originale . . . 85
11.2.1 Il problema . . . 85
11.2.2 Finestre e SENDME . . . 86
11.2.3 SENDME autenticati . . . 86

11.3 Il controllo della congestione . . . 87
11.3.1 Il segnale: il tempo di andata e ritorno . . . 87
11.3.2 Gli algoritmi . . . 87
11.3.3 I parametri: perché 31 . . . 87
11.3.4 Niente più finestre di stream: XON e XOFF . . . 87
11.4 Laboratorio: scaricare un file attraverso Tor . . . 88

12 Il sistema delle directory **93**
12.1 Il problema . . . 93
12.2 Gli attori . . . 94
12.3 I documenti . . . 94
12.4 Il voto . . . 95
12.4.1 Il calendario . . . 95
12.4.2 Come si combinano i voti . . . 96
12.4.3 I flag . . . 96
12.4.4 La banda . . . 97
12.5 Anatomia di un consenso . . . 97
12.6 Verificare un consenso . . . 98
12.7 Come un client resta aggiornato . . . 99
12.8 Laboratorio: scaricare e verificare il consenso . . . 100

13 Selezione del percorso, guard e vanguards **103**
13.1 L'ordine e i vincoli . . . 103
13.2 Pesare con la banda . . . 104
13.2.1 Perché pesare . . . 104
13.2.2 Le posizioni non sono uguali . . . 104
13.2.3 I pesi della rete reale . . . 105
13.3 I guard . . . 105
13.3.1 Il problema che risolvono . . . 105
13.3.2 Tre, uno, due . . . 106
13.3.3 L'algoritmo . . . 106
13.4 Vanguards . . . 107
13.5 Laboratorio: la rete pubblica sotto la lente . . . 108

14 Onion services v3 **111**
14.1 Che cosa offre un onion service . . . 111
14.2 L'indirizzo . . . 112
14.3 Le chiavi accecate . . . 113
14.3.1 Il problema . . . 113
14.3.2 La soluzione . . . 113
14.3.3 Credenziali e chiavi offline . . . 113
14.4 Dove si trova il descrittore . . . 114
14.4.1 I periodi di tempo . . . 114
14.4.2 L'anello degli HSDir . . . 114
14.4.3 Solo richieste anonime . . . 115
14.5 Il descrittore . . . 115
14.6 Introduzione e rendezvous . . . 116
14.7 Scoperta ristretta . . . 118
14.8 Difese . . . 118
14.9 Laboratorio: trovare e leggere un descrittore . . . 119

15 Difese contro DoS e gestione delle risorse **123**
15.1 Perché il DoS è un problema di anonimato . . . 123
15.2 Le risorse fisiche . . . 124
15.2.1 Memoria . . . 124
15.2.2 CPU . . . 124
15.2.3 Banda . . . 124
15.3 Limiti per indirizzo . . . 124
15.3.1 Il codice e il consenso . . . 125
15.4 Onion service sotto attacco . . . 125
15.4.1 Limiti al punto di introduzione . . . 126
15.4.2 La prova di lavoro . . . 126
15.4.3 Uno sforzo lineare . . . 127
15.4.4 Il controllo dello sforzo . . . 127
15.5 Laboratorio: provocare le difese . . . 128

16 Censura, bridge e pluggable transports **131**
16.1 Come si blocca Tor . . . 131
16.2 I bridge . . . 132
16.2.1 Il problema della distribuzione . . . 132
16.2.2 Lox . . . 133
16.3 I pluggable transports . . . 133
16.3.1 L'architettura . . . 133
16.3.2 Il protocollo tra tor e il PT . . . 133
16.3.3 Lyrebird . . . 134
16.4 Le strategie . . . 134
16.4.1 obfs4: non assomigliare a niente . . . 134
16.4.2 meek: dietro un grande servizio . . . 135
16.4.3 Snowflake: tanti proxy effimeri . . . 135
16.4.4 WebTunnel: sembrare un sito web . . . 135
16.4.5 Conjure: nascondersi nello spazio degli indirizzi . . . 135
16.5 Laboratorio: un bridge obfs4 e lo sguardo del censore . . . 136

III Tor in pratica **139**

17 Installazione e torrc **141**
17.1 Quale Tor installare . . . 141
17.2 Il file torrc . . . 142
17.2.1 Controllare prima di usare . . . 142
17.3 Una configurazione da client . . . 142
17.3.1 Isolamento degli stream . . . 143
17.3.2 Scegliere i relay: con prudenza . . . 144
17.3.3 Altre porte . . . 144
17.4 La porta di controllo . . . 144
17.5 I log . . . 145

18 Control port e Stem **147**
18.1 Il protocollo di controllo . . . 147
18.1.1 I circuiti visti da tor . . . 148
18.1.2 I comandi principali . . . 149
18.2 Stem . . . 149

18.3 Laboratorio: l'isolamento degli stream dal vivo 150

19 Una rete Tor privata: Chutney e Shadow **153**
19.1 Perché una rete privata 153
19.2 Emulare o simulare 154
19.3 Il laboratorio del libro 154
19.3.1 Requisiti 155
19.3.2 Costruire l'immagine e avviare la rete 155
19.4 Com'è fatta la rete 156
19.5 Una rete di test non è la rete reale 157
19.6 Dentro la directory di un nodo 158
19.7 Usare la rete 159
19.7.1 Navigare attraverso la rete privata 159
19.7.2 Raggiungere l'onion service 159
19.7.3 Osservare i circuiti con Stem 159
19.7.4 Lavorare dall'interno del container 160
19.8 Un primo contatto con un relay 160
19.9 Shadow: la rete in un simulatore 161

20 Osservare il traffico: Wireshark e log **165**
20.1 Che cosa vede un osservatore 165
20.2 Gli strumenti 166
20.2.1 Decifrare il proprio TLS 166
20.3 Laboratorio: le celle dentro il TLS 167
20.4 I log di tor 168

21 Pubblicare onion service **171**
21.1 Il primo onion service 171
21.1.1 Non tradire il server 172
21.2 Onion-Location 173
21.3 Scoperta ristretta 173
21.4 Altre esigenze 175
21.4.1 Indirizzi vanity 175
21.4.2 Servizi a un salto 175
21.4.3 Onionbalance 175

22 Gestire un relay o un bridge **177**
22.1 Che tipo di nodo 177
22.2 La configurazione essenziale 178
22.2.1 La vita di un relay nuovo 178
22.2.2 Le chiavi 178
22.3 La politica di uscita 179
22.4 Laboratorio: una politica di uscita alla prova 180
22.5 Le famiglie 181
22.6 Un bridge 182

23 Tor Browser, Tails e Whonix **185**
23.1 Perché non basta un proxy 185
23.2 Tor Browser 186
23.2.1 Un circuito per sito 186
23.2.2 Contro le impronte 187

23.2.3 I livelli di sicurezza . . . 187
23.3 Oltre il browser . . . 188
23.3.1 Tails . . . 188
23.3.2 Whonix . . . 188

24 Misurare la rete: Tor Metrics e OONI **191**
24.1 Le fonti dei dati . . . 191
24.2 Contare utenti anonimi . . . 192
24.2.1 Rilevare la censura . . . 193
24.3 Le prestazioni . . . 193
24.4 Laboratorio: leggere un consenso reale . . . 193
24.5 OONI . . . 194

25 Aspetti legali ed etici in Italia e nell'UE **197**
25.1 Uno strumento e i suoi usi . . . 197
25.2 Chi gestisce un relay . . . 198
25.2.1 Il semplice trasporto . . . 198
25.2.2 Responsabilità civile e indagini penali . . . 198
25.2.3 Conservare i dati . . . 199
25.2.4 I log sono dati personali . . . 199
25.3 Chi fa ricerca . . . 200

IV Attacchi e difese **203**

26 Correlazione del traffico, fingerprinting, Sybil **205**
26.1 Dove si trova l'avversario . . . 205
26.2 La correlazione da estremo a estremo . . . 206
26.3 Laboratorio: correlare il proprio traffico . . . 207
26.4 Website fingerprinting . . . 208
26.5 Sybil: moltiplicare i propri relay . . . 208
26.6 Scoprire il guard . . . 209

27 Casi reali contro la rete **211**
27.1 2014: un segnale nel tipo di cella . . . 211
27.2 Laboratorio: il limite delle celle RELAY_EARLY . . . 212
27.3 2020–2021: KAX17 e la salute della rete . . . 213
27.4 2022–2023: negazione del servizio . . . 213
27.5 Lezioni . . . 214

28 Errori operativi e deanonimizzazione: lezioni **217**
28.1 Quattro livelli . . . 217
28.2 Errori dell'applicazione . . . 218
28.3 Laboratorio: tor riconosce la fuga DNS . . . 218
28.4 Errori del contenuto . . . 219
28.5 Errori di comportamento: l'insieme troppo piccolo . . . 220
28.6 Una lista di controllo . . . 220

V Ricerca e futuro **223**

29 La ricerca accademica su Tor **225**
29.1 Dove si fa ricerca su Tor . . . 225
29.2 Misurare l'anonimato . . . 226
29.3 Quattro metodi . . . 226
29.4 Laboratorio: tempo alla prima compromissione . . . 227
29.5 Fare ricerca in modo sicuro . . . 229

30 Arti: Tor in Rust **231**
30.1 Perché riscrivere tor . . . 231
30.2 L'architettura . . . 232
30.2.1 Un confronto: scrivere una cella . . . 233
30.3 Che cosa supporta . . . 233
30.4 Laboratorio: Arti nella rete privata . . . 234

31 Problemi aperti e progetti in corso **239**
31.1 Come cambia il protocollo . . . 239
31.2 Il computer quantistico . . . 240
31.3 Messaggi più grandi, e più piccoli . . . 241
31.4 Prestazioni: più percorsi . . . 241
31.5 Scalabilità: Walking Onions . . . 241
31.6 Altri fronti aperti . . . 242
31.7 Come contribuire . . . 242

A Glossario italiano–inglese **245**

B Complementi di crittografia **249**
B.1 Una curva, due forme . . . 249
B.1.1 I bit fissati delle chiavi X25519 . . . 249
B.1.2 Il punto all'infinito . . . 250
B.2 Da Montgomery a Edwards: il cross-certificato ntor . . . 250
B.3 La malleabilità dei cifrari a flusso . . . 251

C Guida alle specifiche torspec **253**
C.1 Dove si trovano . . . 253
C.2 Le convenzioni . . . 253
C.3 Mappa dei documenti . . . 254
C.4 Come leggere una sezione . . . 254
C.5 Arti e le specifiche . . . 254

D Soluzioni degli esercizi **255**

Bibliografia **277**

Indice analitico **283**

Prefazione

Tor è usato ogni giorno da milioni di persone, ed è raccontato quasi sempre per i suoi usi, veri o presunti: la censura aggirata, le fonti protette, il «dark web». Questo libro racconta invece come funziona. Parte dai byte che due relay si scambiano e risale, capitolo dopo capitolo, fino alle scelte di progetto, agli attacchi, ai casi reali e alle questioni ancora aperte. E chiede al lettore di non fidarsi: ogni meccanismo descritto nella parte tecnica viene costruito e provato in un laboratorio, su una rete Tor privata che gira su un normale computer.

Origine, metodo e invito alla revisione

Questo libro nasce da documenti e appunti di studio che ho prodotto nel corso degli anni, a partire dalla mia tesi di laurea triennale. Per trasformare quei materiali in una trattazione organica ho diretto l'impiego di strumenti di intelligenza artificiale, che hanno contribuito in modo sostanziale a sviluppare le spiegazioni, argomentare e redigere il testo, e realizzare figure, esercizi, codice e laboratori. L'IA non è stata un semplice correttore stilistico e non costituisce un revisore indipendente.

Nonostante i controlli tecnici e la mia revisione, possono rimanere errori, semplificazioni e informazioni non più aggiornate. Invito chi legge a segnalarli e a proporre correzioni attraverso la repository del libro. Ho scelto di distribuire gratuitamente l'opera attraverso i canali ufficiali del progetto, con i relativi sorgenti sotto licenza GNU GPL-3.0-or-later, per favorire la «diffusione dei saperi senza fondare poteri». La licenza stabilisce le condizioni di riuso e redistribuzione, anche commerciale.

A chi è rivolto questo libro

A studenti di informatica e di sicurezza, a professionisti che incontrano Tor nel proprio lavoro, a chi gestisce o vuole gestire un relay, a chi fa ricerca. Servono nozioni di base di reti (TCP/IP, l'idea di TLS), un po' di familiarità con la riga di comando e, per i laboratori, con Python. La crittografia necessaria è introdotta nel capitolo 5, con approfondimenti nell'appendice B.

Come leggere questo libro

Il libro è diviso in cinque parti, che si possono seguire in ordine o scegliendo un percorso:

Percorso storico e concettuale Parte I (capitoli 1–4) e Parte V (capitoli 29–31): che cos'è l'anonimato, da dove viene Tor, contro chi protegge, dove sta andando.

Percorso del protocollo Parte II (capitoli 5–16), da leggere in ordine, con i laboratori: è il cuore del libro.

Percorso pratico Parte III (capitoli 17–25): configurare tor, osservarne il traffico, pubblicare un onion service, gestire un relay, usare Tor Browser, misurare la rete, conoscere gli aspetti legali. Il capitolo 19 è il prerequisito di tutti i laboratori.

Percorso della sicurezza Parte IV (capitoli 26–28), preceduta dal capitolo 4: gli attacchi letti attraverso la ricerca, i casi reali, gli errori operativi.

Convenzioni

Ogni capitolo si apre con gli obiettivi e con una riga *Aggiornato al*, che indica la data e le versioni del software e delle specifiche su cui il contenuto è stato verificato: Tor cambia, e il lettore deve sapere a quale Tor si riferisce il testo. Nel testo *Tor* indica la rete e il progetto, *tor* il programma in C, *Arti* l'implementazione in Rust. I riferimenti alle specifiche hanno la forma [documento § sezione]; le proposte di modifica al protocollo sono indicate con il loro numero. I termini inglesi conservati nel testo sono raccolti, con il corrispondente italiano, nell'appendice A.

I riquadri separano il filo principale dalle digressioni: *approfondimenti*, *note storiche* e *avvertenze*. Gli esercizi sono graduati con stelle: una per la comprensione, due per l'applicazione, tre per un laboratorio o una piccola ricerca; le soluzioni sono nell'appendice D. I laboratori sono numerati da 1 a 20 e costruiscono, nella Parte II, un piccolo client Tor in Python (`torlab`); il codice e la rete di laboratorio accompagnano il libro, e un sistema di prove automatiche permette di rieseguirli dopo gli aggiornamenti. Tutto questo — il pacchetto `torlab`, i venti laboratori, la rete Docker di prova e gli strumenti di verifica — è nel repository del libro: `https://github.com/aleff-github/onion-routing-anatomy`.

Una nota etica

Tutti gli esperimenti del libro si svolgono sulla rete Tor privata del laboratorio, oppure usano la rete reale solo come un normale client, senza osservare né alterare il traffico di altre persone. È la regola che il Tor Research Safety Board raccomanda a chi fa ricerca, ed è anche una questione di legge (capitolo 25). I capitoli sugli attacchi hanno un taglio analitico e difensivo: spiegano che cosa è stato fatto e come ci si protegge, non come colpire qualcuno.

Ringraziamenti

Questo libro poggia sul lavoro del Tor Project e della sua comunità: le specifiche aperte, il codice di `tor` e di Arti, le proposte di modifica e la documentazione sono la fonte di ogni affermazione tecnica che segue, e la loro disponibilità pubblica è ciò che permette di verificare tutto alla fonte, anziché chiedere al lettore di fidarsi. Un grazie va anche a chi costruisce e mantiene gli strumenti liberi con cui il libro è stato scritto e i suoi laboratori messi alla prova: LaTeX, Python, Docker, Chutney, Wireshark e i molti altri progetti citati nel testo.

Nota sulle edizioni tradotte

Quest'opera è stata scritta in italiano; questa è la sua edizione originale.

Le traduzioni di quest'opera nelle lingue inglese, spagnola, russa e cinese semplificato sono state generate interamente mediante strumenti di intelligenza artificiale a partire dall'edizione originale italiana.

Il testo di riferimento dell'opera resta l'edizione italiana.

Parte I

Storia e fondamenti

Capitolo 1

Anonimato, sorveglianza e metadati

Aggiornato al 23 settembre 2026 — dati di Tor Metrics fino al 20/09/2026

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- distinguere riservatezza, anonimato, pseudonimia, non collegabilità e non osservabilità;
- spiegare perché i metadati di una comunicazione rivelano quanto, e talvolta più, del contenuto;
- descrivere che cos'è l'analisi del traffico e perché la cifratura da sola non la impedisce;
- riconoscere chi usa Tor e perché la diversità degli utenti è essa stessa una protezione.

Questo libro parla di un sistema costruito per proteggere un'informazione particolare: non *che cosa* diciamo in rete, ma *con chi* comunichiamo. Prima di vedere come funziona, conviene chiarire il problema che risolve, e perché è diverso da quello, più familiare, della cifratura.

1.1 Contenuto e metadati

Quando due computer comunicano su Internet, ogni pacchetto porta con sé due tipi di informazione. Il **contenuto** è ciò che le applicazioni si scambiano: il testo di un messaggio, la pagina di un sito. I **metadati** sono le informazioni *sulla* comunicazione: chi la invia, a chi, quando, quanto spesso, quanti dati. Una parte dei metadati è indispensabile al funzionamento della rete, perché i router devono sapere dove recapitare i pacchetti: l'indirizzo IP del destinatario, e quello del mittente per la risposta, viaggiano in chiaro in ogni pacchetto.

La cifratura, oggi diffusissima grazie a HTTPS, protegge il contenuto, ma lascia visibili i metadati. Chi osserva la rete, dal fornitore di connettività di un utente fino a chi controlla un grande nodo di interscambio, vede ancora che quell'utente si collega a un certo sito, a che ora, per quanto tempo e con quanto traffico.

1.1.1 Che cosa rivelano i metadati

Si tende a considerare i metadati innocui: «sanno solo con chi parlo, non che cosa dico». È un'impressione ingannevole. Qualche esempio basta a mostrarlo:

- una persona che visita alle due di notte il sito di un centro antiviolenza, poi quello di un avvocato divorzista;

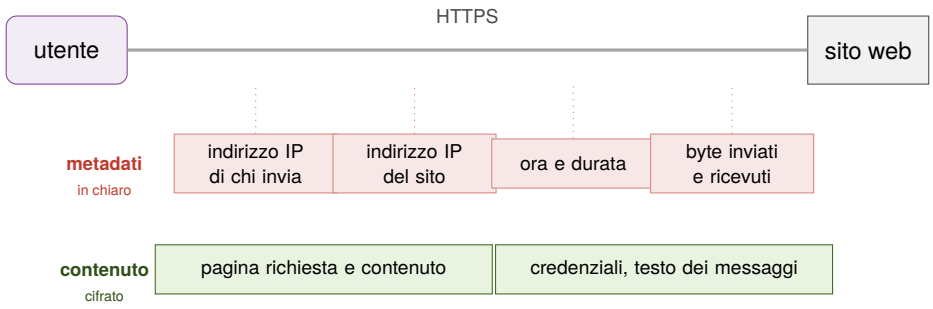


Figura 1.1: Che cosa vede chi osserva una connessione HTTPS. Il contenuto è cifrato, ma i metadati restano in chiaro: sono proprio questi che Tor cerca di proteggere.

- un giornalista che, nei giorni precedenti la pubblicazione di un'inchiesta, scambia messaggi con un dipendente del ministero di cui scrive;
- un dipendente che si collega ogni settimana al sito di un sindacato, o di un'associazione di pazienti di una certa malattia.

In nessuno di questi casi serve leggere il contenuto per capire moltissimo. E i metadati hanno un vantaggio, per chi li raccoglie: sono strutturati, facili da conservare e da analizzare in massa, e si possono incrociare tra loro. Un ex direttore dell'NSA e della CIA statunitensi, il generale Michael Hayden, lo ha riassunto in una frase diventata celebre durante un dibattito alla Johns Hopkins University il 1º aprile 2014: «uccidiamo persone sulla base dei metadati» (trad. nostra), subito precisando però «ma non è questo che facciamo con questi metadati» [Jus14]. La precisazione non toglie nulla al punto: chi raccoglie metadati li considera sufficienti per decisioni di quel peso.

1.2 L'analisi del traffico

L'**analisi del traffico** è lo studio dei metadati di una comunicazione per ricavarne informazioni, senza bisogno di leggerne il contenuto. È antica quanto le telecomunicazioni: già nelle guerre del Novecento, osservare quante trasmissioni partivano da una stazione radio nemica, verso chi e con quale ritmo, permetteva di ricostruire l'organizzazione di un esercito anche senza decifrarne i messaggi.

Su Internet l'analisi del traffico ha due forme principali:

Analisi degli indirizzi Chi osserva un collegamento vede gli indirizzi IP di mittente e destinatario. È la forma più semplice, e quella che Tor contrasta in modo diretto: nessun punto della rete vede entrambi.

Analisi dei volumi e dei tempi Anche senza vedere gli indirizzi completi, chi osserva i flussi di traffico può confrontarne le dimensioni e i tempi. Se un utente scarica 3 MB alle 10:15:02 e, in un altro punto della rete, un server invia 3 MB alle 10:15:02, è probabile che si tratti della stessa comunicazione. Questa **correlazione del traffico** è il limite principale di Tor, e la ritroveremo spesso (capitoli 4 e 26).

1.3 Un vocabolario preciso

La parola «anonimato» viene usata con significati diversi, e le confusioni hanno portato, nella storia di questi sistemi, a errori di progettazione. La letteratura scientifica, in particolare il vocabolario proposto da Pfitzmann e Hansen [PH10], distingue alcune nozioni:

Riservatezza Chi non è autorizzato non può leggere il contenuto. È ciò che offre la cifratura.

Anonimato Un soggetto non è identificabile all'interno di un insieme di soggetti, il suo **insieme di anonimato**. Non si è anonimi in assoluto, ma «tra» altri: più grande e più vario è l'insieme, maggiore è la protezione.

Pseudonimia Si agisce sotto un identificatore diverso dal proprio nome, che può essere stabile nel tempo. Un account con un nome di fantasia è uno pseudonimo, non una forma di anonimato: tutte le sue azioni sono collegabili tra loro.

Non collegabilità Un osservatore non riesce a stabilire se due elementi di interesse (due messaggi, un utente e un sito, due visite) sono legati tra loro. È la proprietà che Tor cerca di garantire tra un utente e le sue destinazioni.

Non osservabilità Un osservatore non riesce nemmeno a stabilire *se* una comunicazione sta avvenendo. È una proprietà molto più forte, che Tor non offre: chi osserva l'utente vede che usa Tor, a meno di usare un pluggable transport (capitolo 16).

Nota storica: Separare l'identificazione dall'instradamento

I tre ricercatori del Naval Research Laboratory che inventarono l'onion routing nel 1995, David Goldschlag, Michael Reed e Paul Syverson, preferivano non parlare di anonimato. Il loro obiettivo, scrivevano già in un articolo del 1996 che Syverson riprende ricordando quegli anni, era «separare l'identificazione dall'instradamento» [Syv11]: un utente può voler dimostrare la propria identità al sistema con cui parla, per esempio il computer del proprio ufficio, senza che tutti i punti intermedi della rete sappiano che qualcuno, da quel luogo, si sta collegando a quell'ufficio.

1.4 Chi ha bisogno di anonimato

Secondo le stime di Tor Metrics, nell'agosto e nel settembre 2026 si sono collegati ogni giorno alla rete Tor, senza usare bridge, tra 2,8 e 3,5 milioni di utenti, con una media di circa 3,2 milioni; dall'Italia, tra 44 000 e 83 000, con una media di circa 60 000 (stime con un ampio margine di incertezza, perché la rete non conta gli utenti, li stima; capitolo 24). Chi sono? Le categorie sono molto diverse:

- persone comuni che non vogliono essere tracciate da inserzionisti, fornitori di connettività o chi controlla la rete in cui si trovano;
- giornalisti e le loro fonti, attivisti, difensori dei diritti umani, soprattutto in paesi con governi autoritari;
- persone che vivono sotto censura e usano Tor per raggiungere siti bloccati (capitolo 16);
- vittime di violenza o di persecuzione che devono nascondere la propria posizione;
- aziende che non vogliono rivelare ai concorrenti quali ricerche stanno facendo;
- forze dell'ordine e militari, che devono poter visitare siti o raccogliere informazioni senza rivelare chi sono: l'onion routing è nato in un laboratorio della marina statunitense proprio per questo.

Una piccola parte degli utenti usa Tor per attività illegali, come accade per qualsiasi strumento di comunicazione. Il dibattito pubblico tende a concentrarsi su questa parte; noi lo affronteremo con i dati e con i casi concreti nella Parte IV e nel capitolo 25.

1.4.1 L'anonimato ama la compagnia

C'è una ragione più profonda per cui la varietà degli utenti conta. Un sistema di anonimato usato solo da una categoria di persone, per esempio solo dai militari, non proteggerebbe nessuno: chi osserva un utente collegarsi alla rete saprebbe subito che è un militare, e chi osserva il traffico in uscita saprebbe che proviene dai militari. Syverson lo osserva per la rete che la marina statunitense avrebbe potuto costruire per sé: tutto ciò che ne usciva sarebbe stato riconoscibile come traffico della marina [Syv11]. L'anonimato di ciascuno dipende da quanti e quanto diversi sono gli altri utenti: la frase che riassume questa idea, titolo di un articolo di Dingledine e Mathewson, è «l'anonimato ama la compagnia» [DM06]. Per questo Tor è progettato per essere usabile da chiunque, gratuito e aperto: più persone lo usano, meglio protegge tutti.

Riepilogo

- La cifratura protegge il contenuto; i metadati (chi, con chi, quando, quanto) restano visibili e rivelano moltissimo.
- L'analisi del traffico ricava informazioni dagli indirizzi, dai volumi e dai tempi delle comunicazioni.
- Anonimato significa non essere identificabili in un insieme; va distinto da pseudonimia, non collegabilità e non osservabilità.
- Tor serve a milioni di utenti molto diversi, e la loro diversità è parte della protezione.

Esercizi

Esercizio 1.1 ⋆ Elenca i metadati che un fornitore di connettività può osservare quando un suo cliente legge un articolo su un sito di notizie in HTTPS. Quali restano visibili se il cliente usa una VPN? E se usa Tor?

Esercizio 1.2 ⋆ Classifica come anonimato, pseudonimia o nessuno dei due: un account social con un nome di fantasia; un messaggio lasciato in una bacheca pubblica senza firma; una lettera firmata con le iniziali; una donazione in contanti in una cassetta.

Esercizio 1.3 ⋆⋆ Un sito vede arrivare, in un'ora, visite da 50 utenti Tor diversi. Uno di essi scrive un commento con un errore di ortografia tipico di chi parla italiano, e il commento è pubblicato alle 3:00 ora italiana. Come cambia l'insieme di anonimato di quell'utente? Che cosa insegna l'esempio sul ruolo del comportamento dell'utente?

Esercizio 1.4 ⋆⋆ Confronta una VPN commerciale e Tor rispetto a chi può collegare l'utente alle sue destinazioni. Dove si concentra la fiducia in ciascun caso?

Per approfondire

- A. Pfitzmann, M. Hansen, *A terminology for talking about privacy by data minimization* [PH10]: il vocabolario di riferimento.
- P. Syverson, *A Peel of Onion* [Syv11]: la storia dell'onion routing raccontata da uno dei suoi inventori.
- Tor Metrics, `https://metrics.torproject.org/`: le stime degli utenti per paese.

Capitolo 2

Le radici: mix network, remailer, Crowds, Freedom

Aggiornato al 23 settembre 2026 — fonti storiche citate nel capitolo

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- spiegare l'idea della *mix* di David Chaum e perché è alla base di tutti i sistemi di anonimato;
- distinguere i sistemi ad alta latenza, per la posta, da quelli a bassa latenza, per la navigazione;
- ricostruire l'evoluzione dei remailer, da anon.penet.fi a Mixminion;
- collocare Crowds, PipeNet e Freedom nella storia che porta a Tor.

Tor non è nato dal nulla. È il punto d'arrivo, per ora, di quarant'anni di ricerca e di sistemi reali, alcuni riusciti e molti falliti, ciascuno dei quali ha insegnato qualcosa. Questo capitolo ripercorre le tappe principali, fino alla soglia dell'onion routing.

2.1 La mix di Chaum (1981)

Nel 1981 David Chaum pubblicò un articolo di cinque pagine destinato a fondare un intero campo di ricerca: *Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms* [Cha81]. Il problema era la posta elettronica: come inviare un messaggio senza che chi osserva la rete possa collegare mittente e destinatario, anche se la rete intera è sotto osservazione?

La soluzione è la **mix**: un server che riceve messaggi cifrati con la propria chiave pubblica, li decifra, ne raccoglie un gruppo (un *lotto*) e li rispedisce tutti insieme, in ordine diverso da quello di arrivo. Tre accorgimenti rendono impossibile collegare un messaggio in ingresso a uno in uscita:

- il messaggio in uscita ha un aspetto completamente diverso da quello in ingresso, perché la mix ha tolto uno strato di cifratura;
- tutti i messaggi hanno la stessa lunghezza;
- l'ordine viene rimescolato, e i messaggi escono insieme, così che nemmeno i tempi li collegano.

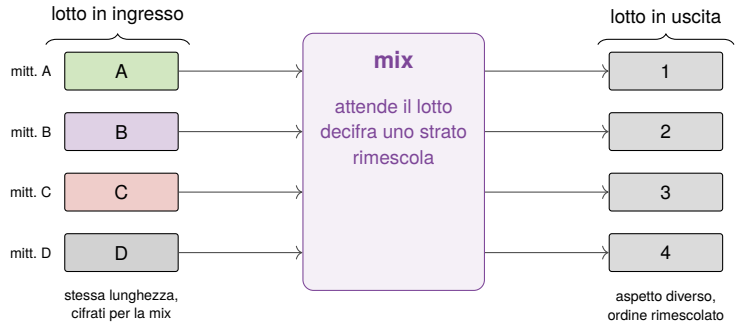


Figura 2.1: Una mix di Chaum. Chi osserva l'ingresso e l'uscita non può collegare i messaggi: sono tutti della stessa lunghezza, cambiano aspetto perché la mix toglie uno strato di cifratura, ed escono insieme in ordine diverso.

Per non dover fidarsi di una sola mix, il mittente può farne attraversare al messaggio una **catena**, cifrandolo a strati con le chiavi di ciascuna: ogni mix toglie il proprio strato e scopre solo la mix successiva. Basta che una sola mix della catena sia onesta perché il collegamento resti nascosto. È, in germe, l'idea della cipolla.

Chaum propose nello stesso articolo gli *indirizzi di ritorno anonimi*, con cui il destinatario può rispondere senza conoscere il mittente, e l'uso di pseudonimi digitali. Nel 1988 aggiunse un secondo meccanismo, le *DC-net* [Cha88], che offrono un anonimato incondizionato al prezzo di costi molto alti e che hanno avuto un'influenza soprattutto teorica.

2.2 Alta e bassa latenza

La mix di Chaum protegge molto bene, ma è **lenta**: un messaggio attende in ogni mix che si formi un lotto, e può impiegare minuti o ore ad arrivare. Per la posta elettronica è accettabile. Per la navigazione web, dove una pagina deve arrivare in pochi secondi, no.

È la grande divisione di questo campo:

Sistemi ad alta latenza (mix, remailer): ritardano e rimescolano i messaggi, e resistono anche a un osservatore che vede tutta la rete, ma sono adatti solo a comunicazioni che possono aspettare.

Sistemi a bassa latenza (onion routing, Tor): inoltrano i dati appena arrivano, e sono quindi usabili per il web, la chat e le connessioni interattive; ma chi osserva entrambi gli estremi può correlare i tempi e i volumi del traffico.

Tor sceglie consapevolmente la seconda strada: un sistema che quasi nessuno usa perché troppo lento non protegge nessuno (sezione 1.4).

2.3 I remailer

Negli anni Novanta le idee di Chaum diventarono servizi reali, i **remailer**: server che ricevono un messaggio di posta e lo inoltrano togliendo le informazioni sul mittente.

anon.penet.fi (1993–1996). Il remailer più famoso fu gestito in Finlandia da Johan «Julf» Helsingius dal 1993 al 1996. Non era una mix: assegnava a ogni utente uno *pseudonimo* e conservava la tabella che collegava gli pseudonimi agli indirizzi reali, per poter inoltrare le risposte. Arrivò a gestire migliaia di messaggi al giorno. Proprio quella tabella ne decretò la

fine: nel 1995 e nel 1996 la Chiesa di Scientology, con l'aiuto della polizia finlandese, ottenne l'identità di alcuni utenti che avevano diffuso documenti della chiesa, e nel 1996 Helsingius chiuse il servizio, citando l'incertezza legale su Internet in Finlandia. La lezione, ripresa esplicitamente dai progettisti dell'onion routing [Syv11], è che un sistema che conserva il collegamento tra utenti e identità è un punto unico di fallimento: chi lo gestisce può essere costretto a consegnarlo.

Cypherpunk, Mixmaster, Mixminion. In parallelo, la comunità dei *cypherpunk* sviluppò remailer senza tabelle, in cui il mittente cifra a strati il messaggio per una catena di server. Le generazioni successive, Mixmaster e poi Mixminion [DDM03], introdussero messaggi di lunghezza fissa, veri lotti e protezioni contro la ripetizione dei messaggi. Mixminion, del 2003, ha tra i suoi autori Roger Dingledine e Nick Mathewson: le stesse persone che nello stesso periodo stavano progettando Tor.

2.4 Verso la bassa latenza

PipeNet (1995). In un messaggio alla lista dei cypherpunk del febbraio 1995, Wei Dai propose PipeNet, un sistema a bassa latenza in cui tutti i partecipanti inviano sempre traffico a velocità costante, così che non ci sia nulla da correlare. È sicuro contro l'analisi dei tempi, ma a un prezzo inaccettabile: se un solo partecipante smette di trasmettere, l'intera rete deve fermarsi [Syv11]. Non fu mai realizzato, ma resta il riferimento di ciò che servirebbe per battere la correlazione del traffico.

Onion routing (1996). Nello stesso anno al Naval Research Laboratory nasceva l'onion routing, che racconteremo nel capitolo 3.

Crowds (1998). Michael Reiter e Aviel Rubin proposero Crowds [RR98]: gli utenti formano una «folla», e ogni richiesta web viene inoltrata a un membro a caso, che con una certa probabilità la invia al sito e altrimenti la passa a un altro membro. Il sito vede la richiesta arrivare da un membro qualsiasi, e ogni membro non sa se chi gli passa la richiesta ne sia l'autore. Crowds non usa cifratura a strati e protegge soltanto dal sito e dagli altri membri, non da chi osserva la rete; ma ha introdotto un modo rigoroso di ragionare sull'anonimato come probabilità, e un'analisi degli attacchi dei membri complici che anticipa quella dei guard.

Freedom (1999–2001). La rete più importante prima di Tor per dimensioni fu Freedom, dell'azienda canadese Zero-Knowledge Systems: annunciata alla fine del 1998, funzionò dalla fine del 1999 alla fine del 2001 [Syv11]. Era un servizio commerciale, in cui gli utenti pagavano un abbonamento e i server erano gestiti dall'azienda e da partner. Chiuse perché gli abbonamenti non bastavano a sostenerla. La sua storia si presta a due lezioni, che Tor, come vedremo, ha di fatto seguito: una rete di anonimato è più solida se è gestita da molti soggetti indipendenti anziché da un'azienda sola, e il modello di finanziamento conta quanto la crittografia (sezione 3.4).

Approfondimento: Molte cipolle

Negli stessi anni furono proposti molti altri sistemi di onion routing, come Tarzan e MorphMix, completamente decentralizzati, o Cebolla, da cui Tor ha preso l'idea di costruire i circuiti un salto alla volta con uno scambio Diffie-Hellman per ogni salto [Syv11]. Altri progetti sono nati in parallelo a Tor seguendo strade diverse: I2P, una rete pensata

soprattutto per servizi interni e tuttora attiva, e AN.ON, nato all'Università di Dresda con il client JAP (e offerto anche come servizio commerciale con il nome JonDonym), basato su catene fisse di mix gestite da operatori noti. Ciascuno sceglie un diverso equilibrio tra decentramento, fiducia e prestazioni.

Riepilogo

- La mix di Chaum (1981) nasconde il legame tra messaggi in ingresso e in uscita cambiandone l'aspetto, uniformandone la lunghezza e rimescolandone l'ordine; una catena di mix cifrata a strati anticipa la cipolla.
- I sistemi ad alta latenza resistono a osservatori molto potenti ma sono lenti; quelli a bassa latenza sono usabili, ma vulnerabili alla correlazione dei tempi.
- anon.penet.fi mostrò il rischio di un sistema che conserva l'identità degli utenti; Mixmaster e Mixminion svilupparono le mix per la posta.
- PipeNet, Crowds e Freedom esplorarono la bassa latenza con compromessi diversi; Freedom mostrò l'importanza di una rete gestita da molti e di un modello sostenibile.

Esercizi

Esercizio 2.1 ⋆ Una mix raccoglie lotti di 100 messaggi. Un osservatore vede entrare i 100 messaggi e uscire i 100 messaggi. Se conosce il mittente di un messaggio in ingresso, qual è la probabilità di indovinare il corrispondente in uscita tirando a caso? Che cosa succederebbe se 99 dei 100 messaggi fossero stati inviati dall'osservatore stesso?

Esercizio 2.2 ⋆⋆ Progetta, a grandi linee, un remailer che permetta di rispondere agli utenti senza conservare una tabella pseudonimo–indirizzo. Quale idea di Chaum ti serve?

Esercizio 2.3 ⋆⋆ In Crowds, con probabilità di inoltro $p_f = 3/4$, quanti membri attraversa in media una richiesta prima di arrivare al sito? Perché un membro che riceve una richiesta non può essere sicuro che chi gliela passa ne sia l'autore?

Per approfondire

- D. Chaum, *Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms* [Cha81].
- M. Reiter, A. Rubin, *Crowds: Anonymity for Web Transactions* [RR98].
- G. Danezis, R. Dingledine, N. Mathewson, *Mixminion: Design of a Type III Anonymous Remailer Protocol* [DDM03].
- La *Anonymity Bibliography* di Free Haven, `https://www.freehaven.net/anonbib/`: la raccolta di riferimento degli articoli del settore.

Capitolo 3

Dall'onion routing del NRL al Tor Project

Aggiornato al 23 settembre 2026 — Syverson, A Peel of Onion (2011); pagina storica del Tor Project

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- ricostruire la nascita dell'onion routing al Naval Research Laboratory e le sue tre generazioni;
- spiegare che cosa distingue Tor dalle generazioni precedenti;
- raccontare le tappe principali della storia del Tor Project, dal 2002 a oggi;
- conoscere l'origine del nome «Tor».

3.1 Al Naval Research Laboratory

Nel 1995 tre ricercatori del Naval Research Laboratory (NRL) statunitense, David Goldschlag, Michael Reed e Paul Syverson, si chiesero se fosse possibile creare connessioni su Internet che non rivelassero chi parla con chi [Theb]. Il loro scopo non era l'anonimato in sé, ma, come abbiamo visto, «separare l'identificazione dall'instradamento» (sezione 1.3). Il primo progetto fu presentato all'*Information Hiding Workshop* nel maggio del 1996 [GRS96]; entro l'anno ne seguì un secondo, che separava il programma dell'utente, che costruisce i circuiti, dai nodi che li trasportano [Syv11]. Già nel 1996 il laboratorio ottenne il permesso di pubblicare il codice, prima che l'idea di software *open source* fosse diffusa: i progettisti sapevano che nessuno si sarebbe fidato di una rete di cui non poteva esaminare il codice.

Il nome viene dalla struttura dei dati: un messaggio avvolto in strati di cifratura, ciascuno dei quali un nodo toglie come si sbuccia una cipolla.

3.2 Tre generazioni

Syverson distingue tre generazioni di progetti dell'NRL [Syv11].

Prima generazione. Circuiti di lunghezza fissa di cinque nodi, pensati anche per collegare tra loro le reti protette di due organizzazioni: con tre salti, un nodo intermedio ostile avrebbe visto subito le due reti che comunicavano; con quattro, chi comprometteva il nodo vicino a un estremo avrebbe saputo quale altro nodo attaccare per ricostruire l'intero circuito; con

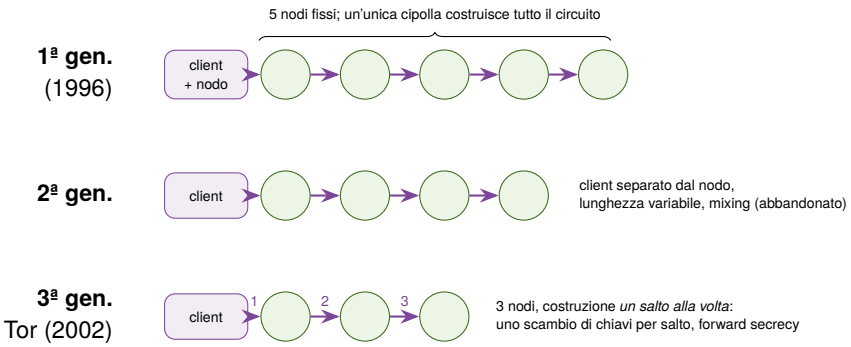


Figura 3.1: Le tre generazioni dell'onion routing. Il cambiamento decisivo di Tor non è il numero di salti, ma il modo di costruire il circuito: non più un'unica cipolla preparata dal client, ma un'estensione alla volta, con uno scambio di chiavi per ciascun nodo.

cinque, nessuna delle due scorciatoie è disponibile senza aver prima compromesso un altro nodo. Il client era integrato nel nodo; la rete era statica, di venti o cento nodi gestiti da organizzazioni stabili, con configurazione e chiavi distribuite a mano. C'erano proxy specifici per ogni applicazione (web, login remoto, posta), perché il protocollo SOCKS non era ancora diffuso. Il circuito veniva costruito con un'unica cipolla, cifrata con le chiavi pubbliche di tutti i nodi.

Seconda generazione. Client separato dal nodo, circuiti di lunghezza variabile, meccanismi di mixing e di padding tra i circuiti, poi abbandonati. La diffusione delle informazioni sulla rete era prevista ma non fu mai completamente realizzata.

Terza generazione: Tor. Il cambiamento più importante è la costruzione dei circuiti **un salto alla volta**, con uno scambio Diffie-Hellman con ciascun nodo (sezione 9.1), un'idea presa dal progetto Cebolla. Ne derivano la forward secrecy e la possibilità di verificare ogni salto man mano. A essa si aggiungono i server di directory, il supporto a SOCKS, i servizi nascosti con punti di rendezvous, e soprattutto un progetto pensato per essere distribuito e gestito da volontari di tutto il mondo.

Nota storica: Il nome

Il titolo dell'articolo del 2004, *Tor: The Second-Generation Onion Router*, ha generato due equivoci che lo stesso Syverson ha poi ammesso [Syv11]. Tor è la *terza* generazione, non la seconda; e la «r» di Tor non sta per *router*. Quando Roger Dingledine iniziò a lavorare al progetto dell'NRL, esistevano molti sistemi di onion routing, e alla domanda «quale?» rispondeva: «*the onion routing*», quello originale. Fu Rachel Greenstadt a notare che le iniziali formavano un bell'acronimo. Dingledine osservò che funzionava anche come acronimo ricorsivo, *Tor's onion routing*, e decise che si sarebbe scritto «Tor», non «TOR»: una parola comune, che in tedesco significa anche *porta*, come quella di una città. Nel resto del libro scriviamo *Tor* per la rete e il progetto, e *tor*, in minuscolo, per il programma.

3.3 La rete Tor

Roger Dingledine si unì al progetto dell'NRL all'inizio degli anni Duemila, seguito poco dopo da Nick Mathewson. La rete Tor fu attivata nell'**ottobre 2002**, con il codice rilasciato fin da

subito con una licenza libera; alla fine del 2003 contava circa una dozzina di nodi gestiti da volontari, quasi tutti negli Stati Uniti e uno in Germania [Theb]. Nell'agosto del 2004 Dingledine, Mathewson e Syverson presentarono il progetto al simposio USENIX Security [DMS04].

Da lì, le tappe principali [Theb]:

Anno	Evento
2002	la rete Tor entra in funzione, con codice libero
2004	la Electronic Frontier Foundation inizia a finanziare il lavoro di Dingledine e Mathewson
2006	nasce The Tor Project, Inc., organizzazione senza scopo di lucro statunitense (501(c)(3))
2007	inizia lo sviluppo dei bridge contro la censura (capitolo 16)
2008	inizia lo sviluppo di Tor Browser (capitolo 23)
2010–2011	Tor diventa uno strumento importante durante le Primavere arabe
2013	le rivelazioni di Edward Snowden sulla sorveglianza di massa, in cui Tor ha un ruolo centrale
2017	onion service v3 (capitolo 14)
2021	disattivazione degli onion service v2
2022–2023	controllo della congestione (sezione 11.3) e prova di lavoro per gli onion service (capitolo 15)
2024	WebTunnel; migrazione della distribuzione dei bridge a rdsys
2025–2026	CGO in tor (sezione 10.4); Arti come client e per gli onion service (capitolo 30)

3.4 Un'organizzazione e una comunità

The Tor Project, Inc. sviluppa il software, mantiene le specifiche, gestisce le directory authority insieme a operatori indipendenti, misura la rete e sostiene la comunità. Ma la rete non gli appartiene: i relay sono gestiti da migliaia di volontari, individui, università, associazioni, in decine di paesi. È una scelta di progetto, maturata guardando all'esperienza di Freedom (sezione 2.4): nessun soggetto singolo deve poter controllare la rete, né essere costretto a compromettere gli utenti.

Il finanziamento dell'organizzazione è un tema ricorrente di discussione pubblica: nel corso degli anni sono arrivati fondi da agenzie governative, soprattutto statunitensi, da fondazioni private e da donazioni individuali. L'organizzazione pubblica ogni anno i propri bilanci. La domanda «ci si può fidare di un software finanziato anche da un governo?» ha una risposta tecnica più che politica: il codice e le specifiche sono pubblici e verificabili, le decisioni di progetto sono discusse apertamente nelle proposte (capitolo 31), e la sicurezza del sistema non dipende dalla fiducia nel Tor Project ma dalla distribuzione della rete tra molti operatori indipendenti (capitolo 4).

Approfondimento: Le rivelazioni del 2013

Tra i documenti diffusi da Snowden nel 2013 c'erano presentazioni interne della NSA dedicate a Tor, che riconoscevano la difficoltà di deanonimizzarne gli utenti in modo sistematico e descrivevano invece attacchi mirati contro i singoli utenti, sfruttando vulnerabilità dei browser anziché della rete. È una conferma indiretta di un principio che ritroveremo spesso: le debolezze più pericolose spesso non stanno nel protocollo, ma attorno a esso (capitolo 28).

Riepilogo

- L'onion routing nasce nel 1995 al Naval Research Laboratory (Goldschlag, Reed, Syverson), con l'obiettivo di separare l'identificazione dall'instradamento.

- La prima generazione usava circuiti fissi di cinque nodi e una rete statica; Tor, la terza, costruisce i circuiti un salto alla volta, con forward secrecy, directory e rendezvous.
- «Tor» viene da *the onion routing*; si scrive con la sola iniziale maiuscola.
- La rete è attiva dal 2002; The Tor Project, Inc. esiste dal 2006; i relay sono gestiti da volontari indipendenti.

Esercizi

Esercizio 3.1 ⋆ Perché la prima generazione usava circuiti di cinque nodi? Ricostruisci il ragionamento su tre e quattro salti riportato nel capitolo, e spiega perché Tor si accontenta di tre.

Esercizio 3.2 ⋆⋆ Confronta la costruzione di un circuito con un'unica cipolla (prima generazione) e quella telescopica di Tor rispetto alla forward secrecy e alla possibilità di scoprire subito un nodo che non risponde.

Esercizio 3.3 ⋆⋆ Scrivi una breve argomentazione (una pagina) sulla domanda «ci si può fidare di Tor, visto chi lo ha finanziato?», distinguendo tra ciò che si può verificare tecnicamente e ciò che richiede fiducia.

Per approfondire

- P. Syverson, *A Peel of Onion* [Syv11].
- D. Goldschlag, M. Reed, P. Syverson, *Hiding Routing Information* [GRS96].
- La pagina storica del Tor Project [Theb].

Capitolo 4

Il modello di minaccia di Tor

Aggiornato al 23 settembre 2026 — Dingledine, Mathewson, Syverson (2004), sezione 3

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- spiegare che cos'è un modello di minaccia e perché ogni sistema di sicurezza ne ha uno;
- elencare gli obiettivi e i «non obiettivi» dichiarati dai progettisti di Tor;
- descrivere l'avversario da cui Tor vuole proteggere e quello da cui non può;
- riconoscere che cosa Tor protegge e che cosa resta a carico dell'utente e delle applicazioni.

Nessun sistema di sicurezza protegge da tutto. Un **modello di minaccia** descrive, in modo esplicito, contro *chi* e contro *che cosa* un sistema vuole proteggere, e quali ipotesi fa. È il criterio con cui giudicarlo: un attacco che rientra nel modello e riesce è un difetto; un attacco che il modello esclude dichiaratamente non lo è, ma è un limite che l'utente deve conoscere. Questo capitolo presenta il modello di Tor come lo formularono i progettisti nel 2004 [DMS04], e come si è evoluto.

4.1 Gli obiettivi

L'obiettivo principale è impedire a un avversario di **collegare** gli interlocutori di una comunicazione, o più comunicazioni di uno stesso utente. Quattro criteri, dichiarati nell'articolo del 2004, hanno guidato le scelte di progetto:

Diffusione Il sistema deve funzionare nel mondo reale: non deve costare troppo ai volontari, non deve esporre gli operatori a responsabilità eccessive, non deve richiedere modifiche al sistema operativo né che i siti web installino qualcosa.

Usabilità Un sistema difficile ha meno utenti, e con meno utenti l'anonimato è minore: l'usabilità, scrivono gli autori, non è una comodità ma «un requisito di sicurezza».

Flessibilità Il protocollo deve essere ben specificato, così da servire come base per la ricerca futura.

Semplicità Solo tecniche comprese e consolidate: ogni funzione in più è una complicazione per l'analisi della sicurezza.

4.2 I non obiettivi

Altrettanto importante è ciò che Tor dichiara di *non* fare:

Non è peer-to-peer Tor non cerca di funzionare con migliaia di nodi di breve durata, molti dei quali potenzialmente ostili; usa un insieme di relay elencati e verificati dalle autorità (capitolo 12).

Non è sicuro contro gli attacchi da estremo a estremo Tor non pretende di risolvere la correlazione dei tempi o gli attacchi di intersezione da parte di chi osserva entrambi gli estremi.

Non normalizza i protocolli Tor trasporta i dati delle applicazioni così come sono: se un'applicazione rivela l'identità dell'utente, Tor non può impedirlo. Allora serviva un proxy filtrante; oggi questo compito è di Tor Browser (capitolo 23).

Non è steganografico Tor non nasconde il fatto che un utente è collegato alla rete. I pluggable transports, nati dopo, attenuano questo limite (capitolo 16).

4.3 L'avversario

Molte analisi teoriche dei sistemi di anonimato assumono un **avversario passivo globale**: qualcuno che osserva tutto il traffico di Internet. Nessun sistema pratico a bassa latenza protegge da un avversario del genere, e Tor lo dice apertamente. Il suo avversario è più realistico, e per certi versi più ampio [DMS04]. Può:

- osservare *una parte* del traffico della rete;
- generare, modificare, cancellare o ritardare traffico;
- gestire relay propri;
- compromettere una parte dei relay degli altri.

Con questi mezzi può tentare molti attacchi: correlare il traffico in ingresso e in uscita osservando i bordi della rete; compromettere relay o chiavi; ripetere traffico; negare il servizio ai relay onesti per spingere gli utenti verso i propri (capitolo 15); marcare il traffico per riconoscerlo altrove (sezione 10.3); dare agli utenti visioni diverse della rete attraverso le directory (capitolo 12); rendere la rete inaffidabile perché gli utenti la abbandonino per sistemi meno sicuri.

4.3.1 Conferma e analisi

I progettisti distinguono due tipi di attacchi basati sul traffico. Negli attacchi di **conferma** l'avversario ha già un sospetto («Alice parla con Bob») e osserva entrambi gli estremi per verificarlo: contro questi Tor, come ogni sistema a bassa latenza, non offre una protezione completa. Negli attacchi di **analisi**, l'avversario usa il traffico per scoprire *dove* guardare, cioè quali punti della rete attaccare: è da questi che Tor cerca di difendersi. Tutta la selezione dei percorsi, con i guard e i pesi di banda (capitolo 13), è pensata per rendere improbabile che un avversario si trovi ai due estremi di un circuito.

Attenzione: La frase da ricordare

Tor protegge da chi osserva *una parte* della rete, non da chi riesce a osservare *entrambi gli estremi* di una comunicazione. Un avversario che controlla, o osserva, sia la connessione dell'utente verso il guard sia quella dell'exit verso il sito può correlare i tempi e i volumi del traffico. Per un'organizzazione con accesso alle reti di un intero paese, sia l'utente sia il sito possono trovarsi nello stesso territorio: è un caso concreto, non teorico.

4.4 Che cosa Tor protegge, e che cosa no

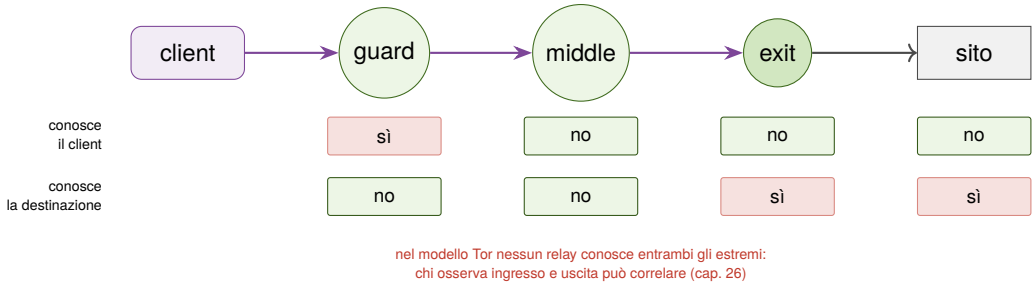


Figura 4.1: Che cosa sa ciascun punto del percorso. La protezione di Tor nasce dalla separazione: il guard conosce il client ma non la destinazione, l'exit la destinazione ma non il client.

Tor protegge da	Tor non protegge da
un sito che vuole conoscere l'indirizzo IP del visitatore	ciò che l'utente stesso rivela (accesso a un account, dati personali)
un osservatore della rete locale o il fornitore di connettività, che vede solo una connessione verso Tor	un osservatore che vede entrambi gli estremi della comunicazione
un singolo relay, che conosce al più un estremo	un avversario che controlla sia il guard sia l'exit di un circuito
un exit che vuole sapere chi ha inviato una richiesta	un exit che legge o modifica traffico non cifrato (senza HTTPS)
la censura per indirizzo, con i bridge	un'applicazione o un browser vulnerabile, compromesso o configurato male

Il modello si è ampliato nel tempo in direzioni che l'articolo del 2004 non poteva prevedere: la censura su scala nazionale (capitolo 16), la protezione dal *fingerprinting* del browser (capitolo 23), la difesa dagli attacchi DoS (capitolo 15), la minaccia futura dei computer quantistici (capitolo 31); resta però fermo il principio: conoscere il modello di minaccia è il primo passo per usare Tor in modo sicuro.

Riepilogo

- Un modello di minaccia dichiara contro chi e contro che cosa un sistema protegge.
- Tor mira a impedire il collegamento tra utenti e destinazioni, privilegiando diffusione, usabilità, flessibilità e semplicità.
- Non è peer-to-peer, non resiste agli attacchi da estremo a estremo, non normalizza i protocolli, non nasconde l'uso di Tor (senza pluggable transports).

- L'avversario osserva una parte della rete, gestisce e compromette relay e può agire attivamente; Tor si difende soprattutto dagli attacchi di analisi, meno da quelli di conferma.

Esercizi

Esercizio 4.1 ⋆ Per ciascuno di questi scenari, di' se Tor protegge l'utente e perché: (a) il gestore del Wi-Fi di un bar vuole sapere quali siti visita un cliente; (b) un sito vuole conoscere l'indirizzo IP dei visitatori; (c) l'utente accede con il proprio account personale a un social network attraverso Tor; (d) un'agenzia che controlla le reti di tutto il paese sospetta che un utente scriva a un giornale dello stesso paese.

Esercizio 4.2 ⋆⋆ Spiega con un esempio perché, in un sistema di anonimato, l'usabilità è un requisito di sicurezza e non solo una comodità.

Esercizio 4.3 ⋆⋆ Scrivi il modello di minaccia di una giornalista che usa Tor Browser per comunicare con una fonte in un altro paese: chi sono i possibili avversari, che cosa possono osservare, e quali aspetti restano fuori dalla protezione di Tor.

Per approfondire

- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04], sezioni 3 e 7.
- R. Dingledine, N. Mathewson, *Anonymity Loves Company: Usability and the Network Effect* [DM06].

Parte II

Il protocollo a basso livello

Capitolo 5

Crittografia essenziale per Tor

Aggiornato al 23 settembre 2026 — tor 0.4.9.13; torspec commit `8e1f245`

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- riconoscere le primitive crittografiche usate da Tor e il ruolo di ciascuna;
- distinguere cifratura, integrità, derivazione di chiavi, scambio di chiavi e firma;
- capire perché Tor usa più famiglie di hash (SHA-1, SHA-2, SHA-3, Blake2b) e che cosa le distingue;
- leggere senza difficoltà le formule dei capitoli successivi.

Questo capitolo è una cassetta degli attrezzi. Non è un corso di crittografia: presenta, con il minimo indispensabile di teoria, gli strumenti che incontreremo nei capitoli della Parte II, e rimanda ogni volta al punto in cui vengono usati. L'Appendice B raccoglie i dettagli per chi vuole approfondire. Tutti gli esempi si possono provare in Python con il modulo standard `hashlib` e con la libreria `cryptography`, le stesse usate nei laboratori.

5.1 Cinque compiti

In un protocollo come Tor la crittografia svolge cinque compiti diversi:

Riservatezza Nessuno oltre ai destinatari deve poter leggere i dati: *cifrari*.

Integrità Nessuno deve poter modificare i dati senza che ce ne si accorga: *funzioni di hash* e *codici di autenticazione dei messaggi* (MAC).

Accordo sulle chiavi Due parti che non condividono segreti devono stabilirne uno comunicando in pubblico: *scambio di chiavi* Diffie-Hellman.

Autenticazione Una parte deve dimostrare di possedere una certa identità: *firme digitali*.

Derivazione Da un segreto condiviso bisogna ricavare più chiavi indipendenti: *funzioni di derivazione di chiavi* (KDF).

Una buona regola di progettazione, che Tor segue sempre più rigorosamente, è non usare mai la stessa chiave per due compiti o per due contesti diversi.

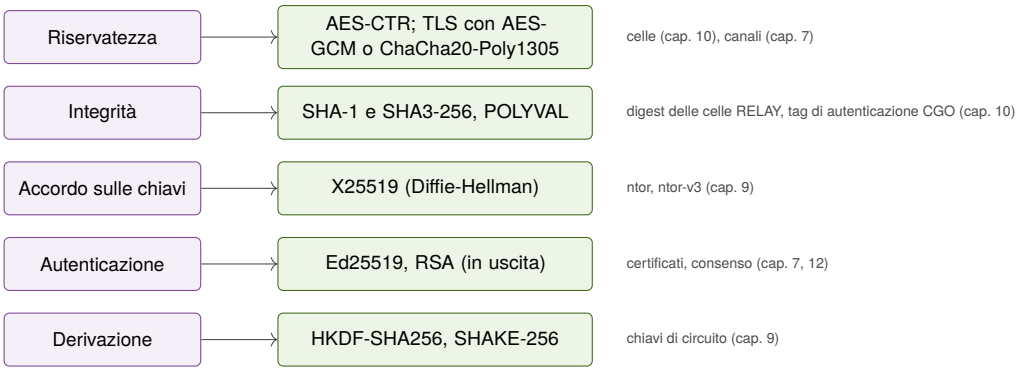


Figura 5.1: I cinque compiti della crittografia in Tor, le primitive che li svolgono e i capitoli in cui le incontreremo all'opera.

5.2 Cifrari

5.2.1 AES in modalità contatore

Tor cifra quasi tutto con **AES**, il cifrario a blocchi standard, con blocchi di 16 byte e chiavi di 128 o 256 bit. Un cifrario a blocchi, da solo, cifra un blocco alla volta; per cifrare flussi di lunghezza qualsiasi Tor lo usa in **modalità contatore** (CTR): AES cifra un contatore che cresce di uno a ogni blocco, e il risultato, il *flusso di chiavi*, viene combinato con i dati con uno XOR:

$$C = P \oplus \mathrm{Trunc}_{|P|}\big(\mathrm{AES}_K(IV) \,\|\, \mathrm{AES}_K(IV+1) \,\|\, \cdots\big)$$

Decifrare è la stessa operazione. La modalità contatore ha due proprietà da tenere sempre presenti:

- **non si deve mai riusare** la stessa coppia chiave–contatore: due messaggi cifrati con lo stesso flusso rivelano lo XOR dei testi in chiaro;
- è **malleabile**: cambiare un bit del testo cifrato cambia lo stesso bit del testo in chiaro, senza conoscere la chiave.

AES-128-CTR cifra i relay cell nello schema tradizionale (sezione 10.1), la cui malleabilità è all'origine degli attacchi di tagging; AES-256-CTR cifra i messaggi di ntor-v3 (sezione 9.7) e i descrittori degli onion service (sezione 14.5); AES-128 è il componente di base di CGO (sezione 10.4).

5.2.2 TLS

Il canale tra due nodi è protetto da TLS (capitolo 7), che negozia per conto suo cifrari autenticati come AES-GCM o ChaCha20-Poly1305. Tor non si affida però a TLS per l'identità dei relay, che dimostra con i propri certificati.

5.3 Funzioni di hash

Una funzione di hash crittografica trasforma un input di qualsiasi lunghezza in un'impronta di lunghezza fissa, in modo che sia impraticabile trovare due input con la stessa impronta (*resistenza alle collisioni*) o un input con un'impronta data. Tor ne usa quattro famiglie, che raccontano la sua storia:

Funzione	Uscita	Dove la incontriamo
SHA-1	20 byte	impronte RSA dei relay, digest progressivo dei relay cell (tor1), KDF-TOR; considerata debole contro le collisioni, sopravvive dove non serve quella proprietà
SHA-256	32 byte	ntor e la sua KDF, certificati Ed25519 e TLS, firme del consenso, impronte dei microdescrittori
SHA3-256, SHAKE-256	32 byte / variabile	ntor-v3, onion service v3 (indirizzi, chiavi accecate, anello degli HSDir, descrittori), digest dei circuiti di rendezvous
Blake2b	variabile	prova di lavoro degli onion service (capitolo 15)

SHA-3 non è una versione migliorata di SHA-2, ma una costruzione completamente diversa (una *spugna*). SHAKE-256 appartiene alla stessa famiglia ed è una funzione a uscita estendibile: produce quanti byte si vuole, il che la rende comoda come funzione di derivazione. Nei protocolli recenti di Tor ogni uso di SHA-3 è preceduto da un'etichetta di dominio (per esempio `"ntor3-curve25519-sha3_256-1:kdf_phase1"`), così che un hash calcolato per uno scopo non possa mai coincidere con uno calcolato per un altro.

Approfondimento: Hash progressivi

Molte librerie permettono di aggiungere dati a una funzione di hash un po' alla volta e di leggerne l'impronta in qualsiasi momento, senza ricominciare. Tor lo sfrutta per il *digest progressivo* dei relay cell (sezione 10.2), che riassume tutte le celle scambiate con un salto. In Python, `hashlib.sha1()` con i metodi `update`, `digest` e `copy` fa esattamente questo.

5.4 MAC e derivazione di chiavi

5.4.1 HMAC

Un MAC è un'impronta che dipende anche da una chiave: solo chi conosce la chiave può calcolarla o verificarla. **HMAC** costruisce un MAC da una funzione di hash; Tor usa HMAC-SHA256 nell'handshake ntor (sezione 9.6) e nella sua funzione di derivazione. Nei protocolli basati su SHA-3 il MAC si ottiene più semplicemente, facendo l'hash della chiave, preceduta dalla propria lunghezza, seguita dal messaggio: la lunghezza serve a stabilire senza ambiguità dove finisce la chiave e comincia il messaggio [rend-spec § Cryptographic building blocks]. In ntor-v3 anche l'etichetta di dominio è anteposta con la sua lunghezza (sezione 9.7).

Un dettaglio pratico: un MAC va sempre confrontato in *tempo costante* (in Python con `hmac.compare_digest`), altrimenti il tempo del confronto può rivelare quanti byte sono corretti.

5.4.2 Funzioni di derivazione

Da un segreto condiviso servono più chiavi: una per cifrare in avanti, una all'indietro, due per l'integrità (sezione 9.8). Una KDF produce da un segreto un flusso lungo quanto serve, da ritagliare. Tor ne usa tre:

KDF-TOR $\mathrm{SHA1}(K_0\|0) \,\|\, \mathrm{SHA1}(K_0\|1) \,\|\, \ldots$, storica, usata ancora da **`CREATE_FAST`**;

HKDF-SHA256 la costruzione standard della RFC 5869, usata da ntor;

SHAKE-256 direttamente come KDF, in ntor-v3 e negli onion service.

5.5 Scambio di chiavi: X25519

Lo scambio Diffie-Hellman permette a due parti di stabilire un segreto comune scambiandosi solo valori pubblici. Nella versione su curva ellittica che Tor usa, **X25519** (lo scambio sulla curva *Curve25519*), ciascuna parte ha una chiave privata x di 32 byte e una pubblica $X = x \cdot G$, dove G è un punto fisso della curva e il prodotto è la moltiplicazione di un punto per un intero. Dalle chiavi x, X e y, Y entrambe le parti calcolano lo stesso valore

$$x \cdot Y = y \cdot X = xy \cdot G,$$

che un osservatore, conoscendo solo X e Y, non può ricavare. Le chiavi sono di 32 byte e lo scambio è molto veloce, due ragioni che hanno portato Tor ad abbandonare il Diffie-Hellman modulare a 1024 bit dell'handshake TAP (sezione 9.4).

Una chiave usata per un solo scambio e poi cancellata si dice **effimera**. Combinare chiavi effimere e chiavi a lungo termine è ciò che permette agli handshake di Tor di offrire insieme autenticazione e *forward secrecy* (sezione 9.3).

5.6 Firme: Ed25519 e RSA

Una firma digitale si produce con una chiave privata e si verifica con la corrispondente chiave pubblica. Tor usa soprattutto **Ed25519**, uno schema di firma su una curva equivalente a Curve25519, con chiavi pubbliche di 32 byte e firme di 64: identità dei relay, certificati di Tor (sezione 7.5), descrittori degli onion service. Ed25519 ha una proprietà che Tor sfrutta in modo originale: moltiplicando una chiave pubblica A per un fattore h si ottiene un'altra chiave pubblica valida, $A' = h \cdot A$, di cui chi conosce la chiave privata può calcolare la privata corrispondente. È la base delle chiavi accecate degli onion service (sezione 14.3).

RSA a 1024 bit è l'eredità dei primi anni: le identità legacy dei relay e le chiavi con cui le directory authority firmano il consenso. Per le autorità le chiavi sono più lunghe, e le due hanno requisiti diversi: la specifica chiede almeno **3072** bit per la chiave d'identità (`KP_auth_id_rsa`) e almeno 2048 per quella di firma (`KP_auth_sign_rsa`) [dir-spec § Directory authority operation and formats]. Tor sta eliminando RSA gradualmente.

5.7 Crittografia recente

Due costruzioni più specialistiche compaiono negli sviluppi recenti:

- **POLYVAL**, una funzione di hash universale (la stessa usata da AES-GCM-SIV), è con AES il mattone di CGO, il nuovo schema di cifratura dei relay cell (sezione 10.4);
- **Elligator2** codifica un punto di Curve25519 in modo che sia indistinguibile da una stringa casuale: obfs4 la usa per non far riconoscere le chiavi pubbliche dello scambio (sezione 16.4).

La crittografia *post-quantistica*, che dovrà sostituire X25519 quando i computer quantistici diventeranno una minaccia concreta, è un tema di ricerca attivo per Tor, discusso nel capitolo 31.

Riepilogo

- AES in modalità contatore cifra i dati; non va mai riusato lo stesso flusso ed è malleabile.

- SHA-1, SHA-256, SHA3-256/SHAKE-256 e Blake2b coesistono in Tor: la scelta dipende dall'epoca del protocollo e dallo scopo.
- HMAC autentica i messaggi, mentre le KDF (KDF-TOR, HKDF, SHAKE) derivano chiavi; i MAC si confrontano in tempo costante.
- X25519 è lo scambio di chiavi di tutti gli handshake moderni; Ed25519 firma identità, certificati e descrittori; RSA è in via di eliminazione.

Esercizi

Esercizio 5.1 ⋆ Con AES-CTR, un avversario conosce il testo in chiaro P_1 di un messaggio e intercetta il suo cifrato C_1 e il cifrato C_2 di un altro messaggio cifrato per errore con lo stesso flusso di chiavi. Come ricava P_2?

Esercizio 5.2 ⋆ In Python, calcola `hashlib.shake_256(b"Tor").hexdigest(16)` e `hexdigest(32)`. Che relazione c'è tra le due uscite? Che cosa implica per l'uso di SHAKE come KDF?

Esercizio 5.3 ⋆⋆ Con la libreria `cryptography`, genera due coppie X25519 e verifica che i due segreti condivisi coincidano. Che cosa succede se una parte usa come chiave pubblica dell'altra 32 byte a zero? Collega il risultato al controllo del «punto all'infinito» di ntor.

Esercizio 5.4 ⋆⋆ Perché i protocolli di Tor basati su SHA-3 fanno precedere ogni input da un'etichetta di dominio? Descrivi un attacco possibile se due usi diversi della stessa funzione non fossero separati.

Per approfondire

- J.-P. Aumasson, *Serious Cryptography* (No Starch Press): un'introduzione moderna e pratica a tutte le primitive di questo capitolo.
- D. J. Bernstein et al., gli articoli originali su Curve25519 ed Ed25519, e le RFC 7748 (X25519) e 8032 (Ed25519).
- La sezione «Preliminaries» della specifica di Tor [Thef], con le convenzioni su cifrari e hash.

Capitolo 6

Architettura della rete

Aggiornato al 23 settembre 2026 — tor 0.4.9.13; consenso pubblico del 23/09/2026 ore 15:00 UTC

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- elencare i ruoli dei nodi della rete Tor e le responsabilità di ciascuno;
- descrivere i livelli del protocollo, dal canale TLS allo stream dell'applicazione;
- seguire, a grandi linee, tutto ciò che accade quando un utente apre una pagina web con Tor;
- capire come implementazioni e versioni diverse convivono nella stessa rete.

Prima di scendere nei dettagli dei capitoli successivi, guardiamo la rete dall'alto. Questo capitolo è una mappa: ogni elemento verrà ripreso e approfondito più avanti, e i rimandi indicano dove.

6.1 I ruoli

La rete Tor è formata da programmi che svolgono ruoli diversi, spesso più di uno contemporaneamente. La fig. 6.1 li riassume.

Client Il programma dell'utente, per esempio quello incluso in Tor Browser. Sceglie i percorsi, costruisce i circuiti e offre alle applicazioni una porta SOCKS. Non accetta connessioni dalla rete e non si autentica (capitolo 7).

Relay Un nodo che inoltra traffico per gli altri. A seconda dei flag assegnati dalle autorità e della sua configurazione può essere:

- **guard**: il primo salto dei circuiti di un client, scelto una volta e usato a lungo (sezione 13.3);
- **middle**: un salto intermedio, che non conosce né l'origine né la destinazione;
- **exit**: l'ultimo salto, che apre le connessioni verso Internet secondo la propria exit policy (capitolo 11);
- **HSDir**: conserva i descrittori degli onion service (sezione 14.4);
- **directory cache**: ridistribuisce il consenso e i descrittori (capitolo 12).

Directory authority Nove server fidati che votano ogni ora sullo stato della rete e firmano il consenso; alcune misurano anche la banda dei relay.

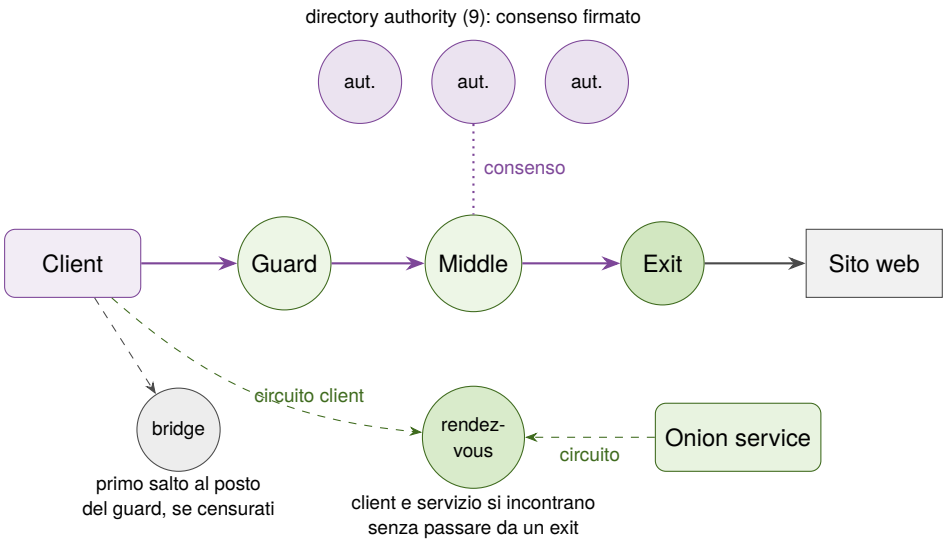


Figura 6.1: I ruoli nella rete Tor. Il client costruisce un circuito attraverso guard, middle ed exit; le directory authority pubblicano il consenso che elenca i relay; i bridge sostituiscono il guard per chi è sotto censura; gli onion service si raggiungono incontrandosi in un punto di rendezvous, senza exit.

Bridge Un relay non pubblicato, usato come primo salto da chi non può raggiungere i relay pubblici, spesso dietro un pluggable transport (capitolo 16).

Onion service Un servizio raggiungibile solo attraverso Tor, senza rivelare la propria posizione (capitolo 14).

Nel consenso pubblico del 23 settembre 2026 i relay sono 9358: 5306 con il flag `Guard`, 3177 con il flag `Exit` (1615 hanno entrambi). La banda dei guard vale il 70 % del totale, quella degli exit il 33 % (capitolo 13).

6.2 I livelli del protocollo

Il protocollo di Tor si può descrivere come una pila di livelli, ciascuno costruito sopra il precedente (fig. 6.2).

Canale Una connessione TLS tra due nodi adiacenti, su cui si verifica l'identità del relay (capitolo 7).

Celle Le unità di 514 byte che viaggiano sul canale (capitolo 8).

Circuito Un percorso di più salti, costruito un salto alla volta con handshake che producono chiavi diverse per ogni salto (capitolo 9).

Cifratura a strati I messaggi relay avvolti in uno strato per ogni salto (capitolo 10).

Stream Le connessioni TCP dell'applicazione, aperte dall'exit e controllate con finestre e controllo della congestione (capitolo 11).

A fianco di questa pila stanno il sistema delle directory, che fornisce ai client l'elenco dei relay, e il protocollo degli onion service, che usa i circuiti per incontrarsi in un punto di rendezvous.

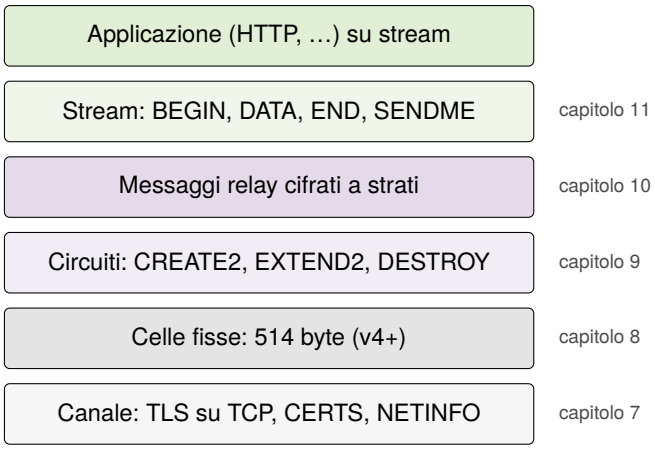


Figura 6.2: I livelli del protocollo di Tor e i capitoli che li descrivono. Ogni livello usa quello sottostante: gli stream viaggiano come messaggi relay, i messaggi relay dentro celle cifrate a strati, le celle dentro canali TLS.

6.3 Una pagina web, dall'inizio alla fine

Seguiamo, a grandi linee, che cosa succede quando un utente avvia Tor Browser per la prima volta e apre una pagina.

1. **Avvio.** tor contatta una directory authority o una delle circa duecento *fallback directory* elencate nel codice e scarica il consenso, di cui verifica le firme; poi scarica i microdescrittori dei relay (capitolo 12).
2. **Scelta dei guard.** Sceglie a caso, in proporzione alla banda, un piccolo campione di guard e ne usa due (sezione 13.3).
3. **Canale.** Apre una connessione TLS verso il guard e ne verifica l'identità con le celle **`CERTS`** (capitolo 7).
4. **Circuito.** Sceglie un exit adatto alla destinazione e un middle, e costruisce un circuito di tre salti con **`CREATE2`** ed `EXTEND2` (capitoli 9 e 13). In realtà tor costruisce alcuni circuiti in anticipo, per averli pronti quando servono.
5. **Stream.** Il browser chiede, attraverso la porta SOCKS, una connessione verso `www.example.com:443`; tor invia un `BEGIN` all'exit, che risolve il nome e apre la connessione TCP (capitolo 11).
6. **Dati.** Il browser parla HTTPS con il sito attraverso lo stream: l'exit vede il nome del sito e il traffico TLS, ma non chi lo richiede; il guard vede chi lo richiede, ma non che cosa; il middle non vede né l'uno né l'altro.

L'intero processo di avvio richiede di solito pochi secondi. Molti dei meccanismi descritti nei capitoli successivi, come il padding, il controllo della congestione o i vanguards, lavorano in silenzio attorno a questa sequenza.

6.4 Implementazioni e versioni

6.4.1 Due implementazioni

L'implementazione storica è **tor**, in C, che svolge tutti i ruoli: client, relay, autorità, onion service. Accanto a essa il Tor Project sviluppa **Arti**, una riscrittura in Rust (capitolo 30), oggi

usata come client e per gli onion service. Nel consenso pubblico del 23 settembre 2026 nessun relay usa Arti: 6027 relay eseguono tor 0.4.9.12, 2520 la 0.4.9.11, e le versioni restanti sono per lo più precedenti della serie 0.4.9; 47 relay eseguono la versione di sviluppo 0.5.0.0-alpha-dev.

6.4.2 Sottoprotocolli

Come fa una rete di migliaia di nodi, aggiornati in tempi diversi, a introdurre nuove funzionalità? Tor non ragiona per numeri di versione del software, ma per **sottoprotocolli**: ogni funzionalità appartiene a un sottoprotocollo con un numero di versione, e ogni relay dichiara nel consenso quali versioni supporta, nella riga `pr` [tor-spec § Subprotocol-based versioning]. Un relay della rete pubblica dichiara, per esempio:

```
pr Conflux=1 Cons=1-2 Desc=1-4 DirCache=2 FlowCtrl=1-2 HSDir=2 HSIntro=4-5
   HSRend=1-2 Link=3-5 LinkAuth=3 Microdesc=1-3 Padding=2 Relay=2-6
```

Un client che vuole usare ntor-v3 con un relay controlla che questo dichiari `Relay=4`; per CGO, `Relay=6`; per il controllo della congestione, `FlowCtrl=2`. Questi numeri sono *capability* indipendenti, non versioni cumulative: `Relay=6` non implica formalmente `Relay=4` (Proposta 346, che per questo preferisce chiamarle «capability di sottoprotocollo»), anche se in pratica tor le annuncia come un intervallo contiguo perché implementa le funzionalità in ordine. Il consenso elenca anche i sottoprotocolli *richiesti*: una versione di tor che non li supporta sa di essere troppo vecchia e si rifiuta di funzionare, invece di mettere a rischio i propri utenti.

Riepilogo

- La rete è fatta di client, relay (guard, middle, exit, HSDir, directory cache), directory authority, bridge e onion service.
- Il protocollo è una pila: canale TLS, celle, circuiti, cifratura a strati, stream.
- Il guard conosce l'utente ma non la destinazione, l'exit la destinazione ma non l'utente.
- tor (C) e Arti (Rust) convivono; le funzionalità si negoziano per sottoprotocolli, dichiarati nella riga `pr` del consenso.

Esercizi

Esercizio 6.1 ⋆ Per ciascun ruolo (guard, middle, exit, HSDir, directory authority) indica che cosa sa di un utente che visita un sito web attraverso Tor.

Esercizio 6.2 ⋆ Nella riga `pr` riportata nel capitolo, quali funzionalità dei capitoli 9–11 supporta il relay? Potresti usarlo come exit con CGO?

Esercizio 6.3 ⋆⋆ Una pagina web di 100 KB viene scaricata attraverso Tor. Stima, livello per livello, quanti byte transitano sul collegamento tra client e guard, tenendo conto di celle, intestazioni dei messaggi relay e SENDME.

Per approfondire

- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04]: l'architettura originale, in gran parte ancora valida.

- Tor Metrics, `https://metrics.torproject.org/`: numeri aggiornati su relay, utenti e versioni.

Capitolo 7

Canali e link protocol

Aggiornato al 23 settembre 2026 — torspec commit `8e1f245`; tor 0.4.9.13; Arti 2.6.0; link protocol 4–5

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- spiegare che cos'è un canale e perché Tor lo costruisce sopra TLS;
- descrivere le chiavi che compongono l'identità di un relay e come si legano tra loro;
- seguire l'handshake di canale cella per cella: VERSIONS, CERTS, AUTH_CHALLENGE, AUTHENTICATE, NETINFO;
- leggere e verificare un certificato Ed25519 di Tor;
- capire perché un client non si autentica e che cosa questo comporta per l'anonimato;
- aprire un canale verificato verso un relay della rete di laboratorio (Laboratorio 1).

Il circuito, l'oggetto centrale di Tor, è una catena di salti. Ogni salto però poggia su qualcosa di più semplice: una connessione diretta e cifrata tra due nodi adiacenti. La specifica chiama questa connessione **canale**. In questo capitolo vediamo come nasce un canale, e soprattutto come le due parti si convincono di stare parlando con chi credono. Il formato delle celle scambiate sul canale è l'argomento del capitolo 8, a cui rimanderemo spesso.

7.1 Che cos'è un canale

Un canale è una connessione diretta e cifrata tra due relay, oppure tra un client e un relay [tor-spec § Channels]. Tecnicamente è una sessione TLS su TCP, all'interno della quale viaggiano celle. Tre proprietà lo caratterizzano.

Asimmetria dei ruoli Chiunque può *aprire* un canale (l'*iniziatore* (initiator)), ma solo un relay può *riceverlo* (il *rispondente* (responder)). Un client non accetta connessioni: non ha una porta in ascolto verso la rete Tor.

Identità verificata Il rispondente dimostra sempre, crittograficamente, di possedere una o più identità di relay. L'iniziatore di solito sa già quale identità aspettarsi: la legge nel consenso, nell'elenco delle autorità compilato nel codice o, quando il canale viene aperto per estendere un circuito, nel messaggio `EXTEND2` stesso. Se l'identità dimostrata non è quella attesa, il canale va chiuso.

Condivisione Un canale trasporta molti circuiti, distinti dal CircID (sezione 8.4). I nodi preferiscono riusare un canale esistente piuttosto che aprirne un secondo verso lo

stesso relay. Un canale senza circuiti viene chiuso dopo un periodo di inattività, scelto a caso in un intervallo che dipende da chi lo usa (`channelpadding_get_channel_idle_timeout` in `src/core/or/channelpadding.c`): 3–4,5 minuti per i canali di un client, 45–75 minuti per quelli tra due relay, dove il valore centrale è il parametro di consenso `nf_conntimeout_relays` (un'ora). Finché il canale è in uso, invece, `KeepalivePeriod` fa inviare una cella di riempimento ogni 5 minuti, perché i firewall non lo chiudano.

Approfondimento: Canale o connessione?

Nella documentazione più vecchia e in molti articoli scientifici i canali sono chiamati *connessioni* (*OR connections*). La specifica ha abbandonato il termine perché si confondeva con le connessioni TCP degli stream in uscita dagli exit. Nel codice di tor sopravvivono entrambi: `or_connection_t` è la connessione TLS, `channel_t` l'astrazione che la usa.

7.2 L'identità di un relay

Prima di vedere l'handshake dobbiamo sapere *che cosa* deve essere dimostrato. Un relay è identificato da chiavi a lungo termine; la specifica le indica con nomi sistematici, in cui il prefisso `KP_` denota una chiave pubblica e `KS_` la corrispondente chiave segreta [tor-spec § Relay keys and identities]. Quelle rilevanti per il canale sono:

`KP_relayid_ed` l'**identità Ed25519**, o «master identity key». Non scade mai e ha un solo compito: firmare la chiave di firma. Può essere tenuta offline.

`KP_relayid_rsa` l'identità RSA a 1024 bit, **legacy**. È ancora la base delle impronte (*fingerprint*) di 40 cifre esadecimali che si vedono su Tor Metrics, ma la sicurezza del protocollo non dipende più da essa.

`KP_relaysign_ed` la **chiave di firma** a medio termine, certificata dall'identità. È quella che il relay usa ogni giorno, online.

`KP_link_ed` una chiave a breve termine usata solo quando è il relay ad aprire un canale e deve autenticarsi (sezione 7.6).

La fig. 7.1 mostra come queste chiavi sono legate da certificati. L'idea è una *catena di fiducia* che parte dall'identità e arriva fino al certificato TLS usato nella connessione.

Nel laboratorio queste chiavi sono file nella cartella `keys/` di ogni relay: `ed25519_master_id_public_key` e la corrispondente chiave segreta per l'identità, `ed25519_signing_cert` e `ed25519_signing_secret_key` per la chiave di firma, `secret_id_key` per l'identità RSA. Il file `fingerprint-ed25519` nella directory del nodo riporta l'identità Ed25519 in base64.

Nota storica: Da RSA a Ed25519

Fino al 2015 l'identità di un relay era soltanto la chiave RSA a 1024 bit, una dimensione ormai considerata insufficiente. La Proposta 220 ha introdotto le identità Ed25519 e il formato di certificato descritto in questo capitolo, supportati a partire da tor 0.2.7.2-alpha. La chiave RSA è rimasta per compatibilità, legata a quella Ed25519 da un certificato incrociato.

7.3 L'handshake in sintesi

La fig. 7.2 riassume l'apertura di un canale [tor-spec § Negotiating and initializing channels].

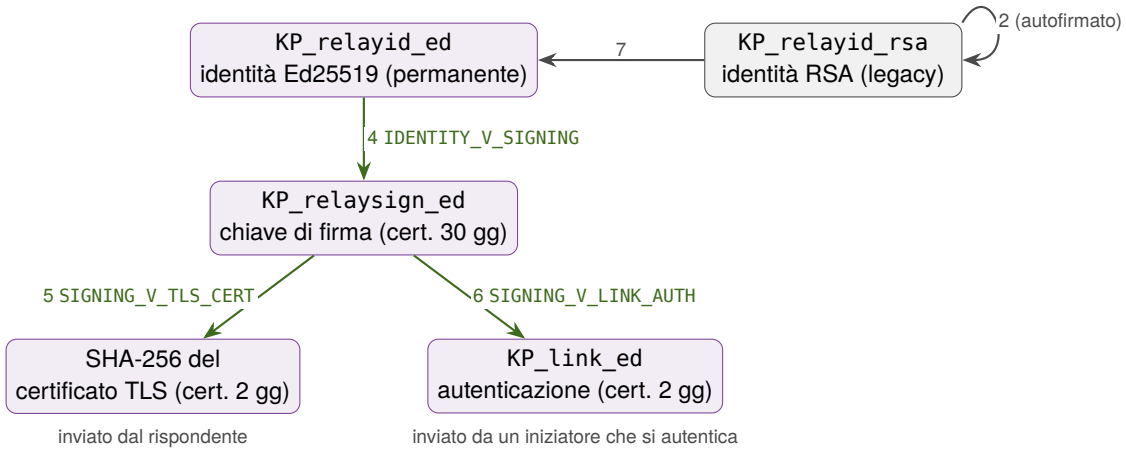


Figura 7.1: La catena di certificati con cui un relay lega la sua identità Ed25519 al certificato TLS della connessione. Ogni freccia è un certificato: la chiave all'origine firma la chiave (o l'impronta) di destinazione; il numero è il CertType usato nella cella CERTS. A destra, la catena RSA legacy, che collega la vecchia identità RSA a quella Ed25519.

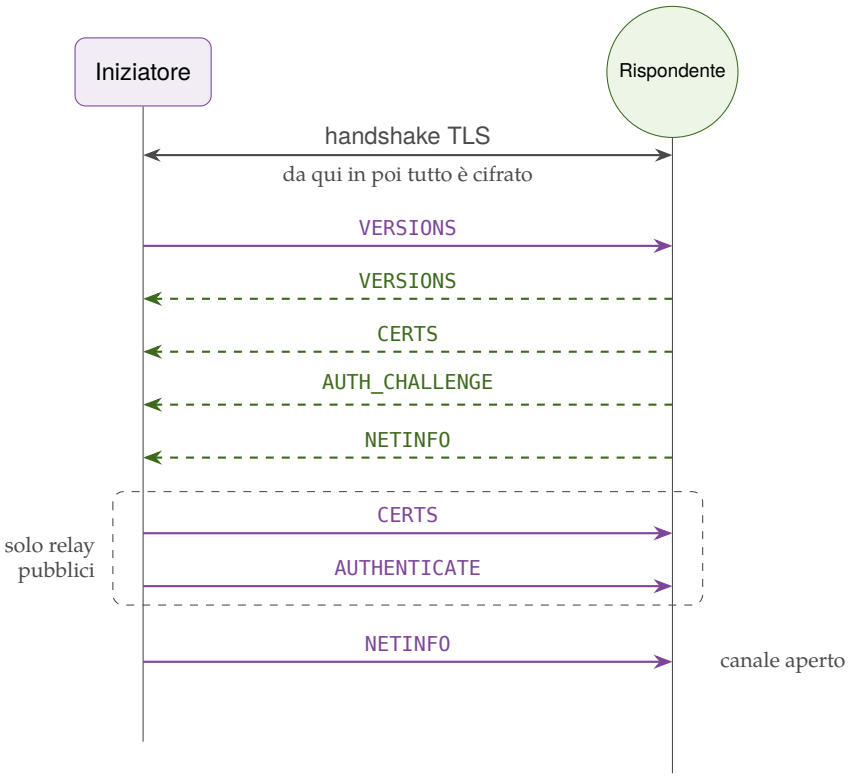


Figura 7.2: L'handshake di canale. Dopo il TLS, tutto viaggia cifrato. Il rispondente invia sempre CERTS, AUTH_CHALLENGE e NETINFO, in quest'ordine. Il riquadro tratteggiato si applica solo se l'iniziatore è un relay pubblico che deve dimostrare la propria identità; un client o un bridge inviano soltanto NETINFO.

1. L'iniziatore apre una connessione TLS.
2. Entrambi inviano una cella **`VERSIONS`** e si accordano sulla versione del link protocol (sezione 8.3).
3. Il rispondente invia **`CERTS`** (i certificati che provano la sua identità), **`AUTH_CHALLENGE`** (una sfida che l'iniziatore firmerà se vuole autenticarsi) e **`NETINFO`** (ora e indirizzi).
4. L'iniziatore verifica i certificati. Se è un relay pubblico, risponde con **`CERTS`** e **`AUTHENTICATE`**; in ogni caso chiude con **`NETINFO`**.

`NETINFO` segna la fine dell'handshake: dopo averla inviata, una parte non invia più celle di handshake. È così che il rispondente capisce se l'iniziatore ha deciso di non autenticarsi. Nel mezzo possono comparire celle **`VPADDING`**, in qualsiasi numero, ma nessun altro tipo di cella.

7.4 Il livello TLS

Tor usa TLS in modo insolito. In un sito web il certificato del server è firmato da un'autorità di certificazione e contiene un nome di dominio: il browser controlla entrambe le cose. Un relay invece presenta un certificato **autofirmato**, senza alcun nome significativo, e il client non lo verifica con le regole di TLS. Nel codice del Laboratorio 1 questo si traduce in due righe che, in qualsiasi altro contesto, sarebbero un grave errore di sicurezza:

```
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE
```

Qui non lo sono, perché l'autenticazione avviene *dopo*, dentro il canale, con la cella **`CERTS`**. TLS fornisce la cifratura e l'integrità del canale e il possesso della chiave del certificato TLS; il resto lo dimostrano le celle.

La specifica aggiunge alcune regole [tor-spec § The TLS handshake]:

- le implementazioni dovrebbero supportare TLS 1.3, e i relay anche TLS 1.2, con suite moderne (AES-GCM, ChaCha20-Poly1305, scambio di chiavi su X25519 o P-256);
- la *session resumption* non dovrebbe essere permessa: indebolisce la forward secrecy e ha reso possibili attacchi come il *Triple Handshake* (2013);
- la compressione TLS non dovrebbe essere usata.

Nella rete di laboratorio, un client Python con OpenSSL 3.5 negozia con tor TLS 1.3 con la suite `TLS_AES_256_GCM_SHA384`.

Nota storica: Perché un handshake così strano

Le prime versioni del link protocol mettevano i certificati di Tor direttamente nell'handshake TLS (versione 1), poi li nascondevano con una rinegoziazione TLS (versione 2), segnalando la variante scelta tramite l'elenco delle ciphersuite offerte. La ragione era la censura: prima dei pluggable transports (capitolo 16) Tor cercava di sembrare una normale connessione HTTPS tra browser e server web, e con TLS 1.2 i certificati viaggiavano in chiaro, facili da riconoscere. Con la versione 3 i certificati si sono spostati dentro il canale cifrato, nelle celle **`CERTS`**. Oggi la resistenza alla censura è affidata ai pluggable transports, e le varianti vecchie sono obsolete; la specifica annota che, a marzo 2025, tor ne accettava ancora alcune, destinate a essere rimosse.

7.5 CERTS: dimostrare l'identità

7.5.1 La cella

CERTS è una cella variabile (comando 129) il cui corpo contiene un elenco di certificati:

Campo	Byte	Contenuto
N	1	numero di certificati
ripetuto N volte:		
CertType	1	tipo di certificato
CertLen	2	lunghezza del certificato
Certificate	CertLen	il certificato

Una cella **CERTS** non può contenere due certificati dello stesso tipo. I tipi definiti sono elencati nella tabella 7.1 [cert-spec § Certificate types (CERT_TYPE field)].

Tipo	Nome	Formato	Che cosa certifica
1	`TLS_LINK_X509`	X.509	chiave TLS, firmata dall'identità RSA (obsoleto)
2	`RSA_ID_X509`	X.509	identità RSA, autofirmata
3	`LINK_AUTH_X509`	X.509	chiave RSA di autenticazione (obsoleto)
4	`IDENTITY_V_SIGNING`	Ed	chiave di firma, firmata dall'identità Ed25519
5	`SIGNING_V_TLS_CERT`	Ed	impronta del certificato TLS, firmata dalla chiave di firma
6	`SIGNING_V_LINK_AUTH`	Ed	chiave `KP_link_ed`, firmata dalla chiave di firma
7	`RSA_ID_V_IDENTITY`	RSA→Ed	identità Ed25519, firmata dall'identità RSA
8–12	onion service (8, 9, 11), certificato incrociato della chiave ntor (10), famiglie (12)		

Tabella 7.1: Tipi di certificato. «Ed» indica il formato di certificato Ed25519 di Tor.

7.5.2 Il certificato Ed25519 di Tor

Tor non usa X.509 per i certificati Ed25519, ma un formato proprio, molto più semplice [cert-spec § Tor Ed25519 Certificates], mostrato nella fig. 7.3.

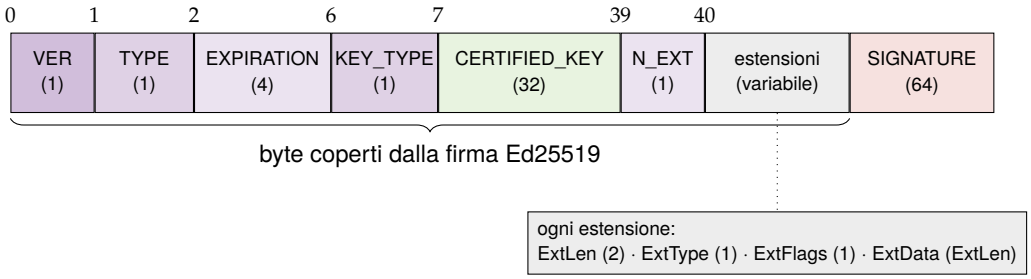


Figura 7.3: Il certificato Ed25519 di Tor. Tutti i campi prima della firma sono coperti dalla firma stessa. La data di scadenza è espressa in *ore* dall'epoca Unix: con 4 byte senza segno il campo non si esaurisce prima di centinaia di migliaia di anni.

VERSION vale 1.

CERT_TYPE lo stesso valore del CertType della tabella 7.1.

EXPIRATION_DATE scadenza in ore dall'1 gennaio 1970.

CERT_KEY_TYPE che cosa contiene il campo successivo: 1 una chiave Ed25519, 3 l'impronta SHA-256 di un certificato X.509. Le versioni di tor fino alla 0.4.5.1-alpha scrivevano sempre 1, e la specifica chiede di tollerarlo.

CERTIFIED_KEY i 32 byte certificati.

Estensioni ognuna con lunghezza, tipo, flag e dati. L'unica definita oggi è la 4, *signed-with-ed25519-key*, che include nel certificato la chiave che lo ha firmato. Il flag 1 (`AFFECTS_VALIDATION`) dice come trattare un'estensione sconosciuta: se è impostato, il certificato va rifiutato; altrimenti l'estensione si ignora.

SIGNATURE firma Ed25519 di tutti i campi precedenti.

7.5.3 La verifica

Il rispondente deve inviare esattamente un certificato di tipo 4 e uno di tipo 5, e l'iniziatore li verifica così [tor-spec § Authenticating the responder from its CERTS]:

1. il certificato di tipo 4 contiene, nell'estensione 4, una chiave Ed25519: è l'identità dichiarata `KP_relayid_ed`. Il certificato deve essere firmato proprio con quella chiave (è *autofirmato*); la chiave certificata è `KP_relaysign_ed`;

2. il certificato di tipo 5 deve essere firmato con `KP_relaysign_ed`, e la sua chiave certificata deve essere l'impronta SHA-256 del certificato TLS ricevuto durante l'handshake TLS;

3. nessuno dei due deve essere scaduto;

4. infine l'identità ottenuta va confrontata con quella attesa.

Perché questa catena dimostra qualcosa? Il punto 2 lega la chiave di firma al certificato TLS di *questa* connessione; l'handshake TLS ha già dimostrato che il rispondente possiede la chiave privata di quel certificato; il punto 1 lega la chiave di firma all'identità. Un impostore potrebbe copiare le celle **`CERTS`** di un relay vero, perché non contengono segreti, ma non potrebbe usarle: dovrebbe presentare nel TLS lo stesso certificato del relay vero, e quindi possederne la chiave privata.

Attenzione: Controllare l'identità attesa

Una catena perfettamente valida dimostra soltanto che il rispondente possiede *una* identità. Se l'iniziatore dimentica il punto 4, chiunque con un'identità qualsiasi potrebbe fingersi il relay desiderato. Nel Laboratorio 1 il parametro `identita_attesa` esiste proprio per questo, e un test verifica che un relay con identità diversa venga rifiutato.

7.5.4 La catena RSA legacy

Facoltativamente, l'iniziatore può verificare anche l'identità RSA: il certificato di tipo 2 contiene la chiave RSA a 1024 bit (autofirmata, esponente 65537) e il certificato di tipo 7, nel formato «cross-certificato», lega quella chiave all'identità Ed25519 già verificata. La firma RSA del cross-certificato è calcolata sull'impronta SHA-256 della stringa `"Tor TLS RSA/Ed25519 cross-certificate"` seguita dai campi del certificato; il prefisso impedisce che la stessa firma possa essere riutilizzata in un altro contesto.

7.5.5 Che cosa invia davvero un relay

Nella rete di laboratorio, il relay `test003r` invia una cella **`CERTS`** con cinque certificati:

```
tipo 1  584 byte
tipo 2  442 byte
tipo 4  140 byte  scade 2026-10-24 16:00  tipo_chiave=1  est=[4]
tipo 5  104 byte  scade 2026-09-29 11:00  tipo_chiave=3  est=[]
tipo 7  165 byte
```

Oltre alla catena Ed25519 (tipi 4 e 5) e a quella RSA (tipi 2 e 7), il relay invia ancora il vecchio certificato di tipo 1, per i client più datati: non ha soltanto la stessa lunghezza del certificato TLS, ne ha anche la stessa impronta: confrontandone i byte si vede che è *esattamente* lo stesso certificato. Questa lunghezza è un numero da non imparare a memoria, e vale la pena capire perché. Il nome comune del certificato è un dominio fittizio, costruito da tor come `www.` più da 4 a 25 caratteri casuali più `.com` (`crypto_random_hostname` in `src/lib/tls/tortls_openssl.c`): già questo fa variare la lunghezza di una ventina di byte da un relay all'altro. Ma cambia anche *nel tempo* per lo stesso relay, perché tor rinnova il contesto TLS, e con esso il certificato, ogni due ore (`MAX_SSL_KEY_LIFETIME_INTERNAL` in `src/core/or/or.h`). Rieseguendo la misura a quaranta minuti di distanza abbiamo infatti ottenuto un corpo di **`CERTS`** di 1456 byte invece di 1451. Se i vostri numeri non coincidono con questi, è il comportamento previsto: ciò che deve tornare è la *struttura*, e che la somma dei certificati, più un byte di conteggio e tre di intestazione per ciascuno, dia il corpo della cella. Le due scadenze mostrano la diversa durata delle chiavi: la chiave di firma è certificata per 30 giorni, il legame con il certificato TLS solo per 2. Sono i valori predefiniti di tor (opzioni `SigningKeyLifetime` e `TestingLinkCertLifetime`, quest'ultima modificabile solo in una rete di test), quindi gli stessi della rete pubblica: la chiave di identità resta ferma, mentre le chiavi che la rappresentano online vengono rinnovate spesso, limitando i danni se un relay viene compromesso.

7.6 AUTH_CHALLENGE e AUTHENTICATE

7.6.1 Chi si autentica

L'autenticazione dell'iniziatore è facoltativa, e la regola su chi debba farla è una delle scelte più importanti del protocollo [tor-spec § Channels]:

- un **relay pubblico** che apre un canale verso un altro relay *dovrebbe* autenticarsi: così il rispondente sa di parlare con un relay della rete, e non con un client;
- un **client** o un **bridge** *non dovrebbe* farlo.

Il motivo della seconda regola è l'anonimato. Un client non ha un'identità, e crearne una sarebbe dannoso: il guard potrebbe riconoscere lo stesso client quando si ricollega da un indirizzo diverso. Un bridge, dal canto suo, non deve rivelare di essere un relay della rete a chi lo usa. Il rovescio della medaglia è che un relay può intuire di essere il primo salto di un circuito se il canale non è autenticato (capitolo 8, sezione 8.6): è uno dei motivi per cui i relay rifiutano di fare da exit a un circuito di un solo salto.

7.6.2 La sfida

Il rispondente invia comunque una cella **`AUTH_CHALLENGE`**, con 32 byte casuali e l'elenco dei metodi di autenticazione accettati:

Campo	Byte	Contenuto
Challenge	32	valore casuale, diverso per ogni canale
N_Methods	2	numero di metodi
Methods	2× N_Methods	elenco dei metodi

I metodi definiti sono 1 (`RSA-SHA256-TLSSecret`, obsoleto), 2 (mai implementato) e 3 (`Ed25519-SHA256-RFC5705`). La cella che riceviamo dai relay della rete di laboratorio è lunga 36 byte: $32 + 2 + 2$, cioè un solo metodo, il 3.

7.6.3 La risposta

Un relay che vuole autenticarsi risponde con la propria cella **`CERTS`** (con i tipi 4 e 6 al posto di 4 e 5, perché un iniziatore non ha necessariamente presentato un certificato TLS) e con **`AUTHENTICATE`**. Per il metodo 3, il corpo di **`AUTHENTICATE`** è un documento di 352 byte firmato con la chiave `KP_link_ed`:

Campo	Byte	Contenuto
TYPE	8	la stringa `AUTH0003`
CID, SID	32 + 32	impronte SHA-256 delle identità RSA di iniziatore e rispondente
CID_ED, SID_ED	32 + 32	identità Ed25519 di iniziatore e rispondente
SLOG	32	SHA-256 di tutti i byte *ricevuti* sul canale fino ad AUTH_CHALLENGE compresa
CLOG	32	SHA-256 di tutti i byte *inviati* sul canale prima di AUTHENTICATE
SCERT	32	SHA-256 del certificato TLS del rispondente
TLSSECRETS	32	valore derivato dai segreti della sessione TLS (*keying material exporter*)
RAND	24	byte casuali
SIG	64	firma Ed25519 di tutti i campi precedenti

Ogni campo lega la firma a un aspetto preciso di *questa* connessione: le identità coinvolte (CID, SID, …), la conversazione avvenuta finora (SLOG, CLOG, che includono la sfida casuale), il certificato TLS (SCERT) e la sessione TLS stessa (TLSSECRETS). Una firma così non può essere riutilizzata su un'altra connessione, né da un intermediario che inoltri i messaggi tra due sessioni TLS diverse (un attacco *man-in-the-middle*), perché i valori di TLSSECRETS e SCERT sarebbero diversi sui due lati.

Approfondimento: Un errore rimasto nella specifica

Il valore TLSSECRETS si calcola con l'*exporter* di TLS (RFC 5705, e per TLS 1.3 RFC 8446, §7.5), con l'etichetta `"EXPORTER FOR TOR TLS CLIENT BINDING AUTH0003"` e con un valore di contesto. Le intenzioni erano di usare come contesto l'identità Ed25519 dell'iniziatore, ma l'implementazione usò per errore il campo CID, l'impronta dell'identità RSA. Poiché non indebolisce la sicurezza, la specifica ha scelto di documentare il comportamento effettivo anziché correggerlo: un buon esempio di come, in un protocollo con migliaia di implementazioni già in uso, sia il codice a definire lo standard.

7.7 NETINFO: ora e indirizzi

L'ultima cella dell'handshake, inviata da entrambe le parti, è **`NETINFO`**. È una cella fissa (comando 8), con questo corpo:

Campo	Byte	Contenuto
TIME	4	secondi dall'epoca Unix, big-endian
OTHERADDR	2 + ALEN	indirizzo dell'altra parte, come lo vede il mittente
NMYADDR	1	numero di indirizzi propri
MYADDR	NMYADDR × (2 + ALEN)	indirizzi propri

Ogni indirizzo è codificato come tipo (1 byte: `0x04` IPv4, `0x06` IPv6), lunghezza (4 o 16) e valore. I campi servono a due scopi:

- **l'orario** permette di accorgersi di un orologio sbagliato: tor registra un avviso se l'orario ricevuto da un relay differisce dal proprio di più di un'ora (costante `NETINFO_NOTICE_SKEW` in `src/core/or/channeltls.c`);
- **gli indirizzi** permettono a un relay di scoprire con quale indirizzo pubblico viene visto, e all'iniziatore di controllare di aver raggiunto il relay a un suo indirizzo *canonico*, cioè quello pubblicato nel suo descrittore.

Attenzione: Un client non dice che ore sono

I relay inviano il proprio orario e i propri indirizzi. Un client invece dovrebbe inviare `00 00 00 00` come orario e nessun indirizzo proprio. Il motivo è il *fingerprinting*: l'orologio di un computer ha quasi sempre un piccolo scarto caratteristico, e se un client inviasse il proprio orario a ogni guard, quello scarto potrebbe servire a riconoscerlo nel tempo. È un esempio di una regola generale che ritroveremo spesso: un client Tor non deve rivelare nulla che non sia strettamente necessario.

Il campo TIME, di 4 byte senza segno, trabocca nel 2106; la Proposta 338 discute l'estensione a 8 byte. Chi riceve una **`NETINFO`** non dovrebbe fidarsi ciecamente dei valori: l'altra parte può mentire sull'orario e sugli indirizzi.

Ecco la **`NETINFO`** che riceviamo dal relay `test003r`, decodificata dal Laboratorio 1:

```
Netinfo(orario=1790511791, tuo_indirizzo='127.0.0.1',
        miei_indirizzi=['127.0.0.1'])
```

L'orario corrisponde al 27 settembre 2026, 12:23:11 UTC; entrambi gli indirizzi sono quelli di loopback, perché tutta la rete gira sulla stessa macchina. La nostra risposta, costruita da `cella_netinfo_client`, ha il corpo `00000000 04 04 7f000001 00`: orario zero, l'indirizzo del relay, nessun indirizzo proprio.

7.8 Dopo l'handshake

A questo punto il canale è *aperto* e può trasportare circuiti. Un esperimento semplice mostra un aspetto interessante. Se, dopo aver aperto il canale, restiamo in ascolto senza creare circuiti, il relay non invia *nulla*: nel laboratorio, nessuna cella in 15 secondi.

Ci si potrebbe aspettare di vedere celle **`PADDING`**: dal link protocol 5, client e guard si scambiano celle di riempimento sui canali inattivi, a intervalli casuali tra 1,5 e 9,5 secondi, così che un osservatore della rete (per esempio un router che registra statistiche NetFlow) non possa distinguere i periodi di attività da quelli di silenzio [padding-spec § Connection-level padding]. Il codice di tor chiarisce perché qui non accade: la funzione che decide se aggiungere padding (`channelpadding_decide_to_pad_channel`, in `src/core/or/channelpadding.c`) lo attiva solo sui canali *già usati* per circuiti completi o per traffico dell'utente. Dal lato del relay,

un canale viene considerato tale solo quando vi transita un circuito di più salti: la prima cella **RELAY_EARLY** (l'estensione verso il secondo salto) o **RELAY** su un circuito che prosegue oltre il relay (`src/core/or/relay.c`). Un canale nuovo resta silenzioso; e, come scopriremo nel capitolo 9, anche un canale con circuiti di un solo salto. Il padding comparirà nel capitolo 10, quando il nostro client estenderà un circuito.

Questo piccolo episodio contiene una lezione di metodo: *l'assenza di un'osservazione non è una prova*. Prima di concludere che il padding non funziona, bisogna verificare che le condizioni in cui dovrebbe comparire siano davvero soddisfatte.

7.9 Laboratorio: aprire un canale verificato

Laboratorio 1: Aprire un canale verso un relay e verificarne l'identità

Obiettivo. Scrivere `labs/it/codice/torlab/canale.py`, che apre un canale come un client Tor: TLS, **VERSIONS**, verifica di **CERTS**, lettura di **AUTH_CHALLENGE** e **NETINFO**, invio della propria **NETINFO**.
Prerequisiti. Laboratorio 2 (`celle.py`, capitolo 8): il codice delle celle è il fondamento del canale, quindi conviene scriverlo prima, anche se lo presentiamo nel capitolo seguente. La spiegazione top-down (prima il canale, poi le celle) e la costruzione bottom-up (prima le celle, poi il canale) qui vanno in ordini opposti, ed è normale. Servono inoltre la rete di laboratorio avviata (capitolo 19) e la libreria `cryptography` per Ed25519.
Tempo stimato. 2–3 ore.

Passo 1: decodificare un certificato Ed25519. Seguendo la fig. 7.3, leggiamo l'intestazione a lunghezza fissa, poi le estensioni, e teniamo da parte i byte coperti dalla firma:

```
def decodifica_cert_ed(dati):
    if len(dati) < 40 + 64 or dati[0] != 1:
        raise ErroreCanale("certificato malformato")
    _, tipo, scadenza, tipo_chiave = struct.unpack(">BBIB", dati[:7])
    chiave = dati[7:39]
    n_est, pos, estensioni = dati[39], 40, {}
    for _ in range(n_est):
        lung, tipo_est, flag = struct.unpack(">HBB", dati[pos:pos+4])
        corpo = dati[pos+4:pos+4+lung]
        if tipo_est != 4 and flag & 1:     # AFFECTS_VALIDATION
            raise ErroreCanale("estensione sconosciuta e vincolante")
        estensioni[tipo_est] = (flag, corpo)
        pos += 4 + lung
    return CertificatoEd(tipo, scadenza, tipo_chiave, chiave,
                         estensioni, firmato=dati[:pos], firma=dati[pos:])
```

Passo 2: verificare la catena. La funzione applica le regole della sezione 7.5.3 e restituisce l'identità dimostrata:

```
def verifica_identita_responder(certificati, cert_tls_der):
    id_v_sign = decodifica_cert_ed(certificati[4])
    sign_v_tls = decodifica_cert_ed(certificati[5])
    identita = id_v_sign.chiave_firmataria        # estensione 4
    id_v_sign.verifica_firma(identita)             # autofirmato
    sign_v_tls.verifica_firma(id_v_sign.chiave_certificata)
    if sign_v_tls.chiave_certificata != sha256(cert_tls_der).digest():
```

```
        raise ErroreCanale("non corrisponde al certificato TLS")
    for c in (id_v_sign, sign_v_tls):
        if c.scaduto():
            raise ErroreCanale("certificato scaduto")
    return identita
```

Il certificato TLS in formato DER si ottiene dal socket con `sock.getpeercert(binary_form=True)`: Python lo restituisce anche se, come qui, non lo abbiamo verificato.

Passo 3: l'handshake. La funzione `apri_canale` invia **`VERSIONS`** offrendo le versioni 4 e 5 (come Arti, rinunciamo alla 3), poi legge celle finché non arriva **`NETINFO`**, gestendo ciascun tipo: ignora **`VPADDING`** e le **`VERSIONS`** successive alla prima, rifiuta qualsiasi altra cella inattesa. Al termine verifica i certificati, confronta l'identità con quella attesa e invia la propria **`NETINFO`**. Il codice completo è nel repository; la parte centrale è:

```
while netinfo is None:
    cella, buffer = _leggi(sock, buffer, versione)
    if cella.comando == Comando.VERSIONS:
        if versione:
            continue
        versione = negozia_versione(VERSIONI_SUPPORTATE, cella.corpo)
    elif cella.comando == Comando.CERTS:
        certificati = decodifica_certs(cella.corpo)
    elif cella.comando == Comando.AUTH_CHALLENGE:
        ...                                  # metodi accettati
    elif cella.comando == Comando.NETINFO:
        netinfo = decodifica_netinfo(cella.corpo)
    elif cella.comando != Comando.VPADDING:
        raise ErroreCanale("cella inattesa durante l'handshake")
```

Passo 4: i test senza rete. In `test/test_canale.py` costruiamo noi stessi, con due chiavi Ed25519 generate al momento, una catena di certificati valida, e verifichiamo che venga accettata. Poi la guastiamo in tutti i modi previsti dalla specifica (firma alterata, chiave di firma non certificata, certificato TLS diverso, certificato scaduto, tipo duplicato) e verifichiamo che ogni guasto venga rilevato. Un test che controlla solo il caso valido non dimostra nulla: un verificatore che accetta tutto lo supererebbe.

Passo 5: contro la rete di laboratorio. Con la rete avviata:

```
from torlab.canale import apri_canale, identita_da_file

atteso = identita_da_file("../lab/net/nodes/003r/fingerprint-ed25519")
c = apri_canale("127.0.0.1", 5103, identita_attesa=atteso)
print(c.versione, c.identita_ed.hex(), c.netinfo, c.metodi_auth)
print(c.registro)
```

L'output (abbreviato):

```
5 785d3199f21d20ad...  Netinfo(orario=1790511793, ...)  [3]
['TLS TLSv1.3 TLS_AES_256_GCM_SHA384', '<- VERSIONS (6 byte)',
 '<- CERTS (1451 byte)', '<- AUTH_CHALLENGE (36 byte)',
 '<- NETINFO (509 byte)', '-> NETINFO: canale aperto']
```

Provando invece a connettersi alla porta 5104 (il relay `test004r`) con l'identità attesa di un altro relay, si ottiene, come deve essere, `il relay ha un'identità diversa da quella attesa`.

Riepilogo

- Un canale è una sessione TLS tra due nodi adiacenti; solo i relay accettano canali, e un canale trasporta molti circuiti.
- L'identità di un relay è la chiave Ed25519 `KP_relayid_ed`, che certifica una chiave di firma `KP_relaysign_ed`; l'identità RSA è mantenuta per compatibilità.
- Il certificato TLS di un relay è autofirmato; l'autenticazione avviene dentro il canale con la cella **`CERTS`**, che lega identità, chiave di firma e impronta del certificato TLS.
- L'iniziatore deve verificare firme, scadenze, legame con il TLS e **identità attesa**.
- I relay che aprono un canale si autenticano con **`AUTHENTICATE`**, una firma legata alla sessione TLS e alla conversazione; client e bridge non si autenticano, per non creare identità riconoscibili.
- **`NETINFO`** chiude l'handshake; un client invia orario zero e nessun indirizzo proprio.
- Il padding di collegamento richiede che il canale abbia già portato un circuito *completo* o traffico dell'utente: un canale nuovo, o che porti solo circuiti di un salto, resta silenzioso. Superata quella soglia il padding parte anche prima del primo traffico applicativo (`nf_pad_before_usage`, che vale 1 per impostazione predefinita).

Esercizi

Esercizio 7.1 ⋆ Perché un client Tor non ha bisogno di una porta in ascolto verso la rete? In quali casi, invece, un relay apre canali come iniziatore?

Esercizio 7.2 ⋆ Un certificato Ed25519 ha il campo EXPIRATION_DATE uguale a `0x000772F7`. Quando scade? Perché la specifica ha scelto le ore anziché i secondi?

Esercizio 7.3 ⋆ Una cella **`AUTH_CHALLENGE`** ha il corpo lungo 38 byte. Quanti metodi offre? Una lunga 36 byte con N_Methods uguale a 2 è valida?

Esercizio 7.4 ⋆⋆ Un avversario registra la cella **`CERTS`** di un relay vero e la riproduce identica su un proprio server, che presenta un proprio certificato TLS. Quale passo della verifica fallisce? E se l'avversario presentasse anche lo stesso certificato TLS del relay vero?

Esercizio 7.5 ⋆⋆ Scrivi in esadecimale il corpo della **`NETINFO`** che un relay con indirizzo `203.0.113.5`, alle 12:00:00 UTC del 1° gennaio 2026, invia a un client che lo ha contattato dall'indirizzo `198.51.100.7`.

Esercizio 7.6 ⋆⋆ Supponi che i client Tor si autenticassero verso il guard con una chiave generata una volta e conservata su disco. Descrivi due modi concreti in cui questo danneggerebbe l'anonimato degli utenti.

Esercizio 7.7 ⋆⋆ Estendi `canale.py` con la verifica facoltativa dell'identità RSA (tipi 2 e 7), seguendo la sezione 7.5. Confronta l'impronta SHA-1 della chiave RSA ottenuta con il file `fingerprint` del relay. *Suggerimento:* l'impronta è lo SHA-1 della codifica DER della chiave pubblica.

Esercizio 7.8 ⋆⋆⋆ Con la rete di laboratorio attiva, apri un canale con `canale.py` e mantienilo aperto mentre il client `test008c` scarica una pagina attraverso la rete. Riceverai celle **`PADDING`** sul *tuo* canale? Formula un'ipotesi, verificala e spiega il risultato alla luce del codice di `channelpadding_decide_to_pad_channel`.

Esercizio 7.9 ⋆⋆⋆ Spiega perché il campo TLSSECRETS di **`AUTHENTICATE`** impedisce a un intermediario di inoltrare l'autenticazione di un relay A verso un relay B su una sessione TLS diversa, anche se l'intermediario inoltra fedelmente tutte le celle. Che cosa succederebbe se TLSSECRETS non esistesse?

Per approfondire

- *Tor Protocol Specification*, capitoli «Channels», «Negotiating and initializing channels» e «Relay keys», e il documento *Tor certificate formats* (cert-spec) [Thef].
- Proposta 220, *Migrate server identity keys to Ed25519*: le motivazioni della transizione da RSA.
- *Tor channel padding*, nella specifica del padding: il modello di avversario basato su NetFlow e la scelta degli intervalli.
- RFC 5705 e RFC 8446 (§ 7.5): i *keying material exporter* di TLS.

Capitolo 8

Le celle

Aggiornato al 23 settembre 2026 — torspec commit `8e1f245`; tor 0.4.9.13; Arti 2.6.0; link protocol 3–5

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- descrivere byte per byte il formato delle celle a lunghezza fissa e variabile;
- spiegare perché Tor usa unità di dimensione costante e che cosa questa scelta protegge (e che cosa no);
- interpretare il CircID e le regole con cui viene scelto su ogni canale;
- riconoscere tutti i comandi di cella e i principali comandi relay;
- leggere l'intestazione di un messaggio relay e sapere a che cosa servono `Recognized` e `Digest`;
- scrivere un codificatore/decodificatore di celle in Python (Laboratorio 2).

Nel capitolo precedente abbiamo visto come due nodi Tor aprono un *canale*: una connessione TLS su cui si accordano sulla versione del protocollo e si autenticano. In questo capitolo guardiamo *che cosa viaggia* dentro quel canale. La risposta è semplice da enunciare: **celle**. Tutto ciò che Tor trasporta, dalla costruzione di un circuito alle pagine web scaricate, viaggia spezzettato in celle. Capire le celle significa avere in mano l'alfabeto con cui è scritto l'intero protocollo.

8.1 Perché le celle

Un sistema di anonimato a bassa latenza non può nascondere *che* due nodi comunicano, ma può cercare di non rivelare *che cosa* si dicono e *quanto* ne sono consapevoli i nodi intermedi. Se ogni nodo inoltrasse i dati così come arrivano, pacchetti di dimensione diversa, un osservatore potrebbe seguire un messaggio di 1 337 byte lungo la rete semplicemente cercando un messaggio di quella lunghezza su ciascun collegamento.

Il progetto originale dell'onion routing, e poi Tor [DMS04], adotta per questo un'unità di trasmissione **di dimensione fissa**: la cella. Con celle tutte uguali:

- la lunghezza del singolo messaggio non rivela il contenuto né identifica il flusso;
- i relay possono gestire code e buffer in modo semplice ed efficiente;
- l'intestazione minima permette di inoltrare una cella leggendo pochissimi byte.

Attenzione: Che cosa le celle *non* nascondono

Le celle uniformano la *dimensione* delle singole unità, non il *numero* di unità né la loro *temporizzazione*. Un avversario che osserva entrambe le estremità di un circuito può ancora correlare i volumi e i tempi del traffico. Questo è il limite fondamentale di Tor, che affronteremo nel capitolo 26; le difese parziali (padding a livello di canale e di circuito) sono discusse nei capitoli 7 e 26 e nella Parte V.

Nota storica: 512 byte

L'articolo del 2004 che presenta Tor descrive celle di 512 byte: 2 byte di identificativo di circuito, 1 byte di comando e 509 byte di dati [DMS04]. I 509 byte di corpo sono sopravvissuti fino a oggi; l'identificativo di circuito, invece, è cresciuto a 4 byte con il link protocol 4, portando la cella a 514 byte.

8.2 Dal flusso TLS alle celle

Una volta completato l'handshake TLS, le due parti si scambiano celle *in serie*, una dopo l'altra, come un flusso continuo di byte [tor-spec § Cells (messages on channels)]. Il confine tra i record TLS e il confine tra le celle sono indipendenti: una cella può essere divisa su più record, e un record può contenere più celle o frammenti di celle. La specifica impone però che «la suddivisione in record TLS non deve rivelare informazioni sul tipo o sul contenuto delle celle» (trad. nostra).

Da questo segue una conseguenza pratica per chi implementa il protocollo: il ricevente non può aspettarsi di leggere «una cella per volta» dal socket. Deve accumulare i byte in un buffer ed estrarre una cella solo quando è completa, lasciando nel buffer l'eventuale avanzo. Il nostro decodificatore del Laboratorio 2 funzionerà esattamente così.

8.3 Il formato delle celle

Esistono due formati di cella: **a lunghezza fissa** e **a lunghezza variabile**. Entrambi iniziano con gli stessi due campi, `CircID` e `Command`.

8.3.1 Costanti

La specifica definisce le seguenti lunghezze [tor-spec § Message lengths], dove v è la versione del link protocol negoziata sul canale:

Nome	Byte	Significato
`CELL_BODY_LEN`	509	corpo di una cella a lunghezza fissa
`CIRCID_LEN`(v), $v < 4$	2	lunghezza del CircID
`CIRCID_LEN`(v), $v \geq 4$	4	
`CELL_LEN`(v), $v < 4$	512	lunghezza di una cella fissa
`CELL_LEN`(v), $v \geq 4$	514	

Per ogni v vale $\texttt{CELL_LEN}(v) = 1 + \texttt{CIRCID_LEN}(v) + \texttt{CELL_BODY_LEN}$.

Approfondimento: Un vecchio nome

Nel codice e nella documentazione più vecchi `CELL_BODY_LEN` compare come `PAYLOAD_LEN`. È lo stesso valore: 509.

8.3.2 Celle a lunghezza fissa

La fig. 8.1 mostra la cella fissa nel link protocol attuale ($v \geq 4$).

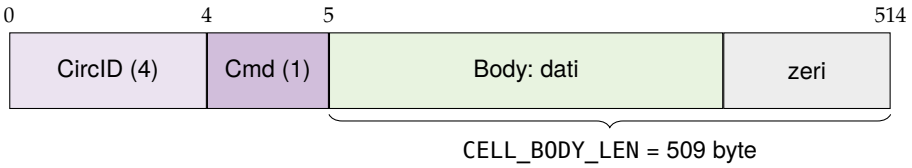


Figura 8.1: Cella a lunghezza fissa (link protocol ≥ 4): 4 byte di CircID, 1 byte di comando e 509 byte di corpo, per un totale di 514 byte. Se il contenuto utile è più corto di 509 byte, il resto del corpo è riempito con zeri. I numeri indicano l'offset in byte dall'inizio della cella.

Quando l'informazione da trasmettere occupa meno di 509 byte, il mittente **deve** riempire la parte rimanente con byte a zero, e il ricevente **deve** ignorarli [tor-spec § Padding fixed-length cell bodies].

8.3.3 Celle a lunghezza variabile

Alcune celle, soprattutto quelle dell'handshake di canale, hanno bisogno di più di 509 byte (un certificato) o di molto meno. Per queste esiste il formato variabile (fig. 8.2), che aggiunge un campo `Length` di 2 byte, intero *big-endian*, e non prevede riempimento.

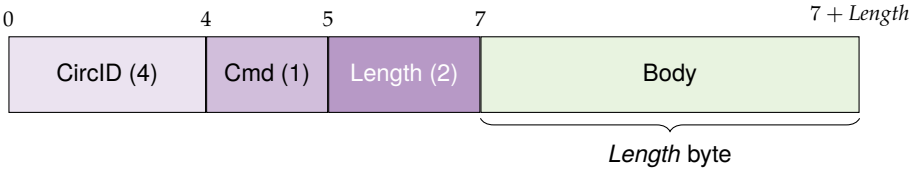


Figura 8.2: Cella a lunghezza variabile (link protocol ≥ 4): dopo CircID e comando, un campo Length di 2 byte (big-endian) indica quanti byte di corpo seguono. Il corpo può quindi essere lungo da 0 a 65 535 byte.

8.3.4 Come distinguere i due formati

Il ricevente deve sapere quanti byte leggere *prima* di aver letto la cella. La regola è interamente basata sul comando [tor-spec § Cells (messages on channels)]:

- il comando 7 (**`VERSIONS`**) è a lunghezza variabile;
- ogni altro comando minore di 128 indica una cella fissa;
- ogni comando maggiore o uguale a 128 indica una cella variabile.

In altre parole, il bit più significativo del byte di comando fa da indicatore del formato, con l'unica eccezione storica di **`VERSIONS`**.

Nota storica: L'eccezione di VERSIONS

Nel link protocol 1 tutte le celle erano fisse. Con il link protocol 2 fu introdotta la negoziazione di versione, e **`VERSIONS`**, con il valore 7 successivo ai comandi già esistenti (0–6), divenne l'unica cella variabile. Con il link protocol 3 si decise di riservare ai comandi variabili l'intervallo ≥ 128. Le versioni 1 e 2 sono oggi obsolete e le implementazioni non dovrebbero supportarle.

8.3.5 Il caso speciale di VERSIONS

C'è un problema di «uovo e gallina»: la lunghezza del CircID dipende dalla versione negoziata, ma la versione viene negoziata proprio con la cella **`VERSIONS`**. La soluzione è che **`VERSIONS`** si invia **sempre** come se fosse $v = 0$, cioè con un CircID di 2 byte. Il suo corpo è una sequenza di interi big-endian di 2 byte, uno per ogni versione supportata. Un client che supporta le versioni 3, 4 e 5 invia quindi:

Byte	Campo	Significato
`00 00`	CircID	0, CircID di 2 byte
`07`	Command	**`VERSIONS`**
`00 06`	Length	6 byte di corpo
`00 03 00 04 00 05`	Body	versioni 3, 4, 5

Entrambe le parti scelgono la versione **più alta presente in tutte e due** le celle **`VERSIONS`**. Se non c'è una versione comune, o se il corpo ha un numero dispari di byte, la connessione va chiusa [tor-spec § Negotiating versions with VERSIONS cells].

Versione	Descrizione
1	(obsoleta) handshake con certificati iniziali
2	(obsoleta) handshake con rinegoziazione TLS; introduce le celle variabili
3	(obsoleta) handshake «in-protocol» attuale (capitolo 7)
4	CircID a 4 byte
5	supporto al link padding e alla sua negoziazione

Nelle implementazioni attuali, tor offre le versioni 3, 4 e 5 (array `or_protocol_versions` in `src/core/or/connection_or.c`), mentre Arti offre solo la 4 e la 5. Due istanze aggiornate negoziano quindi sempre la versione 5.

8.4 Il CircID

Il campo `CircID` indica a quale circuito appartiene la cella; vale 0 per le celle che non appartengono ad alcun circuito (per esempio **`VERSIONS`**, **`NETINFO`**, **`PADDING`**).

Il punto fondamentale è che **il CircID è locale al canale**. Un circuito a tre salti attraversa tre canali (client–guard, guard–middle, middle–exit) e ha un CircID *diverso* su ciascuno. Ogni relay mantiene una tabella che associa la coppia (canale entrante, CircID) alla coppia (canale uscente, CircID), come mostra la fig. 8.3.

Questa scelta ha due vantaggi: nessun nodo deve coordinarsi con gli altri per scegliere gli identificativi, e il CircID non diventa un'etichetta globale che permetta di collegare i tratti di uno stesso circuito.

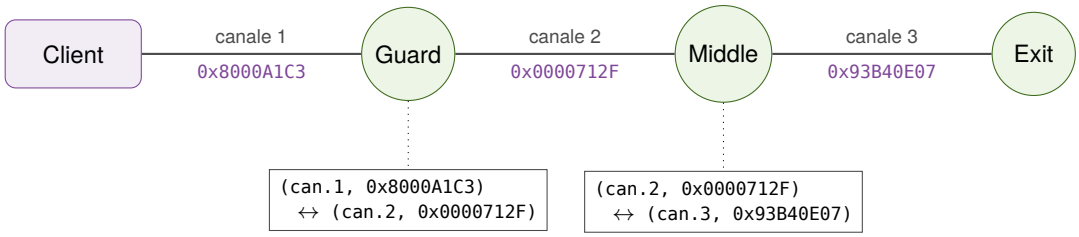


Figura 8.3: Lo stesso circuito ha un CircID diverso su ogni canale. Il guard e il middle traducono l'identificativo tramite una tabella di commutazione. Un osservatore di un singolo collegamento non può quindi riconoscere lo stesso circuito su un altro collegamento confrontando i CircID. I valori sono di esempio, ma rispettano la regola del bit più significativo descritta qui sotto: sul canale 2 il bit è a zero, cioè quella connessione l'aveva aperta il middle e il guard la sta riusando.

8.4.1 Chi sceglie il CircID

Il CircID viene scelto dal nodo che invia la cella di creazione (**`CREATE2`**, capitolo 9). Poiché su un canale entrambe le parti possono creare circuiti, bisogna evitare che scelgano lo stesso valore contemporaneamente. La regola dipende dalla versione [tor-spec § Choosing circuit IDs in create cells]:

- **link protocol** ≥ 4: il nodo che ha *aperto* la connessione usa CircID con il bit più significativo (MSB) a 1; l'altro usa CircID con MSB a 0;
- **link protocol** ≤ 3: la metà dello spazio si sceglieva confrontando le chiavi pubbliche di identità dei due relay (chi ha la chiave minore usa MSB a 0).

Il valore 0 è riservato; nessun altro valore lo è. Le implementazioni esistenti scelgono il CircID **a caso** tra i valori liberi, e la specifica raccomanda alle nuove implementazioni di fare lo stesso per non essere distinguibili. Tor rinuncia a creare nuovi circuiti su un canale dopo 64 tentativi consecutivi di trovare un valore libero.

Approfondimento: Perché 4 byte

Con 2 byte e metà dello spazio disponibile, un canale può ospitare poco più di 32 000 circuiti per lato, e la scelta casuale produce collisioni ben prima di esaurire lo spazio. La Proposta 214 (2012) lo dice senza giri di parole: «i relay stanno esaurendo i circuit ID» (trad. nostra). La proposta ha introdotto il link protocol 4 con CircID a 4 byte, implementato in tor 0.2.4.11-alpha.

8.5 I comandi di cella

Il byte `Command` stabilisce come interpretare il corpo. Le tabelle seguenti riportano tutti i valori definiti [tor-spec § Interpreting the fields: Command]. Nella colonna C, «sì» indica che il comando richiede un CircID diverso da zero e «no» che lo richiede uguale a zero; la colonna P indica la prima versione del link protocol che supporta il comando (se vuota: almeno dalla 3).

Comandi sconosciuti. Non sono ammessi altri valori. Chi riceve un comando sconosciuto *può* scartare la cella in silenzio e *può* chiudere il canale. Il motivo di questa severità è che ogni nuovo comando viene introdotto insieme a una nuova versione del link protocol: un'implementazione corretta non invierà mai un comando che l'altra parte non ha dichiarato di

Val.	Comando	C	P	Funzione
0	**PADDING**	no		padding di collegamento
1	**CREATE**	sì		crea circuito (TAP, deprecato)
2	**CREATED**	sì		risposta a CREATE (deprecato)
3	**RELAY**	sì		dati da estremo a estremo
4	**DESTROY**	sì		distrugge un circuito
5	**CREATE_FAST**	sì		crea circuito senza crittografia a chiave pubblica
6	**CREATED_FAST**	sì		risposta a CREATE_FAST
8	**NETINFO**	no		ora e indirizzi
9	**RELAY_EARLY**	sì		come RELAY, in numero limitato
10	**CREATE2**	sì		crea circuito (handshake estensibile)
11	**CREATED2**	sì		risposta a CREATE2
12	**PADDING_NEGOTIATE**	sì	5	negoziazione del padding

Tabella 8.1: Comandi delle celle a lunghezza fissa.

Val.	Comando	C	Funzione
7	**VERSIONS**	no	negoziazione del link protocol
128	**VPADDING**	no	padding a lunghezza variabile
129	**CERTS**	no	certificati
130	**AUTH_CHALLENGE**	no	sfida per l'autenticazione
131	**AUTHENTICATE**	no	autenticazione dell'iniziatore
132	**AUTHORIZE**	no	riservato

Tabella 8.2: Comandi delle celle a lunghezza variabile.

supportare. Prima che il link protocol fosse versionato, invece, i comandi sconosciuti venivano ignorati nell'ipotesi che provenissero da una versione più recente di Tor.

Possiamo raggruppare i comandi per funzione:

Gestione del canale **VERSIONS**, **CERTS**, **AUTH_CHALLENGE**, **AUTHENTICATE**, **NETINFO**: l'handshake del capitolo 7.

Padding **PADDING**, **VPADDING**, **PADDING_NEGOTIATE**: traffico fittizio per ostacolare l'analisi del traffico sul collegamento.

Ciclo di vita del circuito **CREATE2**/**CREATED2** (e le varianti storiche), **DESTROY**: creazione e distruzione dei circuiti (capitolo 9).

Trasporto **RELAY** e **RELAY_EARLY**: tutto ciò che viaggia *dentro* un circuito.

8.5.1 DESTROY

Il contenuto significativo di **DESTROY** è un solo byte con il *motivo* della chiusura [tor-spec § Tearing down circuits]. La cella rimane però a lunghezza fissa: gli altri 508 byte del corpo sono padding a zero, come richiede la regola generale delle celle fisse. Chi riceve **DESTROY** libera le risorse del circuito e, se non è l'estremo, propaga a sua volta un **DESTROY** al nodo successivo.

0	NONE	7	OR_IDENTITY
1	PROTOCOL	8	CHANNEL_CLOSED
2	INTERNAL	9	FINISHED
3	REQUESTED	10	TIMEOUT
4	HIBERNATING	11	DESTROYED
5	RESOURCELIMIT	12	NOSUCHSERVICE
6	CONNECTFAILED		

Attenzione: Il motivo come canale laterale

La specifica raccomanda di inviare **sempre** il motivo NONE e di non propagare il motivo ricevuto. Se un relay inoltrasse il motivo originale in entrambe le direzioni, due relay complici sullo stesso circuito potrebbero usarlo come segnale per scoprire di trovarsi sullo stesso circuito: un piccolo esempio di come anche un singolo byte possa diventare un *side channel*.

8.5.2 RELAY_EARLY

RELAY_EARLY ha lo stesso formato di **RELAY** ma serve a **limitare la lunghezza dei circuiti** [tor-spec § Handling RELAY_EARLY cells]. Le regole sono:

- un client deve inviare i messaggi di estensione del circuito (`EXTEND2`) solo dentro celle **RELAY_EARLY**; dalla versione 0.2.3.11-alpha i relay rifiutano le estensioni ricevute altrimenti;
- un relay che riceve più di 8 celle **RELAY_EARLY** in avanti su un circuito dovrebbe chiuderlo; se ne riceve anche una sola *all'indietro*, verso il client, deve chiuderlo subito;
- per non rivelare la lunghezza del circuito, i client dovrebbero inviare come **RELAY_EARLY** anche le prime 8 celle circa non destinate al primo salto.

Senza questo limite, un avversario potrebbe costruire circuiti lunghissimi che passano più volte per gli stessi relay e amplificare così il proprio traffico: è l'attacco di congestione con percorsi lunghi descritto da Evans, Dingledine e Grothoff [EDG09].

Nota storica: Un segnale inatteso

La regola che vieta **RELAY_EARLY** all'indietro nasce da un incidente del 2014: alcuni relay usarono celle **RELAY_EARLY** inviate verso il client come canale nascosto per marcare i circuiti degli utenti degli onion service. Ne parleremo in dettaglio nel capitolo 27.

8.6 Dentro le celle RELAY

Le celle **RELAY** e **RELAY_EARLY** trasportano i **messaggi relay**: i comandi che il client scambia con i nodi del circuito, da quelli per estenderlo fino ai dati degli stream TCP. Il loro corpo di 509 byte è cifrato a strati, uno per ogni salto: è la «cipolla» del capitolo 10. Dal punto di vista del canale, quindi, il corpo di una cella RELAY è sempre pieno: non c'è riempimento a zero visibile.

Una volta tolti tutti gli strati di cifratura, il corpo in chiaro ha il formato tradizionale della fig. 8.4 [tor-spec § Relay cells].

Relay command (1 byte) il tipo di messaggio (tabella 8.3).

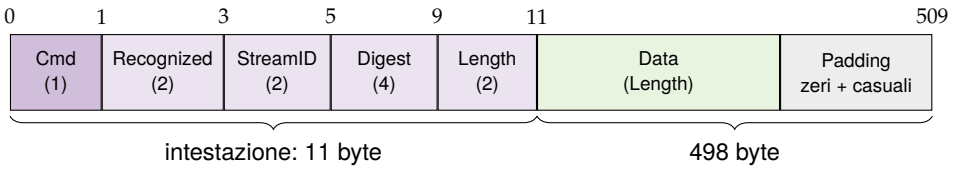


Figura 8.4: Corpo *decifrato* di una cella RELAY nel formato tradizionale: 11 byte di intestazione, fino a 498 byte di dati e padding eventuale fino a 509 byte. La specifica raccomanda fino a quattro byte iniziali a zero e poi byte casuali; se restano meno di quattro byte, sono tutti zero, mentre con 498 byte di dati non c'è padding.

Recognized (2 byte) vale zero nel messaggio in chiaro. Serve al relay per capire rapidamente se la cella è destinata a lui: se dopo aver tolto il proprio strato il campo non è zero, la cella è sicuramente ancora cifrata e va inoltrata.

StreamID (2 byte) lo stream a cui si riferisce il messaggio, scelto dal client; zero per i messaggi che riguardano il circuito nel suo insieme.

Digest (4 byte) i primi 4 byte di un digest *progressivo* di tutti i messaggi scambiati con quel salto (SHA-1 per i circuiti ordinari, SHA3-256 per quelli degli onion service). Conferma che la cella è davvero destinata al nodo e che non è stata alterata.

Length (2 byte) quanti byte del campo Data sono significativi.

Data il contenuto del messaggio.

Padding il resto, fino a 509 byte.

Con 11 byte di intestazione restano al massimo $509 - 11 = 498$ byte di dati per cella.

8.6.1 Recognized e Digest: un filtro a due stadi

Perché due campi? Calcolare il digest per ogni cella in transito sarebbe costoso. Il campo Recognized è un filtro economico: dopo la decifratura di una cella ancora cifrata, i suoi 2 byte sono pseudocasuali e valgono zero solo con probabilità 2^{-16}. Solo quando Recognized è zero il relay calcola il digest; se questo coincide, la cella è «riconosciuta» e viene elaborata, altrimenti viene inoltrata. Vedremo i dettagli del calcolo, e le sue debolezze, nel capitolo 10.

8.6.2 Il padding del messaggio relay

Il padding del messaggio relay è diverso dal riempimento a zero delle celle fisse. La specifica raccomanda 4 byte a zero seguiti da **byte casuali** [tor-spec § Relay cells]. I byte casuali rendono imprevedibile il contenuto della cella: questo è necessario per i SENDME autenticati della Proposta 289, che dimostrano di aver ricevuto davvero i dati includendo il digest delle celle ricevute (capitolo 11). I riceventi non devono fare affidamento sul contenuto del padding.

8.6.3 I comandi relay

La tabella 8.3 elenca i comandi relay definiti. La colonna Dir. indica la direzione: A (*avanti*, solo dal client verso il circuito), I (*indietro*, solo verso il client), A/I (entrambe); C indica un messaggio di *controllo*, che deve avere StreamID zero.

Se una cella è riconosciuta ma il comando relay non è noto, va scartata; il suo contenuto conta comunque per il digest e per le finestre di controllo di flusso.

Val.	Comando	Dir.	Funzione
Protocollo di base			
1	BEGIN	A	apre uno stream
2	DATA	A/I	dati dello stream
3	END	A/I	chiude uno stream
4	CONNECTED	I	stream aperto con successo
5	SENDME	A/I, C?	conferma di ricezione (controllo di flusso)
6–7	EXTEND/EXTENDED	A/I, C	estensione con TAP (obsoleti)
8–9	TRUNCATE/TRUNCATED	A/I, C	troncamento del circuito (non usati)
10	DROP	A/I, C	padding a lungo raggio
11–12	RESOLVE/RESOLVED	A/I	risoluzione DNS remota
13	BEGIN_DIR	A	stream verso la directory cache
14–15	EXTEND2/EXTENDED2	A/I, C	estensione del circuito
16–18	*riservati*		UDP su Tor (Proposta 339)
Conflux			
19–22	CONFLUX_...	A/I, C	LINK, LINKED, LINKED_ACK, SWITCH: circuiti multipli (Proposta 329)
Onion service			
32	ESTABLISH_INTRO	A, C	crea un punto di introduzione
33	ESTABLISH_RENDEZVOUS	A, C	crea un punto di rendezvous
34–35	INTRODUCE1/INTRODUCE2	A/I, C	richiesta di introduzione
36–37	RENDEZVOUS1/RENDEZVOUS2	A/I, C	unione al rendezvous
38–40	..._ESTABLISHED, INTRODUCE_ACK	I, C	conferme
Padding di circuito e controllo di flusso			
41–42	PADDING_NEGOTIATE/...D	A/I, C	padding di circuito
43–44	XOFF/XON	A/I	controllo di flusso dello stream (Proposta 324)

Tabella 8.3: Comandi relay. SENDME è controllo solo quando riguarda il circuito e non uno stream (capitolo 11).

8.7 Verso un nuovo formato: proposta 340 e CGO

Il formato tradizionale ha due limiti noti. Il primo è l'efficienza: un messaggio grande occupa sempre almeno una cella intera, e più messaggi piccoli non possono condividere una cella. Il secondo è la sicurezza: il digest di 4 byte e la cifratura AES-CTR senza autenticazione lasciano spazio ad attacchi di *tagging*, che vedremo nel capitolo 10.

La Proposta 340 (*Packed and fragmented relay messages*) prevede di separare il concetto di *cella* da quello di *messaggio*: in quel progetto un messaggio può occupare più celle e una cella può contenere più messaggi. La Proposta 359 (*Counter Galois Onion, Updated*) introduce invece il nuovo schema di cifratura dei relay, CGO, e riprende dalla 340 *la struttura dell'intestazione*. Sono però due cambiamenti distinti: nel formato CGO implementato da tor e Arti, la presenza della nuova intestazione **non abilita ancora** l'impacchettamento o la frammentazione; ogni cella contiene un solo messaggio relay. Queste funzionalità della proposta 340 restano da distinguere dal supporto già disponibile per CGO. Una cella decifrata con CGO ha questa struttura:

Campo	Byte
tag di autenticazione	16
corpo	$509 - 16 = 493$
nel corpo: comando	1
lunghezza	2
StreamID (solo per alcuni comandi)	2
dati	*lunghezza*
padding (4 zeri, poi byte casuali)	resto

Lo StreamID compare solo per i comandi che lo richiedono (BEGIN, DATA, END, CONNECTED, RESOLVE, RESOLVED, BEGIN_DIR, XON, XOFF); Recognized e Digest spariscono, sostituiti dal tag. Un client usa CGO con un relay solo se entrambi lo supportano (sottoprotocollo `Relay=6`) e lo negoziano durante la creazione del circuito.

Approfondimento: Quanti dati per cella con CGO?

Dal formato si ricava che in un messaggio DATA restano $493 - 1 - 2 - 2 = 488$ byte di dati, 10 in meno del formato tradizionale: è il prezzo dell'autenticazione a 16 byte.

CGO non è più soltanto una proposta: tor lo supporta dalla serie 0.4.9 (introdotto in 0.4.9.3-alpha, settembre 2025), e in Arti il supporto è sempre attivo, al punto che l'opzione di compilazione `counter-galois-onion` è rimasta solo per compatibilità. Nel repository delle specifiche le due proposte risultano ancora «Open»: lo stato di una proposta segue il processo di standardizzazione e non sempre coincide con quello del codice.

8.8 Anatomia di una sessione

Mettiamo insieme i pezzi e seguiamo, cella per cella, che cosa invia un client al proprio guard per costruire un circuito a tre salti e scaricare una pagina web. La fig. 8.5 riassume la sequenza; i dettagli di ogni passo sono nei capitoli successivi.

1. **Handshake di canale**: **`VERSIONS`**, poi dal guard **`CERTS`**, **`AUTH_CHALLENGE`**, **`NETINFO`**; il client risponde con **`NETINFO`**. CircID sempre 0.

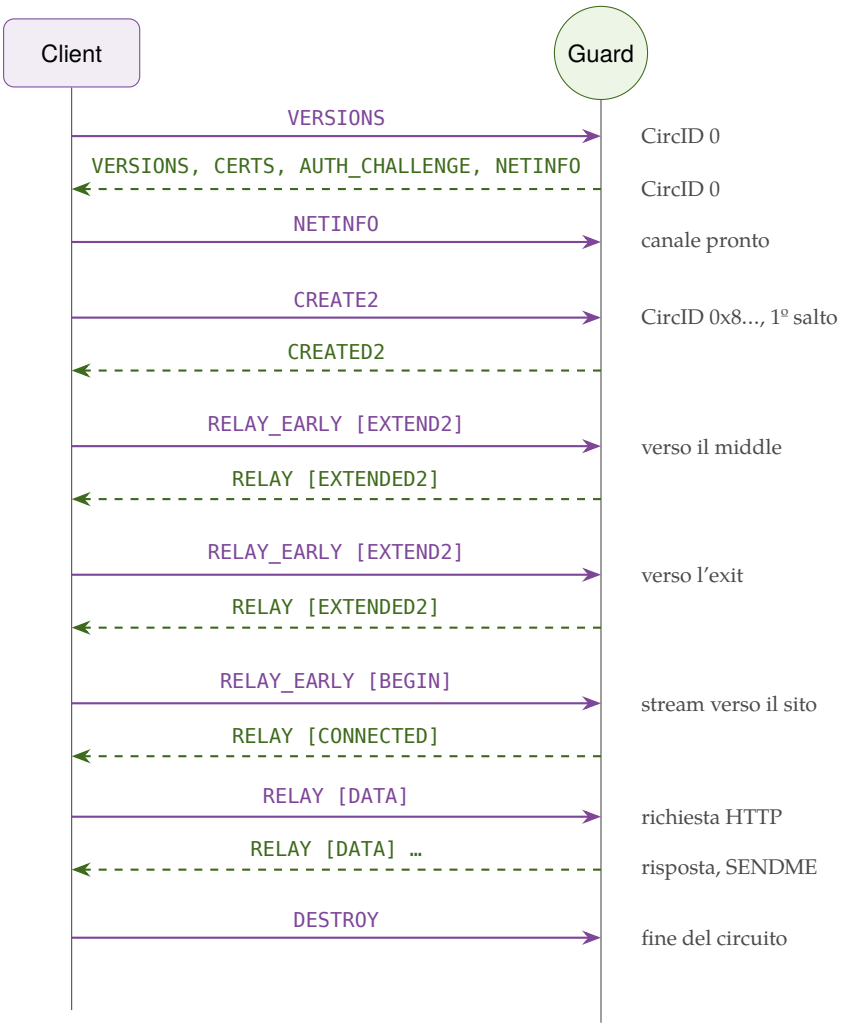


Figura 8.5: Celle scambiate tra client e guard sul canale TLS per creare un circuito a tre salti e aprire uno stream. Il guard vede solo le celle del suo canale: non può leggere il contenuto delle celle RELAY destinate al middle e all'exit, cifrate con chiavi che non possiede.

2. **Primo salto**: **`CREATE2`** con un CircID nuovo (MSB a 1, perché il client ha aperto la connessione); il guard risponde **`CREATED2`**.
3. **Secondo e terzo salto**: due celle **`RELAY_EARLY`** contenenti `EXTEND2`; il guard, e poi il middle, le trasformano in **`CREATE2`** verso il nodo successivo. Le risposte tornano come celle **`RELAY`** contenenti `EXTENDED2`.
4. **Stream**: una cella con `BEGIN` per l'exit, che risponde `CONNECTED`; poi `DATA` in entrambe le direzioni, intervallate da `SENDME`.
5. **Chiusura**: `END` per lo stream e, alla fine della vita del circuito, **`DESTROY`**.

8.9 Laboratorio: un codificatore di celle

Laboratorio 2: Codificare e decodificare celle

Obiettivo. Scrivere il modulo `labs/it/codice/torlab/celle.py`, che userai in tutti i laboratori successivi.
Prerequisiti. Python 3.10 o successivo, `pytest`. Non serve alcuna rete: il laboratorio lavora su byte costruiti a mano.
Tempo stimato. 1–2 ore.

Passo 1: costanti e comandi. Traduciamo le tabelle della specifica in enumerazioni Python, così i valori numerici compaiono in un solo punto:

```
CELL_BODY_LEN = 509
RELAY_HEADER_LEN = 11
RELAY_DATA_MAX = CELL_BODY_LEN - RELAY_HEADER_LEN   # 498

class Comando(IntEnum):
    PADDING = 0
    CREATE = 1
    # ... tutti i valori delle tabelle 8.1 e 8.2
    AUTHORIZE = 132

def circid_len(versione):
    return 2 if versione < 4 else 4

def lunghezza_variabile(comando):
    return comando == Comando.VERSIONS or comando >= 128
```

Passo 2: codifica. Una funzione che, dati CircID, comando, corpo e versione del link protocol, produce i byte da inviare. Nota il trattamento speciale di **`VERSIONS`** e il riempimento con zeri:

```
def codifica(cella, versione):
    if cella.comando == Comando.VERSIONS:
        versione = 0                  # CircID sempre di 2 byte
    n = circid_len(versione)
    intestazione = cella.circ_id.to_bytes(n, "big") + bytes([cella.comando])
    if lunghezza_variabile(cella.comando):
        return (intestazione + struct.pack(">H", len(cella.corpo))
                + cella.corpo)
    return intestazione + cella.corpo.ljust(CELL_BODY_LEN, b"\x00")
```

Passo 3: decodifica da un flusso. Poiché i byte arrivano dal socket TLS a pezzi, la decodifica restituisce `None` finché la cella non è completa, e altrimenti la cella e i byte avanzati:

```
def decodifica(buffer, versione):
    n = circid_len(versione)
    if len(buffer) < n + 1:
        return None
    circ_id = int.from_bytes(buffer[:n], "big")
    comando = buffer[n]
    if lunghezza_variabile(comando):
        if len(buffer) < n + 3:
            return None
        (lunghezza,) = struct.unpack(">H", buffer[n+1:n+3])
        inizio, fine = n + 3, n + 3 + lunghezza
    else:
        inizio, fine = n + 1, n + 1 + CELL_BODY_LEN
    if len(buffer) < fine:
        return None
    return Cella(circ_id, comando, buffer[inizio:fine]), buffer[fine:]
```

Passo 4: VERSIONS e negoziazione.

```
def negozia_versione(mie, corpo_ricevuto):
    if len(corpo_ricevuto) % 2:
        raise ErroreCella("VERSIONS con un numero dispari di byte")
    sue = {v for (v,) in struct.iter_unpack(">H", corpo_ricevuto)}
    comuni = set(mie) & sue
    if not comuni:
        raise ErroreCella("nessuna versione in comune")
    return max(comuni)
```

Passo 5: il messaggio relay. Infine l'intestazione di 11 byte. Il formato `">BHH4sH"` di `struct` corrisponde esattamente ai campi della fig. 8.4:

```
def codifica_relay(msg, casuale=os.urandom):
    intestazione = struct.pack(">BHH4sH", msg.comando, msg.recognized,
                               msg.stream_id, msg.digest, len(msg.dati))
    spazio = CELL_BODY_LEN - RELAY_HEADER_LEN - len(msg.dati)
    padding = b"\x00" * min(4, spazio) + casuale(max(0, spazio - 4))
    return intestazione + msg.dati + padding
```

Il parametro `casuale` permette ai test di sostituire i byte casuali con valori prevedibili.

I listati mostrano il percorso essenziale della codifica, omettendo per brevità alcuni controlli sugli input. Il modulo completo in `labs/it/codice/torlab/celle.py` verifica anche la classe del comando, il valore del CircID e le dimensioni massime dei corpi: 509 byte per una cella fissa, 65 535 per una variabile e 498 byte di dati per un messaggio relay tradizionale. Per trattare dati ricevuti dalla rete va usata la versione completa, non uno di questi frammenti isolato.

Verifica. Il repository contiene i test in `labs/it/codice/torlab/test/test_celle.py`. Eseguili dalla cartella `labs/it/codice`:

```
$ python3 -m pytest -q torlab/test/test_celle.py
........                                                 [100%]
8 passed
```

(Senza il nome del file, `pytest` esegue i test di tutti i laboratori presenti nel repository.) Un test verifica, per esempio, che la cella **`VERSIONS`** per le versioni 3, 4 e 5 corrisponda byte per byte all'esempio della sezione 8.3: `0000 07 0006 0003 0004 0005`.

Estensioni. Nel Laboratorio 1 (capitolo 7) il modulo verrà usato per parlare con un relay vero della rete privata; nel Laboratorio 4 aggiungeremo il calcolo del digest e la cifratura a strati.

Riepilogo

- La cella è l'unità di trasmissione di Tor sul canale. Le celle fisse misurano 514 byte (CircID 4 + comando 1 + corpo 509) dal link protocol 4, 512 byte prima.
- Le celle variabili aggiungono un campo Length di 2 byte; sono variabili **`VERSIONS`** (7) e tutti i comandi ≥ 128.
- **`VERSIONS`** usa sempre un CircID di 2 byte; si sceglie la versione comune più alta.
- Il CircID è locale al canale; chi ha aperto la connessione usa valori con MSB a 1, scelti a caso.
- **`DESTROY`** dovrebbe portare sempre il motivo NONE per non creare canali laterali; **`RELAY_EARLY`** limita la lunghezza dei circuiti.
- Il corpo decifrato di una cella RELAY ha 11 byte di intestazione (comando, Recognized, StreamID, Digest, Length) e al massimo 498 byte di dati; il padding è di 4 zeri seguiti da byte casuali.
- La proposta 359 ha portato al formato CGO con tag di autenticazione di 16 byte e StreamID facoltativo; impacchettamento e frammentazione, previsti dalla proposta 340, non sono implicati dall'adozione di CGO.

Esercizi

Esercizio 8.1 ⋆ Quanti byte misura una cella **`CREATE2`** su un canale che ha negoziato il link protocol 3? E con il 5? Quanti byte misura una cella **`VERSIONS`** con corpo `00 04 00 05`?

Esercizio 8.2 ⋆ Per ciascuno dei comandi 0, 7, 9, 12, 129 e 200, indica se la cella è fissa o variabile. Quale di questi valori non è ammesso?

Esercizio 8.3 ⋆ Un client apre un canale verso un guard con link protocol 5. Quale di questi CircID può usare per un nuovo circuito: `0x00000000`, `0x12345678`, `0x92345678`? Perché?

Esercizio 8.4 ⋆⋆ Decodifica a mano la seguente sequenza di byte ricevuta su un canale con link protocol 4, indicando CircID, comando e lunghezza del corpo di ogni cella:
`00 00 00 00 81 00 03 aa bb cc 80 00 00 07 04 00 …`
(la seconda cella prosegue con il resto del corpo, che non riportiamo).

Esercizio 8.5 ⋆⋆ Scrivi i primi 11 byte, in esadecimale, del corpo in chiaro di un messaggio relay DATA sullo stream 5, con 300 byte di dati, prima del calcolo del digest. Quanti byte di padding seguono, e quanti di essi sono sicuramente zero?

Esercizio 8.6 ⋆⋆ Un relay intermedio elabora $1\,000\,000$ di celle che non sono destinate a lui. In media, per quante di esse dovrà calcolare il digest? Perché non basta il campo Recognized a stabilire che una cella è destinata al relay?

Esercizio 8.7 ⋆⋆ Descrivi uno scenario concreto in cui due relay complici potrebbero sfruttare il motivo di un **`DESTROY`** per scoprire di trovarsi sullo stesso circuito. Perché il motivo NONE neutralizza lo scenario?

Esercizio 8.8 ⋆⋆ Un client scarica un file di $1\,\mathrm{MiB}$ ($1\,048\,576$ byte) attraverso un circuito. Quante celle DATA servono con il formato tradizionale? E con il formato CGO? Qual è, in percentuale, l'overhead delle intestazioni di cella rispetto ai dati utili nei due casi (link protocol 4, senza contare TLS)?

Esercizio 8.9 ⋆ ⋆ ⋆ Estendi il Laboratorio 2 con un test *property-based* (per esempio con la libreria Hypothesis) che generi celle casuali, fisse e variabili, le codifichi, le concateni e le spezzi in frammenti di lunghezza casuale, verificando che la decodifica restituisca esattamente le celle originali.

Esercizio 8.10 ⋆ ⋆ ⋆ Individua nel codice sorgente di Arti (crate `tor-cell`) le funzioni che codificano e decodificano le celle di canale. Confronta la loro gestione del caso «cella incompleta» con la funzione `decodifica` del Laboratorio 2.

Per approfondire

- *Tor Protocol Specification*, capitoli «Cells», «Relay cells» e «Negotiating channels» [Thef]: la fonte primaria di tutto il capitolo.
- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04], sezione 4: il formato originale delle celle e le motivazioni.
- N. Evans, R. Dingledine, C. Grothoff, *A Practical Congestion Attack on Tor Using Long Paths* [EDG09]: l'attacco che motiva **`RELAY_EARLY`**.
- Proposte 359 (CGO, implementato) e 340 (impacchettamento e frammentazione ancora da distinguere dal formato CGO) in torspec.

Capitolo 9

Costruzione dei circuiti: TAP, CREATE_FAST, ntor, ntor-v3

Aggiornato al 23 settembre 2026 — torspec commit `8e1f245`; tor 0.4.9.13; proposte 216, 332, 350

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- descrivere come un client costruisce un circuito un salto alla volta, con CREATE2 ed EXTEND2;
- spiegare che cosa deve garantire un handshake di circuito: autenticazione del relay, segretezza, forward secrecy;
- seguire passo per passo gli handshake CREATE_FAST, ntor e ntor-v3, e sapere perché TAP è stato abbandonato;
- derivare dalle chiavi negoziate le quattro chiavi di un salto (Df, Db, Kf, Kb);
- verificare un'implementazione con i vettori di test ufficiali;
- creare circuiti reali verso un relay della rete di laboratorio (Laboratorio 3).

Con un canale aperto (capitolo 7) e l'alfabeto delle celle (capitolo 8) abbiamo tutto ciò che serve per costruire la struttura centrale di Tor: il **circuito**. In questo capitolo vediamo come il client stabilisce, con ciascun relay del percorso, un segreto condiviso che nessun altro conosce, e come lo trasforma nelle chiavi che cifreranno il traffico. Il modo in cui quelle chiavi vengono usate, la cifratura a strati, è l'argomento del capitolo 10.

9.1 Un circuito, un salto alla volta

Il client costruisce il circuito in modo *incrementale*, o «telescopico» [DMS04]:

1. sceglie il percorso, per esempio guard R_1, middle R_2, exit R_3 (capitolo 13);
2. sul canale verso R_1 invia una cella **`CREATE2`** con la prima metà di un handshake crittografico; R_1 risponde con **`CREATED2`**, e ora client e R_1 condividono delle chiavi;
3. invia, cifrato per R_1, un messaggio `EXTEND2` che chiede a R_1 di estendere il circuito verso R_2 e contiene la prima metà di un nuovo handshake, questa volta destinato a R_2;
4. R_1 apre (o riusa) un canale verso R_2 e gli invia una **`CREATE2`** con quei dati; la risposta **`CREATED2`** torna a R_1, che la inoltra al client dentro un messaggio `EXTENDED2`;

5. il client completa l'handshake con R_2 e ripete il procedimento per R_3, questa volta attraverso R_1 e R_2.

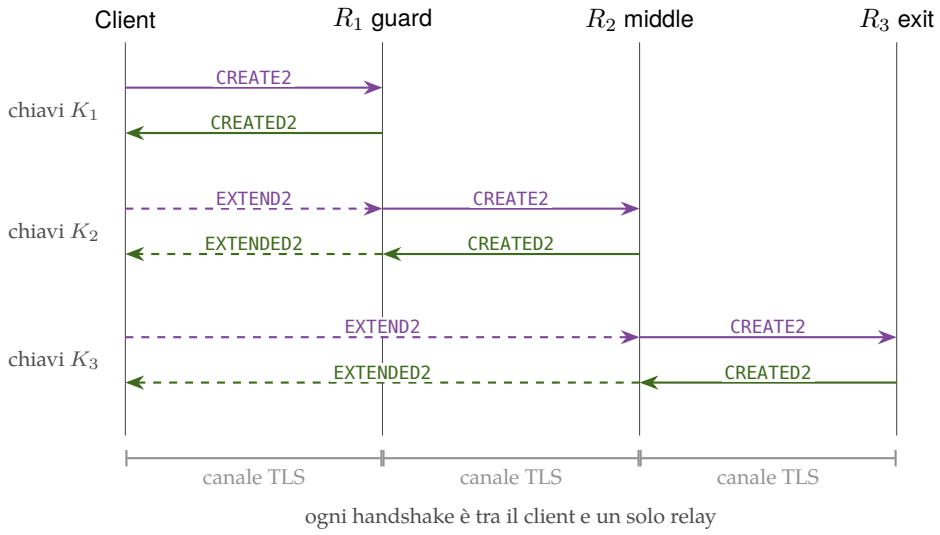


Figura 9.1: Costruzione telescopica di un circuito a tre salti. Ogni handshake è tra il client e un solo relay; i relay intermedi si limitano a trasportarne i messaggi. Le frecce tratteggiate indicano messaggi che viaggiano dentro il circuito, cifrati con le chiavi dei salti già stabiliti.

Questo schema ha una conseguenza fondamentale. L'handshake con R_3 passa attraverso R_1 e R_2, ma è cifrato per la chiave pubblica di R_3: né R_1 né R_2 possono ricavarne il segreto condiviso. E poiché ogni salto usa un segreto diverso, nessun relay può leggere quello che il client scambia con gli altri. Il guard R_1 sa chi è il client ma non dove va il traffico; l'exit R_3 sa dove va il traffico ma non chi lo invia.

Nota storica: Onion routing di prima generazione

Nel sistema originale del Naval Research Laboratory il client costruiva un'unica «cipolla»: un messaggio cifrato a strati con le chiavi pubbliche di tutti i relay, che ciascuno sbucciava per scoprire il salto successivo e la propria chiave di sessione (capitolo 3). Un relay compromesso in futuro avrebbe potuto decifrare le cipolle registrate. Il circuito telescopico di Tor, con uno scambio Diffie-Hellman effimero per ogni salto, ha introdotto la *forward secrecy*: una volta cancellate le chiavi di sessione, nemmeno il possesso delle chiavi a lungo termine permette di decifrare il traffico passato [DMS04].

9.2 Le celle e i messaggi

9.2.1 CREATE2 e CREATED2

Il formato di **`CREATE2`** è volutamente generico [tor-spec § Create and Created cells]: un tipo di handshake (HTYPE, 2 byte), una lunghezza (HLEN, 2 byte) e i dati (HDATA). **`CREATED2`** contiene solo HLEN e HDATA, perché il tipo è già noto. Tutto sta in una cella fissa, quindi i dati di handshake non possono superare $509 - 4 = 505$ byte in **`CREATE2`**.

HTYPE	Handshake	Stato
`0x0000`	TAP	obsoleto, rimosso in tor 0.4.9.1-alpha
`0x0001`	—	riservato
`0x0002`	ntor	in uso
`0x0003`	ntor-v3	in uso, preferito

Se l'handshake riesce, il relay risponde **`CREATED2`**; se fallisce, risponde con **`DESTROY`**. Il CircID della nuova cella segue le regole della sezione 8.4: nel link protocol 4 e successivi, chi ha aperto il canale usa valori con il bit più significativo a 1.

9.2.2 EXTEND2 ed EXTENDED2

Il messaggio relay `EXTEND2` (comando relay 14) contiene, oltre ai campi di **`CREATE2`**, una lista di *link specifier* che dicono al relay come raggiungere il salto successivo e come riconoscerlo [tor-spec § Extend and Extended messages]:

Tipo	Contenuto	Lunghezza
`00`	indirizzo IPv4 e ORPort	6
`01`	indirizzo IPv6 e ORPort	18
`02`	identità RSA legacy (SHA-1, 20 byte)	20
`03`	identità Ed25519	32

Il relay che estende deve verificare *tutte* le identità indicate, e non deve estendere il circuito se il relay raggiunto non dimostra di possederle, con l'handshake di canale del capitolo 7. È così che il client, pur non parlando direttamente con R_2, ottiene la garanzia che il suo handshake arrivi al relay giusto. Il relay rifiuta anche le richieste di estendere il circuito verso sé stesso o verso il relay da cui proviene la richiesta.

Due dettagli mostrano quanta attenzione il protocollo dedichi alla *indistinguibilità*. Primo: la specifica raccomanda di inviare gli specificatori sempre nello stesso ordine (`00`, `02`, `03`, `01`), così che l'ordine non riveli quale implementazione li ha prodotti. Secondo: `EXTEND2` viaggia sempre dentro una cella **`RELAY_EARLY`** (sezione 8.5), il cui numero limitato impedisce di costruire circuiti arbitrariamente lunghi.

`EXTENDED2` (comando relay 15) ha esattamente il corpo di **`CREATED2`**.

9.3 Che cosa deve fare un handshake

Prima di esaminare gli handshake uno per uno, fissiamo che cosa ci aspettiamo da loro. Al termine, client e relay devono condividere un segreto con queste proprietà:

Autenticazione del relay Il client deve sapere che il segreto è condiviso con *quel* relay e con nessun altro. Il relay invece non deve sapere nulla del client: l'handshake è autenticato *in un solo verso*, altrimenti ogni relay conoscerebbe l'identità del client.

Segretezza Un osservatore della rete, o un relay precedente nel circuito che inoltra i messaggi, non deve poter ricavare il segreto.

Forward secrecy Anche chi ottenesse in futuro le chiavi a lungo termine del relay non deve poter ricalcolare i segreti di sessione passati.

Efficienza Un relay molto usato esegue migliaia di handshake al secondo: il costo computazionale conta.

Ogni handshake che vedremo è un compromesso diverso tra queste esigenze.

9.4 TAP: il primo handshake

L'handshake originale di Tor, TAP (*Tor Authentication Protocol*), combinava uno scambio Diffie-Hellman classico, in un gruppo modulare di 1024 bit, con la cifratura a chiave pubblica RSA a 1024 bit: il client cifrava la propria metà Diffie-Hellman con la chiave RSA «onion» del relay, e solo il relay giusto poteva decifrarla e rispondere. Le chiavi venivano poi derivate con la funzione KDF-TOR, basata su SHA-1 (sezione 9.8).

TAP soddisfaceva le proprietà richieste, ma con margini sempre più ridotti: 1024 bit sono insufficienti sia per RSA sia per Diffie-Hellman modulare, la costruzione della cifratura ibrida era considerata fragile, e gli handshake costavano molto in termini di calcolo. Dal 2013 è stato affiancato da ntor, e con tor 0.4.9.1-alpha i relay hanno smesso del tutto di supportarlo (Proposta 350). Una traccia resta visibile: per compatibilità, i relay continuano a pubblicare nei descrittori anche la vecchia chiave RSA «onion». Nella rete di laboratorio la si vede nei microdescrittori, alla voce `onion-key`, accanto alla chiave `ntor-onion-key` che invece viene davvero usata.

9.5 CREATE_FAST: quando la chiave pubblica non serve

C'è un caso in cui l'handshake a chiave pubblica è superfluo: il *primo* salto. Il client ha appena aperto un canale TLS con quel relay e ne ha verificato l'identità; TLS gli offre già un canale cifrato e autenticato. **`CREATE_FAST`** sfrutta questo fatto [tor-spec § CREATE_FAST/CREATED_FAST cells]:

1. il client invia 20 byte casuali X;
2. il relay risponde con 20 byte casuali Y e 20 byte KH;
3. entrambi calcolano con KDF-TOR, a partire da $K_0 = X \| Y$, il flusso di chiavi; i primi 20 byte devono coincidere con KH, e dimostrano che il relay ha ricevuto X.

La sicurezza di **`CREATE_FAST`** poggia interamente su quella del canale TLS: chi potesse leggere il TLS leggerebbe anche X e Y. Per questo va usato solo per circuiti di un salto. Il caso d'uso tipico è il client che, all'avvio, deve scaricare informazioni di directory da un relay di cui non conosce ancora la chiave onion. Il codice di tor lo dice esplicitamente: la funzione `should_use_create_fast_for_circuit` (in `src/core/or/circuitbuild.c`) sceglie **`CREATE_FAST`** solo se il client non dispone di una chiave onion utilizzabile per il primo salto.

Attenzione: CREATE_FAST rivela il primo salto

Un relay che riceve **`CREATE_FAST`** sa con certezza di essere il primo salto del circuito, e quindi che l'altra parte del canale è il client. La specifica suggerisce ai relay di usare proprio questo indizio per rifiutare i circuiti di un solo salto che tentano di uscire verso Internet.

9.6 ntor

L'handshake ntor, proposto da Goldberg, Stebila e Ustaoglu [GSU13] e introdotto in Tor con la Proposta 216 (tor 0.2.4.8-alpha, 2013), sostituisce RSA e il Diffie-Hellman modulare con **Curve25519**, uno scambio di chiavi su curva ellittica veloce e con chiavi di soli 32 byte (capitolo 5).

9.6.1 Gli ingredienti

Il relay ha una chiave onion a medio termine su Curve25519, b (segreta) e $B = bG$ (pubblica), pubblicata nel descrittore come `ntor-onion-key`. Il client conosce B e l'identità del relay (qui l'impronta RSA di 20 byte, `ID`). Indichiamo con $\mathrm{EXP}(P, k)$ lo scambio Diffie-Hellman su Curve25519 tra il punto P e lo scalare k, e con $H(x, t)$ l'HMAC-SHA256 del messaggio x con chiave t. Le chiavi t sono stringhe costanti derivate dall'identificativo del protocollo `"ntor-curve25519-sha256-1"`.

9.6.2 Il protocollo

1. Il client genera una coppia effimera (x, X) e invia in **`CREATE2`**: `ID` (20 byte), B (32 byte, per indicare quale chiave onion ha usato) e X (32 byte). In tutto 84 byte.

2. Il relay controlla `ID` e B, genera una coppia effimera (y, Y) e calcola

$$\textit{secret_input} = \mathrm{EXP}(X, y) \,\|\, \mathrm{EXP}(X, b) \,\|\, \texttt{ID} \,\|\, B \,\|\, X \,\|\, Y \,\|\, \texttt{PROTOID}.$$

   Da questo ricava $\textit{KEY_SEED} = H(\textit{secret_input}, t_{\textit{key}})$ e un valore di conferma AUTH, e risponde con Y (32 byte) e AUTH (32 byte).

3. Il client calcola lo stesso *secret_input* con i propri segreti, $\mathrm{EXP}(Y, x) \,\|\, \mathrm{EXP}(B, x) \,\|\, \ldots$, e controlla che AUTH coincida.

Il cuore dell'handshake sono i due scambi Diffie-Hellman:

- $\mathrm{EXP}(X, y) = \mathrm{EXP}(Y, x)$ combina due chiavi **effimere**: garantisce la forward secrecy, perché x e y vengono cancellate subito dopo l'handshake;
- $\mathrm{EXP}(X, b) = \mathrm{EXP}(B, x)$ coinvolge la chiave onion del relay: garantisce l'**autenticazione**, perché solo chi conosce b può calcolarlo, e quindi produrre un AUTH corretto.

Nessuna delle due metà basta da sola. Senza la prima, chi rubasse in futuro b potrebbe ricalcolare tutto; senza la seconda, un intermediario potrebbe fingersi il relay. Il client invece non usa alcuna chiave a lungo termine: il relay sa di condividere il segreto con «chi ha inviato X», e nient'altro.

Approfondimento: Controllare il punto all'infinito

La specifica chiede a entrambe le parti di verificare che nessuno scambio Diffie-Hellman produca il «punto all'infinito», cioè un risultato di soli zeri. Un avversario potrebbe infatti inviare come Y un punto di ordine piccolo, costringendo il risultato di $\mathrm{EXP}(Y, x)$ a un valore prevedibile. Il controllo è semplice ed è un'alternativa sufficiente, per Curve25519, alla verifica completa che Y appartenga al gruppo giusto. Nel Laboratorio 3 corrisponde a tre righe della funzione `x25519_dh`.

9.7 ntor-v3

ntor ha un limite: non permette di trasmettere altro che le chiavi. Con il tempo è emersa la necessità di *negoziare* delle opzioni al momento della creazione di un salto, per esempio l'uso del controllo della congestione (capitolo 11) o di CGO (capitolo 10). ntor-v3 (Proposta 332) estende ntor con **messaggi cifrati** in entrambe le direzioni, e cambia alcune scelte crittografiche.

9.7.1 Che cosa cambia

- L'identità del relay è quella **Ed25519** (32 byte), non più l'impronta RSA.
- Le funzioni di hash sono della famiglia SHA-3: SHA3-256 per hash e MAC, SHAKE-256 come funzione di derivazione di chiavi a lunghezza arbitraria. Ogni input è preceduto da un'etichetta di dominio codificata con la sua lunghezza (la funzione ENCAP), così che nessun valore calcolato per uno scopo possa essere scambiato per un altro.
- Una stringa di verifica `VER`, nota a entrambe le parti, entra in tutti i calcoli: un handshake eseguito per un contesto non può essere riutilizzato in un altro. Per i circuiti ordinari tor usa la stringa `"circuit extend"` (costante `NTOR3_CIRC_VERIFICATION` in `src/core/crypto/onion_crypto.c`).
- Il client può inviare un messaggio CM e il relay un messaggio SM, entrambi cifrati.

9.7.2 Il protocollo

1. Il client genera (x, X) e calcola *subito* $B_x = \text{EXP}(B, x)$. Da B_x e dai parametri pubblici deriva due chiavi di «fase 1», una per cifrare CM con AES-256-CTR e una per autenticarlo con un MAC. Invia: `ID` (32), B (32), X (32), CM cifrato, MAC (32).
2. Il relay calcola $\text{EXP}(X, b) = B_x$, deriva le stesse chiavi di fase 1, verifica il MAC e decifra CM. Poi genera (y, Y) e combina entrambi gli scambi, $\text{EXP}(X, y)$ e $\text{EXP}(X, b)$, in un *key_seed* da cui deriva, con SHAKE-256, una chiave per cifrare la risposta SM e il flusso di chiavi del circuito. Risponde con Y (32), $AUTH$ (32) e SM cifrato.
3. Il client verifica $AUTH$, decifra SM e ricava lo stesso flusso di chiavi.

Attenzione: Il messaggio del client non ha forward secrecy

CM è cifrato con chiavi che dipendono solo da B_x, cioè dalla chiave onion del relay. Chi registrasse l'handshake e ottenesse in futuro la chiave b potrebbe decifrare CM. La specifica lo dichiara esplicitamente: solo i dati del relay godono di forward secrecy. Per questo CM deve contenere solo parametri di negoziazione, mai dati dell'utente.

9.7.3 Le estensioni

I messaggi CM e SM hanno un formato comune [tor-spec § Sending extensions in the circuit extension handshake]: un byte con il numero di estensioni, poi per ciascuna tipo (1 byte), lunghezza (1 byte) e dati. Quelle definite per la creazione dei circuiti sono:

Tipo	Verso	Significato
1	client → relay	`CC_FIELD_REQUEST`: il client chiede il controllo della congestione della Proposta 324; corpo vuoto
2	relay → client	`CC_FIELD_RESPONSE`: il relay accetta; 1 byte, `sendme_inc`
3	client → relay	richiesta di sottoprotocolli, per esempio `[02 06]` per CGO (`Relay=6`)

9.7.4 Quale handshake usa tor oggi

Nel codice di tor 0.4.9.13 la scelta avviene per ogni salto (funzione `circuit_pick_create_handshake`): si usa ntor-v3 se il relay supporta il controllo della congestione e questo è abilitato, cosa che nella rete attuale è la norma, oppure se si vuole negoziare CGO; altrimenti si ricade su ntor. In pratica, come osserva la specifica, i client moderni usano quasi sempre ntor-v3, e ntor resta come alternativa di compatibilità.

9.8 Dalle chiavi negoziate alle chiavi del circuito

Ogni handshake produce un flusso di byte pseudocasuali, da cui si ritagliano le chiavi usate per cifrare i relay cell con lo schema tradizionale (detto «tor1») [tor-spec § Setting circuit keys]:

Chiave	Byte	Uso	Direzione
Df	20	seme del digest progressivo (SHA-1)	client → relay
Db	20	seme del digest progressivo (SHA-1)	relay → client
Kf	16	chiave AES-128-CTR	client → relay
Kb	16	chiave AES-128-CTR	relay → client

In tutto 72 byte (costante `CPATH_KEY_MATERIAL_LEN` nel codice di tor). Il modo in cui il flusso viene prodotto dipende dall'handshake:

KDF-TOR (CREATE_FAST, e un tempo TAP) $K = \mathrm{SHA1}(K_0 \| \texttt{00}) \,\|\, \mathrm{SHA1}(K_0 \| \texttt{01}) \,\|\, \ldots$; i primi 20 byte formano KH, seguiti da Df, Db, Kf, Kb. Il contatore di un solo byte limita l'uscita a 5120 byte, molto più del necessario.

KDF-RFC5869 (ntor) la funzione HKDF con SHA-256, con `KEY_SEED` come chiave; Df, Db, Kf, Kb occupano i primi 72 byte.

SHAKE-256 (ntor-v3) i primi 32 byte cifrano SM; Df, Db, Kf, Kb seguono.

Due chiavi per direzione non sono un lusso: se client e relay usassero la stessa chiave AES-CTR nei due versi, i due flussi cifrati userebbero lo stesso keystream, e lo XOR di due celle rivelerebbe lo XOR dei due testi in chiaro.

Con CGO lo schema delle chiavi cambia; lo vedremo nel capitolo 10.

9.9 Laboratorio: creare circuiti veri

Laboratorio 3: Handshake di circuito

Obiettivo. Scrivere `labs/it/codice/torlab/ntor.py` con CREATE_FAST, ntor e ntor-v3; verificarlo con i vettori ufficiali; creare circuiti di un salto verso un relay della rete di laboratorio.
Prerequisiti. Laboratori 1 e 2; la rete di laboratorio avviata; la libreria `cryptography` (X25519, AES) e il modulo standard `hashlib` (SHA-3, SHAKE).
Tempo stimato. 3–4 ore.

Passo 1: ntor-v3 in poche righe. La traduzione della specifica è quasi letterale. Le funzioni ausiliarie:

```
NTOR3_PROTOID = b"ntor3-curve25519-sha3_256-1"
def _t(nome):  return NTOR3_PROTOID + b":" + nome.encode()
def _encap(s): return struct.pack(">Q", len(s)) + s
def _h3(s, t): return hashlib.sha3_256(_encap(t) + s).digest()
def _mac3(k, msg, t):
    return hashlib.sha3_256(_encap(t) + _encap(k) + msg).digest()
def _kdf3(s, t, n): return hashlib.shake_256(_encap(t) + s).digest(n)
```

e il lato client dell'handshake:

```
def ntor3_client_inizio(ID, B, messaggio, ver=b"circuit extend", x=None):
    sk, X = x25519_genera(x)
    Bx = x25519_dh(sk, B)
    fase1 = _kdf3(Bx + ID + X + B + NTOR3_PROTOID + _encap(ver),
                  _t("kdf_phase1"), 64)
    enc_k1, mac_k1 = fase1[:32], fase1[32:]
    cifrato = aes_ctr(enc_k1, messaggio)
    mac = _mac3(mac_k1, ID + B + X + cifrato, _t("msg_mac"))
    stato = StatoNtor3(sk, X, B, ID, Bx, mac, ver)
    return stato, ID + B + X + cifrato + mac

def ntor3_client_fine(st, risposta, lunghezza=72):
    Y, auth, msg = risposta[:32], risposta[32:64], risposta[64:]
    secret_input = (x25519_dh(st.x, Y) + st.Bx + st.ID + st.B + st.X + Y
                    + NTOR3_PROTOID + _encap(st.ver))
    key_seed = _h3(secret_input, _t("key_seed"))
    verify = _h3(secret_input, _t("verify"))
    auth_input = (verify + st.ID + st.B + Y + st.X + st.mac
                  + _encap(msg) + NTOR3_PROTOID + b"Server")
    if not hmac.compare_digest(_h3(auth_input, _t("auth_final")), auth):
        raise ErroreHandshake("AUTH ntor-v3 non valido")
    flusso = _kdf3(key_seed, _t("kdf_final"), 32 + lunghezza)
    return aes_ctr(flusso[:32], msg), flusso[32:]
```

Il confronto di *AUTH* usa `hmac.compare_digest`, che impiega lo stesso tempo qualunque sia il punto in cui i due valori differiscono: un confronto ordinario potrebbe rivelare, con misure di tempo, quanti byte iniziali sono corretti.

Passo 2: i vettori di test. Un handshake crittografico non si verifica «facendolo funzionare»: due implementazioni sbagliate nello stesso modo si capirebbero benissimo. Si verifica con i **vettori di test** della specifica: valori di ingresso fissati (comprese le chiavi «casuali») e le uscite attese, byte per byte. La Proposta 332 ne fornisce uno completo per ntor-v3, la Proposta 216 alcuni per la funzione HKDF di ntor. Il file `test/test_ntor.py` li riproduce; per esempio:

```
def test_ntor3_vettori_proposta_332():
    st, hdata = ntor3_client_inizio(h(V["ID"]), h(V["B"]), h(V["CM"]),
                                    ver=h(V["VER"]), x=h(V["x"]))
    assert hdata.hex() == V["client"]
    sm, flusso = ntor3_client_fine(st, h(V["server"]), lunghezza=256)
    assert sm == h(V["SM"])
    assert flusso[:32].hex() == V["keystream_inizio"]
```

Altri test verificano che un *AUTH* alterato, o una stringa `VER` diversa, facciano fallire l'handshake.

Passo 3: le chiavi del relay. Per l'handshake servono l'identità Ed25519 e la chiave `ntor-onion-key` del relay. Un client vero le trova nel consenso e nei microdescrittori (capitolo 12); anche noi le prendiamo da lì, dal file `cached-microdescs.new` del client della rete di laboratorio, dove ogni relay ha una voce come:

```
onion-key
-----BEGIN RSA PUBLIC KEY-----
MIGJAoGBAJet1Oxp+cvg/vgryEmJuVv2s++WxQt5Id1BQ9UNwCOVFcYvEN9KVklM
...
-----END RSA PUBLIC KEY-----
```

```
ntor-onion-key gsi2OisnQE96UkjOVvBKtWDLfwRdYG9d4DScg2wz4Qs
id ed25519 Kz1dj/u6/g7WvhHrt9Ds0jVZjKC+KHo26hWViXAJ1Pg
```

Si notano la vecchia chiave TAP, ancora pubblicata ma inutilizzata, e le due righe che ci servono, in base64.

Passo 4: contro un relay vero. Apriamo un canale verso `test003r` con il Laboratorio 1 e inviamo una **`CREATE2`** per ogni tipo di handshake:

```
c = apri_canale("127.0.0.1", 5103, identita_attesa=ed)
st, hd = ntor3_client_inizio(ed, B, codifica_estensioni([]))
c.invia(Cella(0x80000001, Comando.CREATE2,
              corpo_create2(HTYPE_NTOR_V3, hd)))
r = c.ricevi()
sm, flusso = ntor3_client_fine(st, hdata_created2(r.corpo))
chiavi = ChiaviCircuito.da_flusso(flusso)
```

Ripetendo la prova con le varianti, si ottiene:

```
<- CircID=0x80000001 CREATED2 corpo=509 byte
   ntor-v3 ok, messaggio del relay: []
<- CircID=0x80000002 CREATED2 corpo=509 byte
   ntor-v3 con CC_FIELD_REQUEST ok, risposta: [(2, b'\x1f')]
<- CircID=0x80000003 CREATED2 corpo=509 byte
   ntor ok
<- CircID=0x80000004 CREATED_FAST corpo=509 byte
   CREATE_FAST ok
<- CircID=0x80000005 DESTROY corpo=509 byte     (MAC alterato)
   motivo: 1
```

I primi quattro handshake riescono. Nel secondo caso il relay accetta il controllo della congestione e risponde con l'estensione 2, `sendme_inc` uguale a 31 (capitolo 11). Nel quinto caso abbiamo alterato di proposito un bit del MAC: l'handshake **fallisce**, e il relay risponde con una **`DESTROY`** di motivo 1, `PROTOCOL`. È il comportamento osservato con tor 0.4.9.13 nel laboratorio, non una regola che prescriva di divulgare i motivi: la specifica [tor-spec § Tearing down circuits] raccomanda `NONE` per evitare canali laterali (*SHOULD*, non *MUST*), e raccomanda di non propagare il motivo di un'altra **`DESTROY`**. Qui il motivo proviene invece dall'errore rilevato direttamente dal relay.

Un'osservazione inattesa. Nel capitolo 7 abbiamo visto che un canale nuovo non riceve celle **`PADDING`**. Ora il canale ha quattro circuiti aperti, eppure in 20 secondi di attesa non arriva nessuna cella. La ragione è nel codice di tor (`src/core/or/relay.c`): un relay considera il canale «in uso», e quindi degno di padding, solo quando vi transita un circuito che prosegue oltre di lui. I nostri circuiti di un salto non contano. Nel capitolo 10, quando estenderemo un circuito, il padding comparirà.

Riepilogo

- Un circuito si costruisce un salto alla volta: **`CREATE2`**/**`CREATED2`** con il primo relay, poi `EXTEND2`/`EXTENDED2` attraverso il circuito per i successivi. Ogni handshake coinvolge il client e un solo relay.
- `EXTEND2` contiene gli specificatori di collegamento (indirizzo, identità RSA ed Ed25519); il relay che estende deve verificare le identità.

- Un handshake di circuito autentica il relay ma non il client, e deve garantire segretezza e forward secrecy.
- TAP (RSA e DH a 1024 bit) è stato rimosso in tor 0.4.9.1-alpha; **`CREATE_FAST`** si appoggia al TLS e va usato solo per circuiti di un salto.
- ntor combina due scambi Curve25519: effimero–effimero per la forward secrecy, effimero–chiave onion per l'autenticazione.
- ntor-v3 usa l'identità Ed25519, la famiglia SHA-3 e aggiunge messaggi cifrati per negoziare opzioni come il controllo della congestione e CGO; i dati del client non hanno forward secrecy.
- Il flusso di chiavi fornisce Df, Db (20 byte ciascuno) e Kf, Kb (16 byte ciascuno): una chiave e un digest per direzione.

Esercizi

Esercizio 9.1 ⋆ Calcola la lunghezza di HDATA in **`CREATE2`** per ntor e per ntor-v3 con un messaggio di 3 byte. Quanti byte può contenere al massimo il messaggio CM di ntor-v3 in una **`CREATE2`**?

Esercizio 9.2 ⋆ Perché **`CREATE_FAST`** non va usato per il secondo salto di un circuito? Che cosa potrebbe fare il primo relay?

Esercizio 9.3 ⋆ Perché client e relay usano chiavi AES diverse nei due versi (Kf e Kb)? Descrivi che cosa succederebbe con una chiave sola.

Esercizio 9.4 ⋆⋆ Scrivi in esadecimale l'intestazione del corpo di un messaggio `EXTEND2` (numero di specificatori e specificatore IPv4) per estendere un circuito verso il relay `test004r` della rete di laboratorio. Quanti byte occupa l'intero messaggio con ntor-v3 e un messaggio CM di un solo byte?

Esercizio 9.5 ⋆⋆ Nell'handshake ntor, supponi che il client usi solo $\mathrm{EXP}(Y, x)$ e ometta $\mathrm{EXP}(B, x)$. Descrivi un attacco concreto di un relay intermedio che inoltra l'handshake verso il salto successivo. Supponi invece che ometta $\mathrm{EXP}(Y, x)$: quale proprietà si perde?

Esercizio 9.6 ⋆⋆ A che cosa serve la stringa `VER` di ntor-v3? Immagina un protocollo diverso da Tor che usi lo stesso handshake con le stesse chiavi: che cosa potrebbe succedere se `VER` non esistesse?

Esercizio 9.7 ⋆⋆ Ripeti il passo 4 del laboratorio con la richiesta di sottoprotocollo `[02 06]` (CGO) oltre a `CC_FIELD_REQUEST`. Il relay accetta? Ricava dalla specifica della Proposta 359 perché CGO richiede anche il controllo della congestione.

Esercizio 9.8 ⋆⋆⋆ Il repository di tor contiene un'implementazione di riferimento di ntor in Python (`src/test/ntor_ref.py`). Usala per generare dei vettori di test completi dell'handshake ntor (lato client e lato relay) e aggiungili a `test_ntor.py`.

Esercizio 9.9 ⋆⋆⋆ Implementa il *lato relay* di ntor-v3 (funzioni `server_part1` e `server_part2`) e verificalo con i vettori della Proposta 332. Poi fai eseguire un handshake completo tra il tuo client e il tuo relay, in memoria.

Per approfondire

- *Tor Protocol Specification*, capitoli «Creating and extending circuits» e «Setting circuit keys» [Thef].
- I. Goldberg, D. Stebila, B. Ustaoglu, *Anonymity and one-way authentication in key exchange protocols* [GSU13]: l'articolo che definisce ntor e ne dimostra la sicurezza.
- Proposta 216 (ntor), proposta 332 (ntor-v3), proposta 350 (rimozione di TAP).

Capitolo 10

La cipolla: cifratura dei relay cell e CGO

Aggiornato al 23 settembre 2026 — torspec commit `8e1f245`; tor 0.4.9.13; proposta 359

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- descrivere come il client avvolge un messaggio in uno strato di cifratura per ogni salto e come i relay li tolgono;
- spiegare il ruolo di Recognized e del digest progressivo e perché servono entrambi;
- riconoscere le debolezze dello schema tradizionale: malleabilità e attacchi di *tagging*;
- descrivere a grandi linee Counter Galois Onion (CGO), lo schema che lo sostituisce;
- estendere un circuito reale fino a tre salti con il tuo codice (Laboratorio 4).

Nel capitolo 9 il client ha concordato con ogni relay del circuito quattro chiavi: Kf e Kb per cifrare, Df e Db per il controllo di integrità. In questo capitolo le mettiamo al lavoro. È il meccanismo che dà il nome all'onion routing: il messaggio viene avvolto in tanti strati quanti sono i relay, e ogni relay ne toglie uno, come si sbuccia una cipolla.

10.1 Lo schema tradizionale

Lo schema in uso dalla nascita di Tor, che nel codice più recente prende il nome di *tor1*, combina due strumenti per ciascun salto e ciascuna direzione:

- un cifrario a flusso, **AES-128 in modalità contatore** (CTR), con chiave Kf o Kb;
- un **digest progressivo** SHA-1, inizializzato con Df o Db.

Entrambi hanno uno *stato* che dura quanto il circuito. Il contatore di AES-CTR non riparte da zero a ogni cella: la prima cella usa i primi 509 byte del flusso di chiavi, la seconda i 509 successivi, e così via. Allo stesso modo il digest non è calcolato sulla singola cella, ma su *tutte* le celle scambiate finora con quel salto. Di conseguenza client e relay devono elaborare le celle esattamente nello stesso ordine: una cella persa o duplicata desincronizzerebbe entrambi gli stati. Il canale TLS, che garantisce ordine e integrità, lo rende possibile.

10.1.1 In avanti: dal client verso il circuito

Per inviare un messaggio al salto d di un circuito, il client [tor-spec § Routing relay cells]:

1. prepara il corpo in chiaro del messaggio relay (sezione 8.6), con Recognized e Digest a zero;

2. aggiorna il digest in avanti del salto d con quel corpo e scrive i primi 4 byte del risultato nel campo Digest;

3. cifra il corpo con Kf del salto d, poi con Kf del salto $d-1$, e così via fino al salto 1: lo strato più esterno è quello del primo relay.

Ogni relay che riceve la cella la decifra con la propria Kf, togliendo uno strato, e controlla se la cella è destinata a lui. Se no, la inoltra al salto successivo, che ne toglierà un altro. La fig. 10.1 mostra gli strati.

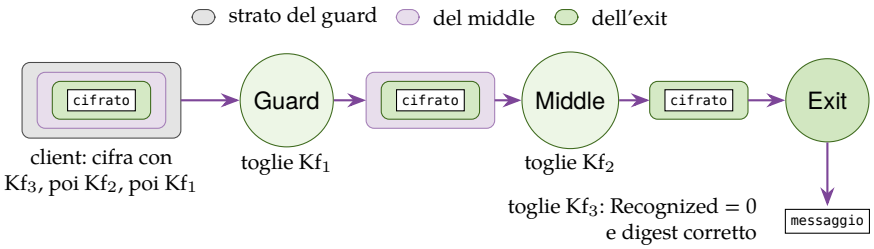


Figura 10.1: Una cella RELAY nella cifratura tradizionale tor1: il client applica Kf_3, Kf_2, Kf_1 e ogni relay rimuove il proprio strato. L'exit riconosce il messaggio con Recognized $= 0$ e digest progressivo corretto; il contenuto applicativo può restare cifrato con HTTPS. Le scatole interne rappresentano schematicamente dati ancora cifrati.

10.1.2 All'indietro: dal circuito verso il client

Nel verso opposto i ruoli si invertono. Il relay che origina un messaggio lo firma con il proprio digest all'indietro e lo cifra con Kb. Ogni relay successivo verso il client *aggiunge* uno strato con la propria Kb. Il client, che conosce tutte le chiavi, toglie gli strati uno per uno, dal primo salto in poi, e dopo ciascuno controlla se la cella è stata originata da quel salto.

10.1.3 Il circuito come «tubo che perde»

Il client può inviare un messaggio a *qualsiasi* salto del circuito, non solo all'ultimo, semplicemente fermando la cifratura a quel salto. È così che l'`EXTEND2` per il terzo salto viene consegnato al secondo relay. La specifica chiama questa proprietà *leaky pipe topology*: il circuito è un tubo da cui i messaggi possono uscire a metà strada. Nell'articolo originale di Tor [DMS04] la si propone anche come difesa: far uscire alcuni stream da un relay intermedio renderebbe più difficile a un osservatore dell'uscita capire di quale circuito si tratta. Nella pratica, però, tor fa uscire gli stream solo dall'ultimo salto.

10.2 Riconoscere una cella

Come fa un relay a sapere, dopo aver tolto il proprio strato, se la cella è per lui? Non può semplicemente «guardare se il contenuto ha senso»: una cella ancora cifrata e una cella in chiaro sono, per un programma, entrambe sequenze di byte. Lo schema tradizionale usa due controlli in cascata [tor-spec § Relay cells]:

1. **Recognized**: i 2 byte devono valere zero. Se la cella è ancora cifrata, questi byte sono pseudocasuali e valgono zero solo una volta su $65\,536$. Il controllo costa quasi nulla e scarta subito la stragrande maggioranza delle celle in transito.

2. **Digest**: se Recognized è zero, il relay calcola il digest progressivo *come se* la cella fosse sua, azzerando il campo Digest, e confronta i primi 4 byte con quelli presenti. Se coincidono, la cella è riconosciuta: il relay aggiorna il proprio stato del digest e la elabora.

Un dettaglio facile da sbagliare: se Recognized è zero ma il digest non coincide, si tratta di un falso positivo, e lo stato del digest **non** deve essere aggiornato. Nel Laboratorio 4 lo si ottiene lavorando su una copia dello stato (`self.digest.copy()`) e conservandola solo in caso di successo.

Il digest copre l'intero corpo, compresi i byte di padding, che per questo sono casuali (sezione 8.6). Per i circuiti degli onion service, dove client e servizio condividono chiavi di un «salto virtuale» (capitolo 14), la funzione è SHA3-256 invece di SHA-1.

Che cosa succede se nessuno riconosce la cella? Se l'ultimo relay del circuito riceve in avanti una cella che non riconosce, c'è stato un errore: distrugge il circuito con **`DESTROY`**. Il client, se nessun salto riconosce una cella in arrivo, fa altrettanto.

10.3 Le debolezze di tor1

10.3.1 Malleabilità

La modalità contatore ha una proprietà scomoda: il testo cifrato è semplicemente $C = P \oplus S$, dove S è il flusso di chiavi. Chi modifica un bit di C modifica lo stesso bit di P, senza conoscere la chiave. Se un avversario conosce, o indovina, una parte del testo in chiaro, può sostituirla con un testo a sua scelta: basta applicare lo XOR con la differenza tra i due. Il Laboratorio 4 contiene un test che fa esattamente questo, trasformando «paga 10 euro» in «paga 99 euro».

In tor1 l'integrità è verificata solo *all'estremo*: il digest viene controllato dal relay destinatario, non dai relay intermedi. Una cella alterata viaggia quindi indisturbata fino alla destinazione, e solo lì il digest non corrisponde.

10.3.2 L'attacco di tagging

Questa combinazione permette un attacco specifico, il *tagging*. Immaginiamo un avversario che controlla il guard e l'exit di un circuito:

1. il guard altera alcuni bit di una cella in transito, secondo uno schema riconoscibile (il «marchio», o *tag*);

2. la cella attraversa il middle, che non può accorgersene;

3. l'exit, prima di verificare il digest, controlla se nella cella c'è il marchio. Se c'è, sa che quel circuito passa dal guard complice, e quindi può associare la destinazione del traffico all'indirizzo del client, noto al guard.

Se invece l'exit non è complice, il digest fallisce, il circuito viene distrutto e il client ne costruisce un altro: l'avversario ci riprova, e ogni tentativo fallito gli dice qualcosa. Il marchio può anche essere applicato all'indietro, dall'exit verso il client.

Approfondimento: Quanto conta il tagging?

Un avversario che controlla sia il guard sia l'exit può già correlare i tempi e i volumi del traffico, senza alcun marchio (capitolo 26). Per questo i progettisti di Tor hanno a lungo considerato il tagging un problema reale ma non prioritario [DMS04]. Il tagging però è più affidabile della correlazione statistica e funziona anche con pochissimo traffico. Le proposte per risolverlo si sono succedute per oltre dieci anni: le proposte 202, 261 (basata sul cifrario AEZ), 295 e 308, fino alla 359, che ha portato a CGO.

10.3.3 Altri limiti

Il digest è di soli 4 byte, e l'algoritmo alla sua base, SHA-1, non è più considerato sicuro per nuove applicazioni. Inoltre lo schema non offre forward secrecy *all'interno* del circuito: chi ottenesse le chiavi di un salto mentre il circuito è attivo potrebbe decifrare anche tutte le celle precedenti che avesse registrato, perché il flusso di chiavi è determinato dalla sola chiave iniziale.

10.4 Counter Galois Onion

CGO (*Counter Galois Onion*) è lo schema di cifratura dei relay cell progettato da Degabriele, Melloni, Münch e Stam e adottato con la Proposta 359. Qui ne diamo una descrizione funzionale; la costruzione crittografica completa è nella proposta e negli articoli degli autori.

10.4.1 Le proprietà

Secondo la proposta, CGO offre tre miglioramenti:

- è **non malleabile**: qualsiasi modifica a una cella la rende indecifrabile, e con essa tutte le celle successive del circuito. Un marchio non può essere letto né rimosso: può solo distruggere il circuito;
- offre **forward secrecy a livello di cella**: dopo aver elaborato una cella, una parte non possiede più il materiale necessario a decifrarla di nuovo;
- l'autenticazione è di 128 bit, invece dei pochi bit dello schema tradizionale.

Non usando più SHA-1, CGO dovrebbe anche essere, nel complesso, più veloce.

10.4.2 Come funziona, a grandi linee

Gli ingredienti di base sono AES-128 e POLYVAL, una funzione di hash universale (la stessa famiglia usata nella modalità AES-GCM-SIV). Con essi la proposta costruisce un cifrario a blocchi *ampio* e *modificabile*: un cifrario che agisce su tutti i 509 byte della cella come se fossero un unico blocco, così che cambiare un solo bit in un punto qualsiasi cambi in modo imprevedibile tutto il risultato. La cella decifrata ha la forma *tag* (16 byte) più corpo (493 byte), come abbiamo visto nella sezione 8.7.

Per ogni salto e ogni direzione, client e relay condividono:

- una chiave K di 64 byte e un valore N di 16 byte, derivati dall'handshake del circuito;
- il *tag* T' dell'ultima cella elaborata, inizialmente zero.

Tre idee rendono lo schema resistente al tagging:

Riconoscimento con il tag Quando un relay ha tolto il proprio strato, la cella è sua se il tag risultante coincide con N. È un controllo di 16 byte, invece dei 2 di Recognized più i 4 del digest.

Concatenamento Il tag dell'ultima cella entra, come parametro del cifrario, nell'elaborazione della cella successiva. Una cella alterata produce un tag sbagliato, che a sua volta rende indecifrabili tutte le celle seguenti: il circuito non può più funzionare.

Aggiornamento delle chiavi Ogni volta che una cella viene riconosciuta, o originata, da un salto, K e N vengono sostituiti da nuovi valori derivati dai precedenti. I vecchi valori si cancellano: è la forward secrecy a livello di cella.

Il byte di comando della cella (`RELAY` o `RELAY_EARLY`) entra nei calcoli come *dato associato*: un relay non può trasformare l'una nell'altra senza che la cella diventi indecifrabile.

Attenzione: Direzioni non intercambiabili

La costruzione alla base di CGO è non malleabile in un verso e malleabile nell'altro. La proposta sceglie con cura quale verso usare nel client e quale nei relay, e avverte esplicitamente gli implementatori di non invertirli: un esempio di come, in crittografia, una scelta che sembra un dettaglio di implementazione possa annullare la sicurezza di tutto lo schema.

10.4.3 CGO in pratica

- **Chiavi**: dal flusso prodotto dall'handshake si ricavano, per ciascuna direzione, K (64 byte) e N (16 byte), invece di Df, Db, Kf, Kb.
- **SENDME**: con tor1 il controllo di flusso autenticato della Proposta 289 usa il digest progressivo; con CGO questo digest non esiste e il `SENDME` riporta invece il tag T della cella che conferma (capitolo 11). La Proposta 359 precisa un limite importante: il tag prova la ricezione della specifica cella, *non* prova da solo che siano state ricevute tutte quelle intermedie o che il contenuto sia stato decifrato. Per confrontare lo stesso valore, chi riceve la cella conserva T *prima* dell'elaborazione UIV, mentre chi la origina lo conserva *dopo*.
- **Negoziazione**: un client usa CGO con un relay solo se questo annuncia il sottoprotocollo `Relay=6` e se lo chiede nell'handshake ntor-v3 (sezione 9.7). Nel capitolo 9 abbiamo visto che il relay rifiuta la richiesta se non è accompagnata da quella del controllo della congestione.
- **Diffusione**: tor supporta CGO dalla serie 0.4.9 e Arti lo ha sempre attivo. In un circuito possono convivere salti che usano CGO e salti che usano ancora tor1: la scelta è fatta salto per salto.

10.5 Laboratorio: un circuito di tre salti

Laboratorio 4: Cifrare a strati ed estendere un circuito

Obiettivo. Scrivere `labs/it/codice/torlab/cifratura.py`, con lo schema tor1 lato client, e usarlo per estendere un circuito reale fino all'exit.
Prerequisiti. Laboratori 1–3; la rete di laboratorio avviata.
Tempo stimato. 3 ore.

Passo 1: una direzione di un salto. Il cuore del laboratorio è una piccola classe che tiene insieme il cifrario AES-CTR e il digest SHA-1 di una direzione:

```
class Direzione:
    @classmethod
    def nuova(cls, chiave, seme_digest):
        d = hashlib.sha1(); d.update(seme_digest)
        return cls(_aes_ctr(chiave), d)

    def applica(self, dati):            # cifra o decifra: in CTR coincidono
        return self.cifrario.update(dati)

    def firma(self, corpo):             # scrive il campo Digest
        self.digest.update(_senza_digest(corpo))
        return corpo[:5] + self.digest.digest()[:4] + corpo[9:]

    def riconosce(self, corpo):
        if corpo[1:3] != b"\x00\x00":
            return False
        prova = self.digest.copy()
        prova.update(_senza_digest(corpo))
        if prova.digest()[:4] != corpo[5:9]:
            return False                # falso positivo: stato invariato
        self.digest = prova
        return True
```

L'oggetto `cifrario` di `cryptography` conserva il contatore tra una chiamata e l'altra: è esattamente il comportamento richiesto.

Passo 2: il client. Con una `Direzione` per verso e per salto, cifrare e decifrare sono due cicli:

```
def cifra(self, msg, destinazione):
    corpo = self.salti[destinazione].avanti.firma(codifica_relay(msg))
    for i in range(destinazione, -1, -1):
        corpo = self.salti[i].avanti.applica(corpo)
    return corpo

def decifra(self, corpo):
    for i, salto in enumerate(self.salti):
        corpo = salto.indietro.applica(corpo)
        if salto.indietro.riconosce(corpo):
            return i, decodifica_relay(corpo)
    raise ErroreCella("cella non riconosciuta")
```

Passo 3: test in memoria, e i loro limiti. Il file `test/test_cifratura.py` costruisce un circuito simulato di tre relay e verifica che ogni salto riconosca solo le proprie celle, che i relay intermedi vedano solo byte pseudocasuali, che una modifica venga rilevata a destinazione, e dimostra la malleabilità di AES-CTR. Questi test però hanno un limite importante: client e relay simulati usano *lo stesso codice*. Se sbagliassimo, per esempio, l'ordine in cui aggiornare il digest, lo sbaglieremmo in modo identico da entrambe le parti, e i test passerebbero comunque. L'unica verifica vera è parlare con un'implementazione indipendente.

Passo 4: estendere un circuito reale. Creiamo il primo salto con ntor-v3 verso `test003r`, come nel Laboratorio 3, poi estendiamo verso `test004r` e verso l'exit `test006e`. Il modulo

`torlab/laboratorio.py` legge per noi identità e chiavi ntor dei relay dai file della rete di laboratorio:

```
for r, est in ((middle, []), (exit_, [(1, b"")])):
    st, hd = ntor3_client_inizio(r.id_ed, r.ntor, codifica_estensioni(est))
    corpo = corpo_extend2("127.0.0.1", r.porta, r.id_rsa, r.id_ed,
                          HTYPE_NTOR_V3, hd)
    msg = MessaggioRelay(ComandoRelay.EXTEND2, 0, corpo)
    c.invia(Cella(CIRC, Comando.RELAY_EARLY, circ.cifra(msg)))
    i, risposta = circ.decifra(c.ricevi().corpo)
    sm, k = ntor3_client_fine(st, hdata_created2(risposta.dati))
    circ.salti.append(Salto.da_chiavi(ChiaviCircuito.da_flusso(k)))
```

Il messaggio `EXTEND2` ha StreamID zero (è un messaggio di controllo) e viaggia in una cella **`RELAY_EARLY`**. Per l'exit chiediamo anche il controllo della congestione, come fa tor. L'esecuzione produce:

```
salto 1: 003r
  <- CircID=0x80000100 RELAY corpo=509 byte
  dal salto 1 EXTENDED2 67 byte
salto 2: 004r  estensioni dal relay: []
  <- CircID=0x80000100 RELAY corpo=509 byte
  dal salto 2 EXTENDED2 70 byte
salto 3: 006e  estensioni dal relay: [(2, b'\x1f')]
```

Il circuito ha tre salti. Ogni riga «dal salto» è la prova che il nostro codice ha cifrato correttamente per il relay destinatario e che il relay ha cifrato correttamente per noi: se una sola chiave, un solo byte del digest o l'ordine degli strati fossero sbagliati, il relay avrebbe distrutto il circuito. Le lunghezze dei messaggi `EXTENDED2` tornano con la sezione 9.7: 2 byte di HLEN, 32 di Y, 32 di $AUTH$ e il messaggio SM, di 1 byte (nessuna estensione) per il middle e di 4 byte (un'estensione di 1 byte) per l'exit.

Passo 5: finalmente il padding. Restiamo in ascolto sul canale dopo l'estensione:

```
  4.1 s  <- CircID=0x00000000 PADDING corpo=509 byte
  7.1 s  <- CircID=0x00000000 PADDING corpo=509 byte
 12.8 s  <- CircID=0x00000000 PADDING corpo=509 byte
 21.5 s  <- CircID=0x00000000 PADDING corpo=509 byte
 28.3 s  <- CircID=0x00000000 PADDING corpo=509 byte
```

È la previsione fatta nei capitoli 7 e 9: ora che sul canale passa un circuito di più salti, il guard lo considera «in uso» e, quando il canale è inattivo, invia celle **`PADDING`** a intervalli casuali. Gli intervalli osservati, tra 3 e 8,7 secondi, cadono nella finestra di 1,5–9,5 secondi prevista dalla specifica del padding.

Riepilogo

- In tor1, ogni salto ha per direzione un flusso AES-128-CTR e un digest SHA-1 progressivo, entrambi con stato che dura quanto il circuito.
- In avanti il client applica gli strati dall'ultimo salto al primo, e ogni relay ne toglie uno; all'indietro ogni relay aggiunge uno strato e il client li toglie tutti.
- Il client può indirizzare qualsiasi salto (*leaky pipe*).

- Una cella è riconosciuta se Recognized vale zero e il digest progressivo corrisponde; un falso positivo di Recognized non deve alterare lo stato del digest.
- AES-CTR è malleabile e l'integrità è controllata solo all'estremo: questo rende possibile il tagging.
- CGO rende la cella non malleabile, concatena i tag delle celle, aggiorna le chiavi dopo ogni messaggio e autentica con 128 bit; si negozia salto per salto con ntor-v3.

Esercizi

Esercizio 10.1 ⋆ Un client con un circuito di tre salti vuole inviare un messaggio al secondo salto. Con quali chiavi cifra, e in quale ordine? Che cosa vede il terzo salto?

Esercizio 10.2 ⋆ Dopo che il client ha inviato 1000 celle al circuito, a quale posizione del flusso di chiavi AES-CTR del primo salto si trova? Perché non si può riusare la stessa chiave per un secondo circuito?

Esercizio 10.3 ⋆⋆ Spiega che cosa succederebbe se un relay aggiornasse il digest progressivo anche quando Recognized vale zero ma il digest non corrisponde. Dopo quante celle, in media, il problema si manifesterebbe?

Esercizio 10.4 ⋆⋆ Un guard complice vuole marcare una cella in avanti in modo che l'exit complice possa riconoscerla. Quali byte della cella conviene alterare, sapendo come è fatto un messaggio `DATA` in chiaro? Che cosa vede un exit onesto?

Esercizio 10.5 ⋆⋆ Ripeti lo scenario dell'esercizio precedente con CGO. Che cosa vede l'exit complice? E che cosa succede alle celle successive del circuito?

Esercizio 10.6 ⋆⋆ Nel passo 5 del laboratorio, calcola gli intervalli tra le celle **`PADDING`** osservate. Secondo la specifica, il ritardo è estratto dalla distribuzione del massimo di due valori uniformi. Qual è il valore atteso di questo ritardo, nell'intervallo di 1,5–9,5 secondi? È compatibile con le osservazioni?

Esercizio 10.7 ⋆ ⋆ ⋆ Estendi il laboratorio inviando al secondo salto un messaggio `DROP` (comando relay 10, padding a lungo raggio). Il relay risponde? Come puoi verificare che lo abbia riconosciuto e non abbia distrutto il circuito?

Esercizio 10.8 ⋆ ⋆ ⋆ Nel laboratorio, altera di proposito un bit di una cella **`RELAY_EARLY`** con `EXTEND2` dopo averla cifrata. Quale relay se ne accorge, e come reagisce? Prevedi il risultato, poi verificalo.

Per approfondire

- *Tor Protocol Specification*, capitoli «Relay cells» e «Routing relay cells» [Thef].
- Proposta 359, *Counter Galois Onion, Updated*, con i riferimenti agli articoli di Degabriele, Melloni, Münch e Stam.
- Proposte 202, 261, 295 e 308: la lunga storia delle soluzioni al tagging.
- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04], sezione 4 (circuiti, leaky pipe e controllo di integrità).

Capitolo 11

Stream, SENDME e controllo della congestione

Aggiornato al 23 settembre 2026 — torspec commit `8e1f245`; tor 0.4.9.13; proposte 289, 324

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- aprire, usare e chiudere uno stream TCP attraverso un circuito, con BEGIN, CONNECTED, DATA ed END;
- spiegare perché la risoluzione dei nomi avviene all'exit e come funziona RESOLVE;
- descrivere il controllo di flusso originale di Tor, con le finestre di circuito e di stream e i SENDME;
- capire perché i SENDME sono autenticati e che cosa dimostrano;
- descrivere il controllo della congestione della proposta 324 e il ruolo di XON e XOFF;
- scaricare un file attraverso un circuito di tre salti costruito da te e misurare le finestre di un exit reale (Laboratorio 5).

Il circuito del capitolo 10 è un condotto cifrato fino all'exit, ma non trasporta ancora nulla di utile. In questo capitolo lo usiamo per il suo scopo: portare connessioni TCP verso Internet, gli **stream**. Vedremo poi un problema che ogni rete deve risolvere, e che in Tor ha una forma particolare: evitare che chi invia dati più velocemente di quanto la rete o il destinatario riescano ad assorbirli riempia le code dei relay.

11.1 Gli stream

Uno stream è una connessione TCP del client, per esempio verso un server web, trasportata attraverso un circuito. Un circuito può trasportare molti stream contemporaneamente, distinti dallo **StreamID** dei messaggi relay (sezione 8.6), scelto dal client e diverso da zero. Il ciclo di vita di uno stream è mostrato nella fig. 11.1.

11.1.1 Aprire uno stream: BEGIN e CONNECTED

Per aprire uno stream il client invia all'exit un messaggio `BEGIN` (comando relay 1) con un nuovo StreamID e un corpo che indica la destinazione [tor-spec § Opening streams and transmitting data]:

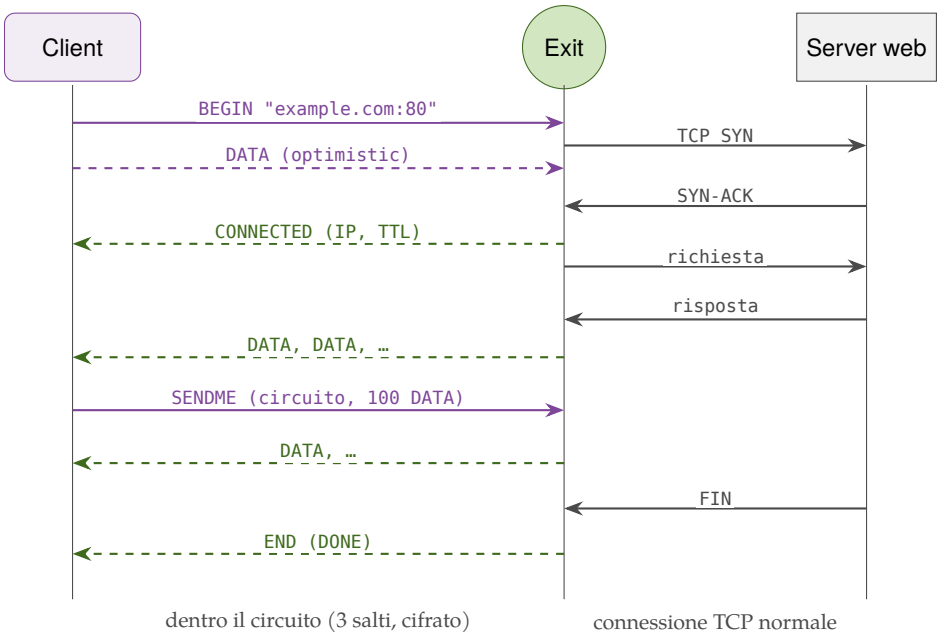


Figura 11.1: Ciclo di vita di uno stream verso un server web. I messaggi tra client ed exit viaggiano nel circuito, cifrati a strati; tra exit e server c'è una normale connessione TCP, in chiaro se l'applicazione non usa TLS. Il client può inviare dati prima di ricevere CONNECTED (*optimistic data*).

Campo	Lunghezza	Contenuto
ADDRPORT	variabile	`indirizzo:porta`, terminato da un byte zero
FLAGS	4 (facoltativo)	preferenze su IPv4 e IPv6

L'indirizzo può essere un nome DNS, un indirizzo IPv4 o un indirizzo IPv6 tra parentesi quadre. Due regole minori sono rivelatrici dello spirito del protocollo: la stringa va inviata **in minuscolo**, per non creare differenze che permettano di distinguere un client dall'altro, e il campo FLAGS va **omesso** quando vale zero. I bit di FLAGS dicono se il client accetta IPv6 (bit 1), se rifiuta IPv4 (bit 2) e se preferisce IPv6 (bit 3).

Ricevuto `BEGIN`, l'exit:

1. controlla di non essere il primo salto del circuito, cioè che il canale da cui arriva il circuito sia stato aperto da un relay autenticato presente nel consenso. Tor non deve funzionare come proxy a un solo salto, che renderebbe gli exit bersagli più appetibili: un tentativo del genere va respinto con un **`DESTROY`**;

2. applica la propria *exit policy*, l'elenco di indirizzi e porte verso cui accetta di collegarsi (capitolo 22);

3. risolve il nome, se necessario, e apre la connessione TCP.

Se tutto va bene risponde con `CONNECTED` (comando relay 4), il cui corpo contiene di solito l'indirizzo IPv4 raggiunto (4 byte) e per quanti secondi il client può ricordarlo (TTL, 4 byte); per IPv6 il formato è un po' diverso, e un corpo vuoto è ammesso. Il client *può* ignorare l'indirizzo, e se lo memorizza non deve riusarlo su altri circuiti: un exit disonesto potrebbe mentire, o restituire un indirizzo unico per riconoscere il client in seguito.

11.1.2 Trasferire dati: DATA

I dati dell'applicazione viaggiano in messaggi `DATA` (comando relay 2), fino a 498 byte ciascuno nel formato tradizionale. L'exit copia il contenuto sulla connessione TCP e viceversa. Il client può inviare `DATA` subito dopo `BEGIN`, senza attendere `CONNECTED`: l'exit li mette in coda e li inoltra appena la connessione è aperta. Questi *optimistic data* risparmiano un intero giro di andata e ritorno attraverso il circuito, che in Tor costa facilmente qualche centinaio di millisecondi.

11.1.3 Chiudere: END

Chi chiude la connessione, o incontra un errore, invia `END` (comando relay 3) con un byte di motivo [tor-spec § Closing streams]. I motivi più comuni sono `DONE` (6, chiusura normale), `CONNECTREFUSED` (3), `RESOLVEFAILED` (2), `TIMEOUT` (7) ed `EXITPOLICY` (4), quest'ultimo seguito dall'indirizzo rifiutato e da un TTL. Il client, per gli stream che ha originato, dovrebbe inviare solo `MISC` (1): come per **`DESTROY`**, non c'è motivo di dare all'altra parte più informazioni del necessario.

11.1.4 Il DNS all'exit

Nel `BEGIN` il client può indicare un *nome* anziché un indirizzo: la risoluzione avviene all'exit. Non è una comodità, è una scelta di sicurezza. Se il client risolvesse i nomi da sé, chiederebbe al proprio resolver DNS, fuori da Tor, l'indirizzo di ogni sito che sta per visitare: chiunque osservi quelle richieste, dal fornitore di connettività in poi, saprebbe dove sta andando. È la classica «fuga DNS», che l'esercizio 19.3 fa provare e che il Laboratorio 18 osserva dal punto di vista di tor (sezione 28.3).

Quando serve solo un indirizzo, senza aprire una connessione, il client usa `RESOLVE` (comando relay 11), con il nome terminato da zero. L'exit risponde con `RESOLVED` (12), che contiene una o più risposte nel formato tipo, lunghezza, valore e TTL: tipo `0x04` per IPv4, `0x06` per IPv6, `0x00` per un nome (nelle risoluzioni inverse), `0xF0` e `0xF1` per un errore temporaneo o permanente [tor-spec § Remote hostname lookup]. È il meccanismo che sta dietro alla porta DNS che un client tor può offrire alle applicazioni locali (opzione `DNSPort`).

11.1.5 Stream verso le directory: BEGIN_DIR

`BEGIN_DIR` (comando relay 13) apre uno stream non verso Internet, ma verso il servizio di directory del relay stesso: è così che i client scaricano consenso e descrittori (capitolo 12) senza aprire connessioni in chiaro. Il corpo è vuoto e la exit policy non si applica, perché lo stream non lascia il processo tor.

11.2 Il controllo di flusso originale

11.2.1 Il problema

Immaginiamo un exit con un collegamento veloce verso Internet e un client lento. Se l'exit leggesse dal server e inviasse nel circuito tutto ciò che riceve, i dati si accumulerebbero nelle code dei relay intermedi, consumando la loro memoria e rallentando gli altri circuiti. TCP risolve il problema su ogni singolo collegamento, ma il circuito è una catena di collegamenti TCP diversi: la contropressione di TCP tra middle e client non arriva all'exit. Serve un meccanismo *da estremo a estremo*.

11.2.2 Finestre e SENDME

Il meccanismo originale di Tor usa delle **finestre**, contate in celle `DATA` [tor-spec § Flow control]:

- a livello di **circuito**, chi invia ha una *package window* che parte da 1000 celle (o dal parametro di consenso `circwindow`); ogni cella `DATA` inviata la decrementa, e a zero smette di inviare. Chi riceve tiene una *deliver window* simmetrica e, ogni 100 celle ricevute, invia un messaggio `SENDME` (comando relay 5) con StreamID zero, che restituisce 100 celle alla finestra del mittente;

- a livello di **stream**, le finestre partono da 500 celle e crescono di 50 per ogni `SENDME` con lo StreamID dello stream.

Poiché il client può scambiare dati con qualsiasi salto (sezione 10.1), tiene una coppia di finestre per ogni salto. Le finestre riguardano solo le celle originate e consumate, non quelle inoltrate.

Il risultato è che in ogni momento non possono esserci in viaggio più di 1000 celle per circuito, e 500 per stream: circa 500 KB e 250 KB. È una garanzia di memoria per i relay, ma anche un limite di velocità: un circuito con un tempo di andata e ritorno di mezzo secondo non può superare 1000 celle ogni mezzo secondo, circa 1 MB/s, qualunque sia la banda disponibile. E se la rete è congestionata, 1000 celle in volo sono troppe: le code crescono, e con esse la latenza.

11.2.3 SENDME autenticati

Un client malintenzionato potrebbe inviare `SENDME` senza aver ricevuto nulla: l'exit continuerebbe a inviare dati, che si accumulerebbero nella rete, e il client potrebbe così esaurire la memoria dei relay, o amplificare il traffico, con uno sforzo minimo. Dalla versione 0.4.1.1-alpha i `SENDME` di circuito sono **autenticati** (Proposta 289): nello schema tradizionale tor1, il corpo contiene una versione (1), una lunghezza (20) e il **digest progressivo** della cella `DATA` che ha fatto scattare il `SENDME`. Chi riceve il `SENDME` conosce quel digest, perché lo ha calcolato inviando la cella, e lo confronta: se non coincide, il circuito viene chiuso.

Il digest dipende dal contenuto di tutte le celle ricevute, ma un client potrebbe provare a prevederlo se conoscesse in anticipo il contenuto delle celle, per esempio scaricando un file noto. La specifica [tor-spec § Flow control] raccomanda quindi di introdurre byte casuali in almeno una cella `DATA` per incremento della finestra di ricezione (100 celle nello schema classico). È una delle motivazioni del padding casuale dei messaggi relay (sezione 8.6); se una cella occupa tutti i 498 byte utili, però, non c'è spazio per il padding, e la casualità non può essere data per scontata.

Approfondimento: Quale digest, esattamente?

La specifica parla del «digest della cella che precede il `SENDME`». Nei circuiti ordinari con tor1 è il digest SHA-1 completo, di 20 byte, del flusso all'indietro dopo l'ultima cella riconosciuta, salvato nel campo `sendme_digest` dalla funzione `tor1_relay_digest_matches_v0` (`src/core/crypto/relay_crypto_tor1.c`). Se il digest progressivo è più lungo, come lo SHA3-256 dei circuiti rendezvous con onion service, per un `SENDME` di versione 1 si trasmettono soltanto i **primi 20 byte** [tor-spec § Flow control]. Con CGO il digest viene invece sostituito dal tag T della cella confermata (sezione 10.4).

11.3 Il controllo della congestione

Le finestre fisse hanno due difetti opposti: sono troppo piccole per i circuiti veloci e troppo grandi per quelli congestionati. La Proposta 324, di Mike Perry, ha introdotto in Tor un vero **controllo della congestione**, analogo a quello di TCP, adottato a partire da tor 0.4.7.7 (aprile 2022).

11.3.1 Il segnale: il tempo di andata e ritorno

Come si accorge l'exit che il circuito è congestionato? Usando i `SENDME` come orologio. L'exit annota quando invia la cella `DATA` che farà scattare un `SENDME`, e quando il `SENDME` arriva calcola il **tempo di andata e ritorno** (RTT) del circuito. Se l'RTT cresce rispetto al minimo osservato, le code lungo il circuito si stanno riempiendo: è il momento di rallentare. Il mittente regola una *finestra di congestione* (*cwnd*), cercando di mantenerla vicina al prodotto banda per ritardo del circuito, cioè alla quantità di dati che il circuito può contenere senza creare code.

11.3.2 Gli algoritmi

La proposta ne descrive tre, ispirati agli algoritmi di TCP: Tor Westwood, Tor Vegas e Tor NOLA. Dopo valutazioni in simulazione con Shadow (capitolo 19) e sulla rete reale, quello predefinito in tor è **Vegas** (costante `CC_ALG_DFLT` in `src/core/or/congestion_control_common.c`). Come TCP Vegas, stima quante celle sono «in coda» confrontando la finestra attuale con quella che servirebbe al minimo RTT osservato, e aumenta o riduce la finestra per mantenere questa stima entro una soglia.

11.3.3 I parametri: perché 31

Il controllo della congestione si negozia salto per salto con ntor-v3 (sezione 9.7): il client lo chiede con l'estensione `CC_FIELD_REQUEST`, e il relay risponde con `CC_FIELD_RESPONSE`, che contiene `sendme_inc`, il numero di celle `DATA` dopo cui chiede un `SENDME`. Nel capitolo 9 l'exit della rete di laboratorio ha risposto 31. Il valore non è arbitrario: nel codice di tor è definito come il numero di celle che stanno in un record TLS di dimensione massima,

$$\texttt{TLS_RECORD_MAX_CELLS} = \left\lfloor \frac{16384}{514} \right\rfloor = 31,$$

e la finestra iniziale vale $4 \times 31 = 124$ celle. SENDME più frequenti che in passato, uno ogni 31 celle invece di 100, significano misure di RTT più frequenti e reazioni più rapide.

11.3.4 Niente più finestre di stream: XON e XOFF

Con il controllo della congestione, i `SENDME` di stream scompaiono. Le finestre per stream imponevano un dilemma: se piccole, limitano la velocità di ogni download; se grandi, permettono a un client malintenzionato di far accumulare all'exit una finestra intera di dati per ciascuno dei tanti stream che apre. La Proposta 324 le sostituisce con due messaggi:

XOFF (comando relay 43): «smetti di inviare su questo stream», inviato quando la coda verso l'applicazione supera una soglia;

XON (comando relay 44): «riprendi», accompagnato dall'indicazione della velocità stimata a cui la coda si sta svuotando, così che l'altra parte possa adeguarsi.

La Proposta 324, *Stream Flow Control Without Windows*, definisce i corpi di questi messaggi byte per byte: `XOFF` contiene soltanto `VERSION` (1 byte, attualmente `0x00`); `XON` contiene `VERSION` (1 byte, `0x00`) seguito da `kbps_ewma` (4 byte, big-endian). **La proposta specifica esplicitamente l'unità**: 1000 *byte al secondo*, non 1000 bit. Il codice tor 0.4.9.13 la conferma in `src/core/or/congestion_control_flow.c`. Il nome `kbps` può indurre in errore: il valore 125 significa 125 000 byte/s e un corpo `XON` corrispondente è `00 00 00 00 7d` (versione, poi il numero 125). Il valore zero del campo di velocità significa *nessun limite*, non «non inviare dati». Un `XON` può inoltre comunicare aggiornamenti di velocità senza che lo stream sia prima stato fermato da `XOFF`.

I `SENDME` di circuito, invece, vengono inviati appena le celle sono decifrate e riconosciute, prima di consegnarle all'applicazione: misurano la rete, non l'applicazione.

Attenzione: Controllo della congestione e anonimato

Qualsiasi meccanismo che reagisce al traffico può diventare un canale laterale: un avversario che osserva l'andamento di una finestra potrebbe dedurne qualcosa sul circuito. La Proposta 324 dedica una sezione a questi rischi e alle mitigazioni, per esempio limitando la frequenza dei messaggi XON e XOFF. È lo stesso genere di cautela che nel capitolo 8 porta a inviare sempre il motivo NONE in una cella **`DESTROY`**: ogni variazione osservabile che dipende dallo stato interno è informazione regalata. Gli attacchi che sfruttano gli indizi ricavabili dal traffico, e che questa cautela cerca di non alimentare, sono il tema del capitolo 26.

11.4 Laboratorio: scaricare un file attraverso Tor

Laboratorio 5: Stream e controllo di flusso

Obiettivo. Scrivere `labs/it/codice/torlab/circuito.py`, che riunisce i laboratori precedenti in una classe `Circuito`, e `labs/it/codice/torlab/stream.py`, che apre stream e trasferisce dati; scaricare un file da un server web locale attraverso un circuito di tre salti; misurare le finestre dell'exit.
Prerequisiti. Laboratori 1–4; la rete di laboratorio avviata; un server web locale (`python3 -m http.server 8080`).
Tempo stimato. 3 ore.

Passo 1: un circuito in una riga. La funzione `costruisci` apre il canale verso il guard, crea il primo salto ed estende il circuito, chiedendo il controllo della congestione solo all'ultimo salto, come fa tor:

```
circ = costruisci([relay("003r"), relay("004r"), relay("006e")])
print(circ.estensioni_exit)            # [(2, b'\x1f')]: sendme_inc = 31
```

Passo 2: aprire lo stream.

```
def corpo_begin(indirizzo, porta, flag=0):
    corpo = f"{indirizzo.lower()}:{porta}".encode() + b"\x00"
    return corpo + (struct.pack(">I", flag) if flag else b"")

s = Stream(circ, 1)
s.apri("127.0.0.1", 8080)       # BEGIN, poi attende CONNECTED o END
```

L'exit risponde con `CONNECTED` e un corpo di 8 byte, `7f000001 00000708`: l'indirizzo 127.0.0.1 e un TTL di `0x708` = 1800 secondi.

Passo 3: i SENDME. Per ogni cella `DATA` ricevuta dall'exit il circuito conta, e ogni `sendme_inc` celle invia un `SENDME` autenticato:

```
def conta_data_ricevuto(self):
    est = dict(self.estensioni_exit)
    sendme_inc = est[2][0] if 2 in est else 100
    self.data_da_confermare += 1
    if self.data_da_confermare == sendme_inc:
        self.data_da_confermare = 0
        digest = self.cripto.salti[-1].indietro.digest.digest()
        corpo = struct.pack(">BH", 1, 20) + digest
        self.invia(MessaggioRelay(ComandoRelay.SENDME, 0, corpo))
```

Attenzione: Un errore istruttivo

Nella prima versione di questo laboratorio il contatore era un attributo dello *stream*. Il primo stream, una pagina di 2 celle, funzionava; il secondo, un file di 1 MB, veniva interrotto dall'exit con un **`DESTROY`**. Il motivo: il `SENDME` di circuito conta le celle `DATA` di *tutti* gli stream del circuito. L'exit aveva già contato le 2 celle del primo stream e si aspettava il `SENDME` dopo la 29ª cella del secondo; noi lo inviavamo dopo la 31ª, con un digest diverso da quello atteso, e l'exit ha fatto esattamente ciò che la Proposta 289 prescrive: ha chiuso il circuito. Spostare il contatore nella classe `Circuito` ha risolto il problema.

Attenzione: Limiti intenzionali del client didattico

Il codice `labs/it/codice/torlab/stream.py` trasferisce uno stream per volta, con l'exit della rete di laboratorio e il controllo della congestione negoziato. Non implementa la gestione concorrente degli stream, la finestra di invio del circuito né la verifica dei `SENDME` ricevuti in direzione opposta. Dimostra il formato dei messaggi e il conteggio dei `SENDME` in uscita, ma non va considerato un client Tor completo né usato come tale sulla rete pubblica.

Passo 4: il download. Inviamo una richiesta HTTP e leggiamo fino a `END`:

```
s.invia(b"GET /grande.bin HTTP/1.0\r\nHost: 127.0.0.1\r\n\r\n")
risposta = s.ricevi_tutto()
```

Con un file casuale di $1\,048\,576$ byte, in una nostra esecuzione:

```
stream 1: 209 byte in 0.05 s, celle DATA 2, SENDME del circuito finora 0,
          altri ['END DONE']
stream 2: 1048781 byte in 0.18 s, celle DATA 2120, SENDME del circuito
          finora 68, altri ['END DONE']
```

L'impronta SHA-256 del file ricevuto coincide con quella dell'originale. I 68 `SENDME` sono esattamente $\lfloor (2 + 2120)/31 \rfloor$: un conteggio per circuito, come abbiamo imparato. Le celle sono 2120 anziché le $\lceil 1048781/498 \rceil = 2106$ minime: l'exit non aspetta di avere 498 byte per spedire una cella, ma impacchetta i dati man mano che li legge dal server.

Passo 5: vedere le finestre. Ora un esperimento che nessun client normale può fare: scarichiamo lo stesso file *senza mai inviare* `SENDME`, e contiamo quante celle l'exit invia prima di fermarsi. Lo ripetiamo con e senza controllo della congestione:

```
circ = costruisci(percorso, cc=cc)
circ.conta_data_ricevuto = lambda: None      # nessun SENDME
```

```
controllo della congestione=True:  l'exit si ferma dopo 124 celle DATA
controllo della congestione=False: l'exit si ferma dopo 500 celle DATA
```

Con il controllo della congestione l'exit si ferma a 124 celle: è la finestra di congestione iniziale, 4×31. Senza, si ferma a 500 celle, non a 1000: la finestra di *stream* (500) si esaurisce prima di quella di circuito (1000), perché nel controllo di flusso originale un client deve inviare anche i `SENDME` di stream. In entrambi i casi l'exit si limita ad aspettare: i dati rimangono nel suo buffer, non nella rete.

Riepilogo

- Uno stream si apre con `BEGIN` (`indirizzo:porta`, in minuscolo), si conferma con `CONNECTED`, trasporta `DATA` e si chiude con `END`; il client può inviare dati prima di `CONNECTED`.
- La risoluzione dei nomi avviene all'exit, per non rivelare le destinazioni al resolver locale; `RESOLVE` e `RESOLVED` permettono risoluzioni senza connessione.
- Il controllo di flusso originale usa finestre di 1000 celle per circuito e 500 per stream, con `SENDME` ogni 100 e 50 celle.
- I `SENDME` di circuito sono autenticati con il digest progressivo della cella che li fa scattare (Proposta 289).
- Con il controllo della congestione (Proposta 324, algoritmo predefinito Vegas) la finestra si adatta all'RTT misurato con i `SENDME`; `sendme_inc` vale 31, le celle di un record TLS, e la finestra iniziale 124.
- Le finestre di stream sono sostituite da XON e XOFF.

Esercizi

Esercizio 11.1 ⋆ Scrivi in esadecimale il corpo di un messaggio `BEGIN` verso `Example.COM` sulla porta 443, per un client che accetta anche IPv6.

Esercizio 11.2 ⋆ Interpreta il corpo di `CONNECTED c0000201 0000012c`. Che cosa dovrebbe fare il client con questa informazione?

Esercizio 11.3 ⋆ Stima il risparmio di tempo degli *optimistic data* per un circuito con un RTT di 600 ms e una pagina che richiede una sola richiesta HTTP. Perché l'exit deve scartare i dati in coda se la connessione fallisce?

Esercizio 11.4 ⋆⋆ Con il controllo di flusso originale, qual è la velocità massima di un singolo stream su un circuito con RTT di 400 ms? E di un circuito con più stream? Confronta con la finestra iniziale del controllo della congestione.

Esercizio 11.5 ⋆⋆ Descrivi un attacco alla memoria di un exit basato su `SENDME` non autenticati. Perché il padding casuale nelle celle `DATA` è necessario perché l'autenticazione funzioni?

Esercizio 11.6 ⋆⋆ Estendi `stream.py` con una funzione `risolvi(nome)` che usi `RESOLVE` e interpreti `RESOLVED`. Provala nella rete di laboratorio con `localhost`. Che cosa risponde l'exit per un nome inesistente?

Esercizio 11.7 ⋆ ⋆ ⋆ Prova ad aprire uno stream verso una porta che l'exit della rete di laboratorio non accetta, dopo aver modificato la sua exit policy (`torrc` di `test006e`, poi `chutney hup`). Quale motivo riporta `END`? Interpreta i campi aggiuntivi.

Esercizio 11.8 ⋆ ⋆ ⋆ Modifica il laboratorio per misurare, lato client, il tempo tra l'invio di ogni `SENDME` e l'arrivo della cella `DATA` successiva. Che cosa stai misurando? Confronta i valori con e senza un secondo download in parallelo sullo stesso circuito.

Per approfondire

- *Tor Protocol Specification*, capitoli «Opening streams and transmitting data», «Closing streams», «Remote hostname lookup» e «Flow control» [Thef].
- Proposta 289, *Authenticating sendme cells to mitigate bandwidth attacks*.
- Proposta 324, *RTT-based Congestion Control for Tor*: motivazioni, algoritmi, parametri e analisi di sicurezza.

Capitolo 12

Il sistema delle directory

Aggiornato al 23 settembre 2026 — torspec commit `8e1f245`; tor 0.4.9.13; consensus method 36

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- spiegare perché Tor ha bisogno di un sistema di directory e quali rischi deve evitare;
- distinguere i documenti del sistema: descrittori, microdescrittori, voti, consenso, certificati delle autorità;
- descrivere come le directory authority votano ogni ora e come combinano i voti;
- conoscere i criteri con cui vengono assegnati i flag;
- leggere un consenso riga per riga e verificarne le firme;
- scaricare e verificare il consenso della rete di laboratorio con il tuo codice (Laboratorio 6).

Nei laboratori precedenti abbiamo sempre saputo in anticipo a quali relay collegarci, con quali chiavi e su quali porte: le leggevamo dai file della rete di laboratorio. Un client vero non ha questo lusso. Deve scoprire da solo quali relay esistono, dove si trovano, quali chiavi usano e quanto sono affidabili. È il compito del **sistema delle directory**, l'argomento di questo capitolo.

12.1 Il problema

Un sistema di directory per Tor deve rispondere a esigenze in parte contrastanti:

Correttezza Le informazioni sui relay (indirizzi, chiavi) devono essere autentiche. Un avversario che riuscisse a far credere a un client che un certo relay ha una chiave diversa potrebbe intercettarne il traffico.

Visione comune Tutti i client devono vedere *la stessa* lista di relay. Se un avversario potesse dare a ciascun client una lista diversa, per esempio con pochi relay scelti da lui, potrebbe riconoscere i client dai relay che usano: è l'attacco di *partizionamento*, o *epistemico*.

Resistenza agli abusi Chiunque può aggiungere relay alla rete. Il sistema deve limitare l'influenza di chi ne aggiunge molti (attacco *Sybil*) e poter escludere quelli che si comportano male.

Scalabilità Milioni di client scaricano le informazioni più volte al giorno: il sistema deve essere economico da distribuire.

La soluzione di Tor, evoluta attraverso tre versioni del protocollo di directory, è un piccolo insieme di server fidati, le **directory authority**, che ogni ora votano e firmano insieme un unico documento: il **consenso** [dir-spec § Outline].

Nota storica: Tre generazioni

Nella prima versione del protocollo ogni autorità pubblicava da sola l'elenco dei relay, e un'autorità disonesta poteva ingannare i client. Nella seconda i client scaricavano l'elenco di ciascuna autorità e ne combinavano le opinioni da sé, con costi di banda moltiplicati. La terza, in uso da oltre quindici anni, affida la combinazione alle autorità stesse: un solo documento, firmato da tutte, che i client devono solo verificare. Con essa le autorità hanno anche smesso di firmare con la stessa chiave che usano come relay, che dovevano tenere in memoria in chiaro.

12.2 Gli attori

Directory authority Server gestiti da persone di fiducia della comunità Tor. La rete pubblica ne ha nove, più un'autorità dedicata ai bridge (capitolo 16); l'elenco è compilato nel codice di tor (`src/app/config/auth_dirs.inc`), così che ogni client parta dalla stessa lista. Ogni autorità ha una chiave di *identità* a lungo termine, tenuta offline, con cui certifica una chiave di *firma* valida da 3 a 12 mesi; è quest'ultima a firmare voti e consensi.

Directory cache Relay che scaricano consenso e descrittori dalle autorità e li ridistribuiscono ai client, alleggerendo le autorità. Hanno il flag `V2Dir` e rispondono agli stream `BEGIN_DIR` (capitolo 11).

Fallback directory Un elenco di circa duecento relay stabili, anch'esso compilato nel codice (`fallback_dirs.inc`, 200 voci in tor 0.4.9.13), a cui un client appena installato chiede il primo consenso senza sovraccaricare le autorità.

Relay Pubblicano periodicamente il proprio descrittore presso le autorità.

Client Scaricano il consenso e i microdescrittori, li verificano e li usano per scegliere i percorsi (capitolo 13).

La fig. 12.1 mostra come circolano i documenti.

12.3 I documenti

Tutti i documenti di directory sono testo, nel formato che la specifica chiama *netdoc*: una sequenza di righe (*item*), ciascuna con una parola chiave e degli argomenti, eventualmente seguite da un oggetto codificato in base64 tra righe `-----BEGIN ...-----` e `-----END ...-----`. È un formato semplice da leggere per le persone e da analizzare per i programmi, come vedremo nel laboratorio.

Descrittore del relay (*server descriptor*) Scritto e firmato dal relay: indirizzi e porte, chiavi (identità, chiave onion ntor, certificati Ed25519), banda dichiarata, exit policy completa, famiglia, piattaforma e versione. Viene ripubblicato almeno ogni 18 ore (`FORCE_REGENERATE_DESCRIPTOR_INTERVAL` nel codice di tor), o prima se cambia qualcosa di importante.

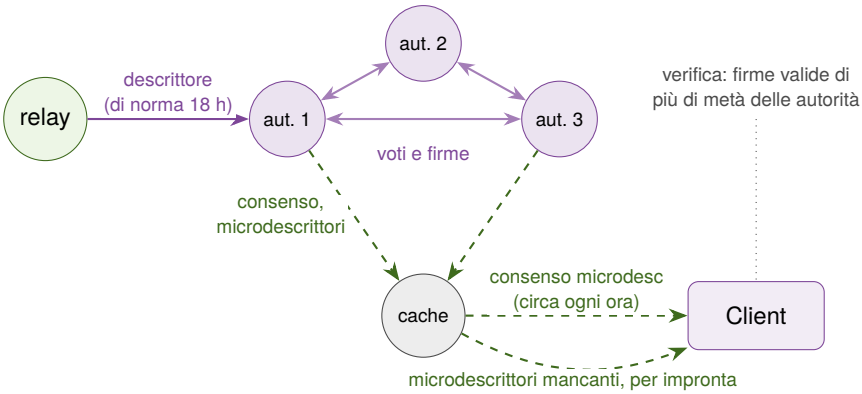


Figura 12.1: Il flusso dei documenti di directory. I relay pubblicano i descrittori presso le autorità; le autorità si scambiano i voti, calcolano il consenso in base ai voti disponibili e si scambiano le firme; le directory cache scaricano consenso e microdescrittori e li servono ai client.

Documento extra-info Statistiche non necessarie al funzionamento del protocollo (per esempio il traffico per paese dei bridge), pubblicate a parte per non appesantire i descrittori. Alimentano Tor Metrics (capitolo 24).

Microdescrittore Il sottoinsieme del descrittore che serve davvero a un client e che cambia raramente: la chiave onion ntor, la famiglia, un riassunto della exit policy, l'identità Ed25519. Lo calcolano le autorità, e ciascuno è identificato dall'impronta SHA-256 del proprio testo.

Voto Il parere di un'autorità sullo stato della rete: per ogni relay, i flag, la banda, la versione.

Consenso Il risultato della combinazione dei voti, firmato dalle autorità. Esiste in due *varianti* (*flavor*): `ns`, che per ogni relay contiene l'impronta del descrittore completo, e `microdesc`, che contiene l'impronta del microdescrittore. I client moderni usano solo la seconda, più compatta.

Certificato di chiave dell'autorità Lega la chiave di firma di un'autorità alla sua identità.

Approfondimento: Perché i microdescrittori

Un descrittore completo è lungo alcuni kilobyte e cambia spesso, per esempio a ogni variazione della banda dichiarata. Scaricarli tutti, per migliaia di relay, più volte al giorno, costerebbe molto ai client, soprattutto su reti mobili. Il microdescrittore contiene solo ciò che non cambia quasi mai; ciò che cambia spesso, come la banda e i flag, sta nel consenso, che il client scarica comunque. Poiché il client chiede i microdescrittori per impronta, e non per identità del relay, una directory cache non può dare a un client un microdescrittore diverso da quello che tutti gli altri ricevono.

12.4 Il voto

12.4.1 Il calendario

Ogni consenso ha tre istanti [dir-spec § Voting timeline]:

`valid-after` da quando è valido;

`fresh-until` fino a quando è il consenso più recente; di norma coincide con il `valid-after` del successivo;

`valid-until` fino a quando un client può ancora usarlo.

Nella rete pubblica l'intervallo di voto è di un'ora e un consenso resta valido per tre intervalli (opzioni `V3AuthVotingInterval` e `V3AuthNIntervalsValid`), così che in ogni momento esistano più consensi validi e un ritardo delle autorità non blocchi i client. Prima dell'inizio di ogni intervallo le autorità hanno 5 minuti per scambiarsi i voti e 5 minuti per scambiarsi le firme (`V3AuthVoteDelay` e `V3AuthDistDelay`). Nella rete di laboratorio tutto è accelerato: un voto ogni 20 secondi, con 4 secondi per fase (sezione 19.5).

12.4.2 Come si combinano i voti

Ogni autorità raccoglie i voti delle altre e calcola il consenso con un algoritmo deterministico, così che tutte ottengano lo stesso documento e possano firmarlo [dir-spec § Computing a consensus from a set of votes]. Le regole principali:

- un relay compare nel consenso solo se abbastanza autorità lo conoscono e, con i consensus method correnti (dal 24 in poi; il laboratorio usa il 36), solo se ha sia il flag `Running` sia `Valid`;
- un relay ha un flag se più della metà delle autorità che si esprimono su quel flag glielo assegnano;
- la banda (`w`) è la mediana inferiore delle bande nei voti, ma se almeno tre autorità ne hanno *misurato* la banda si usa la mediana delle misure;
- i parametri di rete (`params`) sono inclusi se li vota una maggioranza delle autorità totali oppure almeno tre autorità (nel codice `i > total_authorities/2 || i >= 3`), e il valore scelto è la mediana inferiore;
- i tempi sono le mediane dei tempi proposti.

Poiché le autorità possono eseguire versioni diverse di tor, il documento dichiara il *consensus method* usato: un numero che identifica l'insieme esatto di regole, scelto come il più alto supportato da abbastanza autorità. Il consenso della rete di laboratorio usa il metodo 36.

12.4.3 I flag

I flag riassumono il giudizio delle autorità su ciascun relay [dir-spec § Assigning flags in a vote]. La specifica avverte che i criteri esatti possono cambiare e che i client non devono dipenderne; quelli in uso sono riassunti nella tabella 12.1.

`BadExit` e `MiddleOnly` sono gli strumenti con cui le autorità reagiscono a relay sospetti senza rimuoverli del tutto. `Sybil` ha una funzione diversa: compare nei *voti* per segnalare i relay che eccedono il limite per indirizzo, ai quali le autorità non assegnano `Running` o `Valid`; con i metodi di consenso attuali, senza questi flag il relay **non compare nella lista** del consenso. La presenza di `Sybil` nell'elenco globale `known-flags` non implica quindi che esistano voci di relay con quel flag nella sezione `s` [dir-spec § Assigning flags in a vote]. Nella rete di laboratorio, dove tutti i relay stanno su 127.0.0.1, il limite è disattivato dal parametro `AuthDirMaxServersPerAddr=0`, che il laboratorio ci mostrerà nella riga `params`.

Flag	Criterio (in sintesi)
`Running`	l'autorità ha raggiunto il relay su tutte le sue ORPort negli ultimi 45 minuti
`Valid`	versione di tor non nota come difettosa, relay non escluso dalle autorità
`Fast`	banda tra i 7/8 più veloci, o almeno 100 KB/s
`Stable`	tempo medio tra i guasti (pesato nel tempo) almeno pari alla mediana, o almeno 7 giorni
`Guard`	`Fast`, `Stable`, *familiare* (in rete da alcune settimane), buona disponibilità, banda sufficiente, `V2Dir`
`Exit`	permette l'uscita verso almeno un blocco /8 sia sulla porta 80 sia sulla 443
`HSDir`	`Stable` e `Fast`, attivo da almeno 96 ore: può conservare descrittori di onion service
`V2Dir`	serve informazioni di directory (directory cache)
`Authority`	è una directory authority
`BadExit`	exit da non usare come tale (per esempio perché altera il traffico)
`MiddleOnly`	da usare solo come relay intermedio (introdotto in 0.4.7.2-alpha)
`StaleDesc`	descrittore pubblicato più di 18 ore fa
`Sybil`	nei voti: relay eccedenti il limite per indirizzo IP (normalmente 2); generalmente esclusi dal consenso

Tabella 12.1: I flag usati dalle autorità nei voti e, quando applicabile, nel consenso; `Sybil` è principalmente un flag dei voti.

12.4.4 La banda

Quanto è veloce un relay? Il valore dichiarato dal relay nel descrittore non è affidabile: un relay malevolo potrebbe dichiarare una banda enorme per essere scelto più spesso. Per questo alcune autorità, dette *bandwidth authority*, misurano la banda dei relay facendo passare traffico attraverso di essi con uno strumento dedicato (oggi *sbws*, capitolo 24) e la riportano nei voti come `Measured`. Se almeno tre misure sono disponibili, il consenso usa la loro mediana; altrimenti la riga `w` riporta `Unmeasured=1` e un valore limitato. La banda nel consenso non è una velocità in senso stretto, ma un *peso*: è ciò che determina con quale probabilità un relay viene scelto (capitolo 13).

12.5 Anatomia di un consenso

Ecco, abbreviato, il consenso microdesc della rete di laboratorio scaricato nel Laboratorio 6. Gli estratti di questa sezione vengono da una copia conservata nel repository (`labs/it/codice/torlab/test/dati/consenso-microdesc.txt`), su cui girano i test del laboratorio: si possono quindi rileggere riga per riga anche senza avviare la rete. È stata catturata il 23 settembre 2026, quando i relay del laboratorio eseguivano ancora tor 0.4.9.12: è la versione che si legge nella riga `v` della voce di relay, non quella di riferimento del libro, che la rete esegue oggi.

Intestazione.

```
network-status-version 3 microdesc
vote-status consensus
consensus-method 36
valid-after 2026-09-23 15:37:20
fresh-until 2026-09-23 15:37:40
valid-until 2026-09-23 15:38:00
voting-delay 4 4
known-flags Authority Exit Fast Guard HSDir ... Running Stable ... Valid
params AuthDirMaxServersPerAddr=0 cc_alg=2
```

La riga `params` contiene i *parametri di consenso*, con cui le autorità regolano il comportamento di tutta la rete senza bisogno di aggiornare il software: qui, oltre al limite per indirizzo, l'algoritmo di controllo della congestione (`cc_alg=2`, Vegas, sezione 11.3). Nella rete pubblica i parametri sono decine.

Una voce per relay.

```
r test006e ml1aPg9UFy4VCx85YcpsRs/9shc 2038-01-01 00:00:00 127.0.0.1 5106 0
m FfrgO+s4VYS6WKBq3pjfMDs+uZ2uVsvgG88JilbhIOo
s Exit Fast Guard HSDir Running Stable V2Dir Valid
v Tor 0.4.9.12
pr Conflux=1 Cons=1-2 Desc=1-4 ... Link=3-5 ... Relay=2-6
w Bandwidth=4096
```

`r` nome, impronta RSA dell'identità (in base64), un campo di data ormai non significativo nel consenso, indirizzo IPv4, ORPort e DirPort;

`m` impronta del microdescrittore da scaricare;

`s` i flag;

`v`, `pr` versione di tor e *sottoprotocolli* supportati: è qui che un client scopre, per esempio, che il relay supporta CGO (`Relay=6`) o i link protocol 3–5;

`w` il peso di banda, in kilobyte al secondo.

Nella variante `ns` ogni voce ha anche una riga `p` con il riassunto della exit policy per porte; nella variante microdesc questo riassunto si trova invece nel microdescrittore, e il consenso non lo ripete.

Piè di pagina e firme.

```
directory-footer
bandwidth-weights Wbd=821 Wbe=0 Wbg=5000 Wbm=10000 Wdb=10000 ...
directory-signature sha256 3776EF73A9214587AEFB3D732FA4A443CFF491BA
                           79E102D226590BC5D1D1EBBAD8A16122EC0F9801
-----BEGIN SIGNATURE-----
...
-----END SIGNATURE-----
```

I `bandwidth-weights` sono i coefficienti con cui i client bilanciano l'uso dei relay nelle diverse posizioni del circuito (capitolo 13). Seguono le firme, una per autorità: algoritmo di hash, impronta dell'identità dell'autorità, impronta della sua chiave di firma, e la firma.

12.6 Verificare un consenso

Un client accetta un consenso solo se porta le firme valide di **più della metà** delle autorità che conosce. Nel codice di tor la soglia è `n_v3_authorities/2 + 1` (`src/feature/nodelist/networkstatus.c`): 5 firme su 9 nella rete pubblica, 2 su 3 nel laboratorio. Per verificare una firma servono:

1. il **certificato** dell'autorità, che il client scarica (`/tor/keys/all`) e verifica con l'identità dell'autorità nota a priori;

2. il **testo firmato**: tutto il documento dall'inizio fino allo spazio che segue la prima parola chiave `directory-signature`;

3. lo schema di firma: RSA con padding PKCS#1 v1.5 applicato direttamente al digest (SHA-1 o SHA-256), *senza* l'identificativo dell'algoritmo che PKCS#1 prevederebbe [dir-spec § Signing documents].

Attenzione: Una firma corretta non basta a fidarsi del consenso

C'è una differenza fondamentale tra *verificare una firma* e *autenticare il documento*. La firma può essere matematicamente valida rispetto a una chiave pubblica ricevuta insieme al documento, senza che quella chiave appartenga a un'autorità fidata. Un client Tor reale deve controllare anche l'identità dell'autorità rispetto alla propria lista predefinita, la firma `dir-key-certification` apposta al certificato dalla chiave di identità, la *cross-certificate* della chiave di firma e la validità temporale del certificato [dir-spec § Directory authority key certificates]. Il verificatore del Laboratorio 6, per semplicità, **non compie questi controlli**: conta solo le firme matematicamente valide rispetto alle chiavi estratte dai certificati forniti in input. La soglia che calcola non dimostra quindi, da sola, che una maggioranza delle *autorità fidate* abbia approvato il consenso. Non va usato per stabilire l'affidabilità di un consenso sulla rete pubblica.

Approfondimento: Fino allo spazio o fino a capo?

La parte generale della specifica sui documenti firmati dice che la firma copre il testo fino al *ritorno a capo* dopo la riga della firma; la parte sul consenso precisa invece che si arriva allo *spazio* dopo `directory-signature`, così che tutte le autorità firmino esattamente gli stessi byte, anche se le loro righe di firma sono diverse. Nel Laboratorio 6 abbiamo provato entrambe le letture sul consenso reale della rete di laboratorio: con la regola dello spazio tutte e tre le firme sono valide, con quella del ritorno a capo nessuna. In caso di dubbio, la verifica sperimentale contro un'implementazione indipendente resta il giudice finale.

Attenzione: La lista delle autorità è un punto di fiducia

Tutto il sistema poggia sull'elenco delle autorità compilato nel software. Un utente può cambiarlo, ma la specifica lo sconsiglia: un client con autorità diverse vedrebbe un consenso diverso da tutti gli altri, e sarebbe riconoscibile. Le autorità stesse sono un bersaglio: se la maggioranza fosse compromessa, potrebbe firmare un consenso con relay malevoli. Per questo sono gestite da persone diverse, in paesi diversi, e le chiavi di identità restano offline.

12.7 Come un client resta aggiornato

Un client non scarica un nuovo consenso allo scoccare di ogni ora: se tutti lo facessero insieme, le directory cache verrebbero sommerse. Sceglie invece un istante a caso tra i tre quarti del primo intervallo dopo che il consenso ha smesso di essere fresco e i sette ottavi del tempo restante prima che scada [dir-spec § Client operation]. La specifica fa un esempio: con un consenso valido dalle 1:00, fresco fino alle 2:00 e valido fino alle 4:00, il client ne cercherà uno nuovo in un momento a caso tra le 2:45 e le 3:50.

Per risparmiare banda, un client può chiedere solo le *differenze* rispetto al consenso che ha già (*consensus diff*), identificato dalla sua impronta SHA3-256, e scarica solo i microdescrittori che non ha, chiedendoli per impronta (`/tor/micro/d/` seguito dalle impronte separate da `-`).

12.8 Laboratorio: scaricare e verificare il consenso

Laboratorio 6: Il consenso della rete di laboratorio

Obiettivo. Scrivere `labs/it/codice/torlab/directory.py`: scaricare consenso, certificati e microdescrittori da un'autorità attraverso un circuito, analizzarli e verificare le firme.
Prerequisiti. Laboratori 1–5; la rete di laboratorio avviata.
Tempo stimato. 3 ore.

Passo 1: scaricare con BEGIN_DIR. Come fa tor, usiamo un circuito di un solo salto verso una directory, senza controllo della congestione, e apriamo stream `BEGIN_DIR`. Sopra lo stream parliamo HTTP/1.0:

```
def scarica(circ, percorso, stream_id):
    s = Stream(circ, stream_id)
    circ.invia(MessaggioRelay(ComandoRelay.BEGIN_DIR, stream_id, b""))
    _, msg = s._prossimo()
    if msg.comando != ComandoRelay.CONNECTED:
        raise ErroreCircuito("BEGIN_DIR rifiutato")
    s.aperto = True                          # ora Stream.invia è consentito
    s.invia(f"GET {percorso} HTTP/1.0\r\n\r\n".encode())
    intestazione, separatore, corpo = s.ricevi_tutto().partition(b"\r\n\r\n")
    if not separatore:
        raise ErroreCircuito("risposta HTTP incompleta")
    return corpo

circ = costruisci([relay("000a")], cc=False)
testo = scarica(circ, "/tor/status-vote/current/consensus-microdesc", 1)
```

Il listato mostra il percorso minimo: dopo aver ricevuto `CONNECTED` occorre aggiornare lo stato locale dello stream, altrimenti `Stream.invia` rifiuta il `GET`. La funzione completa in `directory.py` controlla anche il codice HTTP (deve essere `200`), i caratteri del percorso e la presenza del separatore fra intestazioni e corpo.

Passo 2: il formato netdoc. Una funzione di una ventina di righe, `items`, divide un documento in item, raccogliendo gli oggetti `-----BEGIN/-----END`. Su di essa si costruiscono `analizza_consenso`, che produce una voce per relay con flag, banda, policy e impronta del microdescrittore, e `analizza_certificati`, che *estrae* le chiavi dai certificati senza verificarne la catena di fiducia.

Passo 3: verificare le firme.

```
def testo_firmato(testo):
    inizio = testo.index("network-status-version ")
    pos = testo.index("\ndirectory-signature ") + len("\ndirectory-signature ")
    return testo[inizio:pos].encode()

def verifica_firma(consenso, firma, cert):
    h = hashlib.sha256 if firma.algoritmo == "sha256" else hashlib.sha1
    atteso = h(testo_firmato(consenso.testo)).digest()
    ricavato = cert.chiave_firma.recover_data_from_signature(
        firma.firma, padding.PKCS1v15(), None)
    return ricavato == atteso
```

Il metodo `recover_data_from_signature`, con algoritmo `None`, restituisce il digest contenuto nella firma senza aspettarsi l'identificativo dell'algoritmo: esattamente lo schema di Tor.

Passo 4: i risultati. In una nostra esecuzione sulla rete di laboratorio (date, nomi e conteggi dipendono dall'istanza della rete):

```
consenso: 6035 byte; 2026-09-23 15:37:20 -> 2026-09-23 15:38:00
metodo 36; firme 3; certificati 3
params: {'AuthDirMaxServersPerAddr': 0, 'cc_alg': 2}
  test006e 127.0.0.1:5106 w=4096 flag=['Exit', 'Fast', 'Guard', ...]
  test000a 127.0.0.1:5100 w=20   flag=['Authority', 'Fast', 'Guard', ...]
  ...
firme valide: 3 (richieste: 2)
microdescrittori: 8; corrispondono agli m del consenso: True
```

Il consenso copre 40 secondi (due intervalli di 20). Le tre firme superano la verifica *matematica* con le chiavi estratte dai certificati dell'istanza di laboratorio, e l'impronta SHA-256 di ciascun microdescrittore scaricato coincide con la riga `m` del consenso. Questo esercizio mostra il legame *crittografico* fra firme del consenso, impronte dei microdescrittori e chiavi ntor; la **catena di fiducia completa** esige anche l'autenticazione dei certificati delle autorità, che il nostro verificatore non implementa.

Passo 5: i controlli negativi. Una verifica che non fallisce mai non verifica nulla. Cambiando una sola cifra del consenso (`Bandwidth=4096` in `4097`) le firme valide scendono a zero; con una sola autorità nota, a una. I file `test/test_directory.py` ripetono questi controlli su una copia del consenso salvata nel repository, così che si possano eseguire anche senza la rete.

Passo 6: il consenso vero. Lo stesso codice funziona sulla rete pubblica. Da CollecTor si scaricano un consenso e i certificati delle autorità dello stesso periodo, e si verifica:

```
c = analizza_consenso(open("consensus-microdesc-2026-09-23-15").read())
cert = analizza_certificati(open("certs").read())
valide, richieste, _ = verifica_consenso(c, cert, 9)
print(valide, richieste)          # 9 5
```

Tutte e nove le firme risultano matematicamente valide con le chiavi del file `certs`, ben oltre le cinque richieste nel semplice conteggio del laboratorio. Questo **non** è sufficiente per accettare il documento come farebbe un client Tor: qui non sono state verificate le identità fidate, le firme di certificazione e le scadenze dei certificati. Un dettaglio fa inciampare al primo tentativo: i file di CollecTor iniziano con una riga di annotazione `@type network-status-microdesc-consensus-3 1.0`, che non fa parte del documento firmato. Includendola, il digest cambia e *tutte* le firme risultano non valide, con un errore che sembra crittografico ma è di formato. La funzione `testo_firmato` parte perciò da `network-status-version`, e un test lo verifica.

Riepilogo

- Il sistema delle directory deve dare a tutti i client la stessa visione autentica della rete, resistere ai Sybil e scalare a milioni di utenti.
- Nove directory authority (più una per i bridge), elencate nel codice, votano ogni ora e firmano un unico consenso, valido per tre ore.
- I relay pubblicano descrittori; i client usano il consenso microdesc e i microdescrittori, scaricati per impronta.

- Un flag è assegnato se lo vota più della metà delle autorità che se ne occupano; la banda misurata da almeno tre autorità prevale su quella dichiarata.
- Un client accetta un consenso con le firme valide di più della metà delle *autorità fidate*: prima deve autenticare i certificati delle loro chiavi di firma. Il Laboratorio 6 verifica le firme, *non* l'intera catena di fiducia. La firma copre il testo fino allo spazio dopo `directory-signature`.
- I parametri di consenso regolano il comportamento di tutta la rete senza aggiornare il software.

Esercizi

Esercizio 12.1 ⋆ Un consenso ha `valid-after` alle 10:00, `fresh-until` alle 11:00 e `valid-until` alle 13:00. In quale intervallo il client cercherà il successivo? Che cosa succede se fino alle 13:00 non ne trova uno nuovo?

Esercizio 12.2 ⋆ Un relay con 500 KB/s di banda, in rete da due giorni e con la exit policy `accept *:443, reject *:*`, quali flag può ottenere? Perché non `Exit`?

Esercizio 12.3 ⋆⋆ Descrivi un attacco di partizionamento in cui una directory cache malevola invia a un client specifico un consenso vecchio ma ancora valido. Che cosa limita l'attacco? Perché chiedere i microdescrittori per impronta aiuta?

Esercizio 12.4 ⋆⋆ Tre autorità misurano la banda di un relay: 5000, 5200 e 90000. Quale valore finisce nel consenso? E se la terza autorità, complice del relay, fosse l'unica a misurarla?

Esercizio 12.5 ⋆⋆ Nella directory di un'autorità della rete di laboratorio si trova il file `v3-status-votes`. Confronta il voto di un'autorità con il consenso: quali righe differiscono, e perché?

Esercizio 12.6 ⋆⋆ Scegli un relay dal consenso scaricato nel laboratorio, calcola a mano (con `sha256sum`) l'impronta del suo microdescrittore e confrontala con la riga `m`. Da quale riga inizia e dove finisce il testo su cui si calcola?

Esercizio 12.7 ⋆⋆⋆ Ferma una delle tre autorità della rete di laboratorio e scarica di nuovo il consenso dopo un minuto. Quante firme contiene? Il tuo verificatore lo accetta? Ripeti fermandone due, e spiega che cosa osservi alla scadenza del consenso.

Esercizio 12.8 ⋆⋆⋆ Scarica anche il consenso di variante `ns` (`/tor/status-vote/current/consensus`) e verificane le firme. Quali algoritmi di hash usano? Che cosa contiene la riga `r` in più rispetto alla variante microdesc?

Per approfondire

- *Tor directory protocol, version 3* (dir-spec) [Thef]: formati, voto, assegnazione dei flag, operazioni di client e cache.
- Il documento sul protocollo di casualità condivisa (srv-spec), usato dalle autorità per produrre ogni giorno un valore casuale comune, di cui avremo bisogno per gli onion service (capitolo 14).
- Tor Metrics e CollecTor, `https://metrics.torproject.org/`: archivi storici di consensi, voti e descrittori.

Capitolo 13

Selezione del percorso, guard e vanguards

Aggiornato al 23 settembre 2026 — torspec commit `8e1f245`; tor 0.4.9.13; consenso pubblico del 23/09/2026 ore 15:00 UTC

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- descrivere in quale ordine e con quali vincoli tor sceglie i relay di un circuito;
- spiegare come la banda e i *bandwidth-weights* del consenso bilanciano il carico tra le posizioni;
- motivare l'uso dei guard e ricostruirne la storia, da tre guard a uno e poi a due;
- descrivere l'algoritmo dei guard: campione, guard filtrati, confermati e primari;
- spiegare che cosa sono i vanguards e perché servono agli onion service;
- simulare la selezione dei percorsi sul consenso reale della rete pubblica (Laboratorio 7).

Il sistema delle directory (capitolo 12) dà al client l'elenco dei relay. Resta la domanda che determina, più di ogni altra, quanto un utente sia protetto: *quali* relay usare. Se un avversario controlla il primo e l'ultimo relay di un circuito, può correlare il traffico in entrata e in uscita e scoprire chi parla con chi (capitolo 26). La selezione del percorso è quindi, in buona parte, l'arte di rendere questo evento il più raro possibile senza sacrificare le prestazioni.

13.1 L'ordine e i vincoli

Per un circuito di tre salti, tor sceglie **prima l'exit**, poi il guard, infine il middle [path-spec § Path selection and constraints]. L'exit viene prima perché è il relay più vincolato: deve accettare la destinazione richiesta, secondo la sua exit policy. Se il client conosce solo il nome della destinazione e non l'indirizzo, sceglie un exit che *potrebbe* accettarla, cioè la cui policy non respinge quella porta prima di accettarla.

Tutti i percorsi, con poche eccezioni, rispettano questi vincoli:

- nessun relay senza il flag `Fast`;
- lo stesso relay non compare due volte;
- al massimo un relay per ciascuna rete /16 IPv4 (o /32 IPv6): due relay nello stesso centro dati sono probabilmente sotto lo stesso controllo;

- al massimo un relay per **famiglia**: gli operatori con più relay devono dichiararli come famiglia, e tor evita di usarne due nello stesso circuito;
- il primo relay è un guard (sezione 13.3);
- nessun exit con il flag `BadExit`.

Per le connessioni che devono restare aperte a lungo (tor considera tali alcune porte, come la 22 di SSH o la 6667 di IRC) servono anche relay con il flag `Stable`. Gli onion service aggiungono un vincolo indipendente dalla porta: i circuiti di introduzione lato servizio e i percorsi di rendezvous sono costruiti come circuiti `Stable`, e i loro endpoint non devono essere `MiddleOnly` [path-spec § Path selection and constraints]. Le «poche eccezioni» sono soprattutto i circuiti degli onion service costruiti con i vanguards (sezione 13.4), che rilassano di proposito alcuni di questi vincoli (per esempio ammettono un guard in una posizione intermedia): impedire quelle combinazioni rivelerebbe all'avversario quali relay *non* sono stati scelti, dandogli informazione sulla selezione del guard.

Approfondimento: Due modi di dichiarare una famiglia

Tradizionalmente un operatore dichiara la famiglia elencando, nel descrittore di ogni relay, tutti gli altri; due relay sono della stessa famiglia solo se si elencano a vicenda. Con molti relay le liste diventano enormi e basta un errore di configurazione per romperle. Il meccanismo più recente, le *happy families*, dà a ogni famiglia una chiave: ciascun relay dimostra l'appartenenza con un certificato firmato da quella chiave (`family-cert` nel descrittore, `family-ids` nel microdescrittore). I parametri di consenso `use-family-lists` e `use-family-ids` stabiliscono quale dei due meccanismi i client applicano.

13.2 Pesare con la banda

13.2.1 Perché pesare

Se i client scegliessero i relay con probabilità uniforme, un relay da 1 MB/s riceverebbe tanto traffico quanto uno da 1 GB/s: il primo sarebbe sommerso, il secondo inutilizzato. Tor sceglie invece ogni relay con probabilità **proporzionale alla sua banda** nel consenso (la riga `w`, sezione 12.5), così che il carico si distribuisca secondo la capacità.

13.2.2 Le posizioni non sono uguali

C'è una complicazione. I relay con il flag `Exit` sono relativamente pochi e il loro traffico in uscita è la risorsa più scarsa della rete: se un exit venisse usato anche come guard o come middle, consumerebbe per quelle posizioni una capacità che serve all'uscita. Lo stesso vale, in misura minore, per i guard. Per questo le autorità calcolano, a ogni consenso, i **bandwidth-weights**: coefficienti che moltiplicano la banda di un relay a seconda del suo *tipo* (solo guard, solo exit, guard ed exit, nessuno dei due) e della *posizione* per cui viene scelto [path-spec § Weighting node selection].

Il nome di ogni peso ha la forma `W`*pt*: *p* è la posizione (`g` guard, `m` middle, `e` exit, `b` richieste di directory) e *t* il tipo (`g`, `e`, `d` per guard ed exit, `m` per nessuno dei due). I valori sono in diecimillesimi: `Wmg=3805` significa che un guard, quando viene scelto come middle, conta per il 38,05 % della sua banda.

13.2.3 I pesi della rete reale

Ecco i *pesi effettivamente utilizzabili per ciascuna posizione* nel consenso pubblico del 23 settembre 2026, ore 15:00 UTC, scaricato da CollecTor nel Laboratorio 7. Il trattino significa che la categoria *non è idonea a quella posizione*, anche se il consenso contiene un coefficiente numerico per essa: non va interpretato come un peso zero.

Posizione \ tipo	guard (`g`)	middle (`m`)	exit (`e`)	guard+exit (`d`)
guard (`Wg`)	6195	—	—	0
middle (`Wm`)	3805	10000	0	0
exit (`We`)	—	—	10000	10000

La distinzione fra *idoneità* e *peso* è essenziale: un relay senza flag `Guard` non può essere scelto come primo salto e uno senza flag `Exit` non può essere scelto per l'uscita di un normale circuito TCP, indipendentemente dai coefficienti `Wgm`, `Weg` e `Wem` riportati nel consenso. In questa rete gli exit, e i relay che sono sia guard sia exit, hanno peso **zero** come middle; quelli che sono sia guard sia exit hanno peso zero anche come guard. La banda di uscita (il 33 % del totale) viene quindi riservata all'uscita. I guard, che rappresentano il 70 % della banda, sono in abbondanza: i loro pesi per le posizioni guard e middle sono rispettivamente 6195 e 3805 su 10000; *non* sono percentuali delle scelte effettive dei client, che dipendono anche dagli altri relay e dai vincoli. Come vedremo nel laboratorio, il risultato è che nel 60 % dei circuiti il middle è a sua volta un relay con il flag `Guard`.

Le formule con cui le autorità calcolano i pesi sono nella specifica delle directory; distinguono diversi casi a seconda di quale tipo di banda sia scarso, e cercano di rendere uguali, dove possibile, le capacità disponibili nelle tre posizioni.

13.3 I guard

13.3.1 Il problema che risolvono

Supponiamo che un avversario controlli una frazione c della banda dei possibili primi relay e dei possibili exit. Se il client scegliesse guard ed exit a caso per ogni circuito, ciascun circuito sarebbe compromesso (guard ed exit dell'avversario) con probabilità circa c^2, e dopo C circuiti la probabilità che almeno uno lo sia sarebbe $1 - (1 - c^2)^C$, che tende a 1 [guard-spec § Introduction and motivation]. Un utente che usa Tor ogni giorno costruisce migliaia di circuiti: prima o poi, con quasi certezza, l'avversario vedrebbe una parte del suo traffico.

L'idea dei **guard** (chiamati *helper node* nella letteratura scientifica) è scegliere il primo relay *una volta*, e poi usarlo a lungo. Se quel guard è onesto, nessun circuito è compromesso in quel modo; se è dell'avversario, lo sono molti. La probabilità di essere compromessi *almeno una volta* scende da quasi 1 a circa c.

Circuiti	Senza guard	Con guard fisso
1	0,0025	0,0025
100	0,221	0,050
1000	0,918	0,050
10 000	1,000	0,050

La tabella, calcolata nel Laboratorio 7 per $c = 5\,\%$, mostra il compromesso: i guard non rendono impossibile la compromissione, ma trasformano un evento quasi certo nel lungo periodo in un evento raro, che colpisce una piccola frazione degli utenti invece di colpirli

prima o poi tutti. Il modello è volutamente semplificato: ignora, per esempio, che il guard scelto può cambiare nel tempo.

13.3.2 Tre, uno, due

I guard sono stati introdotti nel 2006, dopo che Øverlier e Syverson avevano mostrato come localizzare un onion service facendolo connettere ripetutamente a relay dell'avversario [ØS06]. Per anni ogni client ne ha usati tre, sostituiti dopo due o tre mesi. Studi successivi hanno mostrato che tre guard e rotazioni frequenti aumentano l'esposizione: la Proposta 236 (2014) ha proposto di passare a un solo guard, conservato per nove o dieci mesi. Un solo guard, però, rende più fragile la connettività e più facile un attacco che costringa il client a cambiarlo. Con la Proposta 291 (2018) si è passati a **due** guard. Nel codice di tor il valore predefinito è ancora uno (`DFLT_N_PRIMARY_GUARDS_TO_USE` in `src/feature/client/entrynodes.h`), ma il consenso pubblico contiene il parametro `guard-n-primary-guards-to-use=2`, che lo sostituisce per tutti i client. È un esempio concreto di come le autorità regolino la rete attraverso il consenso.

13.3.3 L'algoritmo

L'algoritmo attuale, descritto nella Proposta 271 e nella specifica dei guard, gestisce diversi insiemi [guard-spec § The algorithm]:

Campione (***sampled guards***) Un insieme di guard scelti a caso, pesati per banda, conservato su disco. Cresce solo quando serve, fino a un massimo di 60 relay o al 20 % del numero dei guard presenti nel consenso. Limitare il campione impedisce a un avversario di spingere il client a provare, un guard dopo l'altro, tutti i guard della rete finché non ne trova uno suo.

Filtrati (***filtered guards***) I guard del campione ancora presenti nel consenso e utilizzabili con la configurazione attuale (per esempio con un firewall che permette solo certe porte).

Confermati (***confirmed guards***) I guard effettivamente usati per traffico dell'utente, in ordine di preferenza.

Primari (***primary guards***) Di norma tre guard scelti dai filtrati: prima quelli anche confermati, in ordine di preferenza, poi se necessario altri guard filtrati non ancora confermati. Il client usa il primo primario raggiungibile (o i primi due, con il parametro di consenso visto sopra); se nessuno è raggiungibile, prova gli altri, ma torna ai primari appena possibile.

La permanenza nel campione dipende dallo stato del guard. Un guard **mai confermato** (`CONFIRMED_ON_DATE` non impostata) esce dopo `GUARD_LIFETIME`, cioè 120 giorni dal campionamento; un guard **confermato**, quello che il client usa davvero, esce invece `GUARD_CONFIRMED_MIN_LIFETIME` = 60 giorni dopo la conferma. In più, esce se per 20 giorni non compare nel consenso. Le date memorizzate (`ADDED_ON_DATE` e `CONFIRMED_ON_DATE`) vengono spostate a caso nel passato, fino a `GUARD_LIFETIME`/10 = 12 giorni: il file di stato non rivela con precisione quando il client ha iniziato a usare Tor, e la durata effettiva di un guard confermato scende a 48–60 giorni. È quindi il guard *confermato* a ruotare ogni due mesi circa, non ogni quattro: un dettaglio che conta per la simulazione del capitolo 29.

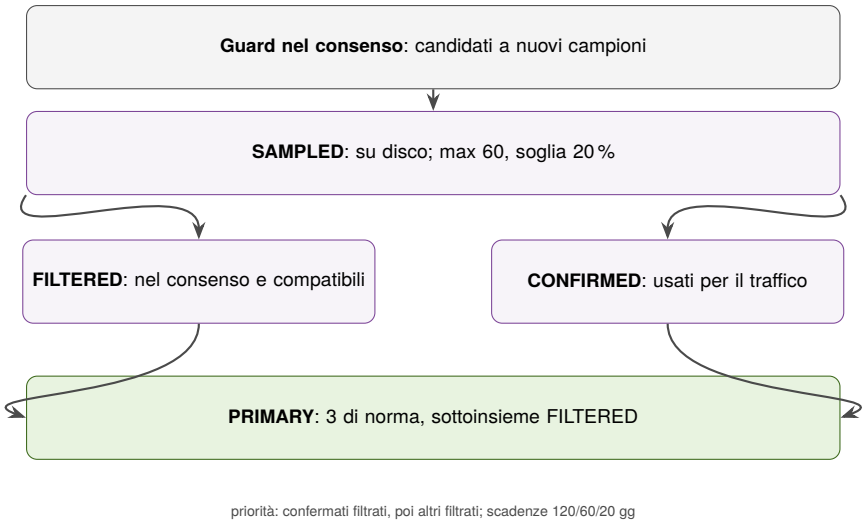


Figura 13.1: Relazioni fra le liste dell'algoritmo dei guard: il campione persistente deriva da guard del consenso, mentre FILTERED e CONFIRMED sono due suoi sottoinsiemi distinti, che possono sovrapporsi. I PRIMARY sono selezionati fra i FILTERED, dando priorità ai confermati utilizzabili e completando con altri filtrati. I limiti del campione riducono l'esposizione a guard nuovi.

Attenzione: Un nuovo guard è un indizio

Tor distingue con cura tra un guard *irraggiungibile* e un circuito *fallito*. Se il client cambiasse guard ogni volta che un circuito fallisce, un avversario potrebbe far fallire di proposito i circuiti che passano per guard onesti finché il client non ne sceglie uno controllato da lui. Proprio su questo punto è intervenuta la versione di sicurezza 0.4.9.13 di tor, uscita il 23 settembre 2026: il client ora attribuisce un fallimento al guard solo quando non riesce a stabilire la connessione, e non più per circuiti, stream o richieste di directory falliti.

13.4 Vanguards

I guard proteggono il *primo* salto. Gli onion service (capitolo 14) hanno però un problema specifico: un client malevolo può far costruire a un servizio quanti circuiti vuole, verso punti di rendezvous scelti da lui. Se il middle di quei circuiti fosse scelto ogni volta a caso, prima o poi sarebbe un relay dell'avversario, che vedrebbe così il guard del servizio: il passo successivo sarebbe attaccare quel guard, o ottenerne i dati. I **vanguards** estendono l'idea dei guard ai salti successivi, fissando anche i relay del secondo (e in alcune configurazioni del terzo) livello.

Vanguards-lite (Proposta 333, tor 0.4.7.1-alpha) È attivo per impostazione predefinita nei client e negli onion service. Usa 4 relay di secondo livello, non salvati su disco, ciascuno con una durata casuale. Qui codice e specifica non coincidono: tor 0.4.9.13 usa un intervallo tra 1 e 12 giorni (in media circa una settimana), fissato dalle costanti `MIN_SECOND_GUARD_LIFETIME` e `MAX_SECOND_GUARD_LIFETIME` in `src/feature/client/entrynodes.c` e sovrascrivibile dai parametri di consenso `guard-hs-l2-lifetime-min` e `-max`, che nel consenso del 23 settembre 2026 non compaiono; la specifica dei vanguards indica invece ancora 1–22 giorni (in media due settimane). È pensato per servizi e utenti che restano attivi al massimo per qualche settimana.

Vanguards completi Relay fissati su due livelli (secondo e terzo salto), con durate più lunghe, pensati per onion service di lunga durata ad alto rischio. In tor si usano tramite il componente aggiuntivo *vanguards*, che imposta le opzioni `HSLayer2Nodes` e `HSLayer3Nodes`; Arti li implementa direttamente, con una modalità `Full` accanto a quella `Lite`.

Nella rete di laboratorio (sezione 19.7.3) abbiamo visto circuiti con lo scopo `HS_VANGUARDS`: sono circuiti preparati in anticipo con i vanguards, pronti a essere estesi verso un punto di introduzione o di rendezvous.

13.5 Laboratorio: la rete pubblica sotto la lente

Laboratorio 7: Selezione pesata dei percorsi

Obiettivo. Scrivere `labs/it/codice/torlab/percorso.py` e usarlo per analizzare un consenso reale della rete pubblica.
Prerequisiti. Laboratorio 6 (`directory.py`). Un consenso pubblico scaricato da CollecTor, l'archivio pubblico del Tor Project (`https://collector.torproject.org/`): sono dati pubblicati apposta per la ricerca, e scaricarli non coinvolge alcun utente della rete. Il comando `make -C labs/it data` salva in `labs/it/dati/collector/` quello usato nel capitolo; ne controlla l'intestazione, non le firme delle autorità, che si verificano con il Laboratorio 6.
Tempo stimato. 2 ore.

Passo 1: il peso di un relay.

```
def _tipo(r):   # g = solo Guard, e = solo Exit, d = entrambi, m = nessuno
    g, e = "Guard" in r.flag, "Exit" in r.flag and "BadExit" not in r.flag
    return {(1, 1): "d", (1, 0): "g", (0, 1): "e", (0, 0): "m"}[(g, e)]

def peso(r, posizione, pesi):
    chiave = f"W{posizione[0]}{_tipo(r)}"          # es. "Wmg"
    return (r.banda or 0) * pesi.get(chiave, 10000) / 10000
```

Passo 2: scegliere un percorso. La classe `Selettore` calcola i pesi una volta per posizione; i vincoli si applicano *per rifiuto*: si estrae un relay e, se viola un vincolo (stesso relay, stessa /16), se ne estrae un altro.

```
def percorso(self, rng, guard=None):
    e = self.scegli("exit", [], rng)
    g = guard or self.scegli("guard", [e], rng)
    m = self.scegli("middle", [e, g], rng)
    return g, m, e
```

Attenzione: Selezione semplificata, non client completo

Il modulo `percorso.py` è un simulatore statistico per studiare i *bandwidth-weights*, non una riproduzione fedele della selezione usata da un client tor. Senza passare esplicitamente il parametro `guard`, sceglie un nuovo guard a ogni percorso anziché gestirne un campione persistente; non legge le famiglie dai microdescrittori, non applica il vincolo di sottorete IPv6 e non verifica che l'exit accetti una specifica destinazione. Gestisce identità distinte e sottoreti /16 IPv4, mentre l'algoritmo vero applica anche i vincoli descritti in questo

capitolo. La simulazione serve a studiare pesi e concentrazione, non a costruire circuiti sicuri sulla rete pubblica.

Una prima versione, che a ogni scelta filtrava gli oltre novemila relay ricalcolandone pesi e reti, impiegava 310 secondi per 2000 percorsi; questa ne costruisce 20 000 in 7 secondi. In una simulazione statistica le prestazioni non sono un dettaglio.

Passo 3: la rete pubblica in numeri. Sul consenso delle 15:00 UTC del 23 settembre 2026:

```
relay: 9358
banda con Guard: 70.1%    banda con Exit: 33.1%
guard : idonei  5306; primi 10 = 1.7%; primi 100 = 12.6%
middle: idonei  8961; primi 10 = 2.8%; primi 100 = 13.7%
exit  : idonei  3126; primi 10 = 2.3%; primi 100 = 13.7%
20000 percorsi: gge 30.9%  ggd 29.4%  gme 20.4%  gmd 19.3%
```

Gli *idonei* sono meno dei relay che portano il flag corrispondente (nel capitolo 6 gli `Exit` sono 3177, qui 3126): la funzione `idoneo` pretende anche `Running`, `Valid` e `Fast`, ed esclude `BadExit` e `MiddleOnly`. La differenza sono 48 exit senza `Fast` e 3 marcati `BadExit`. L'ultima riga è un campione, ottenuto fissando il seme del generatore casuale a 7: senza fissarlo, i valori cambiano di qualche decimo di punto a ogni esecuzione e due prove non sono confrontabili. Le sigle indicano il tipo di guard, middle ed exit di ciascun percorso: per esempio `gme` è un percorso con un guard, un middle senza flag e un exit puro. Nessun percorso usa un exit in prima o seconda posizione, come previsto dai pesi; nel 60 % dei casi il middle è un relay con il flag `Guard`. La concentrazione è bassa: i dieci exit più scelti coprono insieme il 2,3 % delle scelte, i cento più scelti il 13,7 %.

Passo 4: guard e avversari. La classe `Avversario` calcola la tabella della sezione 13.3. Prova a cambiare la quota dell'avversario: con $c = 1\,\%$ o $c = 20\,\%$ l'effetto dei guard resta evidente?

Riepilogo

- tor sceglie prima l'exit, poi il guard, poi il middle, evitando relay ripetuti, della stessa /16 o della stessa famiglia.
- Ogni relay è scelto con probabilità proporzionale alla sua banda, moltiplicata per un peso che dipende dal tipo di relay e dalla posizione; nella rete attuale gli exit sono usati solo come exit.
- I guard trasformano la compromissione, quasi certa nel lungo periodo se si scelgono relay a caso, in un evento raro; oggi i client ne usano due.
- L'algoritmo dei guard mantiene un campione limitato, i guard confermati e i guard primari, e attribuisce i fallimenti ai guard con cautela.
- I vanguards fissano anche il secondo livello per gli onion service; vanguards-lite è attivo per impostazione predefinita.
- I parametri del consenso possono sostituire i valori predefiniti del software.

Esercizi

Esercizio 13.1 ⋆ Perché tor sceglie l'exit per primo? Che cosa potrebbe andare storto scegliendo prima il guard?

Esercizio 13.2 ⋆ Un relay con i flag `Guard` ed `Exit` e banda 20000 nel consenso: con i pesi della rete reale riportati nel capitolo, quanto pesa nelle tre posizioni?

Esercizio 13.3 ⋆⋆ Ripeti il calcolo della tabella dei guard per $c = 1\,\%$ e $c = 20\,\%$. Per quale numero di circuiti la probabilità di compromissione senza guard supera il 50 % nei due casi?

Esercizio 13.4 ⋆⋆ Un avversario vuole aumentare la probabilità che i client scelgano i suoi relay come exit. Quali risorse deve investire, e quali meccanismi (misura della banda, famiglie, limite per indirizzo, pesi) glielo rendono più difficile?

Esercizio 13.5 ⋆⋆ Perché il campione dei guard è limitato a 60 relay o al 20 % del numero dei guard? Descrivi l'attacco che questo limite contrasta.

Esercizio 13.6 ⋆⋆⋆ Estendi `Selettore` con il vincolo delle famiglie, usando le righe `family` e `family-ids` dei microdescrittori. Scarica da CollecTor i microdescrittori corrispondenti al consenso e misura quante estrazioni vengono rifiutate a causa delle famiglie.

Esercizio 13.7 ⋆⋆⋆ Raggruppa i relay del consenso pubblico per famiglia dichiarata e calcola la quota di probabilità di uscita delle dieci famiglie più grandi. Confronta con la quota dei dieci singoli exit più scelti. Che cosa ne concludi sul rischio legato agli operatori con molti relay?

Per approfondire

- *Tor Path Specification* e *Tor Guard Specification* [Thef].
- L. Øverlier, P. Syverson, *Locating Hidden Servers* [ØS06]: l'attacco che ha motivato i guard.
- Proposte 236, 271, 291 e 333: da un guard a due, l'algoritmo attuale e vanguards-lite.
- La specifica dei vanguards (vanguards-spec), con le tabelle sui tempi degli attacchi Sybil.

Capitolo 14

Onion services v3

Aggiornato al 23 settembre 2026 — torspec commit `8e1f245`; tor 0.4.9.13; rend-spec v3

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- spiegare che cosa offre un onion service e in che cosa differisce da un sito raggiunto attraverso un exit;
- ricavare la chiave pubblica di un servizio dal suo indirizzo `.onion` e viceversa;
- descrivere le chiavi accecate, i periodi di tempo e il modo in cui un client trova gli HSDir responsabili;
- leggere la struttura a strati di un descrittore;
- seguire passo per passo il protocollo di introduzione e di rendezvous;
- scaricare, verificare e decifrare il descrittore di un onion service della rete di laboratorio (Laboratorio 8).

Fin qui abbiamo usato Tor per nascondere *chi* si collega a un server. Il server però era pubblico: il suo indirizzo IP era noto e l'exit si collegava direttamente a lui sulla rete pubblica, vedendone la destinazione (il contenuto era in chiaro solo se l'applicazione non usava una cifratura da estremo a estremo come HTTPS). Gli **onion service** ribaltano la prospettiva: permettono di offrire un servizio, per esempio un sito web, senza rivelarne la posizione nella rete. Il client e il servizio si incontrano a metà strada, ciascuno protetto dal proprio circuito, e nessuno dei due conosce l'indirizzo dell'altro.

14.1 Che cosa offre un onion service

Anonimato del servizio L'indirizzo IP del server non compare mai: il servizio parla con la rete solo attraverso circuiti Tor che partono da lui.

Indirizzi autenticanti L'indirizzo `.onion` *codifica* la chiave pubblica del servizio insieme a una versione e a un checksum. Chi si collega a un indirizzo sa crittograficamente di parlare con chi possiede la chiave privata corrispondente, senza bisogno di autorità di certificazione.

Cifratura da estremo a estremo Il traffico tra client e servizio è cifrato con chiavi che solo loro conoscono: non c'è un exit che veda il traffico in chiaro.

Raggiungibilità Il servizio non deve accettare connessioni in ingresso: funziona anche dietro un NAT o un firewall che permettono solo connessioni in uscita.

Il prezzo è la lunghezza del percorso. Nel modello base il client raggiunge un punto d'incontro con un circuito di tre salti e il servizio con un altro circuito di tre salti: sei relay in tutto, contro i tre di una connessione verso un sito normale. Con i vanguards-lite, attivi per impostazione predefinita (sezione 13.4), il percorso non è più simmetrico: il servizio inserisce un relay di secondo livello fisso e il suo lato del circuito di rendezvous diventa di quattro salti ($\texttt{S} \to \texttt{G} \to \texttt{L2} \to \texttt{M} \to \texttt{Rend}$) contro i tre del client.

Nota storica: Da v2 a v3

I *hidden service* esistono dal 2004. La prima versione del protocollo usata a lungo, detta v2, aveva indirizzi di 16 caratteri, derivati da una chiave RSA a 1024 bit troncando un'impronta SHA-1 a 80 bit, e permetteva agli HSDir di raccogliere gli indirizzi dei servizi che ospitavano. La versione 3 (Proposta 224), disponibile da tor 0.3.2.1-alpha (2017), ha introdotto chiavi Ed25519, indirizzi di 56 caratteri che contengono l'intera chiave pubblica, descrittori cifrati e chiavi accecate. I servizi v2 sono stati disattivati del tutto nell'ottobre 2021, con tor 0.4.5.11 e 0.4.6.8. Nel frattempo è cambiato anche il nome: oggi si parla di *onion service*.

14.2 L'indirizzo

Un indirizzo v3 codifica in base32 la chiave pubblica di identità del servizio, un piccolo checksum e il numero di versione [rend-spec § Encoding onion addresses]:

$$\texttt{indirizzo} = \text{base32}(\mathit{PUBKEY} \,\|\, \mathit{CHECKSUM} \,\|\, \mathit{VERSION}) \,\|\, \texttt{".onion"}$$

dove *PUBKEY* è la chiave Ed25519 di 32 byte, *VERSION* è il byte `0x03` e *CHECKSUM* sono i primi 2 byte di SHA3-256(`".onion checksum"` ‖ *PUBKEY* ‖ *VERSION*). I 35 byte diventano esattamente 56 caratteri base32. Il checksum non protegge da un avversario, che può calcolarlo come chiunque, ma intercetta gli errori di battitura.

L'onion service della rete di laboratorio aveva, in una delle nostre esecuzioni, l'indirizzo che segue. È quello dell'esempio conservato in `labs/it/codice/torlab/test/dati/`, su cui girano i test di questo laboratorio: gli output di questa sezione si riferiscono tutti a esso, e sono quindi riproducibili anche senza la rete. La rete che il lettore avvia genera chiavi nuove, e quindi un indirizzo diverso (sezione 19.6).

```
wdjtp7qktdjkkgjjbbx2axvg2gri5p4tuhlc4bq23n7o3uwu4rveppqd.onion
```

da cui la funzione `chiave_da_indirizzo` del Laboratorio 8 ricava la chiave pubblica, verificandone il checksum.

Attenzione: Un indirizzo non si ricorda

Un indirizzo di 56 caratteri casuali non si può memorizzare né riconoscere a colpo d'occhio: gli utenti lo copiano da una fonte che ritengono affidabile, e un indirizzo che differisce di pochi caratteri può passare inosservato. È il principale vettore di *phishing* contro gli onion service. Gli *indirizzi vanity*, generati provando milioni di chiavi finché i primi caratteri non formano una parola, aiutano solo in parte, perché anche l'avversario può generarne uno con lo stesso prefisso. Ne riparleremo nel capitolo 21.

14.3 Le chiavi accecate

14.3.1 Il problema

Il servizio deve pubblicare da qualche parte le informazioni per raggiungerlo: il *descrittore*. Se lo pubblicasse sotto la propria chiave di identità, chiunque gestisse un HSDir vedrebbe passare gli indirizzi di tutti i servizi che ospita, e potrebbe elencarli o collegare nel tempo i descrittori dello stesso servizio. In v2 era esattamente così.

14.3.2 La soluzione

In v3 il servizio firma i descrittori con una **chiave accecata** (*blinded key*), diversa per ogni *periodo di tempo* [rend-spec § Appendix A: Signature scheme with key blinding]. Data la chiave di identità $A = aB$ (B è il punto base di Ed25519), si calcola un fattore

$$h = \text{SHA3-256}(\texttt{"Derive temporary signing key\0"} \,\|\, A \,\|\, B \,\|\, N),$$
$$N = \texttt{"key-blind"} \,\|\, \textit{periodo} \,\|\, \textit{durata}$$

(con alcuni bit forzati, come si fa per le chiavi Ed25519), e la chiave accecata è

$$A' = h \cdot A.$$

La costruzione ha proprietà preziose:

- chi conosce l'indirizzo, e quindi A, può calcolare A' per ogni periodo, e sa dove cercare il descrittore;
- chi vede soltanto A', come un HSDir, non può risalire ad A, né capire che due chiavi accecate di periodi diversi appartengono allo stesso servizio;
- il servizio può firmare con A' usando la chiave privata $a' = h \cdot a$, e la firma si verifica con A' come una normale firma Ed25519.

Un dettaglio curioso mostra quanto la specifica segua da vicino il codice: la stringa costante include un byte zero finale, perché nel codice di tor la sua lunghezza è calcolata con `sizeof` invece che con `strlen`. Un commento nel sorgente (`src/feature/hs/hs_common.c`) ammette che si è preferito documentare la «sorpresa» nella specifica piuttosto che cambiarla nel codice.

14.3.3 Credenziali e chiavi offline

Dalla chiave di identità si ricava la *credenziale* SHA3-256(`"credential"` $\|\, A$), e da questa, per ogni periodo, la *sottocredenziale* SHA3-256(`"subcredential"` $\|$ *credenziale* $\|\, A'$). La sottocredenziale entra nella cifratura del descrittore e nell'handshake di introduzione: dimostra di conoscere l'indirizzo, cosa che un HSDir non può fare.

La catena delle chiavi è pensata anche per permettere di tenere *offline* la chiave di identità. La chiave accecata di ciascun periodo certifica una chiave di firma dei descrittori a breve termine (un certificato Ed25519 di tipo 8, tabella 7.1); in linea di principio il server può ricevere in anticipo, per i periodi futuri, solo i certificati e le chiavi di firma, senza mai possedere la chiave che corrisponde all'indirizzo. Non tutte le implementazioni offrono questa modalità.

14.4 Dove si trova il descrittore

14.4.1 I periodi di tempo

Il tempo è diviso in periodi, di un giorno nella rete pubblica (parametro di consenso `hsdir-interval`) [rend-spec § Dividing time into periods]. I periodi non iniziano a mezzanotte, ma a mezzogiorno UTC: sono sfasati di 12 ore rispetto al protocollo di casualità condivisa delle autorità, che produce ogni giorno a mezzanotte un nuovo **valore casuale condiviso** (SRV) pubblicato nel consenso. Il numero del periodo si calcola dai minuti trascorsi dall'1 gennaio 1970, meno lo sfasamento, diviso per la durata. La specifica fa un esempio: alle 11:15:01 UTC del 13 aprile 2016 si è nel periodo 16903, iniziato alle 12:00 del giorno prima. I test del Laboratorio 8 verificano proprio questo esempio.

Nelle reti di test, dove si vota ogni 20 secondi, aspettare un giorno sarebbe scomodo: tor riduce allora il periodo alla durata di un intero giro del protocollo SRV, 24 voti (funzione `get_time_period_length`). Nel laboratorio un periodo dura 8 minuti ed è sfasato di 4.

14.4.2 L'anello degli HSDir

In quale HSDir si trova il descrittore? Ogni relay con il flag `HSDir` riceve una posizione su un *anello* di valori a 256 bit [rend-spec § Where to publish a hidden service descriptor]:

$$\textit{indice_relay} = \text{SHA3-256}(\texttt{"node-idx"} \,\|\, \textit{identità Ed25519} \,\|\, \textit{SRV} \,\|\, \text{INT}_8(\textit{periodo}) \,\|\, \text{INT}_8(\textit{durata}))$$

e il servizio, per ciascuna di due *repliche*, un indice

$$\textit{indice_servizio}(r) = \text{SHA3-256}(\texttt{"store-at-idx"} \,\|\, A' \,\|\, \text{INT}_8(r) \,\|\, \text{INT}_8(\textit{durata}) \,\|\, \text{INT}_8(\textit{periodo})).$$

Qui $\text{INT}_8(x)$ significa codificare il numero intero x su esattamente otto byte, in ordine di rete (*big-endian*); r è il numero della replica (a partire da 1), *durata* è espressa in minuti e *periodo* è il numero del periodo. Senza questa codifica, le formule descrivono gli indici concettualmente, ma non forniscono gli esatti byte su cui calcolare SHA3-256 [rend-spec § Where to publish a hidden service descriptor].

Il servizio carica il descrittore sui primi 4 HSDir che seguono ciascun indice sull'anello (parametro `hsdir_spread_store`); il client ne sceglie uno a caso tra i primi 3 (`hsdir_spread_fetch`). L'SRV nel calcolo impedisce di prevedere con largo anticipo quali relay saranno responsabili di un servizio, e quindi di posizionare relay malevoli nel punto giusto dell'anello per censurarlo o osservarne le richieste. Il servizio pubblica due descrittori alla volta, per due periodi consecutivi, così da restare raggiungibile anche per i client con un consenso un po' in anticipo o in ritardo. Quali due periodi dipende dal momento nel ciclo: a seconda che ci si trovi prima o dopo la rotazione del valore casuale condiviso, sono il periodo precedente e quello corrente oppure quello corrente e il successivo [rend-spec § Generating and publishing hidden service descriptors].

Approfondimento: Quale SRV usare

Poiché periodi e SRV sono sfasati di mezza giornata, un client deve scegliere con cura quale SRV usare: nella mezza giornata tra l'inizio di un periodo e il successivo SRV usa quello corrente, nell'altra mezza giornata il precedente. Se il consenso non contiene l'SRV richiesto, per esempio subito dopo l'avvio di una rete, tor ne calcola uno di riserva, lo *SRV di emergenza*:

$$\textit{SRV}_{\text{emergenza}} = \text{SHA3-256}(\texttt{"shared-random-disaster"} \,\|\, \text{INT}_8(\textit{durata}) \,\|\, \text{INT}_8(\textit{periodo})).$$

Anche qui *durata* (in minuti) e *periodo* sono interi senza segno codificati su **otto byte big-**

endian ciascuno: non sono stringhe decimali né interi a quattro byte. La scelta è in `hs_get_current_srv` e `hs_get_previous_srv` (`src/feature/hs/hs_common.c`): se il consenso non porta il valore, si passa senza rumore allo SRV di emergenza. È la condizione di una rete di laboratorio appena creata, e dura poco: nei log dell'autorità della nostra rete il primo SRV compare 115 secondi dopo l'avvio, e fino a quel momento il consenso non contiene alcuna riga `shared-rand-current-value`. Chi vuole vederlo deve quindi guardare nei primissimi minuti dopo la creazione della rete; a rete avviata da tempo, come nel sezione 14.9, si usano gli SRV veri.

14.4.3 Solo richieste anonime

Un client scarica il descrittore con una richiesta HTTP `/tor/hs/3/`, seguita dalla chiave accecata in base64 senza i caratteri = finali, su uno stream `BEGIN_DIR` verso l'HSDir. C'è però una condizione: la richiesta deve arrivare su un circuito **anonimo**. Un HSDir che riceve la richiesta su un circuito aperto direttamente da un client, cioè di un solo salto, la rifiuta. Nel laboratorio lo abbiamo scoperto a nostre spese:

```
HTTP/1.0 503 Rejecting single hop HS v3 descriptor request
```

Il controllo è nella funzione `connection_dir_is_anonymous` (`src/feature/dircommon/directory.c`). Impedisce che un HSDir possa associare la richiesta di un descrittore all'indirizzo IP del client che la fa.

14.5 Il descrittore

Il descrittore ha tre livelli [rend-spec § Generating and publishing hidden service descriptors], mostrati nella fig. 14.1.

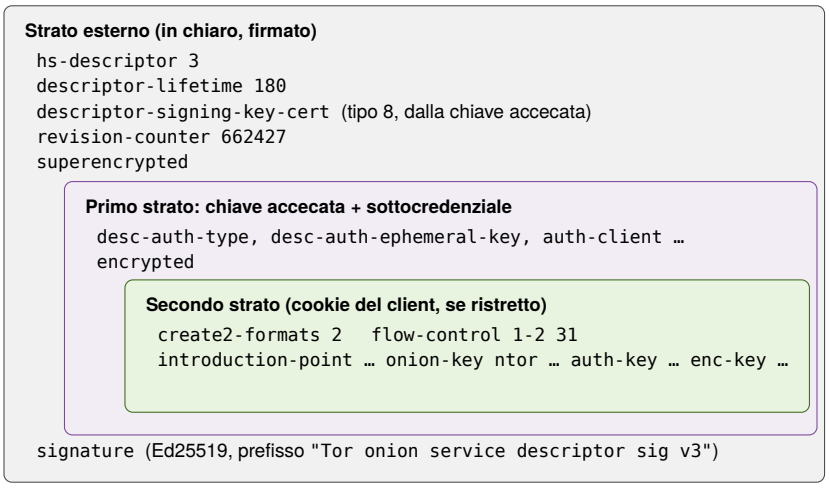


Figura 14.1: La struttura del descrittore di un onion service v3. L'involucro firmato è leggibile dagli HSDir; il primo strato si decifra conoscendo l'identità onion, mentre, nella scoperta ristretta, il client autorizzato usa la propria chiave X25519 per recuperare il descriptor cookie necessario a decifrare il secondo strato.

Lo strato esterno è in chiaro e firmato: versione, durata di validità (`descriptor-lifetime`, 180 minuti nel laboratorio), il certificato della chiave di firma rilasciato dalla chiave accecata,

un contatore di revisione, il blocco cifrato `superencrypted` e la firma. La firma è una firma Ed25519, con la chiave di firma del descrittore, del testo preceduto dalla stringa `"Tor onion service descriptor sig v3"`: il prefisso impedisce che una firma prodotta per un descrittore possa valere per qualcos'altro. Il contatore di revisione deve crescere a ogni nuova versione, così che un HSDir non accetti descrittori più vecchi di quelli che ha.

Il primo strato cifrato è cifrato con una chiave derivata dalla chiave accecata e dalla sotto-credenziale: un HSDir, che non conosce l'indirizzo, non può leggerlo. Contiene i campi della *scoperta ristretta* (sezione 14.7) e il secondo strato. Per non rivelare se la scoperta ristretta è attiva, i campi ci sono sempre, riempiti con dati casuali quando non servono.

Il secondo strato contiene ciò che serve per collegarsi: i formati di handshake supportati, i parametri di controllo di flusso, eventuali parametri anti-DoS e, per ogni **punto di introduzione**, gli specificatori di collegamento per raggiungerlo (in base64), la sua chiave ntor, una chiave di autenticazione certificata e una chiave di cifratura. Un servizio usa per impostazione predefinita 3 punti di introduzione.

Entrambi gli strati usano lo stesso schema: da un sale casuale di 16 byte, dai dati segreti e da una costante diversa per ogni strato si derivano con SHAKE-256 una chiave AES-256, un vettore di inizializzazione e una chiave di autenticazione; il testo è cifrato in modalità contatore e autenticato con SHA3-256. Il testo in chiaro del primo strato viene riempito fino a un multiplo di 10 000 byte, e le righe `auth-client` fino a un multiplo di 16, così che la dimensione del descrittore riveli il meno possibile sul numero di punti di introduzione e di client autorizzati.

14.6 Introduzione e rendezvous

Con il descrittore in mano, il client può collegarsi. Il protocollo, mostrato nella fig. 14.2, coinvolge tre relay «speciali»: il punto di introduzione, scelto dal servizio, il punto di rendezvous, scelto dal client, e gli HSDir.

1. **Il servizio prepara i punti di introduzione.** Sceglie alcuni relay, costruisce un circuito verso ciascuno e invia `ESTABLISH_INTRO` (comando relay 32) con una chiave di autenticazione diversa per ogni punto e una firma. Il relay risponde `INTRO_ESTABLISHED` (38) e da quel momento inoltra al servizio le richieste che portano quella chiave.

2. **Il servizio pubblica il descrittore** sugli HSDir responsabili, come visto sopra.

3. **Il client prepara il rendezvous.** Sceglie a caso un relay come punto di rendezvous, costruisce un circuito verso di esso e invia `ESTABLISH_RENDEZVOUS` (33) con un *cookie* casuale di 20 byte. Il relay risponde `RENDEZVOUS_ESTABLISHED` (39).

4. **Il client si presenta.** Costruisce un circuito verso un punto di introduzione e invia `INTRODUCE1` (34). La parte in chiaro indica la chiave di autenticazione del punto di introduzione; la parte cifrata, leggibile solo dal servizio, contiene il cookie, gli specificatori di collegamento e la chiave ntor del punto di rendezvous, e la prima metà di un handshake. Il punto di introduzione risponde `INTRODUCE_ACK` (40) al client e inoltra il messaggio al servizio come `INTRODUCE2` (35).

5. **Il servizio raggiunge il rendezvous.** Decifra `INTRODUCE2`, controlla che non sia un duplicato di una richiesta già vista (*replay cache*), costruisce un circuito verso il punto di rendezvous indicato e invia `RENDEZVOUS1` (36) con il cookie e la seconda metà dell'handshake.

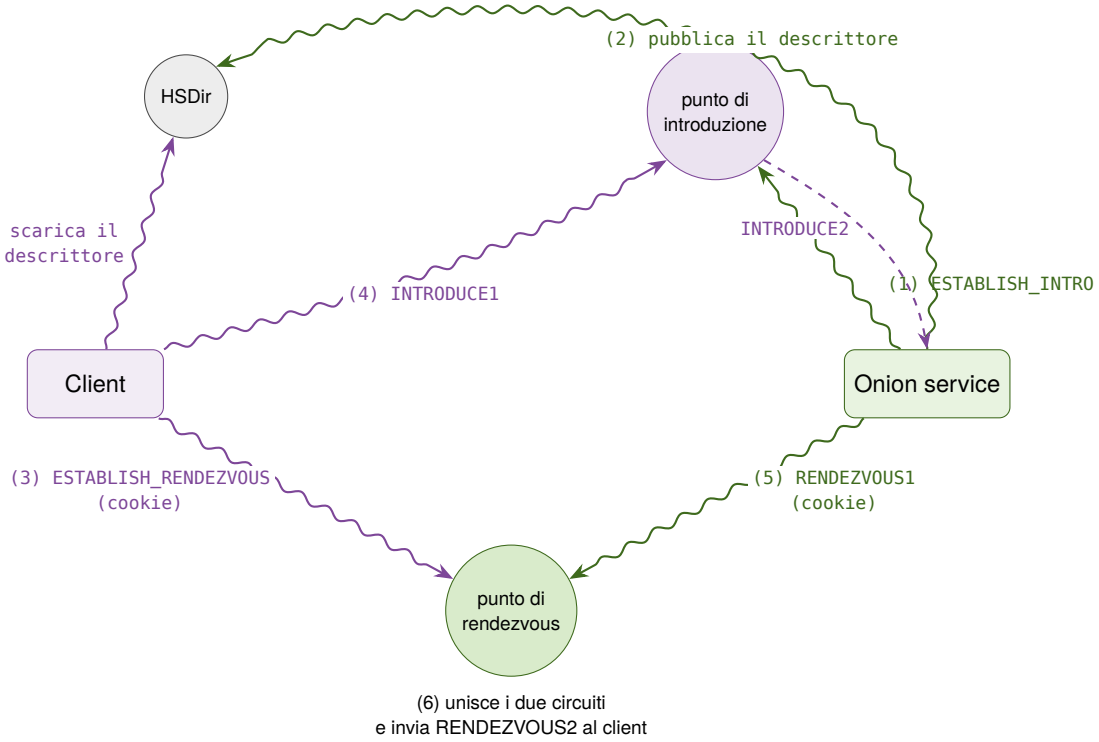


Figura 14.2: Il protocollo di rendezvous. Ogni freccia è un circuito di più salti; nessuna connessione diretta collega client, servizio e punti intermedi. I numeri indicano l'ordine dei passi descritti nel testo.

6. **Il rendezvous unisce i circuiti.** Il punto di rendezvous trova il circuito del client con lo stesso cookie, gli inoltra la seconda metà dell'handshake in un `RENDEZVOUS2` (37), e da quel momento copia le celle da un circuito all'altro.

L'handshake tra client e servizio, chiamato *hs-ntor*, è una variante di ntor (sezione 9.6) che include la sottocredenziale e la chiave di autenticazione del punto di introduzione: solo chi conosce l'indirizzo può produrre un `INTRODUCE1` che il servizio accetti. Le chiavi che ne derivano formano uno strato di cifratura in più, come un *salto virtuale* aggiunto al circuito del client, e usano SHA3-256 per il digest progressivo invece di SHA-1 (sezione 10.2). Né il punto di introduzione né quello di rendezvous possono leggere il traffico.

Sopra il circuito congiunto il client apre stream con `BEGIN`, esattamente come verso un exit (capitolo 11); il servizio li inoltra alle porte locali configurate con `HiddenServicePort`.

Approfondimento: Perché tanti relay

Il punto di introduzione conosce il servizio solo come «la chiave di autenticazione di un circuito», e non sa quale sia l'indirizzo `.onion` per cui lavora. Il punto di rendezvous vede un cookie casuale e due circuiti: non sa quale servizio stia unendo a quale client. Ogni ruolo vede solo un frammento, e nessuno di essi conosce l'indirizzo IP del client o del servizio, che restano dietro ai rispettivi guard.

14.7 Scoperta ristretta

Un servizio può voler essere raggiungibile solo da alcuni client, per esempio un pannello di amministrazione. Con la **scoperta ristretta** (*restricted discovery*, un tempo *client authorization*) ogni client autorizzato ha una coppia di chiavi X25519, e il servizio ne conosce la parte pubblica. Il servizio cifra la chiave del secondo strato del descrittore separatamente per ciascun client, nelle righe `auth-client` del primo strato. Chi conosce l'indirizzo ma non ha una chiave autorizzata non può leggere l'elenco dei punti di introduzione, e quindi non può nemmeno tentare di collegarsi. In tor si configura aggiungendo le chiavi pubbliche dei client nella cartella `authorized_clients` del servizio.

14.8 Difese

Gli onion service sono bersagli naturali di attacchi: un servizio non può semplicemente bloccare gli indirizzi IP di chi lo attacca, perché non li vede. Le principali difese, che approfondiremo altrove, sono:

- i **vanguards**, che proteggono il guard del servizio dagli attacchi basati su circuiti costruiti su richiesta (sezione 13.4);
- la **replay cache**, che impedisce di far ripetere al servizio lo stesso lavoro;
- i parametri anti-DoS che il servizio può impostare nei punti di introduzione con un'estensione di `ESTABLISH_INTRO`, per limitare il numero di `INTRODUCE2` al secondo;
- la **prova di lavoro** (*proof of work*), con cui un servizio sotto attacco chiede ai client di risolvere un piccolo problema computazionale prima di essere serviti (capitolo 15).

14.9 Laboratorio: trovare e leggere un descrittore

Laboratorio 8: Il descrittore dell'onion service di laboratorio

Obiettivo. Scrivere `labs/it/codice/torlab/onion.py`: indirizzo, chiave accecata, anello degli HSDir, download, verifica e decifratura del descrittore.
Prerequisiti. Laboratori 1–6; la rete di laboratorio avviata.
Tempo stimato. 4 ore.

Passo 1: la chiave accecata. Le librerie crittografiche comuni offrono firme Ed25519, ma non la moltiplicazione di un punto per uno scalare, che serve per calcolare $A' = h \cdot A$. Il modulo contiene quindi una piccola implementazione dell'aritmetica di Ed25519, una trentina di righe tratte dalle formule della RFC 8032, sufficiente per questo scopo e da non usare per altro: è lenta e non protegge dagli attacchi basati sui tempi di esecuzione.

```
def fattore_accecante(A, periodo, durata):
    n = b"key-blind" + struct.pack(">QQ", periodo, durata)
    h = bytearray(hashlib.sha3_256(
        b"Derive temporary signing key\x00" + A + BASE_STR + n).digest())
    h[0] &= 248; h[31] &= 63; h[31] |= 64
    return int.from_bytes(h, "little")

def chiave_accecata(A, periodo, durata):
    return _codifica(_moltiplica(fattore_accecante(A, periodo, durata),
                                 _decodifica(A)))
```

Passo 2: il periodo e l'SRV. La funzione `tempo` replica il calcolo di tor a partire dal `valid-after` del consenso e dall'intervallo di voto, e dice se usare l'SRV corrente o il precedente. Se il consenso non contiene quello richiesto, si usa lo SRV di emergenza.

Passo 3: un percorso di ricerca degli errori. La prima esecuzione non ha trovato il descrittore su nessuno degli otto HSDir della rete. Poiché li avevamo provati tutti, la scelta dell'HSDir non poteva essere la causa: restavano la chiave accecata o la richiesta. Per separare le due ipotesi abbiamo chiesto al client vero della rete di scaricare il descrittore, attraverso la porta di controllo (comando `HSFETCH`, poi `GETINFO hs/client/desc/id/...`), e abbiamo letto nel suo certificato la chiave accecata effettivamente usata dal servizio: coincideva con la nostra. Il problema era quindi la richiesta, e il codice di tor ha mostrato il motivo: gli HSDir rifiutano le richieste di un solo salto. Con un circuito di tre salti verso l'HSDir, il descrittore è arrivato. È un metodo che vale in generale: quando un esperimento fallisce, si cerca un punto intermedio in cui confrontare il proprio risultato con quello di un'implementazione di riferimento.

Passo 4: verificare e decifrare. Nel frammento, `percorso` è una lista di tre oggetti `InfoRelay` ricavati dalla rete di laboratorio, con l'HSDir come ultimo relay; `A` è la chiave di identità e `Ap` quella accecata. La preparazione degli oggetti non è mostrata.

```
d = scarica_descrittore(Ap, percorso)
desc = analizza_descrittore(d, A, Ap)
```

`analizza_descrittore` verifica che il certificato della chiave di firma sia emesso dalla chiave accecata attesa, verifica la firma del descrittore e decifra i due strati. **Limite del laboratorio:** per poter esaminare il descrittore storico conservato nel repository, la chiamata mostrata non

impone il controllo di scadenza del certificato (opzione `check_expiration=False`, predefinita). Tale controllo va invece abilitato sui documenti correnti; il modulo non implementa la scoperta ristretta e non costituisce un client onion service completo. In una sessione precedente è stato ottenuto questo risultato abbreviato:

```
lifetime 180 revisione 662427 punti di introduzione 3
create2-formats 2
flow-control 1-2 31
introduction-point AwAGfwAAARPyAhQXE5GoH+wp3MoxW9M1QV6XSxb0fgM...
onion-key ntor yRURxDb2eeEmF/p5M/bmHA+giwVyxjApKYyNY1KkCDo=
auth-key
-----BEGIN ED25519 CERT-----
...
```

La riga `flow-control 1-2 31` dice ai client che il servizio supporta il controllo della congestione, con lo stesso `sendme_inc` di 31 visto nel capitolo 11.

Passo 5: un controllo di coerenza. Nella rete di laboratorio gli HSDir sono otto e il servizio carica il descrittore su $2 \times 4 = 8$ di essi, cioè su tutti: trovare il descrittore non dimostra che il calcolo dell'anello sia giusto. Il log del client (`info.log`) registra però a quale HSDir si è rivolto: `test002a`, alle 16:07. È uno dei tre che il nostro codice indica per quella replica in quel momento. Una sola osservazione non è una prova, ma è coerente; l'esercizio 14.6 propone una verifica più solida.

Riepilogo

- Un onion service offre un servizio senza rivelarne l'indirizzo IP; l'indirizzo `.onion` contiene la chiave pubblica del servizio e lo autentica.
- I descrittori sono firmati con chiavi accecate, diverse per ogni periodo, che solo chi conosce l'indirizzo può collegare al servizio.
- La posizione dei descrittori sull'anello degli HSDir dipende dalla chiave accecata, dal periodo e dal valore casuale condiviso; il client li scarica con richieste anonime.
- Il descrittore ha uno strato esterno firmato e due strati cifrati; il secondo elenca i punti di introduzione.
- Client e servizio si incontrano in un punto di rendezvous scelto dal client, dopo una richiesta passata attraverso un punto di introduzione; l'handshake hs-ntor aggiunge uno strato di cifratura da estremo a estremo.
- Scoperta ristretta, vanguards, replay cache, limiti di frequenza e prova di lavoro proteggono i servizi.

Esercizi

Esercizio 14.1 ⋆ Quanti byte codifica un indirizzo v3, e perché ne escono esattamente 56 caratteri base32? Quale probabilità ha un errore di battitura casuale di superare il controllo del checksum?

Esercizio 14.2 ⋆ Calcola il numero del periodo di tempo della rete pubblica alle 09:00 UTC del 1° gennaio 2026. Quando è iniziato quel periodo?

Esercizio 14.3 ⋆⋆ Un avversario vuole impedire l'accesso a un onion service facendo in modo che gli HSDir responsabili siano suoi. Che cosa deve prevedere, e con quanto anticipo può farlo? Che ruolo ha l'SRV?

Esercizio 14.4 ⋆⋆ Elenca che cosa sanno, di client e servizio, il punto di introduzione, il punto di rendezvous e l'HSDir. Quale di questi ruoli sarebbe più interessante per un avversario che vuole stimare la popolarità di un servizio?

Esercizio 14.5 ⋆⋆ Perché un HSDir rifiuta le richieste di descrittori che arrivano su circuiti di un solo salto? Che cosa potrebbe scoprire altrimenti?

Esercizio 14.6 ⋆⋆⋆ Verifica in modo più solido il calcolo dell'anello: aggiungi alla rete di laboratorio altri relay (esercizio 19.7) fino ad avere più di otto HSDir, poi controlla che il descrittore si trovi *solo* sugli HSDir calcolati dal tuo codice.

Esercizio 14.7 ⋆⋆⋆ Decodifica gli specificatori di collegamento di un punto di introduzione del descrittore del laboratorio (il valore base64 della riga `introduction-point`). Quale relay della rete di laboratorio è? Che cosa contiene il certificato della riga `auth-key`?

Per approfondire

- *Tor Rendezvous Specification - Version 3* (rend-spec) [Thef]: indirizzi, chiavi accecate, descrittori, introduzione e rendezvous, con i vettori di test di hs-ntor.
- Proposta 224: la progettazione dei servizi v3.
- La specifica della casualità condivisa (srv-spec): come le autorità producono l'SRV.

Capitolo 15

Difese contro DoS e gestione delle risorse

Aggiornato al 23 settembre 2026 — torspec commit `8e1f245`; tor 0.4.9.13; dos-spec, hspow-spec

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- spiegare perché un attacco di negazione del servizio contro Tor è anche un attacco all'anonimato;
- descrivere come tor protegge memoria, CPU e banda dei relay;
- conoscere i limiti per indirizzo su connessioni e circuiti e il ruolo dei parametri di consenso;
- spiegare come gli onion service si difendono dalle inondazioni di richieste di introduzione;
- descrivere la prova di lavoro di tipo v1 (Equi-X e Blake2b) e il suo controllo dello sforzo;
- provocare e osservare le difese di un relay e di un onion service nella rete di laboratorio (Laboratorio 9).

Una rete aperta, anonima e gestita da volontari è un bersaglio naturale per chi vuole renderla inutilizzabile. In questo capitolo vediamo come Tor si difende dagli attacchi di *negazione del servizio* (*denial of service*, DoS): tentativi di esaurire le risorse di un relay o di un onion service per impedirne l'uso. Gli episodi concreti di questi anni, come l'ondata di attacchi agli onion service del 2022–2023, sono raccontati nel capitolo 27; qui ci concentriamo sui meccanismi.

15.1 Perché il DoS è un problema di anonimato

In una rete qualsiasi, un attacco DoS è «solo» un problema di disponibilità. In Tor è anche un'arma contro l'anonimato [dos-spec § Overview]:

- un avversario che riesce a spegnere a comando un relay può costringere i client che lo usano come guard a sceglierne un altro, sperando che sia suo. Jansen, Tschorsch, Johnson e Scheuermann hanno mostrato come esaurire la memoria di un relay con poche risorse e usare questa capacità per deanonimizzare gli onion service: il *Sniper Attack* [Jan+14];

- rendere lenti o irraggiungibili i relay onesti spinge il traffico verso quelli dell'avversario, che ne vede una quota maggiore;
- un attacco che fa fallire circuiti in modo mirato può essere usato come segnale, per esempio per capire quali circuiti passano da un certo relay.

Per questo le difese non si limitano a «tenere in piedi» i relay: cercano di farlo senza creare, a loro volta, comportamenti osservabili e sfruttabili da un avversario.

15.2 Le risorse fisiche

15.2.1 Memoria

La memoria è la risorsa più delicata, e quella protetta dai meccanismi più maturi [dos-spec § Memory exhaustion]. Su Linux, un programma che chiede troppa memoria di solito non riceve un errore: il sistema gliela promette e poi, quando la usa davvero, termina il processo. Aspettare l'errore di allocazione non funziona. Tor si impone quindi un limite, `MaxMemInQueues`, impostato dall'operatore o calcolato come frazione della memoria del sistema, e tiene il conto di ciò che occupa: celle in coda sui circuiti, buffer delle connessioni, stati di compressione, descrittori degli onion service in cache, risoluzioni DNS in cache.

Quando il totale supera il limite, tor libera memoria in due passi: prima svuota le cache che occupano più del 20 % del totale, partendo dalle voci più vecchie, finché non scendono al 10 %; poi, se è ancora oltre il 90 % del limite, chiude circuiti e connessioni ordinandoli in base all'età del dato più vecchio che tengono in coda, cominciando da quelli con i dati fermi da più tempo. L'idea è colpire chi sta accumulando dati senza leggerli, cioè proprio il comportamento tipico di un attacco.

15.2.2 CPU

La CPU è il punto debole dell'implementazione in C, che elabora la rete su un unico thread principale. Le operazioni più costose, gli handshake di circuito (capitolo 9), sono delegate a un gruppo di thread di lavoro e messe in coda; ma se ne arrivano troppe, la coda si allunga e i circuiti legittimi scadono. Tor non misura direttamente l'uso della CPU: la protegge indirettamente, limitando le operazioni che la consumano, come la creazione di circuiti e le introduzioni agli onion service.

15.2.3 Banda

Gli operatori possono limitare la banda con opzioni come `RelayBandwidthRate`, ma questi limiti servono a non superare il proprio contratto, non a difendersi. Contro un uso anomalo della banda Tor si affida al controllo di flusso e della congestione (capitolo 11) e a una distribuzione equa delle celle tra i circuiti.

15.3 Limiti per indirizzo

La difesa più efficace contro gli attacchi grossolani è semplice: un guard vede l'indirizzo IP di chi gli si collega, e può limitare quanto ciascun indirizzo può chiedere. È l'unico punto della rete in cui questo è possibile, perché più avanti nel circuito l'origine del traffico è, per costruzione, sconosciuta. Tor applica tre famiglie di limiti ai client (non ai relay autenticati), configurabili con opzioni `DoS…`:

Connessioni (`DoSConnection…`) un massimo di connessioni TLS contemporanee e un ritmo massimo di nuove connessioni per indirizzo; oltre la soglia le connessioni vengono chiuse.

Circuiti (`DoSCircuitCreation…`) un *secchio di gettoni* (*token bucket*) per indirizzo: ogni nuovo circuito consuma un gettone, i gettoni si ricaricano a un ritmo fisso fino a un massimo (il *burst*). Un indirizzo che ha esaurito il secchio *e* tiene aperte almeno un certo numero di connessioni viene marcato, e per un periodo i suoi nuovi circuiti vengono rifiutati.

Rendezvous a un salto (`DoSRefuseSingleHopClientRendezvous`) il rifiuto delle richieste di rendezvous che arrivano su circuiti di un solo salto, come quelli che usavano i gateway Tor2web per raggiungere gli onion service senza anonimato per il client: un modo economico di generare carico per la rete.

15.3.1 Il codice e il consenso

Nel codice di tor queste difese sono **disattivate** per impostazione predefinita (`DOS_CC_ENABLED_DEFAULT 0` in `src/core/or/dos.h`); a regolarle, per tutta la rete, sono i parametri di consenso. Il consenso pubblico del 23 settembre 2026 contiene:

Parametro	Consenso	Predefinito nel codice
`DoSCircuitCreationEnabled`	1	0
`DoSCircuitCreationMinConnections`	2	3
`DoSCircuitCreationRate`	2	3
`DoSCircuitCreationBurst`	60	90
`DoSConnectionEnabled`	1	0
`DoSConnectionMaxConcurrentCount`	50	100
`DoSRefuseSingleHopClientRendezvous`	1	—

I valori della rete sono più severi di quelli del codice: sono il frutto di anni di attacchi e di regolazioni successive. Poterli cambiare dal consenso, in poche ore e senza aggiornare il software di migliaia di relay, è uno degli strumenti più preziosi del Tor Project durante un attacco.

Attenzione: Il limite dei limiti per indirizzo

Un avversario con molti indirizzi IP, come una botnet, sfugge ai limiti per indirizzo; e un limite troppo severo colpisce gli utenti legittimi che condividono lo stesso indirizzo, per esempio dietro il NAT di un operatore mobile o di un'università. I valori sono sempre un compromesso.

15.4 Onion service sotto attacco

Gli onion service sono particolarmente vulnerabili. Ogni richiesta di introduzione costa pochissimo al client: un `INTRODUCE1` (sezione 14.6). Al servizio invece costa molto: decifrare la richiesta, eseguire un handshake e costruire un circuito di più salti verso il punto di rendezvous indicato. È un'**amplificazione**: con poche risorse un avversario costringe il servizio, e tutti i relay dei circuiti che costruisce, a un lavoro molto maggiore. E il servizio non può bloccare gli indirizzi degli attaccanti, perché non li vede.

15.4.1 Limiti al punto di introduzione

Una prima difesa è far limitare le richieste ai punti di introduzione. Con l'opzione `HiddenServiceEnableIntroDoSDefense` e le varianti `…RatePerSec` e `…BurstPerSec`, il servizio comunica a ciascun punto di introduzione, con un'estensione di `ESTABLISH_INTRO` (`DOS_PARAMS`), quante `INTRODUCE2` al secondo è disposto a ricevere. Il limite protegge il servizio, ma non distingue un attaccante da un utente legittimo: sotto attacco, entrambi vengono respinti.

15.4.2 La prova di lavoro

La difesa più recente, proposta con la Proposta 327 e disponibile da tor 0.4.8.4 (agosto 2023), è la **prova di lavoro** (*proof of work*, PoW) [hspow-spec § Motivation]. L'idea è permettere ai client di dimostrare di aver speso una certa quantità di calcolo e di dare precedenza alle richieste che ne dimostrano di più. **Non è una soglia minima obbligatoria**: lo sforzo suggerito dal servizio è un'indicazione per il client, non un requisito d'accesso; le introduzioni senza prova sono ammesse in coda con priorità zero quando la difesa è abilitata. Una prova *presente ma non valida*, invece, è rifiutata. Il costo per il client dipende dallo sforzo scelto e dall'hardware; per un attaccante che vuole inviare richieste in grande quantità il costo si moltiplica.

Il meccanismo ha quattro parti [hspow-spec § Common protocol]:

1. **I parametri nel descrittore.** Il servizio pubblica, nel secondo strato del descrittore (sezione 14.5), una riga `pow-params` con lo schema (`v1`), un *seme* casuale di 32 byte, lo *sforzo suggerito* e la scadenza del seme, rinnovato ogni 105–120 minuti.

2. **Il client risolve il problema.** Con lo sforzo E scelto, il client cerca una soluzione del problema Equi-X per la sfida

   $$\texttt{"Tor hs intro v1\0"} \,\|\, A' \,\|\, \mathit{seme} \,\|\, \mathit{nonce} \,\|\, E,$$

   dove A' è la chiave accecata del servizio. Una soluzione Equi-X non basta: il client calcola un hash Blake2b di 32 bit, R, della sfida e della soluzione, e la soluzione è accettabile solo se $R \cdot E \leq 2^{32} - 1$. Altrimenti cambia nonce e riprova.

3. **La prova nell'introduzione.** Il client inserisce nonce, sforzo, i primi 4 byte del seme e la soluzione in un'estensione della parte cifrata di `INTRODUCE1`, che solo il servizio legge.

4. **La coda a priorità.** Il servizio verifica la prova, che è molto più veloce da verificare che da trovare, e mette la richiesta in una coda ordinata per sforzo. Serve per prime le richieste con lo sforzo più alto; quelle senza prova hanno sforzo zero. Se la coda si riempie, scarta di colpo la metà con lo sforzo più basso.

Approfondimento: Perché Equi-X

Equi-X, progettato da tevador per questo scopo, è un problema *asimmetrico*: trovare una soluzione richiede memoria e molte operazioni, verificarla pochissime. Si basa su HashX, una famiglia di funzioni di hash che cambia implementazione a ogni seme e sfrutta l'esecuzione in parallelo delle CPU moderne: è difficile da accelerare con hardware dedicato, così che un avversario non guadagni molto rispetto a un utente con un normale computer. Esistono due implementazioni, quella in C inclusa in tor e una in Rust scritta per Arti, tenute compatibili con test automatici. Il codice C è distribuito con licenza LGPL: per includerlo, tor va compilato con l'opzione `--enable-gpl`, e un tor compilato senza di essa non offre la PoW: `tor --list-modules` riporta allora `pow: no` invece del `pow: yes` che vedremo nel capitolo 17. Non è però il caso comune: sono compilati con l'opzione sia

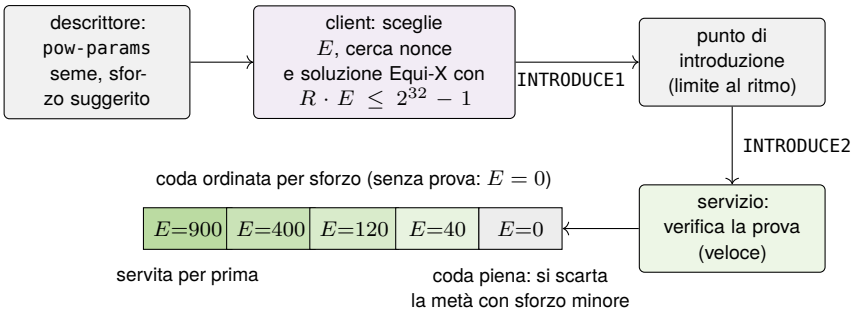


Figura 15.1: La prova di lavoro per gli onion service. Il client legge seme e sforzo suggerito dal descrittore, trova una soluzione che supera il test, la invia nella parte cifrata di `INTRODUCE1`; il servizio la verifica e ordina le richieste per sforzo.

l'immagine di laboratorio sia il pacchetto `tor` di Debian, e in entrambi il modulo risulta presente.

15.4.3 Uno sforzo lineare

Perché il livello Blake2b? Il costo di una singola soluzione Equi-X è grosso modo fisso. Il test $R \cdot E \leq 2^{32} - 1$ è superato da una soluzione casuale con probabilità circa $1/E$: il client deve quindi provare in media E soluzioni. Lo sforzo cresce *linearmente* con E, e questo rende semplice regolarlo. Molti sistemi di prova di lavoro chiedono invece un certo numero di bit a zero, con un costo che raddoppia a ogni bit: una scala troppo grossolana per un controllo fine. Lo sforzo è inoltre dichiarato *prima* di cercare la soluzione e fa parte della sfida: un client non può sperare in un colpo di fortuna e dichiarare poi uno sforzo più alto di quello speso.

15.4.4 Il controllo dello sforzo

Nessuno fissa lo sforzo una volta per tutte. Il servizio parte da uno sforzo suggerito di zero, cioè con la difesa dormiente, e ogni cinque minuti valuta lo stato della coda [hspow-spec § Common protocol]. Aumenta il valore interno se ha scartato richieste con sforzo *strettamente maggiore* di quello suggerito oppure se la coda si è congestionata e contiene ancora richieste con sforzo *almeno pari* al suggerimento. Lo diminuisce soltanto quando nessuna delle due condizioni vale e la coda è abbastanza breve; altrimenti lo lascia invariato. Nell'aumento sceglie il massimo fra *valore precedente più uno* e il rapporto fra sforzo totale accodato e rendezvous effettivamente avviati; nella diminuzione moltiplica per $2/3$. È una logica ispirata al controllo della congestione di TCP, ma aggiornata a intervalli fissi. Non ogni variazione del valore interno comporta la ripubblicazione del descrittore. I client, a loro volta, possono aumentare lo sforzo dopo la scadenza di un tentativo di collegamento. È un sistema a più anelli di retroazione, nel quale un client può scegliere di investire uno sforzo maggiore del valore suggerito per ottenere priorità.

15.5 Laboratorio: provocare le difese

Laboratorio 9: Limiti anti-DoS e prova di lavoro nella rete di laboratorio

Obiettivo. Attivare i limiti sulla creazione di circuiti di un relay e superarli; attivare la prova di lavoro sull'onion service e leggerne i parametri; studiare il livello Blake2b della prova di lavoro con `labs/it/codice/torlab/pow.py`.
Prerequisiti. Laboratori 1–8; la rete di laboratorio avviata, con l'immagine compilata con `--enable-gpl`.
Tempo stimato. 2 ore.

Attenzione: Solo sulla tua rete

Questo laboratorio genera di proposito traffico anomalo. Va eseguito esclusivamente sulla rete privata del libro: fare lo stesso contro relay o servizi reali è un attacco, dannoso per altre persone e, in molti ordinamenti, illegale (capitolo 25).

Passo 1: attivare i limiti su un relay. Aggiungiamo al `torrc` del relay `test003r` limiti molto bassi, per vederli scattare subito, e facciamogli rileggere la configurazione con il segnale `HUP`:

```
DoSCircuitCreationEnabled 1
DoSCircuitCreationMinConnections 2
DoSCircuitCreationRate 1
DoSCircuitCreationBurst 10
DoSCircuitCreationDefenseType 2
DoSCircuitCreationDefenseTimePeriod 60 seconds
```

Il tipo di difesa 2 significa «rifiuta i nuovi circuiti».

Passo 2: superarli. Apriamo uno o due canali verso il relay e chiediamo 30 circuiti in rapida successione con **`CREATE_FAST`**. Per avere contatori separati in ogni prova, facciamo partire le connessioni da indirizzi di loopback diversi (127.0.0.2, 127.0.0.3), con il nuovo parametro `origine` di `apri_canale`:

```
origine 127.0.0.2, 2 canali: 9 ok, 21 DESTROY(5); primo rifiuto al n. 10
origine 127.0.0.3, 1 canale:  30 ok
origine 127.0.0.1, 2 canali: 9 ok, 21 DESTROY(5); primo rifiuto al n. 10
```

Tre osservazioni:

- il relay rifiuta con **`DESTROY`** di motivo 5, `RESOURCELIMIT`;
- con un *burst* di 10 passano 9 circuiti, non 10: il codice (`cc_has_exhausted_circuits` in `src/core/or/dos.c`) marca il client appena il secchio arriva a zero, quindi la decima richiesta è già respinta;
- con un solo canale non scatta nulla, perché la difesa richiede almeno `MinConnections` connessioni contemporanee. Il motivo è evitare di punire un normale client, che di solito tiene aperta una sola connessione al proprio guard.

Avevamo previsto che le connessioni da 127.0.0.1, l'indirizzo dei relay del laboratorio, fossero esenti dai limiti; l'esperimento ha smentito la previsione, e il codice di `dos.c` conferma che un'esenzione del genere non esiste per i circuiti. Alla fine dell'esperimento ripristiniamo il `torrc` originale.

Passo 3: la prova di lavoro dell'onion service. Aggiungiamo, dopo le righe `HiddenService…` del `torrc` di `test009h`:

```
HiddenServicePoWDefensesEnabled 1
```

Dopo il segnale `HUP`, il servizio pubblica un nuovo descrittore. Scaricandolo e decifrandolo con il Laboratorio 8 troviamo, nel secondo strato:

```
pow-params v1 tx2Mtmzw5GHgWjcDoivTvWzwQPNZONwMwSMddOc+ZVQ= 0 2026-09-23T18:20:53
```

Schema `v1`, il seme in base64, uno sforzo suggerito di **zero**, perché il servizio non ha ancora visto una coda, e la scadenza del seme, un'ora e cinquanta minuti dopo la pubblicazione, nella finestra di 105–120 minuti prevista.

Passo 4: lo sforzo è lineare. Il modulo `pow.py` implementa **soltanto il controllo Blake2b** e usa byte casuali come sostituti statistici dei candidati Equi-X: consente di studiare la distribuzione del valore di controllo *assumendo che l'hash sia uniforme*, non di produrre una prova valida per Tor. Non esegue `equix_solve` né `equix_verify`, non controlla la scadenza del seme e non gestisce la cache anti-replay. La prova reale richiede *tutti* questi passaggi [hspow-spec § Onion service proof-of-work: Scheme v1, Equi-X and Blake2b]:

```python
def valore_controllo(id_servizio, seme, nonce, sforzo, soluzione):
    sfida = P_STRING + id_servizio + seme + nonce + struct.pack(">I", sforzo)
    h = hashlib.blake2b(sfida + soluzione, digest_size=4).digest()
    return int.from_bytes(h, "big")

def sforzo_valido(r, sforzo):
    return r * sforzo <= UINT32_MAX
```

Misurando in media quante soluzioni servono per 400 prove a ciascuno sforzo:

```
sforzo    1: 1.0 tentativi
sforzo   10: 9.7
sforzo  100: 102.3
sforzo 1000: 1068.8
```

Il numero di tentativi cresce in proporzione allo sforzo, come previsto: il valore atteso è lo sforzo stesso, perché ogni tentativo riesce con probabilità $1/\mathit{sforzo}$ e il numero di tentativi segue una distribuzione geometrica. Di quella distribuzione lo scarto quadratico medio è grande quanto la media, quindi la media di 400 prove ha uno scarto tipico del $1/\sqrt{400} = 5\,\%$: fra due esecuzioni i valori possono differire del dieci per cento senza che nulla sia cambiato, e rieseguendo la misura abbiamo infatti ottenuto 969,9 dove qui si legge 1068,8. È un buon promemoria: una media va sempre letta insieme all'incertezza del campione. Si noti poi un dettaglio: `blake2b` con `digest_size=4` non equivale ai primi 4 byte di un Blake2b di 64 byte, perché la lunghezza dell'uscita entra nello stato iniziale della funzione. La specifica lo sottolinea, ed è un errore facile da commettere.

Riepilogo

- In Tor gli attacchi DoS sono anche attacchi all'anonimato: spegnere relay a comando o farli fallire può spostare il traffico verso l'avversario.
- La memoria è protetta da un limite autoimposto, `MaxMemInQueues`, oltre il quale tor libera cache e chiude i circuiti che accumulano dati.

- I guard limitano per indirizzo connessioni e circuiti dei client con secchi di gettoni; le difese, spente nel codice, sono attivate e regolate dal consenso.
- Gli onion service subiscono un'amplificazione: introdursi costa poco, rispondere molto.
- La prova di lavoro v1 combina Equi-X e un test Blake2b che rende lo sforzo lineare; il servizio ordina le richieste per sforzo e regola il valore suggerito in base alla coda. Il valore suggerito non è una soglia obbligatoria, e il laboratorio simula soltanto il test Blake2b.

Esercizi

Esercizio 15.1 ⋆ Con `DoSCircuitCreationRate` 2 e `Burst` 60, come nel consenso pubblico, quanti circuiti può creare un client in un minuto partendo da un secchio pieno? E in un'ora? Supponi che la difesa sia già attiva, cioè che l'indirizzo abbia raggiunto `DoSCircuitCreationMinConnections` connessioni contemporanee: sotto quella soglia non c'è alcun limite.

Esercizio 15.2 ⋆ Perché i limiti per indirizzo si applicano solo ai client, e solo sul primo salto? Che cosa succederebbe se un middle cercasse di limitare i circuiti «per indirizzo»?

Esercizio 15.3 ⋆⋆ Stima il rapporto tra il lavoro del client e quello della rete per una singola richiesta di introduzione: quanti circuiti, handshake e celle genera la risposta del servizio?

Esercizio 15.4 ⋆⋆ Confronta la prova di lavoro di Tor con uno schema che richiede k bit iniziali a zero in un hash. Quali valori di sforzo può esprimere ciascuno schema? Perché la scala lineare è preferibile per un controllo a retroazione?

Esercizio 15.5 ⋆⋆ Un client dichiara uno sforzo $E = 50$ ma, per fortuna, trova subito una soluzione con R molto piccolo, che sarebbe valida anche per $E = 5000$. Può inviarla dichiarando $E = 5000$? Perché?

Esercizio 15.6 ⋆⋆⋆ Nella rete di laboratorio, imposta `MaxMemInQueues` al valore minimo su un relay intermedio e costruisci un circuito attraverso di esso da cui scarichi un file grande *senza leggere* i dati in arrivo (e senza inviare SENDME). Che cosa succede alle celle accumulate? Cerca nei log del relay i messaggi del gestore della memoria.

Esercizio 15.7 ⋆⋆⋆ Scrivi un piccolo simulatore della coda a priorità di un onion service: arrivano richieste con sforzo casuale (molte a zero, poche alte), il servizio ne serve un certo numero al secondo e scarta la metà più bassa quando la coda supera una soglia. Misura il tempo d'attesa medio in funzione dello sforzo.

Per approfondire

- *Tor denial-of-service mitigations* (dos-spec) e *Onion service proof-of-work* (hspow-spec) [Thef].
- R. Jansen, F. Tschorsch, A. Johnson, B. Scheuermann, *The Sniper Attack: Anonymously Deanonymizing and Disabling the Tor Network* [Jan+14].
- Il repository di Equi-X di tevador e il suo diario di sviluppo, citati nella specifica.

Capitolo 16

Censura, bridge e pluggable transports

Aggiornato al 23 settembre 2026 — tor 0.4.9.13; pt-spec v1; obfs4proxy 0.0.14 (Debian); documentazione del Tor Project

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- descrivere i modi in cui un censore può bloccare Tor;
- spiegare che cosa sono i bridge e perché la loro distribuzione è un problema difficile;
- descrivere il protocollo con cui tor comunica con un pluggable transport;
- distinguere le strategie di obfs4, meek, Snowflake, WebTunnel e Conjure;
- far funzionare un bridge obfs4 nella rete di laboratorio e osservare che cosa vede un censore (Laboratorio 10).

Tutto ciò che abbiamo visto finora presuppone che il client possa raggiungere la rete. In molti paesi non è così: la connessione verso Tor è bloccata, a volte solo nei periodi di tensione politica, a volte in modo permanente. Questo capitolo chiude la Parte II con il tema dell'**elusione della censura**, che negli ultimi anni è diventato uno dei fronti più attivi del Tor Project.

16.1 Come si blocca Tor

Un censore, di solito un governo che controlla i fornitori di connettività del paese, ha diversi strumenti:

Blocco per indirizzo La lista dei relay è pubblica: è il consenso (capitolo 12). Basta scaricarlo e bloccare gli indirizzi IP di tutti i relay, oppure solo quelli delle directory authority e dei relay da cui i client scaricano il consenso.

Blocco dei siti Bloccare `torproject.org` impedisce di scaricare Tor Browser, anche se non blocca la rete in sé.

Riconoscimento del protocollo Un'ispezione approfondita dei pacchetti (*deep packet inspection*, DPI) può riconoscere una connessione Tor da caratteristiche del suo handshake TLS, dalle dimensioni dei pacchetti o dai tempi, anche verso indirizzi non ancora noti.

Sondaggio attivo Quando il DPI sospetta che un server sia un relay nascosto, il censore può collegarsi a sua volta e provare a parlargli il protocollo di Tor: se risponde, lo blocca.

Liste di permesso Nei casi estremi il censore consente solo protocolli o destinazioni esplicitamente autorizzati, e tutto il resto viene bloccato.

A ciascuna tecnica corrisponde una contromisura, e la corsa tra censori e sviluppatori non si ferma. La misura di dove e come Tor viene bloccato è affidata a progetti come OONI (capitolo 24).

16.2 I bridge

Contro il blocco per indirizzo la risposta è il **bridge**: un relay che non compare nel consenso. Un bridge funziona come un guard (sezione 13.3): il client lo usa come primo salto, e da lì il circuito prosegue normalmente nella rete. Poiché il suo indirizzo non è pubblico, il censore non può bloccarlo semplicemente leggendo una lista. I bridge pubblicano il proprio descrittore presso una **bridge authority**, che ne controlla la raggiungibilità e ne tiene l'elenco, ma non lo pubblica; le statistiche pubbliche sui bridge usano solo un'impronta *di un'impronta* dell'identità, per non rivelarla.

Un client configura un bridge con una *bridge line*. Nella rete di laboratorio il client `test011bc` ha nel suo `torrc`:

```
UseBridges 1
Bridge obfs4 127.0.0.1:9909 2557828B596B33AB02075FE7EAD86F2DC3CEAFD5 cert=4sKDQih...
    iat-mode=0
ClientTransportPlugin obfs4 exec /usr/bin/obfs4proxy
```

La riga indica il tipo di trasporto, l'indirizzo, l'impronta dell'identità del bridge (per verificare di parlare con lui, capitolo 7) e i parametri del trasporto. Il valore `cert=4sKDQih...` è *abbreviato* a scopo tipografico e non è una configurazione da copiare: per un bridge avviato dal lettore servono il certificato e l'impronta reali della nuova istanza, reperibili nella sua bridge line.

16.2.1 Il problema della distribuzione

Un bridge è utile finché il censore non ne conosce l'indirizzo. Ma per essere utile deve essere dato agli utenti, e il censore può fingersi utente. È un problema senza soluzione perfetta: bisogna dare a molti utenti legittimi abbastanza bridge, dandone il meno possibile al censore. Il Tor Project distribuisce i bridge attraverso diversi canali, ciascuno con difese proprie contro la raccolta automatica [Theh]:

- il sito `https://bridges.torproject.org/`;
- la posta elettronica, scrivendo a `bridges@torproject.org` **da un account Gmail o Riseup**, come richiede attualmente il servizio automatico [Theh];
- un bot di Telegram (`@GetBridgesBot`);
- direttamente da Tor Browser, con la funzione che rileva il blocco e chiede i bridge adatti al paese (*Connection Assist*, basata sul canale chiamato *moat*);
- canali privati, gestiti da organizzazioni che conoscono personalmente i propri utenti.

Il sistema che assegna i bridge ai canali, per anni BridgeDB, è stato sostituito da **rdsys**; la migrazione si è conclusa nell'ottobre 2024 [The24b]. L'operatore di un bridge può scegliere il canale con l'opzione `BridgeDistribution`.

16.2.2 Lox

Lox [TG23] affronta la distribuzione con un sistema di *reputazione*: un nuovo utente riceve un bridge e una credenziale anonima; se nel tempo il suo bridge non viene bloccato, la credenziale sale di livello, fino a un massimo di 4, e dal livello 2 permette di invitare altri utenti. Se il bridge viene bloccato, l'utente può ottenerne un altro, ma perde reputazione. L'idea è che il censore, che blocca i bridge che riceve, non riesca mai a salire di livello, mentre gli utenti onesti sì. Le credenziali usano crittografia che impedisce al sistema stesso di sapere chi ha invitato chi, proteggendo il *grafo sociale* degli utenti. Lox è in fase di sperimentazione nelle versioni alfa di Tor Browser.

16.3 I pluggable transports

Contro il riconoscimento del protocollo non basta nascondere l'indirizzo: bisogna cambiare l'aspetto del traffico. Invece di modificare tor ogni volta che un censore impara a riconoscere qualcosa, il Tor Project ha definito un'interfaccia per programmi esterni che trasformano il traffico: i **pluggable transports** (PT) [pt-spec § Architecture Overview].

16.3.1 L'architettura

Il PT è un processo separato, lanciato da tor (per questo si dice *managed*). Dal lato del client offre a tor un proxy SOCKS locale: tor gli affida la connessione verso il bridge, e il PT la trasforma prima di spedirla. Dal lato del bridge il PT ascolta sulla porta pubblica, annulla la trasformazione e passa il traffico all'ORPort locale di tor, eventualmente con un piccolo protocollo, l'*Extended ORPort*, che comunica a tor l'indirizzo reale del client per le statistiche.

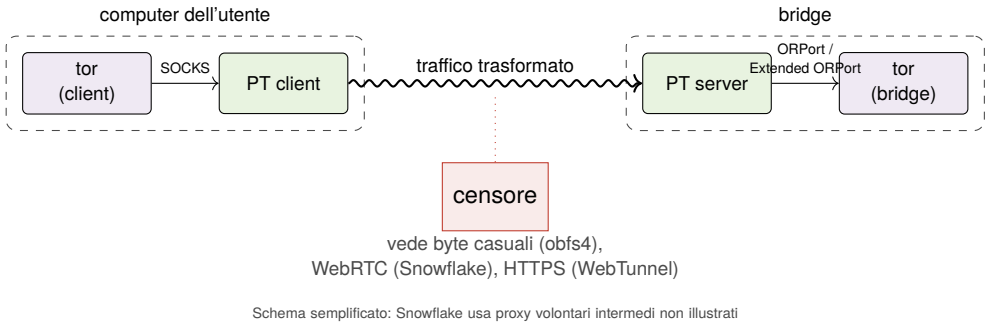


Figura 16.1: Architettura schematica dei pluggable transports. Sul client tor comunica con il PT mediante un proxy SOCKS locale; sul bridge il PT inoltra il traffico ricostruito a tor tramite ORPort o, se configurato, Extended ORPort. Snowflake prevede anche proxy volontari intermedi, omessi nello schema. Un censore osserva il traffico trasformato sul collegamento monitorato, ma ciò non esclude che possa riconoscerlo.

16.3.2 Il protocollo tra tor e il PT

tor configura il PT con variabili d'ambiente e il PT risponde scrivendo righe sul proprio output standard [pt-spec § Specification]. Possiamo provarlo lanciando a mano `obfs4proxy` nel container del laboratorio, come farebbe tor. Come client:

```
$ env TOR_PT_MANAGED_TRANSPORT_VER=1 TOR_PT_STATE_LOCATION=/tmp/pt \
      TOR_PT_EXIT_ON_STDIN_CLOSE=1 TOR_PT_CLIENT_TRANSPORTS=obfs4 obfs4proxy
VERSION 1
```

```
CMETHOD obfs4 socks5 127.0.0.1:40057
CMETHODS DONE
```

Il PT dichiara la versione del protocollo e annuncia di offrire il trasporto `obfs4` come proxy SOCKS5 su una porta locale scelta da lui. Come server, con le variabili che indicano l'indirizzo pubblico e l'ORPort da raggiungere:

```
$ env ... TOR_PT_SERVER_TRANSPORTS=obfs4 \
      TOR_PT_SERVER_BINDADDR=obfs4-127.0.0.1:19999 \
      TOR_PT_ORPORT=127.0.0.1:5104 obfs4proxy
VERSION 1
SMETHOD obfs4 127.0.0.1:19999 ARGS:cert=5
    W6vcbWbFGGZgabwm4C3WipezwnAfadoBhqfNVpY72xnkjB/Szczbxg8fLWJJ1uRlyjPSg,iat-mode=0
SMETHODS DONE
```

Gli argomenti dopo `ARGS:` sono i parametri che il client dovrà conoscere, e che finiscono nella bridge line; il PT li scrive anche nel file `obfs4_bridgeline.txt` della sua cartella di stato. La variabile `TOR_PT_EXIT_ON_STDIN_CLOSE` chiede al PT di terminare quando tor chiude il suo input: così un PT non sopravvive al processo che lo ha lanciato. (L'esempio della specifica, per una svista, dice che il PT scrive i messaggi su «stdin»: in realtà, come vediamo, li scrive sull'output.)

La riga `env ...` del comando server è *schematica*: i puntini sostituiscono variabili d'ambiente obbligatorie e non vanno digitati letteralmente. Anche la porta SOCKS5 nell'output client è un valore osservato in una sessione di riferimento, non una porta fissa.

16.3.3 Lyrebird

Il programma che Tor Browser usa per i trasporti è **lyrebird**, che deriva dal precedente `obfs4proxy` e implementa obfs4, meek, Snowflake e WebTunnel [Theh]. Nel laboratorio usiamo `obfs4proxy` 0.0.14, disponibile nei pacchetti di Debian e ancora compatibile per obfs4.

16.4 Le strategie

I trasporti oggi in uso seguono due strategie opposte: *non assomigliare a niente* o *assomigliare a qualcosa di permesso*.

16.4.1 obfs4: non assomigliare a niente

obfs4, progettato da Yawning Angel, trasforma il traffico in una sequenza di byte indistinguibile da dati casuali: nessuna intestazione riconoscibile, nessun handshake TLS, dimensioni dei pacchetti variabili. Tre scelte lo rendono efficace:

- lo scambio di chiavi è una variante di ntor (sezione 9.6) in cui le chiavi pubbliche Curve25519 sono codificate con Elligator2, una tecnica che rende un punto della curva indistinguibile da una stringa casuale;
- il client deve conoscere il parametro `cert` del bridge, che contiene la sua identità e la sua chiave pubblica: senza di esso non sa costruire un handshake valido, e il bridge non risponde. Questo protegge dal **sondaggio attivo**: un censore che si collega per curiosità non ottiene alcuna risposta riconoscibile;
- il parametro `iat-mode` (*inter-arrival time*) permette di aggiungere, a costo di prestazioni, ritardi e frammentazioni casuali che nascondono anche i tempi.

Il limite di obfs4 è il suo stesso punto di forza: un censore che usa una lista di permesso, o che blocca «tutto ciò che sembra casuale», lo ferma. È quanto accade in alcuni paesi.

16.4.2 meek: dietro un grande servizio

meek [Fif+15] usa il *domain fronting*: il client apre una connessione HTTPS verso un grande servizio di distribuzione di contenuti, con un nome di dominio innocuo nella parte visibile della connessione, e indica il vero destinatario (un server che fa da ponte verso Tor) solo nella parte cifrata. Per bloccarlo il censore dovrebbe bloccare l'intero servizio, con danni enormi. La tecnica dipende però dalla disponibilità dei fornitori a tollerarla, e molti di essi l'hanno resa impossibile negli ultimi anni; meek è inoltre lento e costoso, perché tutto il traffico passa dai server del fornitore.

16.4.3 Snowflake: tanti proxy effimeri

Snowflake [Boc+24] usa come ponte migliaia di volontari che eseguono un piccolo proxy, per esempio un'estensione del browser o un programma su un server. Il client contatta un *broker*, attraverso un canale difficile da bloccare come il domain fronting, per farsi assegnare un proxy, e si collega a lui con WebRTC, il protocollo delle videochiamate nei browser. Il proxy inoltra il traffico a un bridge Snowflake gestito dal Tor Project. I proxy sono tanti e cambiano di continuo: bloccarli tutti per indirizzo è impraticabile, e bloccare WebRTC significa bloccare le videochiamate.

16.4.4 WebTunnel: sembrare un sito web

WebTunnel, annunciato dal Tor Project nel marzo 2024, avvolge il traffico in una connessione HTTPS con un comportamento simile a WebSocket [The24a]. Il bridge può stare dietro un vero sito web, sullo stesso dominio, indirizzo e porta: un reverse proxy inoltra al bridge solo le richieste verso un percorso segreto, e serve il sito normale a tutti gli altri. È pensato per i censori che usano liste di permesso, contro cui obfs4 non funziona: WebTunnel ha la «forma» di un protocollo permesso.

16.4.5 Conjure: nascondersi nello spazio degli indirizzi

Conjure [Fro+19] è un sistema di *refraction networking*: invece di un bridge con un indirizzo, usa stazioni installate presso fornitori di connettività collaborativi, che osservano il traffico diretto a intervalli di indirizzi inutilizzati e lo deviano verso Tor. Il client si collega a un indirizzo «fantasma», scelto tra milioni: per bloccarlo il censore dovrebbe bloccare intere porzioni di Internet. Secondo la documentazione del Tor Project, Conjure è stato provato con successo ma richiede ancora lavoro di stabilizzazione, ed è disponibile solo nelle versioni alfa di Tor Browser e in alcune applicazioni [Theh].

Attenzione: Nessun trasporto funziona ovunque

Ogni trasporto ha i suoi punti deboli, e l'efficacia cambia da paese a paese e di mese in mese. Nel forum del Tor Project, per esempio, gli utenti hanno segnalato nell'aprile 2025 che in Cina obfs4, meek e Snowflake erano inutilizzabili e WebTunnel si collegava ma veniva bloccato rapidamente [Tor25]. Per questo Tor Browser offre più trasporti e una funzione che suggerisce quello adatto al paese dell'utente.

16.5 Laboratorio: un bridge obfs4 e lo sguardo del censore

Laboratorio 10: Un bridge obfs4 nella rete di laboratorio

Obiettivo. Avviare una rete con un bridge obfs4 e un client che lo usa; confrontare, catturando il traffico, una connessione Tor normale e una obfs4.
Prerequisiti. La rete di laboratorio (capitolo 19); l'immagine include `obfs4proxy` e `snowflake-client` dai pacchetti di Debian.
Tempo stimato. 1–2 ore.

Passo 1: la rete con il bridge. Il file `labs/it/laboratorio/chutney/networks/libro-bridge` aggiunge alla rete del libro un'autorità per i bridge (`test003ba`), un bridge obfs4 (`test009br`) e un client che lo usa (`test011bc`):

```
$ make -C labs/it/laboratorio pulisci
$ make -C labs/it/laboratorio avvia RETE=libro-bridge
```

Attenzione: La pulizia distrugge lo stato della rete di laboratorio

Il target `pulisci` elimina la cartella `labs/it/laboratorio/net/`, **incluse le chiavi private e i log dei nodi**. Usalo soltanto con una rete sacrificabile e dopo aver salvato altrove i dati che vuoi conservare. Se la rete `libro` è in uso, segui invece la procedura di scambio e ripristino documentata in `docs/it/procedura-laboratori-dal-vivo.md`; non eseguire `pulisci` alla cieca su un laboratorio da preservare.

Nel log del client di una precedente sessione di riferimento:

```
[notice] new bridge descriptor 'test009br' (fresh): $2557828B...~test009br
[warn] Only one bridge (transport: 'obfs4') is configured. You should
       have at least two for conflux, for any transport that is not 'snowflake'.
[notice] Bootstrapped 100% (done): Done
```

Il client ha ottenuto un descrittore del bridge configurato e ha completato l'avvio. **Il solo log non dimostra da quale canale sia stato scaricato il descrittore**: nella configurazione obfs4 il client conosce in anticipo la bridge line, ma quella informazione non equivale a un record nel consenso pubblico. L'avviso ricorda che Conflux (capitolo 31) usa due circuiti in parallelo, e con un solo bridge non può farlo. Una richiesta web attraverso la porta SOCKS di questo client (9011) funziona come con il client normale.

Passo 2: che cosa vede il censore. Dall'interno del container catturiamo con `tcpdump` il traffico sulla porta obfs4 del bridge (9909) e su una normale ORPort (5104), mentre il client usa il bridge e noi apriamo un canale verso `test004r` con il Laboratorio 1. Poi analizziamo la cattura con `tshark`; ecco il risultato di una nostra sessione:

```
== ORPort 5104 (TLS), primi byte inviati dal client ==
1533  16030105f8010005f403033e1234a0401520d1c52b...
== obfs4 9909, primi byte inviati dal client ==
1957  b566475349099523c53005e1004a392160f15d473e...
== pacchetti riconosciuti come TLS ==
porta 5104: 83
porta 9909: 0
== dimensioni dei segmenti obfs4 (client -> bridge) ==
1957 1677 1888 1891 1716 1716 3436 1395 1716 1748 1500 1194 ...
```

Verso l'ORPort, i primi byte sono inequivocabili: `16 03 01` è l'inizio di un record TLS di handshake, e Wireshark riconosce tutta la conversazione come TLS. Un censore vede un TLS da cui partire per cercare le caratteristiche di Tor, e l'indirizzo del relay è comunque nel consenso. Verso il bridge obfs4, *in questa cattura*, i primi byte non hanno una struttura TLS riconoscibile per `tshark`: nessun frame viene classificato come TLS e le dimensioni dei segmenti non riflettono direttamente le celle da 514 byte. **Non è una prova che obfs4 sia irriconoscibile**: un censore potrebbe comunque usare statistiche su durata, dimensioni e tempi, conoscere l'indirizzo del bridge da un altro canale oppure bloccare tutto il traffico non classificato. Quello osservato è un flusso dall'aspetto casuale, non una garanzia di invisibilità.

Passo 3: sondare il bridge. Proviamo a collegarci alla porta 9909 del bridge con il Laboratorio 1, come farebbe un censore che sospetta un relay:

```
TimeoutError: The handshake operation timed out
```

In questa prova il nostro client **non riceve una risposta utile prima del timeout** di 15 secondi. Per obfs4 il ClientHello TLS è un handshake non valido: il bridge non deve rivelare il proprio protocollo tramite una risposta riconoscibile, e può chiudere la connessione dopo un ritardo. Il risultato mostra una forma di *resistenza al sondaggio attivo*, non silenzio perpetuo né impossibilità di scoprire il servizio: un censore può inferire informazioni da tempi e chiusure, correlare osservazioni o conoscere l'indirizzo da altre fonti. Senza il parametro `cert` corretto non è possibile completare un handshake obfs4 valido.

Riepilogo

- Un censore può bloccare Tor per indirizzo, riconoscerne il protocollo, sondare i server sospetti o permettere solo il traffico autorizzato.
- I bridge sono relay non pubblicati; la loro distribuzione, oggi affidata a rdsys e sperimentata con la reputazione di Lox, è un compromesso tra utenti e censori.
- I pluggable transports sono processi esterni che trasformano il traffico; tor li configura con variabili d'ambiente e ne legge le risposte (`CMETHOD`, `SMETHOD`).
- obfs4 rende il traffico simile a dati casuali e resiste al sondaggio attivo; meek, Snowflake, WebTunnel e Conjure lo nascondono dietro servizi, protocolli o indirizzi che il censore non può bloccare senza danni.
- Nessun trasporto funziona ovunque e per sempre.

Esercizi

Esercizio 16.1 ⋆ In che cosa un bridge differisce da un guard? Perché un client che usa i bridge non usa anche un guard pubblico?

Esercizio 16.2 ⋆ Classifica obfs4, meek, Snowflake, WebTunnel e Conjure secondo la strategia («non assomigliare a niente» o «assomigliare a qualcosa di permesso») e secondo che cosa il censore dovrebbe bloccare per fermarli.

Esercizio 16.3 ⋆⋆ Descrivi come un censore potrebbe scoprire un bridge obfs4 osservando solo il traffico, senza sondarlo. Quali caratteristiche statistiche potrebbe usare, e che cosa fa `iat-mode` per contrastarle?

Esercizio 16.4 ⋆⋆ Nel sistema Lox, perché un censore non riesce a salire di livello? Che cosa potrebbe fare per aggirare il meccanismo, e che cosa glielo rende costoso?

Esercizio 16.5 ⋆⋆ Calcola l'entropia di Shannon dei byte (in bit per byte) dei primi 10 KB di una connessione obfs4 e di una connessione TLS catturate nel laboratorio. Che cosa ti aspetti, e perché la differenza può essere piccola? Che cosa rende allora riconoscibile TLS?

Esercizio 16.6 ⋆⋆⋆ Prova a eseguire il Laboratorio 1 contro la porta obfs4 del bridge. Che errore ottieni? Poi prova a collegarti con `obfs4proxy` come client ma con un parametro `cert` sbagliato: che cosa osservi nei log del bridge?

Per approfondire

- *Pluggable Transport Specification (Version 1)* (pt-spec) [Thef].
- L. Tulloch, I. Goldberg, *Lox: Protecting the Social Graph in Bridge Distribution* [TG23].
- D. Fifield et al., *Blocking-resistant communication through domain fronting* [Fif+15].
- C. Bocovich et al., *Snowflake, a censorship circumvention system using temporary WebRTC proxies* [Boc+24].
- S. Frolov et al., *Conjure: Summoning Proxies from Unused Address Space* [Fro+19].
- Il blog del Tor Project sull'introduzione di WebTunnel [The24a] e le pagine di supporto sui bridge [Theh].

Parte III

Tor in pratica

Capitolo 17

Installazione e torrc

Aggiornato al 23 settembre 2026 — tor 0.4.9.13; manuale `tor(1)` della stessa versione

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- scegliere il modo giusto di installare Tor per il tuo scopo;
- leggere e scrivere un file `torrc`, e sapere dove trovare la documentazione di ogni opzione;
- configurare tor come client, con porte SOCKS, isolamento degli stream e porta di controllo protetta;
- controllare una configurazione prima di usarla e diagnosticare i problemi dai log.

Con questo capitolo inizia la parte pratica del libro. I capitoli precedenti hanno spiegato come funziona il protocollo; qui impariamo a usare e a configurare il programma che lo implementa. Tutte le opzioni citate sono documentate nel manuale di tor (`man tor`), che è la fonte di riferimento: tra una versione e l'altra i valori predefiniti possono cambiare.

17.1 Quale Tor installare

Tor Browser Per navigare, è quasi sempre la scelta giusta: include tor già configurato e un browser modificato per non rivelare informazioni che vanificherebbero la protezione della rete (capitolo 23). Usare tor con un browser qualsiasi, invece, espone a molte forme di identificazione.

Il pacchetto `tor` Per usare Tor con altre applicazioni, per gestire un relay o un onion service. Le distribuzioni Linux lo includono, ma spesso in versioni non recenti; il Tor Project mantiene un proprio repository di pacchetti per Debian e Ubuntu, e distribuisce per Windows e macOS il *Tor Expert Bundle*, con tor e i pluggable transports.

Dai sorgenti Come nel laboratorio del libro (sezione 19.3), per studiare o per avere una versione precisa.

Arti Il client in Rust (capitolo 30), sempre più maturo, ma con una configurazione diversa.

Qualunque sia la scelta, conviene scaricare il software solo dal sito ufficiale e verificarne la firma: un tor modificato da un avversario annullerebbe ogni protezione.

17.2 Il file torrc

tor legge la configurazione da un file di testo, di solito `/etc/tor/torrc` per il servizio di sistema. Il formato è semplice: una opzione per riga, seguita dai suoi valori; le righe che iniziano con # sono commenti. Alcune opzioni si possono ripetere (per esempio più porte SOCKS). Le opzioni si possono passare anche sulla riga di comando, precedute da due trattini:

```
$ tor --SocksPort 9150 --DataDirectory /tmp/tor-prova
```

Il manuale elenca centinaia di opzioni; `tor --list-torrc-options` le stampa tutte. Per le distribuzioni, un secondo file di impostazioni predefinite può essere indicato con `--defaults-torrc`; quello indicato con `-f` ha la precedenza.

17.2.1 Controllare prima di usare

Tre comandi evitano molte sorprese:

```
$ tor --verify-config -f /etc/tor/torrc     # la configurazione è valida?
Configuration was valid
$ tor --dump-config short -f /etc/tor/torrc # che cosa differisce dai predefiniti?
$ tor --list-modules                        # quali moduli sono compilati?
relay: yes
dirauth: yes
dircache: yes
pow: yes
```

L'ultimo comando, come abbiamo visto nel capitolo 15, rivela per esempio se la prova di lavoro degli onion service è disponibile. **Attenzione al significato della verifica**: `--verify-config` controlla che le opzioni siano riconosciute e la configurazione sia accettata da tor, ma non certifica che le porte siano protette, che il client riesca a connettersi o che l'insieme delle impostazioni tuteli davvero l'anonimato.

17.3 Una configurazione da client

Un client ha bisogno di pochissimo. La configurazione minima è addirittura vuota: per impostazione predefinita tor apre una porta SOCKS sulla 9050 di localhost. Un file più esplicito, commentato, pensato per un servizio tor con le autorizzazioni necessarie alle directory `/var/lib/tor` e `/var/log/tor` (non per un avvio da utente ordinario):

```
# Dove tor conserva stato, chiavi e cache
DataDirectory /var/lib/tor

# Porta SOCKS per le applicazioni (solo su localhost!)
SocksPort 127.0.0.1:9050

# Porta di controllo, protetta dal cookie (vedi il capitolo 18)
ControlPort 127.0.0.1:9051
CookieAuthentication 1

# Log a livello notice
Log notice file /var/log/tor/notice.log
```

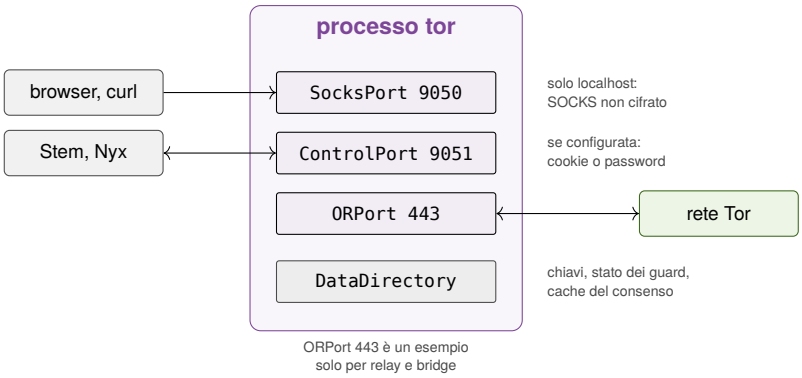


Figura 17.1: Porte e directory di un processo tor, in una configurazione di esempio. La SOCKS sulla 9050 serve applicazioni locali, il ControlPort 9051 è facoltativo e deve essere autenticato; non vanno esposti alla rete. DataDirectory è una cartella, non una porta. L'ORPort 443 è una scelta possibile per relay e bridge, non un valore obbligatorio.

Attenzione: La porta SOCKS non va esposta

Il manuale è esplicito: il protocollo SOCKS non è cifrato né autenticato. Una `SocksPort` in ascolto su un indirizzo raggiungibile dalla rete trasforma il computer in un proxy aperto e fa viaggiare in chiaro, sulla rete locale, le richieste delle applicazioni. Anche con il listener confinato a localhost, **l'applicazione deve affidare a tor la risoluzione dei nomi**: se risolve un dominio con il DNS di sistema *prima* di inviare l'indirizzo IP al proxy SOCKS, quella richiesta DNS può uscire fuori da Tor. Con applicazioni compatibili, preferire SOCKS5 con il nome di dominio passato al proxy (per esempio l'opzione `--socks5-hostname` di `curl`), non una risoluzione DNS preventiva.

17.3.1 Isolamento degli stream

Più stream possono condividere lo stesso circuito. È efficiente, ma permette all'exit di collegare tra loro attività diverse dello stesso utente: se la posta personale e un forum pseudonimo passano dallo stesso circuito, l'exit vede che provengono dalla stessa persona. tor offre quindi l'**isolamento degli stream**: regole su quali stream possono condividere un circuito. Le regole si esprimono come opzioni della `SocksPort`:

`IsolateClientAddr` stream da indirizzi client diversi non condividono circuiti (attivo per impostazione predefinita);

`IsolateSOCKSAuth` stream con credenziali SOCKS diverse non condividono circuiti (attivo per impostazione predefinita);

`IsolateDestAddr`, `IsolateDestPort` isolamento per destinazione;

`IsolateClientProtocol` isolamento per protocollo (SOCKS4, SOCKS5, DNS...).

Inoltre, porte SOCKS diverse sono isolate tra loro per impostazione predefinita: ciascun listener riceve un *session group* distinto, che però si può forzare con l'opzione `SessionGroup` per farle condividere i circuiti. Il meccanismo più usato è il secondo: Tor Browser invia credenziali SOCKS diverse per ogni sito visitato, e ottiene così un circuito diverso per ogni sito. Nel capitolo 18 lo vedremo in azione.

La rotazione dei circuiti dipende anche dal tipo di isolamento. Nella situazione ordinaria, un circuito usato per la prima volta da più di `MaxCircuitDirtiness` (10 minuti per impostazione predefinita) non riceve nuovi stream. **Eccezione importante**: per gli stream dotati di autenticazione SOCKS, l'opzione `KeepAliveIsolateSOCKSAuth` può mantenere utilizzabile il circuito mentre viene usato dalla stessa identità isolata; dopo l'ultimo stream resta ancora disponibile per il periodo di inattività previsto. Non bisogna quindi interpretare i dieci minuti come una durata massima assoluta del circuito [path-spec § Stream isolation and circuit sharing].

17.3.2 Scegliere i relay: con prudenza

Le opzioni `EntryNodes`, `ExitNodes` ed `ExcludeNodes` permettono di forzare o escludere relay, per impronta, nome, indirizzo o codice di paese; `StrictNodes` rende vincolante `ExcludeNodes` anche quando questo rompe delle funzionalità. Sono strumenti utili per esperimenti e per casi particolari, ma usarli «per sentirsi più sicuri» di solito peggiora le cose: restringere la scelta dei relay riduce la varietà dei percorsi, rende l'utente diverso da tutti gli altri e annulla le protezioni della selezione pesata e dei guard (capitolo 13).

17.3.3 Altre porte

`DNSPort` tor risponde a richieste DNS, risolvendo i nomi attraverso la rete (sezione 11.1.4).

`TransPort` riceve connessioni TCP *trasparenti*, dirottate dal firewall del sistema: è il meccanismo con cui sistemi come Whonix fanno passare attraverso Tor tutto il traffico delle applicazioni (capitolo 23).

`AutomapHostsOnResolve` assegna indirizzi fittizi ai nomi `.onion` richiesti via DNS, così che applicazioni che non parlano SOCKS possano raggiungere gli onion service tramite `TransPort`.

17.4 La porta di controllo

La `ControlPort` permette ad altri programmi di comandare tor mentre è in esecuzione: leggerne lo stato, chiedere nuovi circuiti, creare onion service al volo. È potentissima, e per questo va protetta. Due metodi:

Cookie Con `CookieAuthentication` 1, tor scrive nella propria directory un file di 32 byte casuali; chi può leggerlo può autenticarsi. La protezione coincide con i permessi del file.

Password Con `HashedControlPassword`, tor conserva l'hash di una password generato con `tor --hash-password`:

```
$ tor --hash-password prova
16:458528FE0C0AF3626059CFFBB8F30981515C780ACA77694803133AA336
```

L'hash appena mostrato è quello di una prova precedente con la password dimostrativa `prova`; non è una credenziale da riutilizzare. Il comando genera ogni volta un sale casuale, perciò l'output cambia anche usando la stessa password. **Non mettere una password reale in un comando copiato o nella cronologia della shell**: l'argomento può restare registrato o essere visibile temporaneamente ad altri processi. Se un'applicazione offre autenticazione mediante cookie su una porta locale, è preferibile usare quella e proteggere i permessi del file.

Come la porta SOCKS, anche la porta di controllo non va mai esposta sulla rete. Il protocollo di controllo è l'argomento del capitolo 18.

17.5 I log

tor registra ciò che fa a diversi livelli di dettaglio: `err`, `warn`, `notice`, `info`, `debug`. Il livello `notice` basta per l'uso quotidiano; `info` e `debug` sono preziosi per capire il protocollo, come abbiamo fatto nei laboratori, ma sono molto voluminosi. L'opzione `SafeLogging`, attiva per impostazione predefinita, sostituisce nei log gli indirizzi e le destinazioni con `[scrubbed]`: un log dimenticato su disco non deve diventare una prova delle attività dell'utente. La rete di laboratorio la disattiva (`SafeLogging 0`) proprio per poter leggere tutto.

I messaggi più utili all'avvio sono quelli di *bootstrap*, che mostrano le fasi della connessione alla rete fino a `Bootstrapped 100% (done): Done`. Se l'avvio si blocca a una certa percentuale, la fase indica dove cercare: per esempio, un blocco durante la connessione al primo relay indica di solito censura o un firewall, e suggerisce di usare i bridge (capitolo 16).

Riepilogo

- Per navigare si usa Tor Browser; il pacchetto `tor` serve per altre applicazioni, relay e onion service.
- Il `torrc` ha un'opzione per riga; il manuale è la fonte di riferimento, e `--verify-config` controlla la configurazione.
- Porte SOCKS e di controllo vanno tenute su localhost; la porta di controllo va protetta con cookie o password.
- L'isolamento degli stream evita che attività diverse condividano un circuito; Tor Browser lo ottiene con credenziali SOCKS diverse per ogni sito.
- Forzare o escludere relay riduce di solito la sicurezza; `SafeLogging` protegge i log.

Esercizi

Esercizio 17.1 ⋆ Nel container del laboratorio, crea un `torrc` con un errore di battitura in un nome di opzione e controllalo con `--verify-config`. Che messaggio ottieni? Prova poi `--dump-config short` sul `torrc` del client della rete.

Esercizio 17.2 ⋆⋆ Un utente usa la stessa `SocksPort` per il client di posta con il proprio indirizzo reale e per un forum con uno pseudonimo. Che cosa può dedurre un exit? Proponi due configurazioni che evitano il problema.

Esercizio 17.3 ⋆⋆ Perché `ExitNodes {it}`, cioè usare solo exit italiani, peggiora l'anonimato di un utente italiano che visita un sito italiano? Ragiona sul modello di minaccia del capitolo 4.

Per approfondire

- Il manuale di tor, `man tor`, e il file `doc/man/tor.1.txt` nei sorgenti.
- Le pagine di supporto del Tor Project sull'installazione e sui repository dei pacchetti, `https://support.torproject.org/`.

Capitolo 18

Control port e Stem

Aggiornato al 23 settembre 2026 — tor 0.4.9.13; control-spec; Stem 1.8.2

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- parlare con tor attraverso la porta di controllo, anche a mano;
- usare i comandi principali: `GETINFO`, `SETCONF`, `SIGNAL`, `ADD_ONION` e gli eventi;
- scrivere programmi Python con la libreria Stem;
- osservare dal vivo l'isolamento degli stream e l'effetto di una nuova identità (Laboratorio 11).

Tor Browser mostra il circuito della pagina corrente; strumenti come Nyx visualizzano la banda e le connessioni di un relay; i sistemi di misura della rete costruiscono circuiti su richiesta. Tutti usano la stessa interfaccia: il **protocollo di controllo** di tor, raggiungibile dalla `ControlPort` (sezione 17.4).

18.1 Il protocollo di controllo

Il protocollo è testuale, simile a quelli della posta elettronica [control-spec § Message format]: il client invia un comando per riga, tor risponde con righe che iniziano con un codice di tre cifre. Il codice `250` indica successo; un trattino dopo il codice (`250-`) indica che la risposta continua, uno spazio (`250 `) che è l'ultima riga; un `+` introduce un blocco di più righe, terminato da una riga con un solo punto. I codici `5xx` indicano errori, e `650` gli eventi asincroni.

Proviamolo sul client della rete di laboratorio, con una semplice connessione TCP alla porta 8008:

```
> PROTOCOLINFO 1
250-PROTOCOLINFO 1
250-AUTH METHODS=COOKIE,SAFECOOKIE COOKIEFILE="/lab/net/.../008c/control_auth_cookie"
250-VERSION Tor="0.4.9.13"
250 OK
> AUTHENTICATE <COOKIE_HEX>
250 OK
> GETINFO status/bootstrap-phase
250-status/bootstrap-phase=NOTICE BOOTSTRAP PROGRESS=100 TAG=done SUMMARY="Done"
250 OK
```

La trascrizione è un esempio **schematico**, non da incollare letteralmente: `<COOKIE_HEX>` va sostituito con i **64 caratteri esadecimali** dei 32 byte letti dal file `control_auth_cookie` della

propria istanza. La directory e la versione dipendono dalla configurazione. Non pubblicare mai il cookie: chi lo conosce può autenticarsi, finché rimane valido. In un programma preferisci `SAFECOOKIE`, che dimostra il possesso del cookie senza inviarlo a un possibile falso server [control-spec § Authentication]. `PROTOCOLINFO` è uno dei pochi comandi ammessi prima dell'autenticazione (gli altri sono `AUTHCHALLENGE`, `AUTHENTICATE` e `QUIT`) e indica quali metodi di autenticazione sono accettati e dove si trova il file del cookie. Con il metodo `COOKIE` il client invia il contenuto del file in esadecimale. Il metodo `SAFECOOKIE` evita di trasmettere il cookie stesso: client e tor si scambiano valori casuali e dimostrano di conoscere il cookie con un HMAC, così che un programma che si finge tor non possa rubarlo.

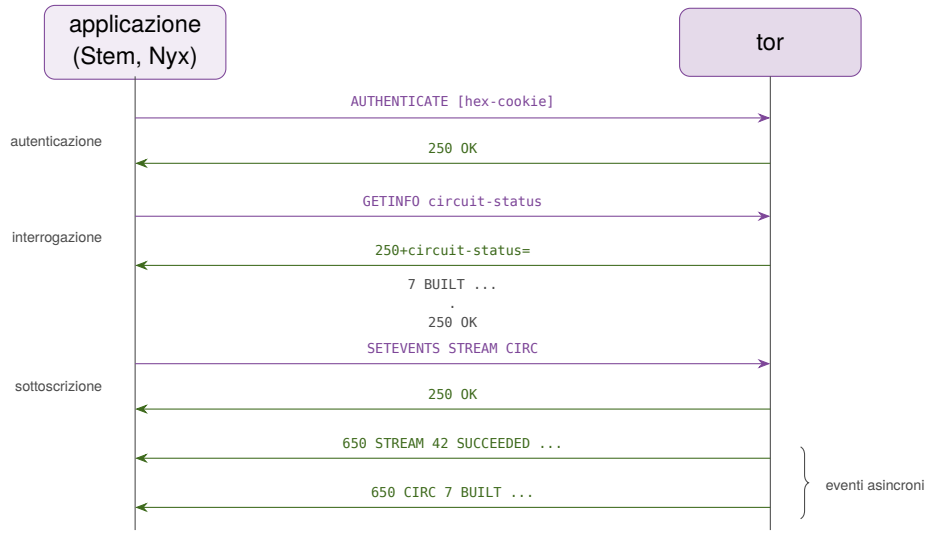


Figura 18.1: Sessione semplificata del protocollo di controllo: dopo l'autenticazione COOKIE (rappresentata da un segnaposto), GETINFO circuit-status può restituire dati multilinea, un punto isolato e 250 OK finale. SETEVENTS conferma la sottoscrizione con 250 OK; gli eventi 650 arrivano asincronicamente e possono intercalarsi ad altre risposte. ID e dati sono illustrativi.

18.1.1 I circuiti visti da tor

`GETINFO circuit-status` elenca i circuiti del client. Una riga, abbreviata:

```
3 BUILT $50160F8F...~test004r,$12415FEB...~test005r,$9449F36E...~test007e
  BUILD_FLAGS=NEED_CAPACITY,NEED_UPTIME PURPOSE=CONFLUX_LINKED
  CONFLUX_ID=DE224696F58F72950D70DDA2D7981726 CONFLUX_RTT=42538
```

Ogni relay è indicato dall'impronta della sua identità RSA e dal nome. In un tunnel Conflux le due «gambe» sono circuiti collegati con lo stesso `CONFLUX_ID`: la Proposta 329 raccomanda di non riutilizzare lo stesso guard o middle *quando possibile*, ma non garantisce in assoluto guard distinti. Tor ne misura il tempo di andata e ritorno (`CONFLUX_RTT`, in microsecondi) per decidere su quale inviare i dati (capitolo 31). Un dettaglio del codice di tor (`src/feature/control/control_fmt.c`): `CONFLUX_ID` mostra solo *metà* dell'identificativo segreto della coppia, perché chi lo conoscesse per intero potrebbe aggiungere «gambe» alla coppia da un exit e iniettare dati.

18.1.2 I comandi principali

Comando	Uso
`GETINFO`	legge informazioni: versione, stato del bootstrap, circuiti, stream, consenso, descrittori…
`GETCONF`, `SETCONF`	legge e modifica la configurazione in esecuzione
`SIGNAL`	invia segnali: `NEWNYM` (nuova identità), `RELOAD`, `SHUTDOWN`…
`SETEVENTS`	chiede di ricevere eventi: `CIRC`, `STREAM`, `BW`, `HS_DESC`…
`EXTENDCIRCUIT`	costruisce un circuito con un percorso scelto dal controllore
`ATTACHSTREAM`	assegna uno stream a un circuito (con `__LeaveStreamsUnattached`)
`ADD_ONION`, `DEL_ONION`	crea e rimuove onion service senza toccare il `torrc`
`HSFETCH`	scarica il descrittore di un onion service (usato nella sezione 14.9)

Attenzione: Chi controlla la porta controlla tor

Con la porta di controllo si può dire a tor quali relay usare, fargli costruire circuiti di un salto, leggere tutte le destinazioni degli stream. Un programma malevolo con accesso alla porta di controllo può annullare l'anonimato dell'utente. Per questo sistemi come Tails e Whonix mettono un filtro (onion-grater) tra le applicazioni e la porta di controllo, e lasciano passare solo i comandi strettamente necessari.

18.2 Stem

Stem è la libreria Python del Tor Project per il protocollo di controllo. Nasconde i dettagli del formato, interpreta le risposte e gli eventi e offre oggetti comodi da usare. La versione più recente, la 1.8.2, risale al giugno 2023: la libreria funziona con le versioni attuali di tor e viene usata da molti strumenti, ma non riceve nuove versioni da tempo, e per le funzionalità più recenti bisogna a volte ricorrere ai comandi grezzi (`Controller.msg`). Per Arti esiste un'interfaccia diversa, l'RPC di Arti (capitolo 30).

Un esempio minimo, che abbiamo già usato nella sezione 19.7.3:

```
from stem.control import Controller

with Controller.from_port(port=8008) as c:
    c.authenticate()                  # trova il cookie o chiede la password
    print(c.get_version())
    for circ in c.get_circuits():
        print(circ.id, circ.purpose, [nome for _, nome in circ.path])
```

Gli eventi si ricevono registrando una funzione. Il seguente è un *programma autonomo*: registra il listener mentre il controller è aperto e lo mantiene attivo per dieci secondi; la durata è solo dimostrativa.

```
import time
from stem.control import Controller, EventType

def su_circuito(evento):
    print(evento.id, evento.status, evento.purpose)

with Controller.from_port(port=8008) as c:
    c.authenticate()
    c.add_event_listener(su_circuito, EventType.CIRC)
    time.sleep(10)
```

La funzione viene chiamata dal thread degli eventi di Stem: ce n'è uno solo, che chiama in sequenza tutti i listener registrati. Un callback lento ritarda quindi la consegna degli eventi successivi, e se serve un'elaborazione pesante conviene delegarla a un thread a parte.

18.3 Laboratorio: l'isolamento degli stream dal vivo

Laboratorio 11: Isolamento degli stream e nuova identità

Obiettivo. Verificare con Stem, sul client della rete di laboratorio, quali richieste condividono un circuito a seconda delle credenziali SOCKS, e che cosa cambia dopo `SIGNAL NEWNYM`.
Prerequisiti. La rete di laboratorio avviata; un server web locale sulla porta 8080. Lo script è `labs/it/laboratorio/isolamento.py`, da eseguire nel container (che include Stem e `curl`).
Tempo stimato. 1 ora.

Passo 1: registrare gli stream. Ascoltiamo gli eventi `STREAM`; quando uno stream riesce, annotiamo su quale circuito è stato attaccato. Il filtro sulla destinazione è necessario: tor apre stream anche per conto proprio, per esempio per scaricare i documenti di directory.

```
assegnati = []
def su_stream(ev):
    if ev.status == StreamStatus.SUCCEEDED and ev.target == DESTINAZIONE:
        assegnati.append((ev.id, ev.circ_id, ev.target))

c.add_event_listener(su_stream, EventType.STREAM)
```

Il frammento mostra soltanto il callback: nel programma completo `labs/it/laboratorio/isolamento.py` sono già definiti `DESTINAZIONE`, `StreamStatus`, la lista `assegnati` e il controller `c` mantenuto aperto. Non eseguire il frammento isolatamente.

Passo 2: richieste con credenziali diverse. Facciamo quattro richieste con `curl`: due con le credenziali SOCKS `sito-a.example:x1`, una con `sito-b.example:x2`, una senza. Il principio riproduce l'isolamento per sito di Tor Browser: l'applicazione usa credenziali SOCKS differenziate secondo il contesto del sito principale. Non significa che *ogni scheda* abbia sempre un circuito distinto, né che due tunnel indipendenti debbano avere exit differenti.

```
$ curl --proxy-user sito-a.example:x1 --socks5-hostname 127.0.0.1:9008 http
    ://127.0.0.1:8080/
```

Passo 3: il risultato. Il seguente è un output di una precedente sessione di riferimento: identificatori Conflux, percorsi e ritardo NEWNYM non sono valori fissi.

```
credenziali sito-a.example  -> conflux AADCE71E  ['test003r', 'test004r', 'test007e']
credenziali sito-a.example  -> conflux AADCE71E  ['test003r', 'test004r', 'test007e']
credenziali sito-b.example  -> conflux 31B1E3E0  ['test003r', 'test004r', 'test007e']
credenziali -               -> conflux 35FD16D0  ['test003r', 'test004r', 'test007e']
(NEWNYM rinviato di 6 s)
dopo NEWNYM, sito-a.example  -> conflux F7011ABE
```

Le due richieste per «sito A» usano lo stesso tunnel, anche se sono state fatte in momenti diversi; quella per «sito B» e quella senza credenziali ne usano altri. Il percorso stampato è

quello di una delle due gambe del tunnel, e qui coincide perché la rete di laboratorio ha pochi relay; l'approfondimento che segue spiega perché conviene guardare il tunnel e non il singolo circuito.

Approfondimento: Il circuito non basta: i tunnel Conflux

Ripetendo il laboratorio molte volte si scopre che le due richieste con le stesse credenziali, ogni tanto, compaiono su circuiti con numeri diversi. Non è un difetto dell'isolamento. Chiedendo a tor l'elenco completo, si vede che quasi tutti i circuiti del client hanno scopo `CONFLUX_LINKED` e che compaiono *a coppie* con lo stesso `CONFLUX_ID`: sono le due «gambe» di un unico tunnel (sezione 31.4), e un evento `STREAM` può riportare l'una o l'altra. Ciò che l'isolamento tiene separato è dunque il *tunnel*, non la singola gamba: lo script del libro raggruppa infatti per `CONFLUX_ID`, quando c'è.
Restano due accorgimenti, entrambi verificabili nei log:

- `SIGNAL NEWNYM` è onorato al massimo una volta ogni dieci secondi (`MAX_SIGNEWNYM_RATE` in `src/core/mainloop/mainloop.c`); se arriva prima, tor lo rinvia e lo scrive nel log («Rate limiting NEWNYM request: delaying by N second(s)»). Chi non aspetta rischia di vedere il NEWNYM della prova precedente arrivare a metà di quella nuova. **NEWNYM non chiude automaticamente le connessioni già aperte**: impedisce invece ai nuovi stream di condividere i circuiti associati all'identità precedente.
- gli eventi `STREAM` riguardano anche gli stream che tor apre per conto proprio, per esempio per scaricare i documenti di directory: vanno filtrati per destinazione.

Anche così, su una rete piccola e molto sollecitata può capitare che tor preferisca per la seconda richiesta un tunnel appena costruito. È il motivo per cui la verifica automatica del libro ripete la prova.

Attenzione: Un errore nel laboratorio

Nella prima esecuzione tutte e quattro le richieste avevano usato lo stesso circuito, come se l'isolamento non funzionasse. Il problema era nel nostro script: avevamo inserito `--proxy-user` subito dopo l'opzione `-o` di `curl`, che l'aveva presa come nome del file di uscita, e le credenziali non erano mai state inviate. Prima di concludere che un sistema non si comporta come dichiarato, bisogna verificare che l'esperimento stia davvero facendo ciò che crediamo.

Riepilogo

- Il protocollo di controllo è testuale: comandi su una riga, risposte con codici a tre cifre, eventi asincroni con il codice 650.
- Ci si autentica con cookie (meglio `SAFECOOKIE`) o password; chi controlla la porta controlla tor.
- Stem offre un'interfaccia Python a comandi ed eventi; è stabile ma non riceve nuove versioni dal 2023.
- L'isolamento per credenziali SOCKS impedisce la condivisione dello stesso circuito fra contesti distinti; con Conflux occorre osservare il tunnel e non una singola gamba. `NEWNYM` influisce sui *nuovi* stream, con un limite di frequenza di circa dieci secondi. Non garantisce exit differenti né interrompe gli stream in corso.

Esercizi

Esercizio 18.1 ⋆ Interpreta la risposta `552 Unrecognized key "pippo"` a un comando `GETINFO pippo`. Che cosa distingue un codice `5xx` da un evento `650`?

Esercizio 18.2 ⋆⋆ Con Stem, crea un onion service temporaneo con `create_ephemeral_hidden_service` sul client della rete di laboratorio, che inoltri la porta 80 al server locale sulla 8080. Raggiungilo con `curl`. Dove sono conservate le sue chiavi?

Esercizio 18.3 ⋆⋆ Scrivi un programma che mostri ogni secondo la banda in ingresso e in uscita del client, usando gli eventi `BW`, mentre scarichi un file grande attraverso la rete di laboratorio.

Esercizio 18.4 ⋆⋆⋆ Con `__LeaveStreamsUnattached` 1, `EXTENDCIRCUIT` e `ATTACHSTREAM`, fai passare una richiesta attraverso un percorso scelto da te (per esempio `test004r`, `test005r`, `test006e`). Perché in un uso reale questa possibilità è pericolosa per l'anonimato?

Per approfondire

- *Tor's control protocol* (control-spec) [Thef]: tutti i comandi e gli eventi.
- La documentazione di Stem, `https://stem.torproject.org/`.

Capitolo 19

Una rete Tor privata: Chutney e Shadow

Aggiornato al 23 settembre 2026 — tor 0.4.9.13; Chutney commit `6cc1588`; Shadow 3.3.0

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- spiegare perché gli esperimenti su Tor si fanno su una rete privata e non sulla rete pubblica;
- costruire e avviare, con due comandi distinti, la rete di laboratorio usata in tutto il libro;
- riconoscere le opzioni di configurazione che rendono una rete di test diversa dalla rete reale;
- orientarti tra i file di stato di un nodo: chiavi, consenso, log;
- osservare i circuiti di un client con Stem e far passare traffico web e onion attraverso la rete;
- capire quando serve un simulatore come Shadow invece di una rete emulata come Chutney.

Questo capitolo si trova nella Parte III, ma conviene leggerlo presto: tutti i laboratori del libro, a partire dal Laboratorio 1 del capitolo 7, si svolgono sulla rete che costruiremo qui. Chi sta seguendo la Parte II può leggere le sezioni 19.1 e 19.3 e tornare al resto più avanti.

19.1 Perché una rete privata

La rete Tor pubblica è fatta di migliaia di relay gestiti da volontari e usata ogni giorno da milioni di persone, molte delle quali dipendono da essa per la propria sicurezza. Sperimentare su di essa ha due problemi, uno etico e uno pratico.

Il problema etico. Ogni esperimento che misura, altera o anche solo osserva il traffico della rete reale coinvolge utenti che non hanno dato il consenso e che potrebbero essere danneggiati. Anche un esperimento innocuo nelle intenzioni, per esempio un relay che registra più dati del necessario, produce informazioni che, se trapelassero, potrebbero servire a deanonimizzare qualcuno. Per questo il Tor Project mantiene il *Tor Research Safety Board*, che offre ai ricercatori una revisione dei piani sperimentali, e raccomanda una regola generale: se una domanda di ricerca può essere affrontata senza toccare utenti reali, va affrontata così. Torneremo su questi principi nel capitolo 29.

Il problema pratico. Sulla rete reale non controlliamo niente: non sappiamo quali relay verranno scelti, i circuiti cambiano, il consenso si aggiorna ogni ora, e ogni esperimento è irripetibile. In una rete privata possiamo invece decidere quanti nodi ci sono, leggere i log con il massimo livello di dettaglio, fermare e riavviare tutto in pochi minuti, e ripetere lo stesso esperimento finché non lo abbiamo capito.

Attenzione: Regola dei laboratori

Tutti gli esercizi pratici di questo libro che inviano celle costruite a mano, modificano il comportamento di un nodo o raccolgono traffico si svolgono **esclusivamente** sulla rete privata. Il codice didattico `torlab` non protegge l'anonimato e non va mai usato sulla rete pubblica.

19.2 Emulare o simulare

Esistono due modi di costruire una rete Tor in laboratorio, e il Tor Project li usa entrambi.

Emulazione Si avviano davvero molti processi `tor` sulla stessa macchina, che comunicano tramite l'interfaccia di loopback. Il tempo è quello reale, il codice è quello reale, la rete è quella del sistema operativo. È l'approccio di **Chutney**, uno strumento del Tor Project nato per i test di integrazione di tor e Arti.

Simulazione Si eseguono sempre i programmi reali, ma dentro un simulatore a eventi discreti che intercetta le loro chiamate di sistema e fornisce una rete virtuale e un orologio virtuale. È l'approccio di **Shadow** [JH12], nato per studiare la rete Tor su larga scala.

	Chutney (emulazione)	Shadow (simulazione)
Scala tipica	da qualche nodo a qualche decina	da centinaia a migliaia di nodi
Tempo	reale	virtuale, riproducibile a parità di condizioni
Rete	loopback, senza latenza né perdite	topologia con latenze e banda configurabili
Ripetibilità	limitata (scheduling del sistema)	progettata per essere deterministica con stessi input, seme e versione
Uso tipico	sviluppo, test, didattica	ricerca sulle prestazioni e sugli attacchi
Nel libro	tutti i laboratori	sezione 19.9 e Parte V

Tabella 19.1: Chutney e Shadow a confronto.

Per imparare il protocollo, Chutney è la scelta naturale: è leggero, si avvia in un paio di minuti e ci lascia interagire con i nodi mentre sono in funzione. Il resto del capitolo è dedicato a Chutney; Shadow è presentato alla fine.

19.3 Il laboratorio del libro

Per evitare differenze tra un sistema e l'altro, l'intero laboratorio è racchiuso in un'immagine Docker, descritta dal file `labs/it/laboratorio/Dockerfile` del repository del libro. L'immagine:

- compila tor **dai sorgenti ufficiali**, alla versione di riferimento del libro (0.4.9.13), con l'opzione `--enable-gpl` che include il modulo di prova di lavoro degli onion service (capitolo 15);

- installa Chutney a una revisione fissata, più Stem e gli strumenti di cattura del traffico (`tcpdump`, `tshark`) usati nel capitolo 20.

Fissare le versioni è importante per replicare formati e comportamenti: **non garantisce che due esecuzioni producano gli stessi byte di chiavi, firme, handshake o traffico**, che dipendono anche dalla casualità e dallo stato di ciascuna rete.

Approfondimento: Quando esce una nuova versione di tor

Gli output riportati in questo capitolo e nella Parte II provengono dalla rete di laboratorio con tor 0.4.9.13, la versione di riferimento del libro. Alcuni sono stati registrati con la precedente 0.4.9.12, uscita pochi giorni prima, e poi riprodotti: lo script `scripts/verify_labs.py`, che riesegue dal vivo tutti i laboratori e ne controlla i risultati, dà gli stessi esiti con entrambe le versioni (oggi 53 controlli, più 14 per il laboratorio sul bridge). Cambiano solo gli elementi che dipendono dalla singola esecuzione: chiavi, impronte, indirizzi `.onion`, orari e dimensioni dei certificati, che variano perfino da un relay all'altro. Lo stesso script ti permette di verificare, con la versione che userai tu, che quanto descritto nel libro sia ancora vero.

19.3.1 Requisiti

Serve un sistema Linux con Docker e circa 1 GB di spazio libero; su macOS e Windows si può usare Docker Desktop, ma la rete «host» usata dal laboratorio funziona in modo diverso e richiede di lavorare dall'interno del container (sezione 19.7.4). La rete occupa sulla macchina porte nelle serie 5100, 7100, 8000 e 9000: conviene verificare che siano libere. **Su Linux il container usa `--network host`: condivide la rete dell'host, quindi non crea una barriera di isolamento dal traffico esterno.** Il laboratorio va eseguito su un ambiente fidato, senza esporre i suoi servizi e senza indirizzare gli esperimenti alla rete Tor pubblica.

19.3.2 Costruire l'immagine e avviare la rete

Dalla cartella `labs/it/laboratorio`:

```
$ make immagine        # compila tor e prepara Chutney (una volta sola)
$ make avvia           # configura e avvia la rete, attende il bootstrap
```

La compilazione dell'immagine richiede alcuni minuti. L'avvio della rete, sulla macchina su cui è stato scritto questo capitolo, richiede circa due minuti e mezzo e termina con un riepilogo di questo tipo (abbreviato):

```
  test006e:
    bootstrap progress: 100 (SUCCESS), done, Done
    desc publishing: 100 (SUCCESS), all nodes, DESC MD MD_CONS NS_CONS
  test008c:
    bootstrap progress: 100 (SUCCESS), done, Done
  test009h:
    bootstrap progress: 100 (SUCCESS), done, Done
    oniondesc publishing: 200 (ONIONDESC_PUBLISHED), hidden service v3
```

Ogni nodo ha raggiunto il 100 % del bootstrap, ogni relay vede il proprio descrittore pubblicato nel consenso e l'onion service ha pubblicato il suo descrittore. Il comando `make verifica` esegue poi il test di Chutney, che fa passare dati attraverso un exit e attraverso l'onion service e controlla che arrivino intatti:

```
  not-done:0 successes:4 failures:0
Transmission: Success
```

`make stato` mostra lo stato dei nodi, `make shell` apre una shell nel container, `make ferma` spegne la rete e `make pulisci` cancella anche tutti i dati generati. **Attenzione**: `pulisci` elimina `labs/it/laboratorio/net/` con chiavi di identità, cookie e log; usalo solo su una rete sacrificabile o dopo averne fatto un backup. Il target `avvia` non è pensato per essere lanciato una seconda volta su un container con lo stesso nome ancora attivo.

19.4 Com'è fatta la rete

La rete del libro è descritta da un piccolo file Python, `labs/it/laboratorio/chutney/networks/libro`, nel linguaggio di configurazione di Chutney:

```
Authority = Node(tag="a", authority=1, relay=1)
Middle    = Node(tag="r", relay=1)
Exit      = Node(tag="e", relay=1, exit=1)
Client    = Node(tag="c", client=1, launch_phase=2)
Onion     = Node(tag="h", hs=1, launch_phase=2)

NODES = (Authority.getN(3) + Middle.getN(3) + Exit.getN(2)
         + Client.getN(1) + Onion.getN(1))
ConfigureNodes(NODES)
```

Ogni `Node` è un modello; `getN` ne crea più copie. Il client e l'onion service partono in una seconda fase (`launch_phase=2`), quando le autorità hanno già prodotto un consenso. Chutney numera i nodi in ordine e dà a ciascuno un nome che combina il numero e il tag: `test000a`, ..., `test009h`. Il numero determina anche le porte, come mostra la fig. 19.1.

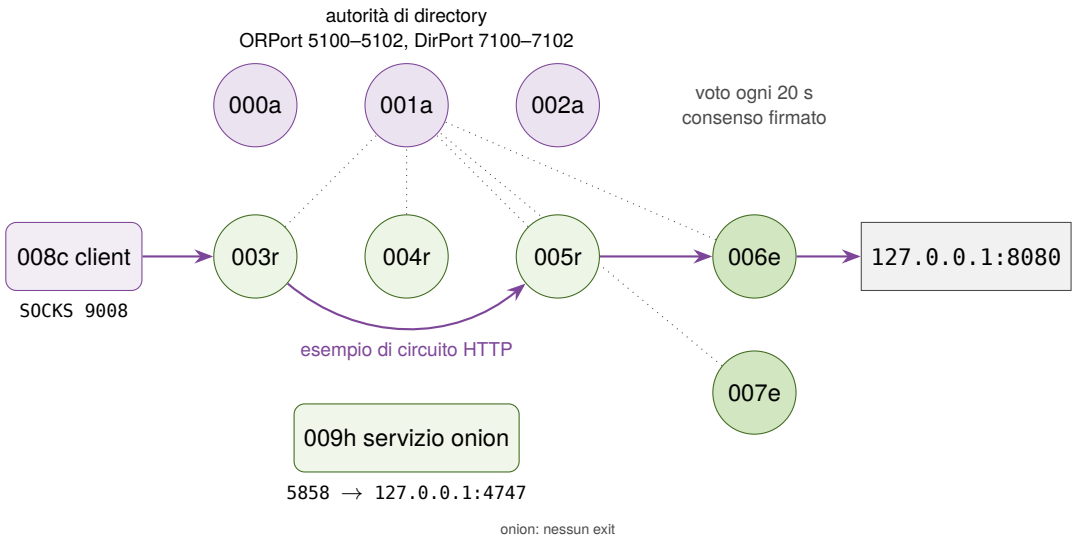


Figura 19.1: Topologia Chutney del laboratorio sulla sola interfaccia `127.0.0.1`: tre directory authority/relay, tre relay intermedi, due exit, un client e un servizio onion. Le ORPort $5100+n$ riguardano soltanto autorità e relay (inclusi gli exit); tutti i nodi hanno ControlPort $8000+n$. Le autorità usano DirPort $7100+n$, il client SocksPort 9008 e il servizio onion la porta virtuale 5858 verso `127.0.0.1:4747`. Il percorso HTTP via exit e il servizio onion sono distinti: quest'ultimo non impiega exit relay. La rete usa il loopback, non un isolamento di rete del container.

Nodo	Ruolo	ORPort	ControlPort	Altro
`test000a–002a`	directory authority	5100–5102	8000–8002	DirPort 7100–7102
`test003r–005r`	relay intermedio	5103–5105	8003–8005	
`test006e–007e`	exit	5106–5107	8006–8007	
`test008c`	client	—	8008	SocksPort 9008
`test009h`	onion service	—	8009	porta virtuale 5858

Approfondimento: Perché tre autorità

Il consenso è un documento firmato dalla maggioranza delle directory authority (capitolo 12). Con una sola autorità non vedremmo il processo di voto; con tre possiamo osservarlo e, in un esercizio, vedere che cosa succede se una delle tre si ferma. La rete pubblica ne ha nove, più un'autorità dedicata ai bridge; l'elenco è compilato direttamente nel codice di tor (`src/app/config/auth_dirs.inc`).

19.5 Una rete di test non è la rete reale

Chutney genera un file `torrc` per ogni nodo. Leggerli è istruttivo, perché mostrano esattamente che cosa bisogna cambiare per far funzionare una rete Tor in miniatura, e quindi quali ipotesi della rete reale vengono meno. Ecco le opzioni più significative del torrc di un'autorità (`net/nodes/000a/torrc`):

```
TestingTorNetwork 1
AuthoritativeDirectory 1
V3AuthoritativeDirectory 1
AssumeReachable 1
V3AuthVotingInterval 20 seconds
V3AuthVoteDelay 4
V3AuthDistDelay 4
TestingDirAuthVoteGuard *
TestingDirAuthVoteHSDir *
```

`TestingTorNetwork` Attiva la modalità di test. Può essere usata solo insieme a un elenco esplicito di directory authority (`DirAuthority`), così che un nodo di test non possa mai unirsi per errore alla rete pubblica. Cambia i valori predefiniti di molte opzioni per rendere tutto più rapido.

`V3AuthVotingInterval` Nella rete reale le autorità votano un nuovo consenso ogni ora; qui ogni 20 secondi. Lo si vede nel consenso stesso:

```
valid-after 2026-09-23 14:37:40
fresh-until 2026-09-23 14:38:00
valid-until 2026-09-23 14:38:20
```

`AssumeReachable` Un relay normalmente verifica di essere raggiungibile dall'esterno prima di pubblicarsi. Su loopback questo test non ha senso e viene saltato.

`TestingDirAuthVoteGuard *` Nella rete reale il flag Guard si guadagna con settimane di stabilità e banda sufficiente. Qui le autorità lo assegnano a tutti. Conseguenza: **nel nostro consenso ogni relay è un possibile guard**, cosa impossibile nella rete reale.

Il torrc degli exit contiene inoltre, *esclusivamente per l'ambiente privato*:

```
ExitPolicyRejectPrivate 0
ExitPolicy accept private:*
```

Per impostazione predefinita un exit rifiuta le connessioni verso indirizzi privati e di loopback, per non esporre la rete locale su cui si trova. Nel laboratorio le *destinazioni degli esperimenti* sono servizi sulla `127.0.0.1`, e la regola va invertita. **Questo non rende il container isolato da Internet**: con `--network host` l'ambiente condivide lo stack di rete dell'host; la sicurezza dipende dalla configurazione delle destinazioni e dalle restrizioni dell'host.

Attenzione: Non copiare queste opzioni altrove

Le opzioni `Testing…`, `AssumeReachable` e soprattutto `ExitPolicy accept private:*` sono pensate solo per le reti di test. Su un relay pubblico renderebbero il nodo scorretto o pericoloso per la rete su cui è ospitato.

19.6 Dentro la directory di un nodo

Ogni nodo ha la propria directory dati in `labs/it/laboratorio/net/nodes/NNNx/`. Quella di un'autorità contiene, tra l'altro:

File o cartella	Contenuto
`torrc`	configurazione generata da Chutney
`keys/`	chiavi di identità e di onion del nodo (capitoli 5 e 6)
`fingerprint`, `fingerprint-ed25519`	impronte delle identità RSA ed Ed25519
`cached-consensus`, `cached-microdesc-consensus`	i due tipi di consenso (capitolo 12)
`cached-microdescs.new`	microdescrittori scaricati
`v3-status-votes`	i voti delle autorità (solo autorità)
`sr-state`	stato del protocollo di casualità condivisa (solo autorità)
`notice.log`, `info.log`	log ai livelli *notice* e *info*
`control_auth_cookie`	cookie per l'autenticazione sulla porta di controllo

La directory dell'onion service contiene in più `hidden_service/`, con le chiavi e il file `hostname` che riporta l'indirizzo `.onion`, per esempio:

```
rpns2r36zn5u46wctlgnvtwjotxaf7iflsc3k5eas3lqoxeesv4zwyad.onion
```

L'indirizzo cambia a ogni nuova configurazione, perché deriva da una chiave generata al momento (capitolo 14).

Il consenso è un file di testo leggibile. Estraendo le righe `r` (identità e indirizzo del relay) e `s` (flag) otteniamo, per due dei nostri relay:

```
r test000a H+9DDHx4rEo6I6Jdn8gIyKyi3jk ... 127.0.0.1 5100 7100
s Authority Fast Guard HSDir Running Stable V2Dir Valid
r test006e dEPHoFFHamQyy6pRzlsLB9/ilzU ... 127.0.0.1 5106 0
s Exit Fast Guard HSDir Running Stable V2Dir Valid
```

Notiamo il flag Guard ovunque, come previsto, e il flag Exit solo sui due exit. Il significato di ogni campo è l'oggetto del capitolo 12.

19.7 Usare la rete

19.7.1 Navigare attraverso la rete privata

Per evitare differenze tra le variabili d'ambiente dell'host (in particolare `NO_PROXY`) e il container, esegui entrambi gli esempi HTTP **nella medesima shell** del laboratorio (`make shell`). L'opzione `--noproxy ""` forza curl a usare il proxy SOCKS anche quando la destinazione è `127.0.0.1`. **Non avviare `python -m http.server` nella cartella `/lab`**: per impostazione predefinita servirebbe i file presenti in quella directory, che comprende la rete Chutney con chiavi e cookie. Creiamo invece una cartella temporanea contenente *solo* il file innocuo della prova, e diciamo esplicitamente al server di servire quella. Il server della porta 4747 userà la stessa cartella.

Il client `test008c` espone una porta SOCKS sulla 9008. Qualsiasi programma che parli SOCKS5 può usarla. Avviamo un piccolo server web locale e raggiungiamolo attraverso Tor:

```
$ WEBROOT=$(mktemp -d /tmp/torlibro-web.XXXXXX)
$ printf 'Ciao dalla rete privata\n' > "$WEBROOT/index.html"
$ python3 -m http.server 8080 --bind 127.0.0.1 --directory "$WEBROOT" &
$ SERVER_HTTP_PID=$!
$ curl --noproxy "" --socks5-hostname 127.0.0.1:9008 http://127.0.0.1:8080/index.html
Ciao dalla rete privata
```

La richiesta ha attraversato un circuito di tre relay ed è uscita da uno dei due exit, che si è collegato al server. L'opzione `--socks5-hostname`, invece di `--socks5`, fa sì che sia l'exit, e non il nostro sistema, a risolvere il nome di destinazione: nella rete reale è essenziale per non rivelare al proprio resolver DNS i siti visitati (capitolo 11).

19.7.2 Raggiungere l'onion service

L'onion service è configurato da Chutney con la direttiva

```
HiddenServicePort 5858 127.0.0.1:4747
```

che inoltra la porta virtuale 5858 dell'indirizzo `.onion` alla porta locale 4747. Basta quindi un server su quella porta:

```
$ python3 -m http.server 4747 --bind 127.0.0.1 --directory "$WEBROOT" &
$ SERVER_ONION_PID=$!
$ curl --noproxy "" --socks5-hostname 127.0.0.1:9008 \
    http://$(cat net/nodes/009h/hidden_service/hostname):5858/index.html
Ciao dalla rete privata
```

Questa volta nessun exit è coinvolto: client e servizio si incontrano in un punto di rendezvous all'interno della rete (capitolo 14). Terminata la prova, **nella stessa shell** ferma i due server con `kill "$SERVER_HTTP_PID" "$SERVER_ONION_PID"` e rimuovi soltanto la cartella temporanea, dopo averne verificato il percorso. Non cancellare `/lab/net`: contiene lo stato della rete.

19.7.3 Osservare i circuiti con Stem

La porta di controllo permette di interrogare un nodo mentre è in funzione. Con la libreria Stem (capitolo 18), installata nell'immagine, elenchiamo i circuiti del client:

```
from stem.control import Controller

with Controller.from_port(port=8008) as c:
    c.authenticate()                    # usa il cookie in automatico
```

```
    print(c.get_version())
    for circ in c.get_circuits():
        print(circ.id, circ.status, circ.purpose,
              [nome for _, nome in circ.path])
```

Un estratto dell'output, dopo l'esecuzione di `make verifica`:

```
0.4.9.13 (git-3c575400909efe65)
10 BUILT GENERAL ['test004r', 'test005r', 'test007e']
124 BUILT CONFLUX_LINKED ['test006e', 'test004r', 'test007e']
114 BUILT CONFLUX_UNLINKED ['test007e', 'test004r', 'test006e']
128 BUILT CIRCUIT_PADDING ['test004r', 'test001a', 'test007e', 'test006e']
163 BUILT HS_CLIENT_REND ['test004r', 'test001a', 'test006e']
166 BUILT HS_CLIENT_HSDIR ['test004r', 'test006e', 'test007e', 'test003r']
181 BUILT HS_VANGUARDS ['test004r', 'test007e', 'test006e']
```

Anche in una rete così piccola, questo elenco è un'anteprima di buona parte del libro:

- molti circuiti iniziano da `test004r`: è uno dei **guard** del client, riusato fra i percorsi (capitolo 13). Nell'output di questa sessione compaiono anche altri primi hop, incluso nelle gambe Conflux; la Proposta 329 raccomanda guard e middle distinti tra le gambe quando possibile, ma ciò non implica una regola assoluta né permette di dedurre tutti i circuiti Conflux da queste sole righe (capitolo 18);
- i circuiti `GENERAL` a tre salti servono il traffico ordinario; quelli a un solo salto (qui non mostrati) servono a scaricare informazioni di directory;
- i circuiti `CONFLUX_LINKED` sono coppie di circuiti usati insieme per lo stesso traffico, la funzionalità Conflux (Proposta 329, capitolo 31);
- `HS_VANGUARDS`, `HS_CLIENT_HSDIR` e `HS_CLIENT_REND` sono i circuiti usati per raggiungere l'onion service (capitolo 14);
- `CIRCUIT_PADDING` indica, secondo la specifica del protocollo di controllo, un circuito tenuto aperto per mascherare il momento in cui è stato davvero chiuso: un osservatore non vede la chiusura coincidere con la fine dell'uso.

19.7.4 Lavorare dall'interno del container

Il container usa la rete dell'host, quindi le porte dei nodi sono raggiungibili direttamente dalla macchina. Se questo non è possibile (per esempio con Docker Desktop), `make shell` apre una shell nel container, dove sono disponibili tor, Chutney, Stem, Python e gli strumenti di cattura, e i file del laboratorio sono in `/lab`.

19.8 Un primo contatto con un relay

Il listato Python seguente usa il pacchetto didattico `torlab`, che si trova in `labs/it/codice/` e *non* viene montato automaticamente nel container Chutney. Su Linux con rete host puoi salvarlo come script ed eseguirlo dalla radice del repository con `PYTHONPATH=labs/it/codice python3 script.py`; in alternativa, rendi esplicitamente disponibile il pacchetto nel container prima di eseguirlo. È soltanto un esempio di rete privata.

Chiudiamo il cerchio con il capitolo 8. Con il modulo `torlab/celle.py` del Laboratorio 2 possiamo aprire una connessione TLS verso l'ORPort di un relay della rete, inviare una cella **`VERSIONS`** e decodificare quello che risponde. Il codice completo, con la verifica dei certificati, è l'oggetto del Laboratorio 1 del capitolo 7; qui ci basta osservare:

```python
import socket, ssl
from torlab.celle import *

ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE   # solo rete privata, nessuna verifica TLS
s = ctx.wrap_socket(socket.create_connection(("127.0.0.1", 5103)))
s.sendall(codifica(cella_versions([3, 4, 5]), 0))

buf, versione, ricevute = b"", 0, 0
while ricevute < 4:
    chunk = s.recv(4096)
    if not chunk:
        raise ConnectionError("TLS channel closed early")
    buf += chunk
    while (r := decodifica(buf, versione)) is not None:
        cella, buf = r
        ricevute += 1
        print(descrivi(cella, versione))
        if cella.comando == Comando.VERSIONS:
            versione = negozia_versione([3, 4, 5], cella.corpo)
s.close()
```

Attenzione: questa breve dimostrazione disattiva intenzionalmente la verifica TLS con `ssl.CERT_NONE` e *non* implementa la verifica delle identità tramite la cella CERTS. Non usare il frammento verso relay pubblici; per una sessione autenticata occorre la verifica completa illustrata nel Laboratorio 1.

L'output seguente è una registrazione di riferimento precedente: impronte, ID dei circuiti, indirizzi onion, orari e dimensioni dei certificati possono variare nei tuoi esperimenti.

Il relay `test003r` risponde:

```
CircID=0x0000 VERSIONS corpo=6 byte
CircID=0x00000000 CERTS corpo=1451 byte
CircID=0x00000000 AUTH_CHALLENGE corpo=36 byte
CircID=0x00000000 NETINFO corpo=509 byte
```

È esattamente la sequenza descritta nella sezione 8.8: la cella **`VERSIONS`** con CircID di 2 byte, poi, negoziata la versione 5, le celle con CircID di 4 byte. **`CERTS`** e **`AUTH_CHALLENGE`** sono variabili e hanno il loro corpo esatto; **`NETINFO`** è fissa e arriva con tutti i suoi 509 byte. Nota l'assegnamento `versione = ...` subito dopo **`VERSIONS`**: la decodifica delle celle successive dipende dalla versione appena negoziata.

19.9 Shadow: la rete in un simulatore

Chutney non basta quando la domanda riguarda il comportamento della rete *nel suo insieme*: come cambia la latenza se si modifica l'algoritmo di controllo della congestione? Quanto traffico serve a un avversario per un certo attacco? Servono migliaia di nodi, latenze realistiche tra continenti diversi, e soprattutto esperimenti ripetibili.

Shadow, sviluppato originariamente da Rob Jansen e Nicholas Hopper [JH12], risponde a queste esigenze. Nelle versioni attuali esegue i programmi reali, tor compreso, come normali processi Linux, ma ne intercetta le chiamate di sistema legate a rete e tempo: la rete è un grafo virtuale con latenze e bande definite dall'utente, e il tempo avanza solo quando il simulatore lo decide. Il risultato è che un'ora di rete simulata può durare più o meno di un'ora reale, a seconda della potenza di calcolo. Shadow è progettato per fornire esecuzioni deterministiche *a parità*

di input, configurazione, software e seme casuale; la ripetibilità va comunque verificata, perché bug, dipendenze non modellate e modifiche all'ambiente possono introdurre differenze.

Per costruire reti Tor realistiche su Shadow si usa **tornettools**, che a partire dai dati pubblici di Tor Metrics (capitolo 24) genera una rete in scala ridotta con la stessa distribuzione di banda, flag e ruoli della rete reale.

Nel capitolo 29 useremo Shadow per un esperimento guidato; per ora ci basta sapere che esiste e quando serve.

Riepilogo

- Gli esperimenti su Tor si fanno su reti private: sulla rete pubblica coinvolgerebbero utenti reali senza consenso e sarebbero irripetibili.
- Chutney emula una rete Tor avviando processi `tor` veri su loopback; Shadow li esegue in un simulatore a eventi discreti, adatto a reti grandi e a esperimenti ripetibili.
- Il laboratorio del libro (`labs/it/laboratorio/`) compila tor 0.4.9.13 dai sorgenti e avvia con `make avvia` una rete di 10 nodi: 3 autorità, 3 relay intermedi, 2 exit, un client e un onion service.
- La numerazione assegna ControlPort $8000 + n$ a tutti i nodi; ORPort $5100 + n$ *solo* alle autorità e ai relay, e SocksPort $9000 + n$ *solo* ai client configurati.
- Le opzioni di test (`TestingTorNetwork`, voto ogni 20 secondi, Guard a tutti, exit verso indirizzi privati) rendono la rete funzionante in miniatura ma diversa da quella reale: le misure fatte qui non si trasferiscono alla rete pubblica senza cautela.
- Con la SocksPort 9008 si naviga attraverso la rete; con Stem si osservano i circuiti; con `torlab` si parla direttamente con i relay.

Esercizi

Esercizio 19.1 ⋆ Senza guardare il torrc, indica ORPort e ControlPort del nodo `test006e`. Il nodo `test009h` ha una ORPort? Perché?

Esercizio 19.2 ⋆ Nella rete privata tutti i relay hanno il flag Guard. Quale opzione lo causa, e perché nella rete reale non accade? Quale conseguenza ha sulle misure della selezione del percorso fatte in laboratorio?

Esercizio 19.3 ⋆ Ripeti la richiesta `curl` della sezione 19.7 usando `--socks5` invece di `--socks5-hostname` e con `localhost` al posto di `127.0.0.1`. Che cosa cambia in ciò che l'exit riceve? Perché nella rete reale è importante?

Esercizio 19.4 ⋆⋆ Osserva il campo `valid-after` del consenso del client per alcuni minuti (`grep valid-after net/nodes/008c/cached-microdesc-consensus`). Ogni quanto cambia? Calcola quanti consensi vengono prodotti in un giorno nella rete di laboratorio e nella rete reale.

Esercizio 19.5 ⋆⋆ Ferma una delle tre autorità (`make shell`, poi `kill` del processo corrispondente, usando il file `pid`). La rete continua a produrre consensi? E se ne fermi due? Spiega il risultato; ne ritroverai la ragione nel capitolo 12.

Esercizio 19.6 ⋆⋆ Modifica lo script Stem della sezione 19.7.3 perché stampi, per ogni circuito a tre salti, anche le ORPort dei relay attraversati, ricavandole dal consenso con `get_network_status`.

Esercizio 19.7 ⋆⋆ Crea una nuova rete `labs/it/laboratorio/chutney/networks/libro-grande` con 5 autorità, 10 relay intermedi e 4 exit, e avviala impostando `RETE=libro-grande` nel `make`. Quanto tempo impiega il bootstrap rispetto alla rete del libro?

Esercizio 19.8 ⋆⋆⋆ Nel file `info.log` del client individua i messaggi relativi alla costruzione di un circuito, dalla scelta dei nodi fino al completamento. Collega ogni messaggio a una delle celle della fig. 8.5.

Esercizio 19.9 ⋆⋆⋆ Dall'interno del container cattura con `tcpdump -i lo` il traffico verso la porta 5103 mentre esegui lo script della sezione 19.8. Riesci a vedere le celle? Perché no? Che cosa resta osservabile (dimensioni, tempi)? Il capitolo 20 mostra come andare oltre.

Per approfondire

- Chutney, `https://gitlab.torproject.org/tpo/core/chutney`: il README e la cartella `networks/` contengono molte reti di esempio, anche con bridge e Arti.
- R. Jansen, N. Hopper, *Shadow: Running Tor in a Box for Accurate and Efficient Experimentation* [JH12].
- Documentazione di Shadow e tornettools, `https://shadow.github.io/`.
- Tor Research Safety Board, `https://research.torproject.org/safetyboard/`: linee guida per la ricerca etica su Tor.

Capitolo 20

Osservare il traffico: Wireshark e log

Aggiornato al 23 settembre 2026 — tor 0.4.9.13; tshark (Debian trixie); rete di laboratorio

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- catturare il traffico di Tor con `tcpdump` e analizzarlo con `tshark` o Wireshark;
- distinguere ciò che un osservatore esterno vede da ciò che resta nascosto;
- decifrare il TLS del *tuo* canale con le chiavi di sessione e leggere le celle che contiene (Laboratorio 12);
- usare i log di tor per seguire il protocollo passo per passo.

Nei capitoli precedenti abbiamo visto il protocollo «dall'interno», scrivendo il codice che lo parla. In questo capitolo lo guardiamo «dall'esterno», come lo vede chi osserva la rete. È un esercizio utile per due ragioni opposte: capire che cosa un avversario può ricavare dal traffico, e disporre di uno strumento di verifica indipendente per il proprio codice.

Attenzione: Solo il tuo traffico, solo nel laboratorio

Catturare il traffico di altre persone senza autorizzazione è illecito in Italia come nella maggior parte dei paesi (capitolo 25), e su una rete di anonimato è anche contrario all'etica della ricerca (sezione 19.1). Tutte le catture di questo capitolo riguardano la rete privata del libro o connessioni aperte da noi.

20.1 Che cosa vede un osservatore

Chi osserva il collegamento tra un client e il suo guard, per esempio il fornitore di connettività, vede:

- gli indirizzi IP e le porte: il client si collega a un indirizzo che, se non è un bridge, compare nel consenso pubblico ed è quindi riconoscibile come relay Tor;
- l'handshake TLS: con TLS 1.3 i certificati viaggiano cifrati, ma l'inizio del messaggio `ClientHello` resta visibile; nei collegamenti che negoziano TLS 1.2 i certificati non hanno la stessa protezione. L'osservazione precisa dipende dalla versione TLS effettivamente negoziata;
- la dimensione dei record TLS e i loro tempi.

Non vede il contenuto delle celle, né i circuiti, né le destinazioni. Ma dimensioni e tempi sono proprio il materiale su cui si basano gli attacchi di *website fingerprinting* e di correlazione del capitolo 26: il fatto che il contenuto sia cifrato non significa che il traffico non dica nulla.

Nella sezione 16.5 abbiamo già confrontato una connessione Tor normale, riconoscibile come TLS, con una obfs4, che non ha struttura riconoscibile. Qui facciamo un passo in più: guardiamo dentro il TLS.

20.2 Gli strumenti

`tcpdump` cattura i pacchetti da un'interfaccia e li salva in un file `pcap`. Per esempio, nel container del laboratorio, il traffico verso la ORPort di `test004r`:

```
$ docker exec -u 0 torlibro-rete tcpdump -i lo \
    -w /lab/net/tls.pcap "tcp port 5104"
```

Wireshark e `tshark` analizzano le catture: il primo con un'interfaccia grafica, il secondo da riga di comando. Riconoscono TLS, ma non esiste un analizzatore ufficiale del protocollo di Tor: le celle, anche una volta decifrate, appaiono come dati grezzi. Il nostro modulo `celle.py` farà da analizzatore.

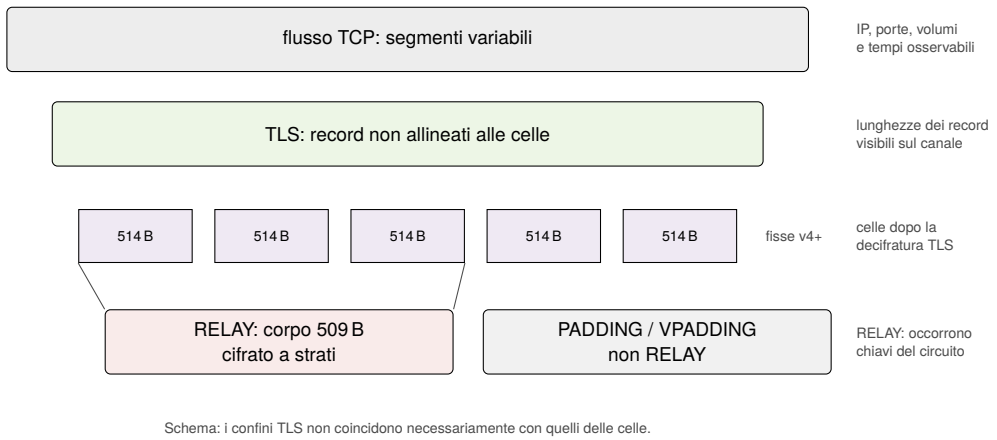


Figura 20.1: Schema concettuale del canale Tor: i segmenti TCP trasportano un flusso TLS, i cui record non coincidono necessariamente con i confini delle celle. IP, porte, lunghezze e tempi sono osservabili; la decifratura TLS consente di leggere le celle. Nel protocollo di collegamento v4+ una cella a lunghezza fissa misura 514 byte (5 di intestazione, 509 di corpo), ma esistono anche celle variabili. Solo i corpi delle celle `RELAY` sono cifrati a strati dalle chiavi del circuito; le celle di padding non vanno confuse con i messaggi RELAY.

20.2.1 Decifrare il proprio TLS

TLS usa chiavi di sessione che nessun osservatore conosce. Chi però è *una delle due parti* della connessione può esportarle: i browser e le applicazioni Python che attivano il registro delle chiavi possono scriverle in un file. La nostra `canale.py` imposta esplicitamente `SSLContext.keylog_filename` quando è definita la variabile `SSLKEYLOGFILE`. Wireshark e `tshark` usano quel file per decifrare la cattura. Il Laboratorio 1 (`canale.py`) rispetta questa convenzione: basta impostare la variabile prima di eseguirlo. Il file contiene righe come

```
# TLS secrets log file, generated by OpenSSL / Python
SERVER_HANDSHAKE_TRAFFIC_SECRET 4878224d3f9fb9aedb96d... 0da89dff6feb...
```

cioè, per ogni sessione, il valore casuale del client e i segreti delle varie fasi di TLS 1.3.

Approfondimento: Perché questo non è un attacco a Tor

Decifrare il TLS del proprio canale non rivela nulla che il client non sappia già. E anche con le chiavi TLS, le celle **`RELAY`** restano cifrate a strati con le chiavi dei singoli salti (capitolo 10): per leggerle servirebbero le chiavi dei circuiti, che solo il client e i relay coinvolti possiedono. TLS è il guscio più esterno della cipolla, non il solo.

20.3 Laboratorio: le celle dentro il TLS

Laboratorio 12: Catturare, decifrare e leggere un canale

Obiettivo. Catturare il canale aperto dal nostro client verso un relay, decifrarlo con le chiavi di sessione e ricostruire la sequenza delle celle con `labs/it/codice/torlab/traffico.py`.
Prerequisiti. Laboratori 1–3; la rete di laboratorio avviata; `tcpdump` e `tshark` nel container.
Tempo stimato. 1–2 ore.

Per eseguire il Laboratorio 12, avvia la rete privata e apri due terminali dalla radice del repository. Il comando di cattura qui sotto va lanciato sul sistema host e avvia `tcpdump` come utente root *nel solo container del laboratorio*; il Makefile esegue infatti la normale shell con i permessi dell'utente. Lascia attiva la cattura mentre esegui il client nel secondo terminale, poi interrompila con Ctrl+C. Il file viene salvato nel volume condiviso `labs/it/laboratorio/net/`.

Passo 1: catturare. Avviamo la cattura nel container, poi, dall'host, apriamo un canale verso `test004r` e creiamo un circuito di un salto, con le chiavi esportate:

```
$ umask 077  # limita la lettura dei nuovi file di chiave all'utente
$ PYTHONPATH=labs/it/codice SSLKEYLOGFILE=labs/it/laboratorio/net/chiavi-tls.log
    python3 -c "
from torlab.laboratorio import relay
from torlab.circuito import costruisci
costruisci([relay('004r')], cc=False)"
```

Il comando Python va eseguito dalla *radice del repository* su Linux con rete host, dove `labs/it/codice/` e `labs/it/laboratorio/net/` sono accessibili. `PYTHONPATH` serve per trovare `torlab`; la variabile `SSLKEYLOGFILE` viene letta esplicitamente dall'implementazione `labs/it/codice/torlab/canale.py`. **Prima di aprire il file** è opportuno applicare `umask 077` nella shell che lancerà Python: il log contiene segreti TLS utilizzabili per decifrare la sessione. Non pubblicare né il log né la cattura, che può contenere metadati di rete; cancellali dopo l'esperimento, una volta chiusi gli strumenti che li usano.

Passo 2: trovare la nostra connessione. Sulla stessa porta si collegano anche gli altri relay della rete, che hanno le proprie chiavi. Chiediamo a `tshark` di seguire, decifrato, ciascuno stream TCP, e teniamo quello per cui la decifratura riesce:

```
$ docker exec torlibro-rete tshark -r /lab/net/tls.pcap \
    -o tls.keylog_file:/lab/net/chiavi-tls.log \
```

```
    -q -z follow,tls,raw,2
```

Il numero di stream `2` è soltanto un esempio di una precedente cattura: per individuare quello corretto puoi elencare gli identificatori con `tshark` usando `-T fields -e tcp.stream` e provare il follow TLS sugli stream candidati. L'output seguente è storico e i valori dei certificati, delle chiavi e del CircID variano per esecuzione. L'uscita contiene, in esadecimale, i dati decifrati dei due versi della connessione; quelli di un verso sono rientrati con una tabulazione.

Passo 3: leggere le celle. La funzione `celle_da_follow` ricompone i due flussi e li passa a `decodifica` del Laboratorio 2. Il collegamento negozia **una sola versione comune**: ciascuna parte invia la propria **`VERSIONS`** con CircID di 2 byte, ma entrambe usano poi la *massima versione condivisa* (5 nella cattura di riferimento), con CircID di 4 byte. Il parser legge entrambe le offerte e rifiuta una cattura priva delle celle iniziali, non assumendo più la versione 5 a priori. Il risultato:

```
client -> relay  CircID=0x0000 VERSIONS corpo=4 byte
relay  -> client CircID=0x0000 VERSIONS corpo=6 byte
relay  -> client CircID=0x00000000 CERTS corpo=1443 byte
relay  -> client CircID=0x00000000 AUTH_CHALLENGE corpo=36 byte
relay  -> client CircID=0x00000000 NETINFO corpo=509 byte
client -> relay  CircID=0x00000000 NETINFO corpo=509 byte
client -> relay  CircID=0xc244f1f6 CREATE2 corpo=509 byte
relay  -> client CircID=0xc244f1f6 CREATED2 corpo=509 byte
```

È, osservata dall'esterno, esattamente la sequenza della fig. 8.5 e del capitolo 7. La **`VERSIONS`** del client ha un corpo di 4 byte, perché `canale.py` offre solo le versioni 4 e 5; il CircID del circuito, `0xc244f1f6`, ha il bit più significativo a 1, perché il canale l'ha aperto il client (sezione 8.4).

Attenzione: Due errori nella prima versione

Nelle prime prove l'analisi aveva due errori: interpretava le **`VERSIONS`** con il formato dei CircID *successivo* alla negoziazione, e attribuiva i blocchi al verso sbagliato. Il parser ora riconosce le **`VERSIONS`** iniziali con CircID di due byte e seleziona una **versione comune** per i due versi. Nel formato `follow,tls,raw` di una cattura *completa dall'inizio*, il primo blocco della nostra sessione deve contenere la **`VERSIONS`** del client; senza questo prerequisito il parser rifiuta l'analisi, invece di indovinare i versi. Non è un dissector generico per catture iniziate a metà del flusso.

20.4 I log di tor

Una cattura mostra *che cosa* passa sulla rete; i log di tor spiegano *perché*. Al livello `info` tor registra le fasi della costruzione dei circuiti, le scelte dei guard, i descrittori scaricati e pubblicati; al livello `debug` arriva fino alle singole celle. Abbiamo già usato i log come strumento di verifica: per scoprire a quale HSDir si era rivolto il client (sezione 14.9), o quando l'onion service aveva ruotato i suoi descrittori.

Qualche consiglio pratico:

- il livello `debug` produce moltissimo testo: conviene attivarlo solo per un dominio, per esempio `Log [circ]debug file circ.log`, che limita il dettaglio ai messaggi sui circuiti;
- nella rete reale `SafeLogging` maschera gli indirizzi: i log di un client non dovrebbero mai contenere le destinazioni visitate;

- i log di un relay pubblico sono dati personali di terzi, con le conseguenze legali del capitolo 25: vanno tenuti al minimo necessario.

Riepilogo

- Un osservatore del collegamento vede indirizzi, handshake TLS, dimensioni e tempi, ma non le celle né le destinazioni.
- Con `SSLKEYLOGFILE` si può decifrare il TLS del proprio canale e leggerne le celle; le celle **`RELAY`** restano comunque cifrate a strati.
- L'analisi di una cattura è un modo indipendente per verificare un'implementazione.
- I log di tor, a livello `info` o `debug`, spiegano le scelte del programma; in un uso reale vanno ridotti e protetti.

Esercizi

Esercizio 20.1 ⋆ Nella cattura di un download attraverso Tor, i record TLS non hanno tutti dimensioni multiple di 514 byte. Perché, se le celle sono di dimensione fissa?

Esercizio 20.2 ⋆⋆ Estendi il Laboratorio 12 decifrando anche le celle **`RELAY`** del tuo circuito, usando le chiavi di `circuito.py`. Quali messaggi relay vedi durante la costruzione di un circuito di tre salti?

Esercizio 20.3 ⋆⋆ Cattura un canale di un client della rete di laboratorio per un minuto senza generare traffico, e conta i record TLS in ciascun verso. Collega il risultato al padding di canale del capitolo 7.

Esercizio 20.4 ⋆⋆⋆ Scrivi un piccolo analizzatore per Wireshark in Lua che riconosca, nei dati decifrati, le celle di Tor e ne mostri CircID e comando.

Per approfondire

- La documentazione di Wireshark sulla decifratura di TLS con il file delle chiavi di sessione.
- Il manuale di tor, alla voce `Log`, per i domini e i livelli di registrazione.

Capitolo 21

Pubblicare onion service

Aggiornato al 23 settembre 2026 — tor 0.4.9.13; documentazione di Onionbalance

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- configurare un onion service e proteggerne le chiavi;
- evitare gli errori di configurazione che rivelano il server;
- offrire a chi visita un sito normale la sua versione `.onion` con Onion-Location;
- limitare l'accesso ai soli client autorizzati con la scoperta ristretta (Laboratorio 13);
- valutare indirizzi vanity, servizi a un salto e bilanciamento del carico con Onionbalance.

Il capitolo 14 ha spiegato come funzionano gli onion service. Questo capitolo è la loro controparte pratica: come pubblicarne uno, come non comprometterne l'anonimato con un errore banale e quali strumenti esistono per le esigenze più comuni.

21.1 Il primo onion service

Bastano due righe nel `torrc`:

```
HiddenServiceDir /var/lib/tor/mio_servizio/
HiddenServicePort 80 127.0.0.1:8080
```

La directory indicata da `HiddenServiceDir` deve essere accessibile in lettura e scrittura al processo tor; nel laboratorio è un percorso *interno al container* Chutney. Questo primo esempio configura un servizio *pubblico*: non richiede alcun file `alice.auth`. L'autorizzazione dei client, compresi i file `authorized_clients/*.auth`, è trattata più avanti in questo capitolo. La prima indica la cartella in cui tor conserva le chiavi del servizio; la seconda dice che la porta *virtuale* 80 dell'indirizzo `.onion` va inoltrata al server locale sulla porta 8080. Al riavvio, tor crea nella cartella:

`hostname` l'indirizzo `.onion` (sezione 14.2);

`hs_ed25519_public_key`, `hs_ed25519_secret_key` la coppia di chiavi di identità del servizio. **Chi possiede la chiave segreta possiede l'indirizzo**: può pubblicare descrittori e impersonare il servizio. Va protetta come la chiave privata di un certificato, e salvata in un backup sicuro, perché se si perde l'indirizzo è perso per sempre.

La cartella deve essere accessibile solo all'utente che esegue tor; tor rifiuta di usarla altrimenti. Si possono configurare più servizi, ciascuno con la propria cartella, nello stesso processo: nel laboratorio abbiamo aggiunto un secondo servizio al nodo `test009h` con altre due righe.

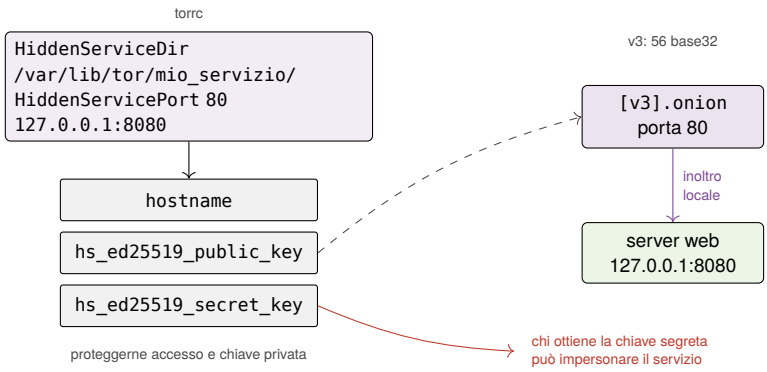


Figura 21.1: Dalle due direttive `HiddenServiceDir` e `HiddenServicePort` al servizio onion v3 pubblico. Tor genera nella directory `hostname` e le chiavi di identità Ed25519; l'indirizzo contiene 56 caratteri base32 derivati dalla chiave pubblica, da un checksum e dalla versione (il nome nella figura è un segnaposto). La porta virtuale 80 viene inoltrata al server locale `127.0.0.1:8080`. La chiave segreta permette di impersonare il servizio e va protetta; in questo esempio non serve `alice.auth`.

21.1.1 Non tradire il server

Il protocollo nasconde l'indirizzo del server; la configurazione può rivelarlo. Gli errori più comuni:

- **servizio raggiungibile anche dall'esterno**: se il server web ascolta su tutte le interfacce e non solo su 127.0.0.1, chi scansiona Internet può trovarlo e confrontarne il contenuto con quello dell'onion service. Il server deve ascoltare solo su localhost, o meglio su un socket Unix (`HiddenServicePort 80 unix:/percorso/socket`);

- **informazioni nelle risposte**: pagine di errore con il nome del server, intestazioni HTTP con indirizzi interni, certificati TLS emessi per un dominio pubblico, pagine di stato del server web lasciate accessibili;

- **risorse esterne**: una pagina che carica immagini o script da siti normali fa sì che quel traffico, invece di restare dentro il servizio onion, esca da un exit (in Tor Browser passa comunque per Tor, quindi non svela l'IP del visitatore, ma non beneficia più dell'autenticazione e della protezione end-to-end proprie del servizio onion: l'exit può osservare i metadati della connessione e, se è HTTP non cifrato, anche il contenuto (HTTPS continua a proteggere il contenuto applicativo)); e un'applicazione lato server che contatta servizi esterni (aggiornamenti, statistiche) può rivelare l'indirizzo del server;

- **orologio e tempi**: un server che risponde con l'ora locale sbagliata di un fuso preciso, o che è irraggiungibile ogni volta che un certo server pubblico è in manutenzione, lascia tracce correlabili.

Nel capitolo 28 vedremo casi reali in cui errori di questo tipo hanno fatto localizzare dei servizi.

21.2 Onion-Location

Molti siti pubblici offrono anche una versione `.onion`: grandi testate giornalistiche, social network, il Tor Project stesso. Per segnalarla ai visitatori che usano Tor Browser, il sito normale invia l'intestazione HTTP

```
Onion-Location: http://<indirizzo>.onion/percorso
```

(o l'equivalente elemento `meta` nella pagina), e Tor Browser mostra nella barra degli indirizzi un pulsante per passare alla versione onion. Per chi visita, il vantaggio è evitare l'exit e ottenere un collegamento autenticato dall'indirizzo stesso; per il sito, ricevere il traffico di Tor senza che passi dagli exit, che spesso vengono bloccati o trattati con sospetto.

21.3 Scoperta ristretta

Un servizio personale, per esempio l'accesso SSH a un server di casa o un pannello di amministrazione, non deve essere raggiungibile da chiunque ne conosca l'indirizzo. La scoperta ristretta (sezione 14.7) cifra il secondo strato del descrittore per i soli client autorizzati. La configurazione usa coppie di chiavi X25519 [Thef]:

Lato servizio nella cartella `authorized_clients/` del servizio, un file `.auth` per ogni client, con una riga `descriptor:x25519:`*chiave pubblica del client in base32*;

Lato client l'opzione `ClientOnionAuthDir` indica una cartella con un file `.auth_private` per ogni servizio, con la riga *indirizzo senza .onion*`:descriptor:x25519:`*chiave privata del client in base32*.

Tor Browser permette di inserire la chiave privata quando visita un servizio che la richiede.

Avvertenza per il laboratorio: lo script `labs/it/laboratorio/scoperta_ristretta.py` ora riconosce le proprie cartelle temporanee tramite un file marcatore e **rifiuta di cancellare cartelle preesistenti non riconosciute** `hs_ristretto/` o `onion_auth/`. Conserva comunque altrove i dati importanti e usalo solo nella rete Chutney dedicata; non su un servizio reale. La chiave privata X25519 viene scritta con permessi `0600` e senza sovrascrivere file esistenti. I passi manuali richiedono la creazione corretta delle chiavi e la rilettura della configurazione dei nodi interessati; per *revocare* una credenziale su un servizio effettivo, segui invece la procedura di **riavvio** documentata dal Tor Project, senza considerare `HUP` una revoca sicura.

Laboratorio 13: Un onion service con scoperta ristretta

Obiettivo. Configurare sul nodo `test009h` un secondo onion service accessibile solo a un client autorizzato, e verificarlo con e senza la chiave.
Prerequisiti. La rete di laboratorio avviata; un server web locale sulla porta 8080.
Tempo stimato. 1 ora.

Passo 1: la coppia di chiavi del client. Con la libreria `cryptography`: Il frammento seguente è un programma Python autonomo che stampa la riga di autorizzazione pubblica. Occorre conservare separatamente `priv`, senza divulgarla, per creare successivamente il file `.auth_private`.

```
1 import base64
2 from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
3 from cryptography.hazmat.primitives.serialization import (
4     Encoding, PrivateFormat, PublicFormat, NoEncryption
```

```
)

sk = X25519PrivateKey.generate()
priv = sk.private_bytes(Encoding.Raw, PrivateFormat.Raw, NoEncryption())
pub = sk.public_key().public_bytes(Encoding.Raw, PublicFormat.Raw)
b32 = lambda b: base64.b32encode(b).decode().rstrip("=")
print("descriptor:x25519:" + b32(pub))
```

La base32 va scritta senza i caratteri = di riempimento. Il frammento stampa soltanto la riga pubblica e *non salva* la chiave privata: se lo esegui come script separato, `priv` andrà persa alla sua terminazione. Per completare manualmente la procedura, conserva la chiave con permessi restrittivi prima che il processo termini oppure usa il laboratorio automatizzato, che crea il file client e rimuove le chiavi temporanee alla fine. I passi manuali e quelli automatici sono alternative, non vanno sovrapposti.

Passo 2: il servizio. Creiamo `hs_ristretto/authorized_clients/alice.auth` con la chiave pubblica, aggiungiamo al `torrc` di `test009h`

```
HiddenServiceDir /lab/net/nodes/009h/hs_ristretto
HiddenServicePort 80 127.0.0.1:8080
```

e inviamo il segnale `HUP`. tor crea l'indirizzo e pubblica i descrittori.

Il comando seguente legge il vero indirizzo di questa istanza dal file `hostname` che tor crea insieme al servizio: ogni rete ha il proprio indirizzo. Il messaggio SOCKS e i log sotto riportati provengono da una precedente sessione di riferimento e possono variare.

Passo 3: senza chiave. Dal client della rete:

```
$ ONION_ADDR=$(cat labs/it/laboratorio/net/nodes/009h/hs_ristretto/hostname)
$ curl --noproxy "" --socks5-hostname 127.0.0.1:9008 \
    "http://${ONION_ADDR}/index.html"
(curl: errore 97 dal proxy SOCKS)
```

e nel log del client:

```
[notice] Fail to decrypt descriptor for requested onion address.
         It is likely requiring client authorization.
```

Il client ha trovato e scaricato il descrittore, ma non riesce a decifrarne il secondo strato: non conosce nemmeno i punti di introduzione, e non può tentare alcuna connessione.

Attenzione: Cartella vuota: rischio di rendere pubblico il servizio

In tor 0.4.9.13, **la sola presenza** di `authorized_clients/` **non abilita l'autorizzazione**: la funzione `is_client_auth_enabled` in `src/feature/hs/hs_service.c` restituisce vero soltanto se è stata caricata *almeno una* chiave pubblica valida. Se la cartella è vuota o tutti i file `.auth` sono illeggibili o non validi, il servizio viene configurato **senza scoperta ristretta**: chi conosce l'indirizzo può raggiungerlo. Tor genera comunque le voci `auth-client` fittizie richieste dal formato del descrittore; non è corretto dedurre che l'HSDir rifiuti necessariamente il descrittore per assenza di chiavi autorizzate. Il codice è stato confrontato con Tor 0.4.9.13; la documentazione Tor conferma che l'autorizzazione si attiva solo caricando un file valido. Una risposta HTTP 400 storicamente osservata nel laboratorio non prova, da sola, quale fosse la causa del rifiuto del descrittore.
Non revocare l'ultimo client lasciando la cartella vuota su un servizio sensibile: la scoperta ristretta potrebbe disattivarsi. Per mantenere l'accesso ristretto, configura prima

almeno un altro client autorizzato, verifica l'effettivo caricamento delle chiavi, poi rimuovi la credenziale da revocare e riavvia tor secondo la procedura documentata. Considera ancora utilizzabili le connessioni già stabilite e i descrittori precedentemente recuperati finché non scadono o non vengono invalidati; per revoche urgenti è necessaria anche una strategia applicativa.

Passo 4: con la chiave. Scriviamo il file `.auth_private` in una cartella indicata da `ClientOnionAuthDir`, facciamo rileggere la configurazione al client e ripetiamo la richiesta: questa volta la pagina arriva. Lo script `labs/it/laboratorio/scoperta_ristretta.py` automatizza l'esperimento, esegue la pulizia e ripristina i file `torrc` al termine; fa parte di `make verifica-lab`.

21.4 Altre esigenze

21.4.1 Indirizzi vanity

Un indirizzo che inizia con una parola riconoscibile aiuta gli utenti a riconoscerlo. Si ottiene generando chiavi a caso finché l'indirizzo non inizia con il prefisso voluto, con programmi specializzati come *mkp224o*. Ogni carattere in più del prefisso moltiplica per 32 il numero medio di tentativi: sei o sette caratteri si trovano in tempi ragionevoli su un computer normale, ogni carattere successivo richiede decine di volte più tempo. Il limite, già visto nella sezione 14.2, è che anche un avversario può generare un indirizzo con lo stesso prefisso: il prefisso aiuta a riconoscere, non autentica. Una chiave generata da un programma di terzi va inoltre considerata sensibile quanto qualsiasi altra.

21.4.2 Servizi a un salto

Un sito che non ha bisogno di nascondere la propria posizione, come la versione onion di un giornale noto, può rinunciare all'anonimato del server per guadagnare velocità: con `HiddenServiceSingleHopMode` e `HiddenServiceNonAnonymousMode`, il servizio costruisce circuiti di un solo salto verso i punti di introduzione e di rendezvous. Il client resta anonimo come sempre. I descrittori, invece, vengono pubblicati ancora con circuiti di tre salti: come abbiamo scoperto nella sezione 14.9, gli HSDir rifiutano le richieste che arrivano direttamente, e il manuale di tor lo cita proprio come motivo. Il nome dell'opzione, che include «NonAnonymous», è voluto: un'istanza di tor configurata così non può essere usata anche come client anonimo. E la scelta è difficile da annullare: una volta che il servizio ha operato in modo non anonimo, il suo indirizzo IP è collegabile all'indirizzo `.onion`; per tornare a un servizio anonimo non basta togliere l'opzione, servono un nuovo indirizzo (nuove chiavi, nuova `HiddenServiceDir`) e un'istanza pulita.

21.4.3 Onionbalance

Un solo processo tor è un punto di fallimento e ha un limite di prestazioni. Onionbalance [Thec] permette di servire lo stesso indirizzo da più server: ciascun server gestisce un proprio onion service, con i propri punti di introduzione, e Onionbalance, che possiede la chiave dell'indirizzo principale, pubblica un descrittore che raccoglie i punti di introduzione di tutti. I client si distribuiscono così tra i server senza saperlo. Il Tor Project lo usa per i propri siti.

Riepilogo

- Un onion service richiede due righe di configurazione; la chiave segreta nella sua cartella *è* l'indirizzo, e va protetta e salvata.
- Il server deve ascoltare solo su localhost o su un socket Unix e non deve rivelare informazioni su di sé nelle risposte o nei collegamenti esterni.
- Onion-Location segnala ai visitatori di Tor Browser la versione onion di un sito.
- La scoperta ristretta usa chiavi X25519: senza la chiave, il client non riesce nemmeno a decifrare il descrittore.
- Indirizzi vanity, servizi a un salto e Onionbalance rispondono a esigenze di riconoscibilità, velocità e scalabilità.

Esercizi

Esercizio 21.1 ⋆ Quanti tentativi servono in media per un prefisso di 6 caratteri base32? E di 8? Se un computer ne prova 10 milioni al secondo, quanto tempo serve nei due casi?

Esercizio 21.2 ⋆⋆ Un onion service mostra, nelle pagine di errore, la riga `Apache/2.4.62 (Debian) Server at 192.0.2.10 Port 80`. Che cosa rivela? Come si evita?

Esercizio 21.3 ⋆⋆ Nel Laboratorio 13, come revochi l'accesso ad Alice? Che cosa succede ai client che hanno già scaricato il descrittore?

Esercizio 21.4 ⋆⋆⋆ Modifica il servizio del laboratorio perché inoltri a un socket Unix invece che a una porta TCP locale. Quali vantaggi offre questa scelta, oltre a non esporre una porta?

Per approfondire

- Il manuale di tor, alle voci `HiddenServiceDir`, `HiddenServicePort` e alla sezione sull'autorizzazione dei client.
- La documentazione del Tor Project sugli onion service e su Onionbalance [Thec].

Capitolo 22

Gestire un relay o un bridge

Aggiornato al 23 settembre 2026 — tor 0.4.9.13 (manuale); rete di laboratorio

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- scegliere tra relay intermedio, exit e bridge in base alle tue risorse e ai rischi che puoi sostenere;
- scrivere la configurazione essenziale di ciascuno, con limiti di banda e chiavi custodite offline;
- leggere e scrivere una politica di uscita, e verificarne l'effetto nella rete di laboratorio (Laboratorio 14);
- dichiarare correttamente una famiglia di relay.

La rete Tor esiste perché migliaia di persone e organizzazioni mettono a disposizione banda e computer. Nei capitoli precedenti abbiamo guardato i relay dal punto di vista del client; qui passiamo dall'altra parte. Non è un manuale di amministrazione, che il Tor Project mantiene aggiornato nella propria guida per gli operatori, ma una spiegazione di che cosa significano le scelte principali alla luce del protocollo.

22.1 Che tipo di nodo

Relay intermedio (*middle*, e col tempo eventualmente guard): inoltra celle tra altri relay tramite connessioni Internet, ma non apre connessioni verso i siti di destinazione dei client. È il ruolo meno impegnativo: il suo indirizzo compare nel consenso, ma nessun sito vede traffico provenire da esso.

Exit apre, per conto dei client, le connessioni verso le destinazioni. È il ruolo più prezioso per la rete, perché gli exit sono pochi rispetto alla domanda (sezione 13.2), ed è quello più esposto: per chi riceve un abuso, l'autore apparente è l'indirizzo dell'exit.

Bridge è un relay non elencato nel consenso, usato come primo salto da chi non può raggiungere la rete pubblica (sezione 16.2). Con un pluggable transport come obfs4 o WebTunnel, anche un collegamento domestico modesto è utile.

Attenzione: Prima di cominciare

Gestire un exit ha conseguenze legali e pratiche (segnalazioni di abuso, richieste delle autorità) che discutiamo nel capitolo 25. Conviene iniziare da un relay intermedio o da

un bridge, informare il fornitore di connettività, e non usare mai per un relay la stessa connessione da cui si usa Tor come client per attività sensibili.

22.2 La configurazione essenziale

Un relay intermedio richiede poche righe di `torrc`:

```
Nickname        MioRelay
ContactInfo     operatore@example.org
ORPort          443
ExitRelay       0
SocksPort       0
RelayBandwidthRate  4 MBytes
RelayBandwidthBurst 8 MBytes
```

`ContactInfo` serve a essere contattati se il relay ha problemi; il manuale ricorda che i descrittori sono archiviati e pubblicati, quindi l'indirizzo sarà pubblico per sempre. `ExitRelay 0` rende esplicito ciò che nelle versioni recenti è già il comportamento predefinito: con il valore `auto`, tor fa da exit solo se l'operatore ha scritto una politica di uscita. `RelayBandwidthRate` e `RelayBandwidthBurst` limitano la banda media e di picco; `AccountingMax` fissa invece un tetto al traffico di un periodo (per esempio un mese), raggiunto il quale il relay smette di accettare nuovo traffico fino al periodo successivo.

Nota operativa. Con il valore automatico di `ExitRelay`, anche `ReducedExitPolicy` o `IPv6Exit` possono abilitare il ruolo di uscita: non conta soltanto una riga `ExitPolicy`. La direttiva esplicita `ExitRelay 0` dell'esempio impedisce invece qualunque uscita. Verifica sempre il risultato prima di pubblicare un relay reale.

22.2.1 La vita di un relay nuovo

Un relay appena avviato pubblica il proprio descrittore alle autorità, che lo inseriscono nel consenso e iniziano a misurarlo (sezione 12.4). Nei primi giorni riceve pochissimo traffico: i client scelgono i relay in proporzione ai pesi di banda, e finché le misure non sono stabili il peso resta basso. I flag arrivano col tempo (sezione 12.4.3): `Stable` richiede che il relay resti acceso a lungo senza interruzioni, e `Guard` richiede anche un'anzianità minima e un'alta disponibilità. Diventato guard, il relay vedrà prima calare il traffico, perché i client che già hanno i propri guard non lo scelgono come intermedio con la stessa frequenza, e poi crescere lentamente, man mano che nuovi client lo adottano (sezione 13.3). I due casi che fanno perdere terreno non sono equivalenti: chi riavvia spesso il relay, senza cambiarne l'identità, conserva le misure di banda ma perde la disponibilità continua richiesta per i flag `Stable` e `Guard`, e deve riconquistarli; chi ne cambia le *chiavi d'identità* crea invece un relay del tutto nuovo per le autorità, che riparte da zero anche nelle misure di banda.

22.2.2 Le chiavi

L'identità di un relay è la sua chiave ed25519 principale (sezione 7.2). Per default tor la tiene sul disco del server; con `OfflineMasterKey 1` la chiave principale resta su un altro computer, e sul server vanno solo chiavi di firma temporanee, certificate con `tor --keygen`. Se il server viene compromesso, l'attaccante ottiene chiavi che scadono, non l'identità del relay. È un accorgimento che richiede disciplina: bisogna rinnovare le chiavi temporanee prima che scadano, altrimenti il relay smette di funzionare.

22.3 La politica di uscita

La politica di uscita è una lista di regole `accept` e `reject`, ciascuna su un indirizzo (o una rete) e una porta (o un intervallo). Tor le esamina **nell'ordine** e applica la prima che corrisponde alla destinazione. Le regole scritte dall'operatore precedono quelle predefinite, che il manuale elenca: vengono rifiutate le porte della posta (25), di NNTP (119), di NetBIOS e SMB (135–139, 445), di alcuni programmi di condivisione file, e poi tutto il resto è accettato. Indipendentemente da questo, tor rifiuta per default le destinazioni private (`127.0.0.0/8`, `10.0.0.0/8`, `192.168.0.0/16` e simili) e il proprio indirizzo pubblico, per non diventare un accesso alla rete interna dell'operatore (opzione `ExitPolicyRejectPrivate`, che la rete di laboratorio disattiva per poter usare `127.0.0.1`).

Ordine delle protezioni. Quando `ExitPolicyRejectPrivate` è attivo (valore predefinito), i rifiuti verso reti private e indirizzi locali vengono anteposti alle regole definite dall'operatore, non semplicemente aggiunti in fondo. La rete Chutney dell'esperimento disabilita esplicitamente questo controllo: non replicare questa impostazione su un exit pubblico.

Un exit che accetta solo il web si scrive così:

```
ExitRelay  1
ExitPolicy accept *:80
ExitPolicy accept *:443
ExitPolicy reject *:*
```

Il Tor Project suggerisce, come compromesso diffuso, la `ReducedExitPolicy`, che accetta un elenco di porte di servizi comuni ed esclude quelle che generano più segnalazioni.

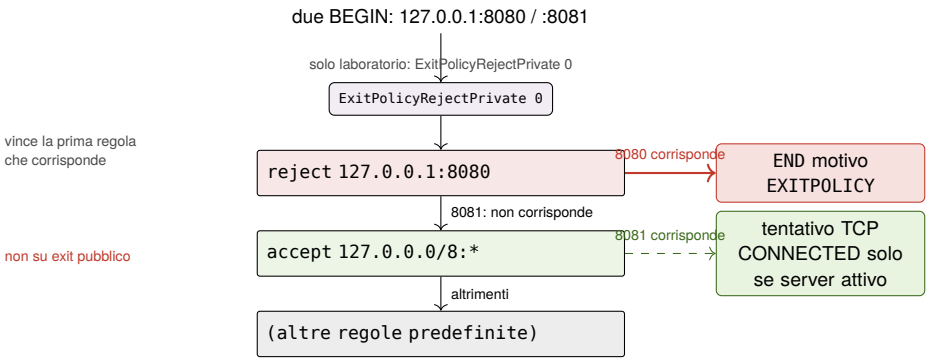


Figura 22.1: Esempio del laboratorio Chutney: `ExitPolicyRejectPrivate 0` è impostata soltanto per consentire indirizzi privati nella rete di prova; normalmente Tor antepone il loro rifiuto alle regole dell'operatore. La richiesta `BEGIN` verso `127.0.0.1:8080` corrisponde a `reject` e produce `END EXITPOLICY`; la porta 8081 supera tale regola e viene accettata. `accept` consente il tentativo TCP, non garantisce `CONNECTED` senza un server in ascolto. Invertire le regole renderebbe inefficace il rifiuto.

Che cosa arriva ai client. Il descrittore del relay contiene la politica completa; il consenso e i microdescrittori ne portano un riassunto, solo per porte (`p accept 80,443`), da cui il client decide se un exit può servire una richiesta (sezione 13.1). Il riassunto non contiene indirizzi: se l'exit rifiuta una destinazione per indirizzo, il client lo scopre solo provando, come mostra il laboratorio.

22.4 Laboratorio: una politica di uscita alla prova

Laboratorio 14: Una regola di uscita e le sue conseguenze

Obiettivo. Aggiungere una regola a un exit della rete di laboratorio e osservare, con `labs/it/codice/torlab`, la risposta a destinazioni accettate e rifiutate.
Prerequisiti. Laboratori 3 e 4; la rete di laboratorio avviata.
Tempo stimato. 1 ora.

Passo 1: la regola. Nel `torrc` di `test006e` aggiungiamo, prima delle regole esistenti che accettano la rete `127.0.0.0/8`, una riga che rifiuta una sola porta:

```
ExitPolicy reject 127.0.0.1:8080
ExitPolicy accept 127.0.0.0/8:*
```

Il listato mostra l'ordine delle due regole interessate: quando modifichi il `torrc` già esistente, aggiungi soltanto la nuova riga `reject` prima della prima `ExitPolicy`, senza duplicare la regola `accept`. L'ordine conta: messa dopo la regola `accept`, la nostra riga non avrebbe alcun effetto, perché la prima regola che corrisponde vince. È esattamente l'errore registrato nel precedente esperimento di riferimento. Dopo aver riavviato il relay, avviamo due piccoli server HTTP sulle porte 8080 e 8081 del container.

Passo 2: tre richieste. Con `costruisci` e il modulo `stream.py` apriamo un circuito che termina su `006e` e inviamo tre `BEGIN`, verso le porte 8080, 8081 e 9999 (su quest'ultima non ascolta nessuno). Le risposte, in una nostra sessione (codici, indirizzo e TTL possono variare secondo la versione e l'ambiente):

```
127.0.0.1:8080 -> END  motivo=EXITPOLICY indirizzo=127.0.0.1 TTL=1800
127.0.0.1:8081 -> CONNECTED
127.0.0.1:9999 -> END  motivo=MISC
```

Passo 3: leggere le risposte. Il rifiuto per politica è comunicato con il motivo `EXITPOLICY`, e la cella `END` porta con sé l'indirizzo risolto e un TTL: così il client può ricordare che quell'exit non serve quella destinazione e sceglierne un altro, senza aspettare un nuovo descrittore. La porta senza server, invece, produce un generico `MISC`, e non il `CONNECTREFUSED` che la specifica elenca per le connessioni rifiutate [tor-spec § Closing streams].

Approfondimento: Perché `MISC` e non `CONNECTREFUSED`

Il motivo si trova nei log dell'exit, al livello `info`, e vale la pena seguirlo perché mostra come si indaga una differenza tra specifica e implementazione:

```
[info] connection_handle_write_impl(): in-progress connect failed.
       Removing. (Connection refused)
[info] errno_to_stream_end_reason(): Didn't recognize errno 115 (Operation
       now in progress); telling the client that we are ending a stream
       for 'misc' reason.
```

L'exit *sa* che la connessione è stata rifiutata, ma sceglie il motivo a partire da un altro valore. La connessione verso la destinazione non è bloccante: `connect()` ritorna subito con `EINPROGRESS` (errno 115) e l'esito arriva più tardi, come evento di scrittura. A quel punto `connection.c` legge l'errore vero con `getsockopt(SO_ERROR)`, lo registra nel log («Connection refused») e chiama `connection_edge_end_errno()`. Questa funzione, però,

ricalcola il motivo con `errno_to_stream_end_reason(tor_socket_errno(…))`: su Unix `tor_socket_errno` è definito come la macro `(errno)` (`src/lib/net/socket.h`), cioè la variabile globale, che conserva ancora `EINPROGRESS`. Il valore 115 non compare nell'elenco di `src/core/or/reasons.c`, e il ramo predefinito restituisce `MISC`. L'informazione giusta esiste, ma non è quella che viene usata. Nello stesso blocco di codice le due diramazioni non concordano: quella per le connessioni tra relay passa il valore letto, quella per gli stream lo ignora. Secondo il resoconto e la patch già presenti in `docs/it/segnalazioni/`, passando il valore corretto anche lì, nella precedente prova privata su tor 0.4.9.13 la terza richiesta ha prodotto `CONNECTREFUSED`. È un difetto minore, senza conseguenze per l'anonimato: il client sa comunque che lo stream è fallito, ma perde un'informazione su cui agirebbe, perché `CONNECTREFUSED` gli direbbe che l'exit ha raggiunto la destinazione e che riprovare altrove è inutile. Analisi e patch sono in `docs/it/segnalazioni/`, pronte per il Tor Project. È anche un buon esempio di perché conviene verificare sul vivo ciò che si legge nelle specifiche.

Lo script `labs/it/laboratorio/politica_uscita.py`, eseguibile dall'host Linux dalla radice della repository con `PYTHONPATH=labs/it/codice python3 labs/it/laboratorio/politica_uscita.py`, esegue i tre tentativi e tenta di ripristinare il `torrc` originale di `006e` tramite un blocco `finally`. Usalo soltanto nella rete Chutney dedicata: modifica temporaneamente la configurazione dell'exit e apre server HTTP locali. Fa parte di `make verifica-lab`; il target `make verifica` della cartella del laboratorio, invece, controlla soltanto la rete Chutney. Un dettaglio necessario: il circuito ha tre salti, perché un exit rifiuta di far uscire il traffico da un circuito di un solo salto e risponderebbe `END` con motivo `TORPROTOCOL` (sezione 15.3).

22.5 Le famiglie

Chi gestisce più relay deve dichiararli come **famiglia**: il client non usa mai due relay della stessa famiglia nello stesso circuito (sezione 13.1), perché chi li controlla entrambi potrebbe correlare il traffico. Il meccanismo storico è `MyFamily`: ogni relay elenca le impronte di tutti gli altri, e la famiglia vale solo se le dichiarazioni sono reciproche. Con dieci relay sono dieci elenchi da tenere allineati, e ogni aggiunta richiede di modificarli tutti.

Il meccanismo introdotto con tor 0.4.9 usa invece una **chiave di famiglia**: l'operatore la genera una volta con `tor --keygen-family MyFamilyKey`, copia il file risultante `MyFamilyKey.secret_family_key` nella directory delle chiavi (`KeyDirectory`) di ogni relay, mantenendo il suffisso `.secret_family_key`, e inserisce in ciascun `torrc` la riga `FamilyId` generata dallo strumento. Proteggi e conserva questa chiave segreta: se viene compromessa, va ruotata sull'intera famiglia. Ogni relay firma con quella chiave un certificato che lo lega alla famiglia, e chi verifica non ha più bisogno di confrontare elenchi. Nella fase di transizione alcune versioni hanno supportato entrambi i meccanismi. Per nuove installazioni occorre seguire la guida aggiornata del Tor Project: FamilyId e la chiave condivisa sono il metodo corrente, mentre MyFamily è stato rimosso nelle versioni più recenti. Il meccanismo precedente resta descritto qui per interpretare le configurazioni storiche.

Approfondimento: Perché dichiararsi conviene

Nessuno obbliga un operatore a dichiarare la famiglia, e un avversario non lo farebbe. Ma le autorità e i volontari che sorvegliano la rete cercano relay con caratteristiche comuni (stessa rete, stessa versione, avvio simultaneo, stesso contatto): una famiglia non dichiarata viene notata e può portare all'esclusione dei relay dal consenso. Dichiararla è parte del

patto di fiducia su cui si regge la rete.

I bridge non appartengono agli elenchi pubblici `MyFamily`: associarli ai relay pubblici potrebbe comprometterne la riservatezza. La guida di Tor raccomanda di non usare questa opzione sui bridge. Inoltre `tor --keygen-family MyFamilyKey` può sovrascrivere file esistenti con quel nome: usa un nome nuovo e custodisci il file segreto con permessi stretti.

22.6 Un bridge

Un bridge obfs4 si configura come nel Laboratorio 10 (sezione 16.5):

```
BridgeRelay 1
ORPort 9001
ServerTransportPlugin obfs4 exec /usr/bin/obfs4proxy
ServerTransportListenAddr obfs4 0.0.0.0:443
ExtORPort auto
BridgeDistribution any
```

`BridgeDistribution` indica al servizio di distribuzione (oggi rdsys) come vorremmo che il bridge fosse dato ai client: `any` lascia decidere al servizio, `none` lo esclude del tutto, utile per un bridge privato da dare a mano a persone di fiducia; i canali disponibili (sito web, posta, Telegram e altri) sono elencati nella pagina informativa dei bridge. Un bridge non compare nel consenso, ma il suo descrittore va comunque all'autorità dei bridge, che ne verifica la raggiungibilità.

Conviene che la porta del transport sia diversa dall'ORPort e, se possibile, una porta comune come 443: è quella che i client usano e che un censore vede (sezione 16.4).

Riepilogo

- Un relay intermedio è il modo meno impegnativo per contribuire; un exit è il più utile e il più esposto; un bridge aiuta chi è censurato.
- Poche opzioni bastano: identità, contatto, porte, limiti di banda; la chiave principale può restare offline.
- La politica di uscita è una lista ordinata in cui vince la prima regola che corrisponde; le destinazioni private sono rifiutate per default.
- Un rifiuto per politica arriva al client come `END` con motivo `EXITPOLICY`, indirizzo e TTL.
- Più relay dello stesso operatore vanno dichiarati come famiglia, con `MyFamily` o con la nuova chiave di famiglia.

Esercizi

Esercizio 22.1 ⋆ Data la politica `accept *:80`, `reject 192.0.2.0/24:*`, `reject *:*`, un client può uscire verso `192.0.2.7:80`? Riscrivi la politica perché non possa.

Esercizio 22.2 ⋆ Perché il riassunto per porte nel consenso non basta a sapere se un exit accetterà una connessione? Che cosa fa il client quando riceve `END` con motivo `EXITPOLICY`?

Esercizio 22.3 ⋆⋆ Un operatore gestisce cinque relay con `MyFamily` e ne aggiunge un sesto, ma aggiorna la configurazione solo sul nuovo. Che cosa vedono i client? E con la chiave di famiglia?

Esercizio 22.4 ⋆⋆ Verifica la spiegazione dell'approfondimento: attiva sull'exit i log di livello `info` e ripeti le tre richieste. Che cosa cambierebbe se `connection_edge_end_errno()` ricevesse l'errore letto con `getsockopt`? Quali altri motivi di chiusura sarebbero riportati correttamente?

Esercizio 22.5 ⋆⋆⋆ Un relay ha `AccountingMax 1 TBytes` su base mensile e una banda di 10 MB/s. Dopo quanti giorni, a pieno regime, esaurisce il tetto? Quali effetti ha sulla rete un relay che resta spento per metà del mese, rispetto a uno che funziona tutto il mese con banda dimezzata?

Per approfondire

- La guida per gli operatori di relay del Tor Project, `https://community.torproject.org/relay/`.
- Il manuale di tor, alle voci `ExitPolicy`, `ReducedExitPolicy`, `FamilyId`, `OfflineMasterKey` e `BridgeDistribution`.
- La specifica dei motivi di chiusura degli stream, [tor-spec § Closing streams].

Capitolo 23

Tor Browser, Tails e Whonix

Aggiornato al 23 settembre 2026 — documento di progetto di Tor Browser (2018); manuale di Tor Browser; documentazione di Tails e Whonix

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- spiegare perché un browser qualsiasi configurato per usare Tor non basta;
- elencare i requisiti di sicurezza e di privacy su cui è progettato Tor Browser;
- collegare l'isolamento per sito di Tor Browser all'isolamento degli stream di tor;
- confrontare Tor Browser, Tails e Whonix per ciò che proteggono e per il prezzo che chiedono.

Il modello di minaccia del 2004 dichiarava che Tor non normalizza i protocolli delle applicazioni (sezione 4.2): se il browser rivela chi sei, la cipolla non serve a niente. Questo capitolo descrive gli strumenti che si occupano proprio di quel livello: Tor Browser, che riduce le fughe e la tracciabilità a livello applicativo, e due ambienti, Tails e Whonix, che estendono le protezioni a più componenti del sistema senza eliminare ogni rischio.

23.1 Perché non basta un proxy

Impostare un browser comune perché usi la SocksPort di tor nasconde l'indirizzo IP in molti casi, ma non in tutti, e non nasconde quasi nient'altro:

- alcune componenti possono aprire connessioni fuori dal proxy: risoluzioni DNS locali, WebRTC, estensioni, programmi esterni aperti dal browser;
- i cookie e le altre forme di memoria del browser collegano le visite di oggi a quelle di ieri, fatte magari senza Tor;
- un sito può raccogliere decine di caratteristiche del browser (dimensioni della finestra, caratteri installati, lingua, fuso orario, dettagli del rendering) e combinarle in un'**impronta** che distingue un utente tra milioni;
- se tutte le schede usano lo stesso circuito, un tracciatore presente su più siti vede le stesse richieste arrivare dallo stesso exit nello stesso momento.

Il primo problema mette a rischio l'indirizzo; gli altri il *collegamento* tra attività diverse, che è ciò che Tor promette di impedire.

23.2 Tor Browser

Tor Browser è una versione di Firefox ESR, il ramo a supporto esteso di Mozilla, modificata dal Tor Project e distribuita insieme a tor. Il suo progetto è descritto in un documento che, pur non più aggiornato nei dettagli, resta il riferimento per i principi [Per+18]. Distingue due gruppi di requisiti.

Requisiti di sicurezza. Riguardano l'uso sicuro di Tor, e una loro violazione espone subito l'utente:

Obbedienza al proxy nessun contenuto deve aggirare tor;

Separazione dello stato nessuna informazione condivisa con altri browser o con la navigazione senza Tor;

Nessuna scrittura su disco i dati della navigazione non devono essere conservati oltre la sessione senza un'azione esplicita dell'utente; i file scaricati o salvati, i log di sistema e le attività esterne al browser richiedono cautele proprie;

Isolamento dell'applicazione tutto ciò che il browser lascia sul sistema deve poter essere rimosso cancellandolo.

Requisiti di privacy. Riguardano il collegamento tra attività diverse, e sono organizzati attorno all'**origine nella barra degli indirizzi**, cioè il sito che l'utente sta visitando:

Identificatori ciò che un terzo (un tracciatore incorporato in più siti) memorizza visitando un sito non deve essere leggibile quando lo stesso terzo compare su un altro sito;

Impronte le caratteristiche del browser non devono permettere di collegare le attività su siti diversi;

Lungo periodo l'utente deve poter cancellare tutto e ottenere una nuova identità con un gesto semplice.

Il documento aggiunge un principio di metodo che vale la pena ricordare: le protezioni non devono cambiare il modo in cui l'utente si aspetta che un browser funzioni. Il predecessore di Tor Browser, l'estensione Torbutton, permetteva di attivare e disattivare Tor nello stesso browser, e gli utenti si confondevano tra schede aperte nei due stati: un'interfaccia che induce in errore è un difetto di sicurezza.

23.2.1 Un circuito per sito

L'isolamento per origine arriva fino alla rete. Tor Browser apre gli stream verso la SocksPort indicando nelle credenziali SOCKS il sito della barra degli indirizzi, e tor, che per default isola gli stream con credenziali diverse (`IsolateSOCKSAuth`), li assegna a circuiti diversi. È il meccanismo che abbiamo verificato dal vivo nel Laboratorio 11 (sezione 18.3). Il manuale lo descrive dal punto di vista dell'utente [Thee]: due siti che incorporano lo stesso tracciatore sono serviti da circuiti diversi, mentre tutte le schede dello stesso sito condividono il circuito.

Due comandi agiscono su questo schema:

Nuovo circuito per questo sito ricarica il sito su un circuito nuovo, per esempio se l'exit non riesce a raggiungerlo; non cancella nulla e non cambia i circuiti degli altri siti.

Nuova identità chiude tutte le schede, cancella cookie, cronologia e ogni altro stato, e fa usare circuiti nuovi per tutto: è l'attuazione del requisito di lungo periodo.

Nessuno dei due cambia il guard, che resta lo stesso per mesi per le ragioni della sezione 13.3.

L'isolamento per sito va inteso come *separazione tra siti*, non come garanzia che ogni sito possieda sempre un solo circuito per tutta la sessione: circuiti vecchi possono scadere o essere sostituiti. Il comando «Nuovo circuito per questo sito» ricarica la scheda corrente; le altre schede dello stesso sito useranno il nuovo circuito quando saranno ricaricate, mentre le connessioni già in corso non vengono necessariamente interrotte. «Nuova identità» interrompe anche attività e download in corso; nessuna delle due opzioni è pensata per forzare il cambio del guard, che tor gestisce separatamente.

23.2.2 Contro le impronte

La strategia di Tor Browser contro il fingerprinting non è rendere ogni utente diverso, ma renderli tutti *uguali*: se tutti gli utenti di Tor Browser presentano gli stessi valori, l'impronta distingue soltanto «un utente di Tor Browser». Ne derivano scelte visibili: la finestra si apre con dimensioni arrotondate e, quando l'utente la ridimensiona, il contenuto è circondato da margini che ne arrotondano le misure (il *letterboxing*); il fuso orario dichiarato è UTC; molte interfacce che espongono dettagli del sistema sono disattivate o uniformate. È anche la ragione per cui installare estensioni, o modificare impostazioni, è sconsigliato: ogni differenza riduce il gruppo in cui ci si nasconde, lo stesso principio dell'anonimato come insieme del capitolo 1.

L'obiettivo è *ridurre le differenze osservabili*, non garantire l'identità perfetta delle impronte. Piattaforma, lingua, livello di sicurezza e altre caratteristiche possono continuare a distinguere gruppi di utenti: la protezione va valutata come riduzione dell'unicità, non come promessa di un'impronta identica per tutti.

23.2.3 I livelli di sicurezza

Molte vulnerabilità dei browser sfruttano JavaScript e le funzioni più complesse. Tor Browser offre tre livelli [Thee]:

Standard tutte le funzioni sono attive; è il livello predefinito.

Più sicuro JavaScript è disattivato sui siti senza HTTPS; alcuni caratteri e simboli matematici sono disattivati; audio e video partono solo con un clic.

Il più sicuro JavaScript è disattivato ovunque, e sono limitati anche immagini e altri contenuti: restano utilizzabili i siti statici.

La scelta è un compromesso tra sicurezza e usabilità, e ha a sua volta un costo in termini di impronta: il livello scelto è uno dei valori che un sito può dedurre.

Approfondimento: Build riproducibili

Chi scarica Tor Browser deve fidarsi che il programma corrisponda al codice pubblico. Il Tor Project produce build *riproducibili*: chiunque, ricompilando gli stessi sorgenti con lo stesso ambiente, ottiene file identici bit per bit a quelli distribuiti. Il documento di progetto descrive il lavoro necessario per eliminare le fonti di differenza (date, percorsi, ordine dei file negli archivi) [Per+18]. Così un attacco che inserisse codice malevolo nei computer di compilazione verrebbe scoperto da chi confronta i risultati.

23.3 Oltre il browser

Tor Browser protegge solo il proprio traffico: le altre applicazioni del computer continuano a usare la rete direttamente, e un errore, o una vulnerabilità sfruttata nel browser, può ancora rivelare l'indirizzo reale. Due sistemi affrontano il problema a livello dell'intero computer, con architetture diverse (fig. 23.1).

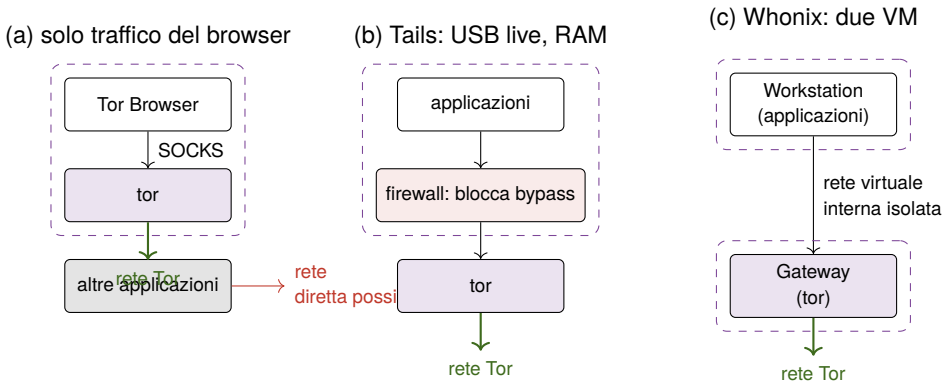


Figura 23.1: Tre modelli di instradamento e isolamento, non garanzie assolute di anonimato. (a) Tor Browser instrada il proprio traffico tramite tor, ma le altre applicazioni dell'host possono collegarsi direttamente. (b) In Tails le regole firewall impediscono normalmente alle applicazioni di aggirare tor; l'*Unsafe Browser*, previsto per i captive portal, è un'eccezione esplicita e non anonima. Tails è un sistema live su USB, normalmente eseguito in RAM, con persistenza opzionale. (c) Whonix separa Workstation e Gateway (tor) in due VM collegate tramite rete virtuale isolata; una compromissione dell'host o dell'hypervisor può comunque vanificarne la protezione.

23.3.1 Tails

Tails è un sistema operativo basato su Debian che si avvia da una chiavetta USB [Taia]. Due proprietà lo caratterizzano:

Tutto attraverso Tor le applicazioni che provano a collegarsi a Internet senza passare per tor vengono bloccate.

Amnesia il sistema gira in memoria, non scrive sul disco del computer e dimentica tutto allo spegnimento; ciò che l'utente vuole conservare va in una *memoria persistente* cifrata sulla chiavetta, che è facoltativa.

Tails include Tor Browser e altre applicazioni configurate in modo prudente. Il progetto è confluito nel Tor Project nel settembre 2024 [The24c]. La sua documentazione è esplicita sui limiti: Tails non protegge da un computer con hardware manomesso, per esempio un registratore di tasti, né da una chiavetta preparata su un computer infetto.

Questa è una proprietà del normale funzionamento di Tails, non una garanzia assoluta che nessun supporto possa mai contenere tracce: l'utente può montare esplicitamente dischi o creare dati persistenti; malware, hardware compromesso e comportamenti esterni al sistema possono inoltre invalidare le ipotesi di sicurezza.

23.3.2 Whonix

Whonix separa le funzioni in due macchine virtuali. Il **Gateway** esegue tor ed è l'unico collegato alla rete; la **Workstation** esegue le applicazioni ed è collegata soltanto al Gateway,

attraverso una rete interna. Tutto ciò che esce dalla Workstation passa per tor, per costruzione e non per configurazione. Il vantaggio rispetto a Tails è l'isolamento: anche un attacco che prenda il pieno controllo della Workstation, con privilegi di amministratore, non vede l'indirizzo reale, perché la macchina non lo conosce. Il prezzo è che la sicurezza dipende dal software di virtualizzazione, e che non c'è amnesia, a meno di usare Whonix all'interno di sistemi che la forniscono.

La separazione Gateway/Workstation è una barriera importante ma *non una garanzia assoluta* contro una Workstation compromessa. L'avversario può ancora usare dati applicativi per identificare l'utente e, se compromette host, hypervisor o Gateway, può compromettere la protezione della rete. Il firewall e la corretta configurazione dell'ambiente restano essenziali.

La control port filtrata. Alcune applicazioni della Workstation, compreso Tor Browser per il comando Nuova identità, hanno bisogno della ControlPort di tor, che si trova sul Gateway. Ma il protocollo di controllo (sezione 18.1) permette anche di chiedere a tor il suo indirizzo pubblico, con `GETINFO address`: esattamente ciò che la Workstation non deve sapere. Whonix interpone quindi *onion-grater*, un proxy che lascia passare solo i comandi di una lista consentita [Who]. È un esempio chiaro di come una funzione innocua in un contesto diventi una fuga in un altro.

Attenzione: Nessuno strumento sostituisce le abitudini

Tor Browser, Tails e Whonix riducono gli errori tecnici possibili, ma non quelli di comportamento: accedere a un account personale, aprire documenti scaricati fuori dal sistema protetto, raccontare dettagli di sé. Il capitolo 28 è dedicato a questo livello.

Riepilogo

- Un browser comune dietro Tor rischia fughe fuori dal proxy e, soprattutto, collegamenti tra attività attraverso stato, impronte e circuiti condivisi.
- Tor Browser è progettato su requisiti espliciti: obbedienza al proxy, niente stato condiviso o su disco, nessun collegamento tra siti diversi, nuova identità a portata di mano.
- L'isolamento per sito passa per le credenziali SOCKS e l'isolamento degli stream di tor.
- Contro le impronte, Tor Browser uniforma molte caratteristiche per rendere più difficile distinguere gli utenti, senza garantire un'impronta perfettamente identica per tutti; i livelli di sicurezza riducono la superficie d'attacco.
- Tails instrada normalmente il traffico attraverso Tor e limita le tracce persistenti, con eccezioni documentate; Whonix separa Workstation e Gateway, ma non protegge da ogni compromissione dell'host o dalle fughe applicative.

Esercizi

Esercizio 23.1 ⋆ Che differenza c'è, in termini di circuiti e di stato del browser, tra «Nuovo circuito per questo sito» e «Nuova identità»? Perché nessuno dei due cambia il guard?

Esercizio 23.2 ⋆ Un utente installa in Tor Browser un'estensione per bloccare la pubblicità, convinto di migliorare la propria privacy. Che cosa guadagna e che cosa perde?

Nota metodologica. Lo script del Laboratorio 11 simula credenziali SOCKS distinte: non avvia tre vere schede di Tor Browser. Con credenziali uguali si può osservare il riuso di un circuito, ma non è garantito in ogni momento; circuiti scaduti o politiche di isolamento aggiuntive possono produrre circuiti diversi.

Esercizio 23.3 ⋆⋆ Con la rete di laboratorio e lo script del Laboratorio 11, simula tre schede: due sullo stesso sito e una su un sito diverso, usando come credenziali SOCKS il nome del sito. Verifica con la ControlPort quali stream condividono un circuito.

Esercizio 23.4 ⋆⋆ Per ciascuno di questi casi, indica tra Tor Browser, Tails e Whonix lo strumento più adatto e giustifica la scelta: (a) una persona che consulta siti censurati dal computer di casa; (b) una giornalista che deve lavorare su documenti riservati da computer di terzi; (c) un ricercatore che esegue a lungo programmi diversi che devono tutti usare Tor.

Esercizio 23.5 ⋆⋆⋆ Scegli cinque caratteristiche del browser che un sito può leggere con JavaScript e stima quanti bit di informazione porta ciascuna in un browser comune. Come le uniforma Tor Browser, e quali restano diverse tra gli utenti?

Per approfondire

- M. Perry, E. Clark, S. Murdoch, G. Koppen, *The Design and Implementation of the Tor Browser* [Per+18].
- Il manuale di Tor Browser [Thee].
- La documentazione di Tails [Taia] e la documentazione di Whonix, in particolare la pagina su onion-grater [Who].

Capitolo 24

Misurare la rete: Tor Metrics e OONI

Aggiornato al 23 settembre 2026 — dati di Tor Metrics fino al 20 settembre 2026; consenso del 23 settembre 2026, ore 15; specifiche OONI ts-023 e ts-030

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- descrivere quali dati pubblica il Tor Project e da dove vengono;
- spiegare come si stima il numero di utenti di una rete progettata per non conoscerli, e quanto è incerta la stima;
- leggere un consenso reale per ricavarne statistiche sulla rete (Laboratorio 15);
- spiegare come OONI misura il blocco di Tor dal punto di vista degli utenti.

Una rete di anonimato pone un problema curioso a chi la gestisce: per migliorarla bisogna misurarla, ma misurarla significa raccogliere dati proprio su ciò che si vuole proteggere. La soluzione del Tor Project è misurare poco, in modo aggregato e pubblico, e documentare ogni passaggio perché chiunque possa rifare i conti [Thed]. Questo capitolo segue i dati dalla loro origine ai grafici, e poi guarda la rete dall'esterno, con le misure di OONI.

24.1 Le fonti dei dati

Quasi tutto ciò che si sa della rete Tor viene da documenti che abbiamo già incontrato:

Consenso e descrittori (capitolo 12): elenco dei relay, flag, pesi di banda, versioni, politiche di uscita.

Descrittori extra-info che ogni relay pubblica accanto al proprio descrittore: storico dei byte trasferiti e statistiche aggregate, tra cui il numero di richieste di directory ricevute, suddivise per paese.

Misure attive del servizio OnionPerf, che scarica periodicamente file attraverso Tor da alcuni punti del mondo e ne registra tempi e fallimenti.

CollecTor archivia tutti questi documenti, fin dai primi anni della rete, e li rende scaricabili; **Tor Metrics** ne ricava grafici e tabelle, ciascuno con il file CSV dei dati e la descrizione del metodo; **Relay Search** permette di cercare un singolo relay. Il consenso usato nei laboratori del libro viene da CollecTor: `make -C labs/it data` lo scarica in `labs/it/dati/collector/`.

24.2 Contare utenti anonimi

Tor non registra gli utenti, e nessun relay sa se due connessioni vengono dalla stessa persona. Come si stima, allora, che la rete abbia circa tre milioni di utenti? L'idea è indiretta (fig. 24.1). Ogni client scarica periodicamente il consenso, e le richieste di directory sono contabili senza sapere chi le fa. I relay che servono le directory contano le richieste riuscite in un giorno, per paese di provenienza stimato dall'indirizzo, e pubblicano i totali nel descrittore extra-info, arrotondati per eccesso (a un multiplo di 8 quelli per paese) perché un conteggio piccolo e preciso non riveli troppo [dir-spec § Extra-info document format].

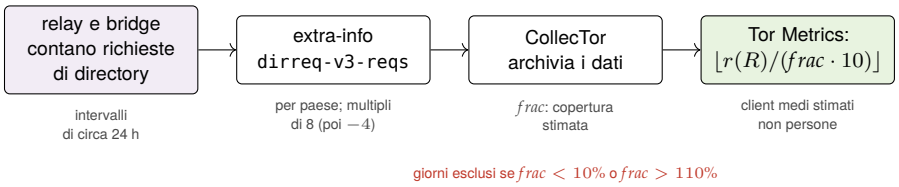


Figura 24.1: Dalle richieste riuscite di directory alla stima indiretta dei *client medi connessi*, non delle persone. Relay che forniscono directory e bridge producono serie distinte di statistiche aggregate in descrittori extra-info, tra cui `dirreq-v3-reqs`, con conteggi per paese arrotondati al multiplo di 8 superiore. CollecTor archivia i documenti; Metrics corregge l'arrotondamento (sottraendo 4 ai conteggi validi), stima la frazione di copertura $frac$ e applica $\lfloor r(R)/(frac \cdot 10) \rfloor$, dove 10 è un'ipotesi sulle richieste giornaliere medie per client. I giorni con $frac < 10\%$ o $frac > 110\%$ sono scartati.

Metrics raccoglie questi numeri e compie quattro passi [Thed]:

1. toglie l'effetto dell'arrotondamento sottraendo metà del passo a ogni valore;
2. per le stime per paese, usa i conteggi geografici per ricavare la proporzione delle richieste provenienti da ciascun paese, e applica tale proporzione al totale delle richieste riuscite dello stesso relay: non somma direttamente i conteggi per paese, perché amplificherebbe l'errore di arrotondamento;
3. stima la **frazione** $frac$ delle richieste dell'intera rete che è coperta dai relay che hanno pubblicato statistiche, tenendo conto di quanto a lungo ciascuno è rimasto in servizio e di quanto traffico di directory ha servito;
4. divide le richieste per la frazione, per ottenere un totale di rete, e poi per 10:

$$r(N) = \left\lfloor \frac{r(R)}{frac \cdot 10} \right\rfloor .$$

Il 10 è un'ipotesi: un client sempre acceso fa circa 15 richieste al giorno, ma molti sono accesi solo per una parte della giornata, e il Tor Project ha scelto 10 come media. Le stime con frazione inferiore al 10% o superiore al 110% vengono scartate.

Attenzione: Che cosa significa «utenti»

Il numero è una stima di client *connessi in media*, non di persone. Un client attivo per due ore al giorno genera normalmente meno richieste di uno attivo 24 ore su 24, ma il rapporto non è fisso; più programmi che usano Tor sullo stesso computer contano come più client; e un cambiamento nel comportamento del software (per esempio, scaricare il consenso meno spesso) cambia la stima senza che cambino gli utenti. I grafici vanno letti

soprattutto per le *variazioni*, non per i valori assoluti.

I numeri, oggi. Tra il 1º agosto e il 20 settembre 2026 la stima giornaliera degli utenti collegati direttamente ai relay è oscillata tra 2,8 e 3,5 milioni, con una frazione tra il 57% e il 73%. Per l'Italia la media è di circa 60 000, ma i dati riportano per ogni giorno un intervallo atteso molto ampio (per esempio tra 34 000 e 111 000 il 20 settembre): variazioni del genere non indicano da sole un evento. Per gli utenti dei bridge, lo stesso giorno, obfs4 contava circa 98 000 utenti, Snowflake 60 000 e WebTunnel 35 000.

24.2.1 Rilevare la censura

Dalle stesse serie Metrics ricava un indicatore di possibili eventi di censura: per i paesi con più utenti confronta la variazione di ciascun paese rispetto alla settimana precedente con la variazione tipica degli altri, e segnala i giorni in cui un paese esce dall'intervallo atteso [Thed]. Un crollo improvviso degli utenti diretti, magari accompagnato da un aumento degli utenti dei bridge, è la firma tipica di un blocco. È un indizio, non una prova: va confermato con misure dall'interno del paese, che è ciò che fa OONI.

24.3 Le prestazioni

OnionPerf misura, da alcuni punti fissi, il tempo per scaricare file di varie dimensioni attraverso Tor, i fallimenti, e la latenza di circuito: il tempo tra l'invio della richiesta HTTP e l'arrivo dell'intestazione della risposta [Thed]. Il 20 settembre 2026 la latenza mediana verso un server pubblico era di circa 110–140 ms dal punto di misura tedesco, 280 ms da quello statunitense e oltre 500 ms da quello di Hong Kong: la distanza dal cuore della rete, che è concentrata in Europa e Nordamerica, si sente. Alcuni punti di misura usano Conflux, il multipercorso del capitolo 31, e permettono di confrontare le due modalità nelle stesse condizioni.

24.4 Laboratorio: leggere un consenso reale

Laboratorio 15: Statistiche della rete da un consenso

Obiettivo. Ricavare da un consenso reale di CollecTor le statistiche principali della rete, con il nuovo modulo `labs/it/codice/torlab/metriche.py`.
Prerequisiti. Laboratorio 6 (`directory.py`) e Laboratorio 7 (`percorso.py`); nessuna rete necessaria.
Tempo stimato. 1 ora.

Il modulo aggiunge una funzione `riassunto`, che riusa l'analisi del consenso del Laboratorio 6, e le due funzioni della stima degli utenti. I conteggi sono ordinati anche per nome, a parità di valore: senza questo accorgimento l'ordine cambierebbe a ogni esecuzione, perché dipende da come Python itera gli insiemi. Applicato al consenso del 23 settembre 2026 alle 15:

```
$ PYTHONPATH=labs/it/codice python3 -m torlab.metriche labs/it/dati/collector/
    consensus-microdesc-2026-09-23-15
relay: 9358
flag: {'Running': 9358, 'Valid': 9358, 'Fast': 8961, 'Stable': 8199,
       'V2Dir': 7674, 'Guard': 5306, 'HSDir': 4050, 'Exit': 3177,
       'BadExit': 32, 'MiddleOnly': 28, 'StaleDesc': 21, 'Authority': 10}
```

```
banda_totale: 198927900
quota_banda_exit: 0.3308947161257923
quota_banda_guard: 0.7010271560701139
versioni: [('Tor 0.4.9.12', 6027), ('Tor 0.4.9.11', 2520), ...]
reti16: [('64.65', 771), ('192.42', 636), ('204.8', 598), ...]
```

Qualche osservazione:

- i 9358 relay sono coerenti con il grafico di Metrics, che per il 20 settembre riporta 9416 relay e 2333 bridge;
- i relay con flag `Exit` sono circa un terzo del totale e portano circa un terzo della banda misurata; i guard ne portano il 70%. Le due quote si sovrappongono, perché un relay può avere entrambi i flag, ed è questo squilibrio che i pesi di banda della sezione 13.2 compensano;
- il flag `BadExit` (32 relay) e `MiddleOnly` (28) sono decisioni delle autorità contro relay sospetti o mal configurati;
- i relay con flag `Authority` sono dieci, ma le autorità che votano il consenso sono nove (sezione 6.1): la decima è `Serge`, l'autorità dei bridge, che ha il flag ma non partecipa al voto;
- quasi tutti i relay usano versioni 0.4.9 recenti;
- alcune reti /16 ospitano centinaia di relay: sono tipicamente grandi operatori di exit, e il vincolo di un solo relay per /16 per circuito (sezione 13.1) serve proprio a non dipendere troppo da un'unica rete.

La somma dei pesi del consenso è un numero in unità di banda misurata dalle autorità, non il traffico reale: per quest'ultimo Metrics usa gli storici dei byte dei descrittori extra-info.

24.5 OONI

L'*Open Observatory of Network Interference* (OONI) misura la censura di Internet dal punto di vista degli utenti: volontari di tutto il mondo installano OONI Probe, che esegue periodicamente dei test e ne pubblica i risultati in forma aperta. Tra i test ce ne sono alcuni dedicati a Tor [OON]:

`tor` verifica la raggiungibilità di alcune directory authority e di alcuni bridge, non limitandosi ad aprire una connessione TCP ma svolgendo operazioni specifiche per ciascuno, come l'handshake obfs4;

`torsf` prova ad avviare tor con Snowflake e registra se e in quanto tempo il bootstrap riesce.

Le specifiche dei test prevedono accorgimenti per non pubblicare gli indirizzi dei bridge privati: pubblicarli nei risultati li consegnerebbe ai censori. Metrics e OONI si completano: il primo vede un calo degli utenti da un paese, il secondo può dire se, in quel paese, i relay o i bridge erano davvero irraggiungibili e con quale tecnica venivano bloccati.

Approfondimento: Misurare senza danneggiare

Ogni statistica pubblicata da un relay è un compromesso: più è dettagliata, più è utile e più rischia di rivelare qualcosa. Per questo i conteggi sono arrotondati, e aggregati per giorno e per paese. La ricerca sulla rete segue gli stessi principi (sezione 19.1): misurare il

minimo necessario, aggregare prima di pubblicare, non conservare ciò che non serve.

Riepilogo

- I dati sulla rete vengono da consenso, descrittori ed extra-info, archiviati da CollecTor, e dalle misure attive di OnionPerf; Metrics li elabora con metodi documentati e ripetibili.
- Gli utenti si stimano dalle richieste di directory: richieste riportate, divise per la frazione coperta e per 10; il risultato è una stima di client medi, utile soprattutto per le variazioni.
- Lo stesso dato serve a segnalare possibili eventi di censura, da confermare con misure locali.
- Un consenso reale basta a ricavare numero di relay, flag, quote di banda, versioni e concentrazione per rete.
- OONI misura dall'interno dei paesi la raggiungibilità di autorità e bridge e il funzionamento di Snowflake.

Esercizi

Esercizio 24.1 ⋆ Se tutti i client iniziassero a scaricare il consenso la metà delle volte, che cosa succederebbe alla stima degli utenti, a parità di persone? Come se ne accorgerebbe chi legge i grafici?

Esercizio 24.2 ⋆ Un giorno i relay che riportano statistiche hanno registrato 20 milioni di richieste, e la frazione stimata è del 60%. Quanti utenti stima Metrics? E con una frazione dell'8%?

Esercizio 24.3 ⋆⋆ Estendi `riassunto` perché calcoli la quota di banda dei relay con i flag `Guard` ed `Exit` insieme, e la probabilità complessiva, con i pesi del consenso, che un circuito abbia un exit in una delle cinque reti /16 più rappresentate.

Esercizio 24.4 ⋆⋆ Scarica da Metrics le serie degli utenti diretti e degli utenti dei bridge di un paese in cui sia stato documentato un blocco di Tor, e descrivi che cosa accade alle due serie nei giorni del blocco.

Esercizio 24.5 ⋆⋆⋆ I conteggi per paese sono arrotondati per eccesso a multipli di 8. Descrivi un attacco che l'arrotondamento rende più difficile, e discuti perché, invece, non protegge contro un avversario che controlla molti relay.

Per approfondire

- La pagina *Reproducible Metrics* del Tor Project [Thed], con il metodo di ogni grafico.
- Le specifiche dei test di OONI [OON] e OONI Explorer, `https://explorer.ooni.org/`.
- Il formato dei descrittori extra-info, [dir-spec § Extra-info document format].

Capitolo 25

Aspetti legali ed etici in Italia e nell'UE

Aggiornato al 23 settembre 2026 — Regolamento (UE) 2022/2065; d.lgs. 196/2003 (testo coordinato del Garante); linee guida del Tor Research Safety Board

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- distinguere la legittimità di uno strumento dalla liceità di ciò che se ne fa;
- leggere la regola europea del «semplice trasporto» alla luce del funzionamento di un exit;
- individuare le domande aperte per chi gestisce un relay in Italia: responsabilità, conservazione dei dati, sequestri;
- applicare alla ricerca su Tor i principi di sicurezza del Tor Research Safety Board.

Attenzione: Questo capitolo non è una consulenza legale

Il diritto cambia, dipende dai fatti del caso concreto e, su Tor, conta pochissime decisioni. Questo capitolo inquadra i problemi e cita le norme che abbiamo potuto leggere nel testo ufficiale, indicando dove la risposta non è certa. Chi deve prendere una decisione, per esempio aprire un exit, farebbe bene a parlarne con un avvocato esperto di diritto delle tecnologie.

25.1 Uno strumento e i suoi usi

In Italia, come nel resto dell'Unione europea, non esiste un divieto generale di usare Tor o altri strumenti che proteggono la riservatezza delle comunicazioni. Al contrario, la segretezza della corrispondenza e di ogni altra forma di comunicazione (art. 15 della Costituzione) e la libertà di manifestazione del pensiero (art. 21) sono diritti fondamentali, come il rispetto della vita privata e la protezione dei dati personali nella Carta dei diritti fondamentali dell'Unione (artt. 7 e 8).

Ciò che si fa attraverso Tor resta invece soggetto alle stesse regole di ciò che si fa senza: un accesso abusivo a un sistema informatico (art. 615-*ter* del codice penale) non diventa lecito perché passa per tre relay. La distinzione tra strumento e uso è la chiave dell'intero capitolo, e vale anche per chi offre lo strumento: chi gestisce un relay mette a disposizione capacità di trasmissione, come un fornitore di connettività, e non sceglie che cosa vi transita.

25.2 Chi gestisce un relay

25.2.1 Il semplice trasporto

Il regolamento europeo sui servizi digitali (*Digital Services Act*, DSA), applicabile dal 17 febbraio 2024, ha ripreso e sostituito le regole sulla responsabilità degli intermediari che dal 2000 stavano nella direttiva sul commercio elettronico, recepita in Italia con il d.lgs. 70/2003 [22]. L'art. 4 riguarda il **semplice trasporto** (*mere conduit*): chi fornisce un servizio che consiste nel trasmettere su una rete informazioni fornite da altri non è responsabile di quelle informazioni a tre condizioni: non dà origine alla trasmissione, non ne seleziona il destinatario, non seleziona né modifica le informazioni trasmesse. La fig. 25.1 mette a confronto le tre condizioni con il comportamento di un exit, come lo abbiamo visto a livello di celle.

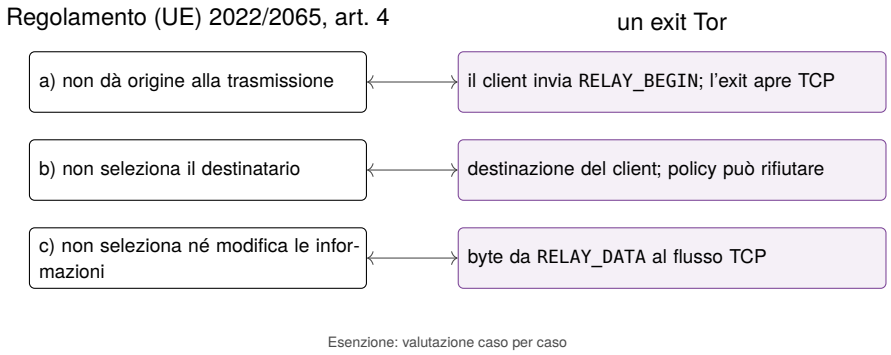


Figura 25.1: Confronto tecnico, non conclusione legale automatica, tra le tre condizioni di *mere conduit* dell'art. 4 del regolamento (UE) 2022/2065 e un exit Tor. Il client specifica la destinazione tramite `RELAY_BEGIN`; l'exit apre il collegamento TCP e può negarlo in base alla propria politica. I byte applicativi di `RELAY_DATA` vengono trasferiti al flusso TCP, non le celle Tor intatte. L'eventuale esenzione richiede la valutazione del servizio concreto e di tutti i presupposti legali.

Due precisazioni dello stesso regolamento sono rilevanti. L'art. 8 esclude un obbligo *generale* di sorvegliare le informazioni trasmesse o di cercare attivamente indizi di attività illecite: un operatore non è tenuto a ispezionare indiscriminatamente il traffico e, per un relay intermedio, il contenuto applicativo rimane cifrato. E l'art. 4, paragrafo 3, lascia alle autorità giudiziarie o amministrative il potere di ordinare al prestatore di impedire o far cessare una violazione.

La domanda aperta. Il regolamento si applica ai «servizi della società dell'informazione», una nozione che il diritto europeo definisce come servizi prestati *normalmente dietro retribuzione*. Un relay gestito gratuitamente da un volontario vi rientra? Ci sono buoni argomenti per un'interpretazione ampia, e l'esenzione esprime comunque un principio (chi trasporta non risponde di ciò che trasporta) che un giudice può tenere presente; ma non ci risulta che la questione sia stata decisa per Tor in Italia. È una delle ragioni per cui molti exit sono gestiti da associazioni o organizzazioni che offrono già servizi di rete.

25.2.2 Responsabilità civile e indagini penali

L'esenzione riguarda la responsabilità per le informazioni trasmesse; non impedisce che un'indagine penale parta dall'indirizzo di un exit. Un reato commesso attraverso Tor appare, a chi lo subisce, come proveniente dall'exit, e la prima mossa di chi indaga può essere

una perquisizione o il sequestro dei computer dell'operatore. La FAQ legale preparata dalla Electronic Frontier Foundation per gli operatori, scritta per il diritto statunitense ma utile come esperienza, riporta di non conoscere casi di operatori condannati per il solo fatto di gestire un relay, ma ricorda indagini e sequestri basati su un fraintendimento del funzionamento di Tor [Ele]. Alcune precauzioni riducono il rischio:

- gestire l'exit su un server dedicato, in un centro dati, non a casa propria e non sul computer con cui si lavora;
- rendere evidente che l'indirizzo è un exit: un nome di rete (DNS inverso) che lo dica, e una pagina informativa, che tor può servire da solo sulla DirPort con l'opzione `DirPortFrontPage`;
- informare prima il fornitore di connettività e concordare come gestire le segnalazioni di abuso;
- rispondere alle segnalazioni spiegando che cos'è Tor, e conservare la corrispondenza.

25.2.3 Conservare i dati

L'art. 132 del Codice in materia di protezione dei dati personali impone al *fornitore* di conservare i dati di traffico telematico per dodici mesi a fini di accertamento e repressione dei reati [03] (ventiquattro mesi per il traffico telefonico). Lo stesso articolo, al comma 5-*bis*, fa salva una disciplina speciale: l'art. 24 della legge 20 novembre 2017, n. 167 porta il termine a **settantadue mesi** per l'accertamento dei reati indicati negli artt. 51, comma 3-*quater*, e 407, comma 2, lettera a), del codice di procedura penale, cioè terrorismo e altri delitti gravi [17]. Il titolo del Codice in cui si trova si applica ai servizi di comunicazione elettronica *accessibili al pubblico su reti pubbliche di comunicazioni* (art. 121). Se l'operatore di un relay volontario sia un fornitore di questo tipo è, di nuovo, una questione non risolta, che dipende anche dalla forma in cui il servizio è offerto; va valutata con un legale prima di aprire un exit, soprattutto se lo gestisce un'organizzazione che è già un fornitore.

I termini della normativa italiana non risolvono, da soli, la questione della sua **compatibilità con il diritto dell'Unione**. Il Garante ha segnalato al Parlamento e al Governo criticità di proporzionalità nella conservazione generalizzata fino a 72 mesi; la Corte di giustizia dell'UE, nelle cause riunite C-793/19 e C-794/19 (*SpaceNet*, 20 settembre 2022), ha ribadito i limiti alla conservazione generalizzata e indiscriminata dei dati di traffico e di ubicazione, distinguendola da misure specifiche consentite a determinate condizioni [Gar21; Cor22]. Questi rilievi non permettono di concludere automaticamente né che ogni conservazione sia illecita né che un volontario che gestisce un relay sia un soggetto obbligato.

Dal punto di vista tecnico, con le impostazioni di registrazione raccomandate tor non mantiene normalmente una cronologia delle destinazioni degli utenti. Un exit vede le destinazioni delle connessioni che inoltra, ma non conosce l'indirizzo IP originario del client. I dati concretamente disponibili dipendono dal ruolo del relay e dalla sua configurazione: non si deve presumere che l'exit possa identificare i singoli utenti Tor.

25.2.4 I log sono dati personali

Gli indirizzi IP sono dati personali, e il Regolamento generale sulla protezione dei dati (GDPR) chiede di trattarne solo quelli necessari, per il tempo necessario (art. 5, principio di minimizzazione). Per un relay la scelta coerente è la stessa che suggerisce la sicurezza: log al livello `notice`, con `SafeLogging` attivo, e nessuna cattura di traffico (sezione 20.4).

25.3 Chi fa ricerca

La ricerca su Tor ha una storia di esperimenti discutibili, alcuni dei quali racconteremo nel capitolo 27: relay che osservavano o manipolavano il traffico degli utenti reali per dimostrare un attacco. Oltre al problema etico, in Italia un esperimento del genere può ricadere in norme penali, come l'intercettazione illecita di comunicazioni informatiche (art. 617-*quater* c.p.), e nella disciplina dei dati personali.

Il **Tor Research Safety Board** è un gruppo di ricercatori che offre consigli sulla sicurezza degli esperimenti e ha formulato alcune linee guida [Tor]:

- usare una rete di prova ogni volta che è possibile, e attaccare solo il proprio traffico;
- raccogliere solo dati che sarebbe sicuro rendere pubblici, e solo quelli che servono;
- proteggere i dati raccolti e gli strumenti dell'esperimento;
- ridurre il dettaglio dei dati, con arrotondamenti o rumore;
- considerare i dati di terzi che, incrociati con i propri, potrebbero rivelare qualcosa;
- chiedersi se gli utenti intendevano quei dati come privati;
- fare in modo che i benefici superino i rischi.

Il Board chiede a chi propone un esperimento di descrivere che cosa vuole imparare, come lo farà, quali rischi introduce e perché i benefici li giustificano. È la stessa disciplina che questo libro ha seguito: i laboratori che generano traffico Tor attivo usano la rete privata (sezione 19.1), mentre le analisi di documenti pubblici, come quelle del capitolo 24, si svolgono offline su dati aggregati e pubblici.

Riepilogo

- Usare Tor è lecito; ciò che si fa attraverso Tor resta soggetto alle regole ordinarie.
- L'art. 4 del DSA esenta dalla responsabilità chi si limita a trasportare informazioni altrui, a condizioni con cui il normale funzionamento di un exit può essere compatibile; resta da valutare caso per caso se un relay volontario rientri nei servizi a cui il regolamento si applica.
- L'esenzione non evita indagini e sequestri: server dedicato, identificazione chiara dell'exit e rapporti con il fornitore riducono i problemi.
- Gli obblighi di conservazione dei dati di traffico riguardano i fornitori di servizi di comunicazione accessibili al pubblico; la loro applicazione ai relay è da valutare caso per caso.
- La ricerca su Tor deve evitare per quanto possibile esperimenti sul traffico di utenti reali: rete di prova, minimizzazione, dati pubblicabili, bilancio tra benefici e rischi.

Esercizi

Esercizio 25.1 ⋆ Per ciascuna delle tre condizioni dell'art. 4 del DSA, descrivi una modifica al comportamento di un exit che la violerebbe.

Esercizio 25.2 ⋆ Spiega con un esempio la differenza tra la liceità di uno strumento e la liceità dei suoi usi, applicandola sia a un utente di Tor sia a un operatore di relay.

Esercizio 25.3 ⋆⋆ Scrivi la risposta a una segnalazione di abuso ricevuta dal fornitore di connettività di un exit, in cui un sito lamenta tentativi di accesso ripetuti dal tuo indirizzo. Che cosa puoi spiegare, e che cosa puoi fare?

Esercizio 25.4 ⋆⋆ Una ricercatrice vuole misurare quanti utenti di Tor visitano siti di notizie italiani, aprendo un exit e contando i nomi di dominio delle connessioni. Valuta la proposta con le linee guida del Research Safety Board e proponi un'alternativa più sicura.

Esercizio 25.5 ⋆⋆⋆ Cerca le decisioni dei giudici europei, italiani o di altri paesi dell'Unione che riguardano operatori di relay Tor, e riassumi per ciascuna il problema giuridico e l'esito, citando le fonti primarie.

Per approfondire

- Regolamento (UE) 2022/2065, in particolare gli artt. 4, 8 e 9 [22].
- D.lgs. 196/2003, artt. 121 e 132, nel testo coordinato pubblicato dal Garante [03].
- Electronic Frontier Foundation, *Legal FAQ for Tor Relay Operators* [Ele].
- Il sito del Tor Research Safety Board [Tor].

Parte IV

Attacchi e difese

Capitolo 26

Correlazione del traffico, fingerprinting, Sybil

Aggiornato al 23 settembre 2026 — articoli citati nel capitolo; tor 0.4.9.13; rete di laboratorio

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- classificare gli attacchi all'anonimato di Tor secondo la posizione dell'avversario;
- spiegare la correlazione da estremo a estremo e misurarla sul tuo traffico nella rete di laboratorio (Laboratorio 16);
- descrivere il website fingerprinting e i limiti delle sue valutazioni;
- riconoscere gli attacchi Sybil e le difese della rete, dai guard alla misura della banda.

Il modello di minaccia del capitolo 4 dichiarava che Tor non resiste a chi osserva entrambi gli estremi di una comunicazione. Vent'anni di ricerca hanno trasformato quella frase in numeri, e hanno mostrato che le posizioni da cui osservare sono più numerose e più facili da raggiungere di quanto sembrasse. Questo capitolo segue gli attacchi dal punto di vista di chi difende: che cosa richiedono, che cosa ottengono, e quale parte del protocollo che abbiamo studiato serve a contrastarli.

26.1 Dove si trova l'avversario

Conviene ordinare gli attacchi in base a ciò che l'avversario vede:

Un estremo il collegamento tra client e guard (il fornitore di connettività, la rete locale, un censore) oppure quello tra exit e destinazione. Sul collegamento client–guard osserva tempi, volumi e direzioni di traffico cifrato; tra exit e destinazione la cifratura applicativa dipende dal protocollo usato. Il website fingerprinting può sfruttare i metadati del lato client, senza leggere il contenuto.

Entrambi gli estremi un avversario che osserva sia l'ingresso sia l'uscita, perché gestisce relay in entrambe le posizioni o perché controlla le reti che le collegano. È la posizione della correlazione.

Dentro la rete relay ostili, anche in posizione intermedia, che possono misurare, ritardare, marcare o scartare celle.

Sopra la rete Internet stessa: sistemi autonomi (AS) e punti di interscambio (IXP) attraversati dai collegamenti tra i relay e verso di essi.

26.2 La correlazione da estremo a estremo

Tor non applica deliberatamente lunghi ritardi o rimescolamenti di tipo mix alle celle, per restare usabile (sezione 2.2). La conseguenza è che la *forma* del traffico, cioè quanti byte passano in ciascun istante, resta spesso abbastanza riconoscibile ai due estremi, pur risentendo di ritardi, buffering, multiplexing e traffico aggiuntivo (fig. 26.1). Chi vede i due estremi confronta le forme e decide se appartengono allo stesso flusso: non ha bisogno di rompere nessuna cifratura.

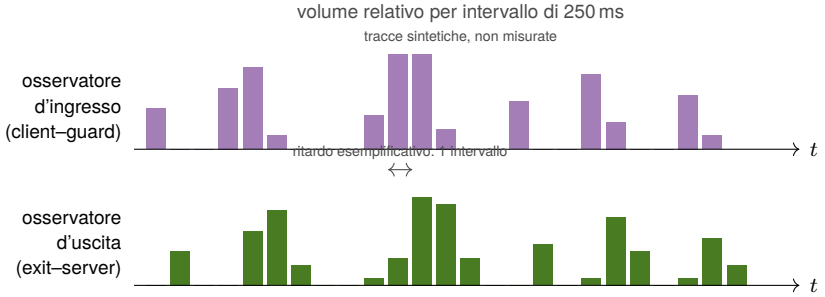


Figura 26.1: Due profili *sintetici*, non misure reali, del volume di traffico osservato in una direzione su client–guard ed exit–server. I picchi si somigliano, ma le serie non sono identiche: lo sfasamento di un intervallo da 250 ms è puramente illustrativo. La cifratura protegge i contenuti, non necessariamente le caratteristiche temporali e volumetriche; ritardi, overhead, buffering, multiplexing e traffico concorrente possono modificare le tracce. Una somiglianza statistica non prova da sola l'identità del flusso.

Quanto è facile. Nel 2018 DeepCorr ha addestrato una rete neurale a riconoscere coppie di flussi Tor: osservando circa 900 pacchetti per flusso ha ottenuto un'accuratezza del 96%, contro il 4% del metodo statistico precedente nelle stesse condizioni [NBH18]. La difficoltà vera, per l'avversario, non è confrontare i flussi ma trovarsi nel posto giusto.

Quanto è probabile trovarsi nel posto giusto. Johnson e coautori hanno simulato per mesi il comportamento di utenti tipici con il vero algoritmo di selezione dei percorsi [Joh+13]. Con relay ostili di banda moderata, l'80% degli utenti di ogni tipo veniva deanonimizzato almeno una volta entro sei mesi; contro un singolo AS, per utenti in alcune località comuni, quasi il 100% entro tre mesi. Il risultato non dice che ogni connessione è esposta, ma che nel lungo periodo le occasioni si accumulano: è il ragionamento che ha portato a guard più longevi e a meno guard per client (sezione 13.3).

Internet come avversario. RAPTOR ha mostrato che un AS può aumentare le proprie occasioni in tre modi [Sun+15]: sfruttando l'asimmetria dei percorsi di Internet (basta vedere *una* direzione a ciascun estremo, anche i soli riscontri TCP); sfruttando i cambiamenti naturali dei percorsi nel tempo; e dirottando o intercettando prefissi con BGP. Queste mosse non toccano Tor: la difesa passa per il monitoraggio dell'instradamento e, in prospettiva, per una selezione dei percorsi consapevole della topologia.

Attacchi attivi. Un avversario che controlla un estremo può anche *imprimere* una forma riconoscibile al traffico, ritardandolo o raggruppandolo, per ritrovarla all'altro estremo: la marcatura che la sezione 10.3 discuteva a livello di cifratura è un caso estremo di questa idea. Già nel 2005 Murdoch e Danezis mostrarono che non serve nemmeno osservare i collegamenti:

un server ostile imprime un ritmo al traffico diretto alla vittima, e l'avversario misura la latenza di propri circuiti attraverso i vari relay; quelli che rallentano con quel ritmo sono i relay del circuito della vittima [MD05].

26.3 Laboratorio: correlare il proprio traffico

Laboratorio 16: Correlazione da estremo a estremo

Obiettivo. Verificare sulla rete di laboratorio che la forma del traffico sopravvive a un circuito, e che basta a riconoscere quale flusso d'uscita corrisponde a quale client.
Prerequisiti. Laboratorio 11; la rete di laboratorio avviata. Tutti i flussi sono generati dallo script stesso.
Tempo stimato. 1 ora.

Lo script `labs/it/laboratorio/correlazione.py` avvia un server sulla porta 8095 del container e sei client, ciascuno con credenziali SOCKS diverse: con `IsolateSOCKSAuth` attivo, i loro flussi non condividono lo stesso circuito (sezione 18.3). Per 20 secondi ogni client alterna, a caso, raffiche di blocchi da 2 KB e silenzi. Due «osservatori» registrano i dati:

- l'osservatore d'ingresso annota quando ogni client scrive: è una traccia idealizzata di ciò che vedrebbe chi osserva il collegamento verso il guard, dove il traffico sarebbe incapsulato in celle e in TLS, ma la struttura temporale che serve alla correlazione sopravvive a entrambi;

- l'osservatore d'uscita, il server, annota quando arrivano i byte di ciascuna connessione, e riceve anche un identificatore di controllo nei primi quattro byte, escluso dall'algoritmo di correlazione e usato solo per verificare gli abbinamenti.

Entrambi raggruppano i byte in intervalli di 250 ms, e per ogni client si calcola la correlazione di Pearson con ognuno dei sei flussi d'uscita. Il flusso più correlato è l'abbinamento proposto; lo script conosce la risposta giusta, che il client comunica nei primi 4 byte, ma la usa solo per valutare, non per scegliere il flusso più correlato.

```
$ docker exec -w /tmp torlibro-rete python3 /lab/correlazione.py 1
client 0: +0.79 -0.11 +0.12 +0.17 +0.11 -0.01  -> uscita 0 OK
client 1: -0.11 +0.80 -0.02 +0.07 +0.18 +0.08  -> uscita 1 OK
client 4: +0.13 +0.02 +1.00 +0.21 +0.10 +0.21  -> uscita 2 OK
client 3: +0.21 +0.09 +0.33 +0.81 -0.07 +0.03  -> uscita 3 OK
client 2: +0.17 +0.05 +0.10 -0.09 +0.85 -0.03  -> uscita 4 OK
client 5: -0.00 +0.01 +0.23 +0.06 -0.13 +0.92  -> uscita 5 OK
abbinamenti corretti: 6/6
```

Le colonne sono i flussi d'uscita, numerati nell'ordine in cui le connessioni arrivano al server, che non coincide con quello dei client. Nell'output illustrativo il valore corretto spicca in ogni riga, tra 0,79 e 1,00; ordine dei flussi e coefficienti possono cambiare tra esecuzioni. Nessuna chiave, nessuna cella decifrata: solo quantità e tempi. La rete di laboratorio è piccola e poco carica, quindi il ritardo è minimo; ma, come dicono i risultati di DeepCorr, anche nella rete reale bastano poche centinaia di pacchetti a un classificatore migliore del nostro. Questo è un estratto di un'esecuzione di riferimento, non una previsione esatta: possono comparire anche 5/6 abbinamenti. Il laboratorio fa parte di `make verifica-lab`.

26.4 Website fingerprinting

Un avversario che vede soltanto il collegamento tra client e guard non può correlare, ma può *riconoscere*. Ogni pagina web produce una sequenza caratteristica di celle nelle due direzioni: quante, in che ordine, con che raffiche. L'avversario visita in anticipo i siti che gli interessano attraverso Tor, registra queste sequenze e addestra un classificatore; poi confronta con esse il traffico della vittima.

La storia degli attacchi è una corsa tra classificatori e difese:

- nel 2011 Panchenko e coautori, con caratteristiche basate su volumi, tempi e direzioni, portarono il riconoscimento su un insieme chiuso di 775 siti dal 3% al 55%, e mostrarono per primi un attacco efficace in un *mondo aperto*, dove la vittima può visitare anche siti sconosciuti [Pan+11];
- nel 2018 Deep Fingerprinting, con una rete neurale convoluzionale, superò il 98% di accuratezza su 95 siti senza difese, e il 90% contro WTF-PAD, una difesa basata sul padding adattivo considerata per Tor [Sir+18].

Approfondimento: La fallacia del tasso di base

Le accuratezze altissime vanno lette con cautela. Juarez e coautori hanno mostrato che molti studi assumevano condizioni favorevoli: un utente che visita una pagina alla volta, la stessa versione di Tor Browser dell'avversario, dati raccolti poco prima dell'attacco [Jua+14]. E soprattutto, nel mondo aperto, i siti sorvegliati sono una frazione minuscola di quelli visitati: anche un classificatore con pochissimi falsi positivi produce, su milioni di visite innocenti, più falsi allarmi che riconoscimenti corretti. È la stessa trappola dei test medici per malattie rare, e un avversario reale deve farci i conti.

Le difese. Il padding di canale del capitolo 7 rende più difficile riconoscere i tempi di inattività, ma non è pensato contro il fingerprinting delle pagine. tor contiene un sistema più generale, il *circuit padding*, che permette di definire «macchine» che inviano celle finte secondo automi a stati; oggi le macchine attive servono a far somigliare i circuiti di introduzione e di rendezvous degli onion service (capitolo 14) ai circuiti ordinari (Proposta 302). Una difesa generale contro il website fingerprinting, efficace e con un costo di banda accettabile, resta un problema di ricerca aperto.

26.5 Sybil: moltiplicare i propri relay

Un attacco **Sybil** consiste nel presentarsi alla rete con molte identità. Per Tor significa aggiungere molti relay, in modo da essere scelti spesso come guard e come exit. Tre linee di difesa lo rendono costoso:

La banda misurata Un relay che dichiara più banda di quella reale sarebbe scelto più spesso: Bauer e coautori mostrarono nel 2007 che bastava mentire nel descrittore [Bau+07]. Oggi la banda usata dal consenso per la scelta dei relay deriva dalle misure delle autorità di banda (sezione 12.4), ma è soggetta anche ai limiti dichiarati dal relay: il valore misurato non è una semplice autodichiarazione. Per ottenere più traffico non basta quindi aumentare il numero scritto nel descrittore. Le misure rendono l'attacco più difficile, senza dimostrare che sia impossibile manipolare la capacità apparente durante una prova; il valore di consenso non garantisce una banda reale e costante.

I vincoli sui percorsi Due relay della stessa famiglia o della stessa rete /16 non stanno nello stesso circuito (sezione 13.1): un operatore onesto si dichiara, e uno disonesto è limitato almeno dagli indirizzi.

La sorveglianza della rete Le autorità possono escludere relay o assegnare `BadExit`; volontari e ricercatori cercano gruppi di relay con caratteristiche comuni e controllano il comportamento degli exit. Nel 2014, per esempio, uno scanner degli exit trovò 25 relay che manipolavano il traffico o erano mal configurati: intercettazioni HTTPS, alterazioni del DNS, *sslstrip* [WL14].

Il capitolo 27 racconta alcuni gruppi di relay ostili scoperti negli anni successivi, e mostra che questa sorveglianza è un lavoro continuo.

Sybil contro gli onion service. Nel sistema v2 la posizione degli HSDir responsabili di un servizio era prevedibile, e un avversario poteva creare relay con identità calcolate per diventarlo: Biryukov e coautori lo usarono nel 2013 per misurare la popolarità dei servizi, renderli irraggiungibili o avvicinarsi alla loro localizzazione [BPW13]. Nel sistema v3 la posizione dipende dalle chiavi accecate e da un valore casuale condiviso che cambia ogni giorno (capitolo 14): la collocazione mirata con identità precalcolate diventa molto più difficile, senza eliminare il rischio di HSDir ostili una volta pubblicati i valori del periodo.

26.6 Scoprire il guard

Il guard protegge il client, ma è anche il bersaglio più interessante: conoscerlo significa sapere dove guardare. Per un onion service l'avversario può far costruire al servizio molti circuiti, per esempio chiedendo molte connessioni, finché un proprio relay non viene scelto come intermedio, accanto al guard: il primo attacco del genere, del 2006, portò all'introduzione dei guard stessi [ØS06]. I vanguard (sezione 13.4) aggiungono uno strato di relay intermedi fissi per un certo tempo, riducendo la velocità con cui un avversario può risalire verso il guard; non garantiscono che sia impossibile scoprirlo.

Riepilogo

- Gli attacchi si distinguono per ciò che l'avversario vede: un estremo, entrambi, l'interno della rete o i percorsi di Internet.
- La correlazione confronta la forma del traffico ai due estremi e non richiede di rompere la cifratura; il problema dell'avversario è essere nella posizione giusta, e nel lungo periodo le occasioni si accumulano.
- Il website fingerprinting riconosce le pagine dal solo traffico cifrato; le valutazioni vanno lette alla luce del tasso di base.
- Contro Sybil: pesi misurati, famiglie e /16, sorveglianza degli exit e dei gruppi di relay; nel sistema v3 la scelta degli HSDir è meno manipolabile in anticipo, ma gli HSDir responsabili sono calcolabili nel periodo corrente.
- Guard e vanguard rendono lenta e costosa la scoperta del primo salto.

Esercizi

Esercizio 26.1 ⋆ Per ciascun attacco del capitolo, indica la posizione minima che l'avversario deve occupare e la difesa di Tor che lo rende più difficile.

Esercizio 26.2 ⋆ Un classificatore riconosce un sito sorvegliato con probabilità 0,95 e scambia per quel sito una visita innocente con probabilità 0,001. Se una visita su 100 000 è davvero al sito sorvegliato, quale frazione degli allarmi è corretta?

Esercizio 26.3 ⋆⋆ Ripeti il Laboratorio 16 con intervalli di 2 secondi, con una durata di 5 secondi, e con tutti i client che usano lo stesso ritmo. Quale modifica peggiora di più gli abbinamenti, e perché?

Esercizio 26.4 ⋆⋆ Modifica i client del Laboratorio 16 perché inviino blocchi a ritmo costante, anche quando non hanno dati (padding). Quanto costa in banda la difesa, e che cosa succede alla correlazione?

Esercizio 26.5 ⋆ ⋆ ⋆ Con `percorso.py` e il consenso reale del Laboratorio 15, calcola la probabilità che un avversario che controlla il 5% della banda di guard e il 5% della banda di exit si trovi ai due estremi di almeno un circuito di un client in sei mesi, con un guard fisso e con un guard cambiato a ogni circuito. Esplicita un numero ipotetico di circuiti nel semestre e assumi, solo per il modello, selezioni indipendenti in base alle quote di banda; spiega perché il calcolo non predice la rete reale.

Per approfondire

- A. Johnson e coautori, *Users Get Routed* [Joh+13].
- Y. Sun e coautori, *RAPTOR: Routing Attacks on Privacy in Tor* [Sun+15].
- M. Juarez e coautori, *A Critical Evaluation of Website Fingerprinting Attacks* [Jua+14].
- La *Anonymity Bibliography*, `https://www.freehaven.net/anonbib/`, per la letteratura successiva.

Capitolo 27

Casi reali contro la rete

Aggiornato al 23 settembre 2026 — comunicazioni del Tor Project (2014, 2022, 2023); tor 0.4.9.13; rete di laboratorio

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- ricostruire l'attacco del 2014 basato sulle celle **RELAY_EARLY** e la difesa introdotta in tor, verificandola dal vivo (Laboratorio 17);
- descrivere il caso KAX17 e il lavoro di sorveglianza dei relay che ne è seguito;
- collegare gli attacchi di negazione del servizio del 2022–2023 alle difese del capitolo 15;
- trarre da questi casi alcune lezioni generali sulla sicurezza di una rete aperta.

Il capitolo precedente ha descritto gli attacchi come li presenta la ricerca: modelli, esperimenti, percentuali. Qui raccontiamo tre episodi di sicurezza della rete reale, ricostruiti sulle comunicazioni pubbliche del Tor Project. Nel caso KAX17, intenzioni ed eventuali attività ostili dell'operatore non sono state accertate. Ci interessano per ciò che insegnano sul protocollo e sulla gestione di una rete aperta, non per i dettagli operativi.

27.1 2014: un segnale nel tipo di cella

Che cosa accadde. Il 4 luglio 2014 il Tor Project individuò un gruppo di relay che, secondo la sua ricostruzione, cercava di deanonimizzare chi pubblicava descrittori o richiedeva informazioni sugli onion service allora chiamati *hidden service*, e lo rimosse dalla rete. I relay erano entrati il 30 gennaio: circa 115 relay veloci, nessuno exit, tutti in due reti /16, che insieme valevano circa il 6,4% della capacità dei guard. Il 30 luglio il Tor Project pubblicò un avviso di sicurezza che descriveva l'attacco e le contromisure [Din14].

Come funzionava. Era la combinazione di un attacco Sybil (sezione 26.5) e di una conferma del traffico attiva (sezione 26.2). I relay erano abbastanza stabili da ottenere sia il flag `HSDir` sia il flag `Guard`. Quando uno di essi veniva interpellato come HSDir per un descrittore, rispondeva inviando verso il client una sequenza di celle in cui l'alternanza tra **RELAY** e **RELAY_EARLY** codificava il nome del servizio (fig. 27.1). Quando un altro relay del gruppo era il guard dello stesso client, leggeva la sequenza e poteva così collegare il client alla richiesta del descrittore di quel servizio, senza per questo dimostrare che il servizio fosse stato visitato.

Perché proprio quel campo? Le celle **RELAY_EARLY** servono a limitare la lunghezza dei circuiti: il client deve usarle per le richieste di estensione, e un relay non ne accetta più di

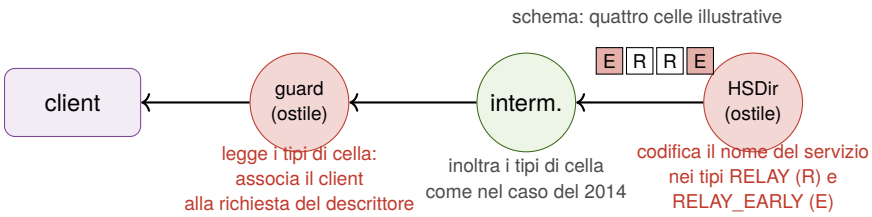


Figura 27.1: L'attacco storico del 2014: l'HSDir ostile codificava il servizio richiesto nei comandi esterni RELAY e RELAY_EARLY, leggibili dai relay attraversati ma protetti dal TLS sui collegamenti. Il guard ostile poteva così associare il client alla richiesta del descrittore, non dimostrare una visita. Le quattro celle sono schematiche, non la codifica integrale di un indirizzo.

otto per circuito ([tor-spec § Handling RELAY_EARLY cells]). Ma il controllo riguardava solo la direzione dal client verso la rete; nella direzione opposta nessuno le inviava, e nessuno le controllava. E, a differenza del contenuto, che è cifrato a strati (capitolo 10), il comando della cella è visibile a ogni relay: un segnale scritto lì attraversa il circuito senza essere alterato. L'avviso osservava anche che il segnale conteneva il nome del servizio in modo leggibile, quindi un relay che avesse ricevuto le celle avrebbe potuto leggerlo; un osservatore esterno con sole registrazioni di rete avrebbe dovuto prima decifrare il TLS dei collegamenti per recuperare il segnale a posteriori.

La risposta. Tre misure: i relay ostili furono esclusi; una nuova versione di tor (0.2.4.23 e 0.2.5.6-alpha) iniziò a chiudere i circuiti su cui arriva una **`RELAY_EARLY`** in direzione del client; e si preparò il passaggio da tre guard a uno solo per client, per ridurre il numero di relay in posizione di vedere il traffico di ciascun utente. L'avviso ricordava infine che la conferma del traffico *passiva* resta un problema aperto: il canale nel tipo di cella la rendeva più comoda, non possibile.

27.2 Laboratorio: il limite delle celle RELAY_EARLY

Laboratorio 17: Otto sì, nove no

Obiettivo. Verificare con `labs/it/codice/torlab` come un relay attuale applica il limite alle celle **`RELAY_EARLY`**.
Prerequisiti. Laboratori 4 e 5; la rete di laboratorio avviata.
Tempo stimato. 30 minuti.

Il nostro `circuito.py` rifiuta già di inviare più di otto celle **`RELAY_EARLY`**. Lo script `labs/it/laboratorio/relay_early.py` lo scavalca. Costruisce un circuito di un salto verso `test004r` e invia n celle **`RELAY_EARLY`** che contengono un messaggio `DROP`, che il relay scarta senza rispondere. Poi chiede una connessione di directory con `BEGIN_DIR`, per vedere se il circuito è ancora vivo:

```
$ PYTHONPATH=labs/it/codice python3 labs/it/laboratorio/relay_early.py 0 8 9
0 celle RELAY_EARLY -> CONNECTED
8 celle RELAY_EARLY -> CONNECTED
9 celle RELAY_EARLY -> circuito distrutto, motivo 1
```

La nona cella chiude il circuito con una **DESTROY** di motivo 1, PROTOCOL. Il listato è un esempio storico, non un risultato ricalcolato qui: indirizzi, identificatori di circuito e log variano tra esecuzioni. Un log di riferimento del relay mostra:

```
[warn] Received too many RELAY_EARLY cells on circ 4045911407 from
       127.0.0.1:58048.  Closing circuit.
```

Il controllo si trova in `src/core/or/command.c`, prima ancora di decifrare la cella. Poche righe sopra c'è quello introdotto nel 2014: una **RELAY_EARLY** che viaggia verso il client produce un avviso nel log e la chiusura del circuito. Il commento nel codice spiega che, un tempo, celle del genere potevano arrivare per un vecchio errore di tor, ma che i relay con quell'errore sono spariti da tempo: oggi una cella così è «sorprendente», e va trattata come un attacco. Anche questo laboratorio fa parte di `make verifica-lab`.

27.3 2020–2021: KAX17 e la salute della rete

Un secondo episodio mostra il lato meno visibile della difesa: accorgersi degli avversari. Il Tor Project lo ha raccontato nell'aprile 2022, in un articolo sul lavoro della propria squadra dedicata alla salute della rete [The22].

Nel dicembre 2020 i grafici dei relay mostravano una forma strana: ogni primo giorno del mese, a mezzanotte, decine di relay entravano nella rete e poi sparivano lentamente. Sembrava un errore nel codice di ibernazione (quello di `AccountingMax`, sezione 22.2). Nel settembre 2021 comparvero gruppi di relay che sembravano di un unico operatore, senza dichiarazione di famiglia e senza contatto; le autorità li esclusero, ma ne arrivavano sempre di nuovi. A fine ottobre una segnalazione anonima portò a identificare un gruppo, poi chiamato KAX17, che fu rimosso; da allora cessarono sia i picchi mensili sia le ondate di relay, e il Tor Project ha ritenuto ragionevole attribuirli allo stesso operatore. Chi fosse e che cosa cercasse di ottenere non è noto.

L'articolo è insolitamente franco sulle cause del ritardo: strumenti di sorveglianza trascurati per anni, sfiducia tra volontari, personale e autorità, un'organizzazione non pensata per questo lavoro. E descrive il processo attuale: chi sorveglia la rete propone l'esclusione di un relay, ma a decidere sono le directory authority, a maggioranza, e non il personale del Tor Project. È la stessa distribuzione della fiducia del capitolo 12, applicata alla gestione della rete.

Approfondimento: Relay senza famiglia

Il primo segnale d'allarme nel caso KAX17 non era un comportamento ostile, ma una violazione di una regola sociale: molti relay dello stesso operatore senza dichiarazione di famiglia (sezione 22.5). È una lezione generale: un avversario non ha bisogno di rompere il protocollo se può aggirare le convenzioni su cui si basa la selezione dei percorsi.

27.4 2022–2023: negazione del servizio

Dall'estate del 2022 la rete fu colpita per mesi da attacchi di negazione del servizio di vario tipo; nel febbraio 2023 il Tor Project scriveva che in alcuni momenti gli utenti non riuscivano a caricare le pagine né a raggiungere gli onion service, che metodi e bersagli cambiavano nel tempo e che non era possibile stabilire con certezza chi li conducesse [The23b]. Gli onion service erano il bersaglio più esposto: chi li attacca non ha un indirizzo da limitare, perché ogni richiesta arriva da un circuito anonimo.

La risposta di più lungo periodo è la prova di lavoro della sezione 15.4, introdotta con tor 0.4.8 nell'agosto 2023 [The23a]. La disponibilità del modulo non equivale alla prote-

zione attiva di ogni onion service: l'operatore deve abilitarla per il singolo servizio con `HiddenServicePoWDefensesEnabled` impostato a `1` (il valore predefinito è `0`). Quando la difesa è abilitata resta inattiva in condizioni normali; durante il sovraccarico può chiedere ai client di risolvere un problema di difficoltà crescente e dare priorità a chi presenta una soluzione valida con più sforzo. Il Tor Project indicava tempi di pochi millisecondi per un client normale, fino a circa un minuto sotto un attacco intenso. Il principio è quello del Laboratorio 9: trasformare un attacco che costa poco all'aggressore in uno che costa, senza chiedere a nessuno di identificarsi.

27.5 Lezioni

- **Ogni campo visibile è un canale.** L'attacco del 2014 non ha rotto la crittografia: ha usato un campo che i relay vedono e inoltrano. Un protocollo di anonimato deve chiedersi, per ogni bit non cifrato, chi può scriverlo e chi può leggerlo; è una delle motivazioni del nuovo schema di cifratura dei relay (sezione 10.4).
- **Sybil è economico, la sorveglianza no.** Aggiungere relay costa poco; accorgersene richiede strumenti, persone e fiducia reciproca.
- **Le difese vanno misurate sul lungo periodo.** Un guard solo, guard più longevi, vanguard: tutte scelte che riducono le occasioni dell'avversario nel tempo (sezione 26.2).
- **La trasparenza è una difesa.** In tutti e tre i casi il Tor Project ha pubblicato che cosa sapeva e che cosa no: è ciò che permette a utenti, operatori e ricercatori di valutare il rischio.

Riepilogo

- Nel 2014 un gruppo di relay Sybil, guard e HSDir insieme, codificò nel tipo di cella il nome degli onion service di cui venivano richiesti i descrittori; tor chiude oggi i circuiti con **`RELAY_EARLY`** in direzione del client, e nello stesso periodo iniziò la riduzione del numero di guard (da tre a uno, poi a due dal 2018), per limitare le occasioni di scegliere quello ostile (sezione 13.3).
- Il limite di otto **`RELAY_EARLY`** per circuito è verificabile nella rete di laboratorio: la nona chiude il circuito.
- KAX17 fu individuato dopo mesi grazie a segnali indiretti (picchi mensili, relay senza famiglia) e a una segnalazione; l'esclusione dei relay spetta alle directory authority.
- Gli attacchi di negazione del servizio del 2022–2023 hanno portato alla prova di lavoro per gli onion service.

Esercizi

Esercizio 27.1 ⋆ Perché un segnale scritto nel contenuto di una cella dall'HSDir non avrebbe potuto essere letto dal guard, mentre uno scritto nel tipo di cella sì?

Esercizio 27.2 ⋆ Spiega come il passaggio da tre guard a uno riduce il danno di un attacco come quello del 2014, e quale svantaggio comporta.

Esercizio 27.3 ⋆⋆ Modifica il Laboratorio 17 per mostrare che anche una cella **`RELAY_EARLY`** che non contiene una richiesta di estensione conta nel limite. Verifica poi nel codice di tor che cosa accade a una cella **`RELAY_EARLY`** che arriva in un circuito in direzione del client.

Esercizio 27.4 ⋆⋆ Dai dati di CollecTor, quali segnali potrebbero rivelare un gruppo di relay dello stesso operatore senza famiglia dichiarata? Proponi tre criteri e discuti i falsi positivi.

Esercizio 27.5 ⋆⋆⋆ Elenca i campi delle celle e dei messaggi relay che un relay intermedio vede in chiaro (capitolo 8) e, per ciascuno, discuti se un relay ostile potrebbe usarlo come canale verso un altro relay dello stesso circuito, e quale difesa esiste o è proposta.

Per approfondire

- Tor Project, *Tor security advisory: «relay early» traffic confirmation attack* [Din14].
- Tor Project, *Malicious relays and the health of the Tor network* [The22].
- Tor Project, *Introducing Proof-of-Work Defense for Onion Services* [The23a].

Capitolo 28

Errori operativi e deanonimizzazione: lezioni

Aggiornato al 23 settembre 2026 — supporto del Tor Project; avvertenze di Tails; tor 0.4.9.13; rete di laboratorio

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- distinguere i livelli a cui un'identità può trapelare, e chi protegge ciascuno;
- riconoscere gli errori operativi più comuni, dalla fuga DNS all'insieme di anonimato troppo piccolo;
- verificare con tor stesso se un'applicazione gli affida la risoluzione dei nomi (Laboratorio 18);
- ragionare su un caso reale nei termini del modello di minaccia.

Il capitolo 26 si chiudeva con un'osservazione scomoda: gli attacchi al protocollo sono costosi e richiedono posizioni privilegiate. In molti casi documentati, la perdita di anonimato dipende anche da errori nell'uso degli strumenti, che il solo protocollo di rete non può prevenire. Questo capitolo è dedicato a quegli errori: non per raccontare come scoprire qualcuno, ma per capire dove finisce ciò che Tor protegge e dove inizia ciò che resta a carico di chi lo usa.

Attenzione: Taglio del capitolo

Parliamo di errori e di difese dal punto di vista di chi vuole proteggersi. Non descriviamo tecniche per identificare persone, e i casi citati sono pubblici e ricostruiti sulle fonti indicate.

28.1 Quattro livelli

Un'identità può trapelare a quattro livelli (fig. 28.1). Il primo, la rete, è quello che tor aiuta a proteggere: in una connessione ordinaria attraverso Tor il destinatario non vede l'IP originario del client e un singolo relay del circuito non dispone normalmente della vista completa di entrambi gli estremi. Questa separazione non elimina attacchi di correlazione o altri rischi. Il secondo, l'applicazione, è compito di Tor Browser, Tails e Whonix (capitolo 23). Gli ultimi due, il contenuto e il comportamento, non sono protetti automaticamente da tor: strumenti dedicati possono ridurre alcuni rischi, ma contano anche ciò che l'utente scrive, carica e fa.

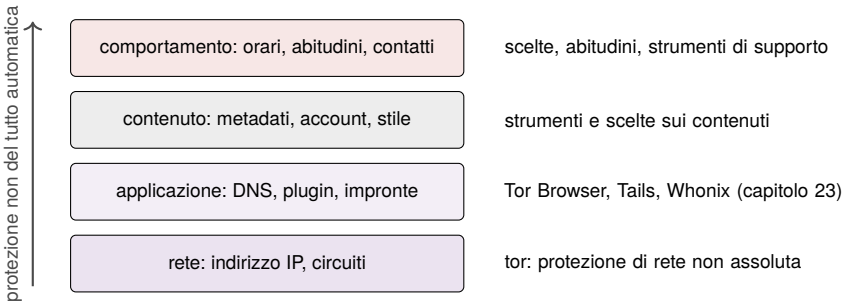


Figura 28.1: Schema didattico dei quattro livelli a cui un'identità può essere esposta. Tor attenua alcuni rischi di rete; le protezioni applicative e gli strumenti per il contenuto aiutano, ma le informazioni condivise e le abitudini restano importanti a ogni livello. Nessuna fascia garantisce da sola l'anonimato.

28.2 Errori dell'applicazione

Traffico fuori da Tor. tor protegge solo le applicazioni configurate per usarlo [Theg]. Un programma che apre connessioni dirette, anche solo per un aggiornamento o un'anteprima, rivela l'indirizzo reale. Il supporto del Tor Project cita due casi tipici. Il primo sono i programmi di condivisione di file BitTorrent, che possono ignorare le impostazioni del proxy e che comunque comunicano ai tracker il proprio indirizzo. Il secondo sono i documenti scaricati e aperti mentre si è in rete: un file DOC o PDF può contenere risorse che il programma che lo apre scarica fuori da Tor.

La fuga DNS. Un caso più sottile riguarda il protocollo SOCKS. Un'applicazione può passare a tor il *nome* della destinazione, e lasciare che la risoluzione avvenga all'exit (sezione 11.1.4); oppure può risolvere il nome da sola, con il DNS del sistema, e passare a tor soltanto l'indirizzo IP. Nel secondo caso la connessione può comunque passare per Tor, ma la risoluzione precedente può aver rivelato il dominio a un resolver esterno al circuito, associandolo alla connessione o all'identità di rete del client. Non è però automatico che sia partita una query DNS in chiaro: il nome potrebbe essere in cache, in un file locale oppure risolto con DNS cifrato. tor non può accertare una fuga già avvenuta, ma può segnalare che la richiesta SOCKS contiene soltanto un IP, come mostra il laboratorio.

28.3 Laboratorio: tor riconosce la fuga DNS

Laboratorio 18: SOCKS con e senza nome

Obiettivo. Osservare come tor distingue un'applicazione che gli affida la risoluzione dei nomi da una che la fa da sola, e come può rifiutare la seconda.
Prerequisiti. Laboratorio 11; la rete di laboratorio avviata.
Tempo stimato. 30 minuti.

Il comando `curl` offre entrambe le varianti: con `--socks5` risolve il nome in locale e passa a tor l'indirizzo; con `--socks5-hostname` passa il nome. Lo script `labs/it/laboratorio/fuga_dns.py` si collega alla ControlPort del client `008c` e attiva l'opzione `TestSocks`, che fa annotare a tor il tipo di ogni richiesta SOCKS. Poi ascolta i messaggi di log e prova le due varianti verso `http://localhost:8080/`. Il laboratorio mostra il *meccanismo*, non una fuga osservata sul filo: `localhost` viene risolto localmente senza interrogare alcun resolver, quindi qui nessuna

query lascia la macchina — ciò che tor rileva è di aver ricevuto un indirizzo IP invece di un nome, un possibile indizio di risoluzione precedente, non una prova di fuga DNS (il programma potrebbe usare direttamente un IP). Con un dominio reale al posto di `localhost`, la risoluzione locale potrebbe produrre una query esterna, eventualmente in chiaro secondo la configurazione DNS: è il rischio che qui evitiamo di provocare deliberatamente, restando sulla rete privata. Lo script usa anche `--noproxy` con argomento vuoto per evitare che le variabili `NO_PROXY` saltino il SOCKS per `localhost`. Infine attiva `SafeSocks`, che rifiuta le richieste con il solo indirizzo, anche quando non vi è stata alcuna fuga DNS:

```
--socks5           HTTP 200  tor: Your application (using socks5 to port
                   8080) is giving Tor only an IP address. Applications
                   that do DNS resolves themselves may leak information...
--socks5-hostname  HTTP 200  tor: Your application (using socks5 to port
                   8080) instructed Tor to take care of the DNS resolution
                   itself if necessary. This is good.
--socks5 SafeSocks HTTP 000
```

Nell'esempio illustrativo entrambe le prime richieste riescono, ma solo la seconda delega la risoluzione a tor; la prima non dimostra una fuga DNS, dato che usa `localhost`. Nel codice di tor (`src/core/proto/proto_socks.c`) il primo avviso è un messaggio di livello `warn`, limitato nel tempo per non riempire il log, accompagnato da un evento `STATUS_CLIENT` con azione `DANGEROUS_SOCKS` sulla ControlPort: un'applicazione di controllo può quindi segnalarlo all'utente. Con `SafeSocks` attivo, la stessa richiesta viene rifiutata e `curl` non ottiene risposta. Attenzione: il fatto che un'applicazione passi correttamente il nome a Tor non dimostra, da solo, che non faccia anche altre risoluzioni DNS fuori dal proxy, per esempio tramite componenti o estensioni. `TestSocks` osserva le richieste inviate alla SocksPort e `SafeSocks` può rifiutare quelle classificate come insicure, ma nessuna delle due opzioni intercetta eventuali interrogazioni DNS separate: il test qui verifica il comportamento SOCKS, non l'assenza di qualsiasi fuga DNS [Thea]. Il laboratorio fa parte di `make verifica-lab`.

28.4 Errori del contenuto

Metadati. Molti file portano con sé informazioni che l'autore non vede: le fotografie la posizione e il modello della macchina, i documenti l'autore e le date di creazione. Tails include uno strumento per rimuoverli, e le sue avvertenze ricordano casi in cui i metadati di una fotografia hanno rivelato dove si trovava chi l'aveva scattata [Taib].

Account e identità. Se si accede a un account personale, il sito sa chi siamo, qualunque sia l'indirizzo da cui ci colleghiamo [Theg]. E se nella stessa sessione si usano due identità diverse sullo stesso sito, il sito può collegarle attraverso cookie, sessione, identificatori applicativi e altri indizi: non può però osservare direttamente l'identificatore del circuito Tor. Le avvertenze di Tails consigliano di riavviare il sistema tra attività che non devono essere collegate [Taib]. È il requisito di non collegabilità di Tor Browser (sezione 23.2), che però vale tra siti diversi, non tra identità diverse sullo stesso sito.

Lo stile. Anche senza metadati, il modo di scrivere, gli argomenti, gli errori ricorrenti e gli orari di pubblicazione possono collegare un testo anonimo a uno firmato. Nessuno strumento automatico protegge da questo.

28.5 Errori di comportamento: l'insieme troppo piccolo

Il capitolo 1 ha definito l'anonimato come l'essere indistinguibili all'interno di un insieme. tor può ridurre diversi elementi di distinguibilità a livello di rete, ma non può garantire un insieme di anonimato numeroso: se in un certo luogo e in un certo momento gli utenti di Tor sono pochissimi, basta sapere *chi usava Tor* per restringere la ricerca.

Un caso. Nel dicembre 2013 un'università statunitense ricevette per posta elettronica false minacce di bomba, poco prima di un esame. Il mittente aveva usato un servizio di posta temporanea e Tor. Secondo la dichiarazione giurata allegata alla denuncia dell'FBI, riportata concordemente dalla stampa, l'università fu in grado di stabilire chi si fosse collegato a Tor attraverso la propria rete senza fili nelle ore precedenti l'arrivo dei messaggi [WBU13; San13]. Lo studente, interrogato, ammise. Non risultò necessario violare la cifratura di Tor per restringere i sospetti: l'osservatore locale poteva conoscere l'uso di Tor, senza che ciò implicasse la conoscenza dei contenuti o delle destinazioni. Ma l'osservatore della rete locale vedeva che *lui* usava Tor, in quell'intervallo, e l'insieme dei sospetti si riduceva a pochi nomi.

Il caso illustra tre principi del modello di minaccia (capitolo 4):

- Tor, senza bridge, non nasconde di essere usato: chi osserva la rete locale lo vede [Taib]. Un bridge con un pluggable transport (capitolo 16) attenua il problema.
- La correlazione non richiede un avversario globale se l'insieme è piccolo e l'evento è preciso nel tempo: è una forma elementare di attacco di intersezione.
- L'anonimato ama la compagnia [DM06]: più utenti con motivazioni diverse usano Tor nello stesso luogo, meno la sola presenza è un indizio. Il supporto del Tor Project lo dice in modo diretto: convincete altri a usare Tor [Theg].

28.6 Una lista di controllo

Senza pretesa di completezza, e in ordine di livello:

1. usare Tor Browser o un sistema come Tails o Whonix, non un browser qualsiasi con un proxy;
2. non installare estensioni né cambiare impostazioni che rendano il browser diverso dagli altri;
3. per altre applicazioni, verificare che passino i nomi a tor (`SafeSocks`, `TestSocks`);
4. non aprire documenti scaricati mentre si è collegati, o aprirli in un ambiente isolato;
5. preferire HTTPS e gli onion service, perché l'exit vede ciò che non è cifrato;
6. togliere i metadati dai file prima di condividerli;
7. non mescolare nella stessa sessione identità che non devono essere collegate;
8. se anche solo l'uso di Tor è rischioso, valutare un bridge con trasporto adatto, tenendo presente che non garantisce invisibilità;
9. ricordare che orari, abitudini e stile di scrittura sono informazioni.

Riepilogo

- L'identità può trapelare dalla rete, dall'applicazione, dal contenuto e dal comportamento; tor protegge soltanto il primo livello.
- Applicazioni che escono da Tor, documenti aperti in rete e fughe DNS sono errori dell'applicazione; tor segnala e può rifiutare le richieste SOCKS con il solo indirizzo, ma queste opzioni non escludono risoluzioni DNS compiute separatamente.
- Metadati, account e stile collegano il contenuto a una persona.
- Se l'insieme degli utenti di Tor è piccolo, sapere chi usa Tor e quando può restringere i sospetti: bridge e un'ampia comunità di utenti possono attenuare, non cancellare, questo rischio.

Esercizi

Esercizio 28.1 ⋆ Per ciascuno degli errori elencati nel capitolo, indica il livello della fig. 28.1 e lo strumento o l'abitudine che lo previene.

Esercizio 28.2 ⋆ Spiega perché una fuga DNS rivela informazioni anche se la connessione successiva passa per Tor. Chi le vede?

Esercizio 28.3 ⋆⋆ Nel Laboratorio 18, iscriviti con Stem agli eventi `STATUS_CLIENT` e mostra l'azione `DANGEROUS_SOCKS` generato dalla richiesta con il solo indirizzo. Come potrebbe usarlo un'interfaccia grafica?

Esercizio 28.4 ⋆⋆ Rileggi il caso del 2013 immaginando che lo studente avesse usato un bridge obfs4 dalla stessa rete. Che cosa avrebbe visto l'osservatore della rete locale, e quali altre informazioni avrebbero potuto restringere comunque l'insieme?

Esercizio 28.5 ⋆⋆⋆ Scegli una persona reale o immaginaria con esigenze di anonimato (una fonte giornalistica, un'attivista, un ricercatore) e scrivi il suo piano operativo: minacce, livelli coinvolti, strumenti e abitudini, e i rischi che restano scoperti.

Per approfondire

- Il supporto del Tor Project sull'anonimato [Theg].
- Le avvertenze di Tails, *Tails is safe but not magic* [Taib].
- R. Dingledine, N. Mathewson, *Anonymity Loves Company* [DM06].

Parte V

Ricerca e futuro

Capitolo 29

La ricerca accademica su Tor

Aggiornato al 23 settembre 2026 — articoli citati nel capitolo; portale di ricerca del Tor Project; consenso del 23 settembre 2026

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- orientarti tra i luoghi in cui si pubblica la ricerca su Tor e le risorse che il Tor Project mette a disposizione;
- misurare l'anonimato con strumenti quantitativi, dall'entropia al tempo alla prima compromissione;
- scegliere il metodo sperimentale adatto a una domanda: analisi, simulazione, emulazione, misura;
- simulare per mesi il comportamento di migliaia di client sul consenso reale (Laboratorio 19).

Tor è, fin dall'inizio, un progetto nato dalla ricerca: l'onion routing viene da un laboratorio scientifico (capitolo 3), il progetto di Tor è stato presentato a una conferenza accademica [DMS04], e molte delle scelte che abbiamo studiato, dai guard al controllo della congestione, sono risposte ad articoli scientifici. Questo capitolo guarda la ricerca dal punto di vista di chi vuole farla: dove si pubblica, come si misura l'anonimato, con quali strumenti si sperimenta e con quali regole.

29.1 Dove si fa ricerca su Tor

La ricerca sulle reti di anonimato si pubblica soprattutto in due tipi di sedi:

Il Privacy Enhancing Technologies Symposium (PETS) con la sua rivista *Proceedings on Privacy Enhancing Technologies* (PoPETs): la sede specializzata della comunità: il lavoro sulla misura dell'anonimato che vedremo tra poco fu presentato nel 2002 al predecessore workshop PET, prima dell'attuale rivista PoPETs [Día+02].

Le grandi conferenze di sicurezza USENIX Security, ACM CCS, IEEE Symposium on Security and Privacy, NDSS: vi sono apparsi quasi tutti gli articoli del capitolo 26.

La *Anonymity Bibliography* di Free Haven raccoglie da oltre vent'anni gli articoli del settore, ed è il punto di partenza di ogni ricerca bibliografica.

Il Tor Project, dal canto suo, mantiene un portale per i ricercatori con dati, strumenti, rapporti tecnici, idee di ricerca e il Research Safety Board. Il portale insiste su un punto che

ogni ricercatore dovrebbe prendere sul serio: il protocollo e la rete cambiano continuamente, e misurare qualcosa senza capire che cosa succede dietro le quinte porta a conclusioni sbagliate. Per questo chiede ai ricercatori di contattare il progetto il prima possibile, per sapere quali altri esperimenti sono in corso e quali cambiamenti sono previsti.

29.2 Misurare l'anonimato

Nel capitolo 1 abbiamo definito l'anonimato come l'essere indistinguibili in un insieme. Ma non tutti i membri dell'insieme sono uguali agli occhi di un avversario: dopo un'osservazione, alcuni possono essere molto più probabili di altri. La dimensione dell'insieme non basta.

Il grado di anonimato. Nel 2002 Díaz e coautori proposero di misurare l'anonimato con l'entropia della distribuzione di probabilità che l'avversario assegna ai possibili mittenti [Día+02]. Se p_i è la probabilità che l'avversario attribuisce all'utente i di essere il mittente, l'entropia è

$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i,$$

e il suo massimo, $H_M = \log_2 N$, si ha quando tutti gli N utenti sono ugualmente probabili. Il **grado di anonimato** è il rapporto

$$d = \frac{H(X)}{H_M}, \qquad 0 \le d \le 1 :$$

per $N \ge 2$, vale 1 se la distribuzione a posteriori è uniforme (non necessariamente se l'avversario non ha imparato nulla) e 0 quando un solo mittente ha probabilità 1. Per $N = 1$ il rapporto è indefinito, perché $H_M = 0$.

Un modo intuitivo di leggere l'entropia è $2^{H(X)}$: il numero di utenti ugualmente probabili che darebbe la stessa incertezza. Un insieme di 1000 utenti in cui l'avversario ha ristretto i sospetti a due, con probabilità 0,5 ciascuno, vale quanto un insieme di due: $H = 1$ bit.

Il limite delle misure istantanee. L'entropia descrive una singola osservazione. Ma, come abbiamo visto nella sezione 26.2, per Tor il rischio si accumula nel tempo: ogni circuito è una nuova occasione per l'avversario. Johnson e coautori hanno quindi proposto misure *nel tempo*, come la probabilità di essere deanonimizzati almeno una volta entro un certo periodo, o il tempo mediano alla prima compromissione, calcolate simulando il comportamento degli utenti su dati reali della rete [Joh+13]. È ciò che faremo nel laboratorio.

29.3 Quattro metodi

Analisi Un modello matematico, come quello del grado di anonimato o quello della sezione 26.5 per gli attacchi Sybil. È rapida e generale, ma dipende dalle ipotesi, e il sistema reale ne viola sempre qualcuna.

Simulazione del comportamento Si riproduce in modo semplificato una parte del sistema, per esempio la scelta dei percorsi, con dati reali. È il metodo di TorPS, il simulatore di percorsi di Johnson e coautori [Joh+13], e del nostro Laboratorio 19: veloce, ma cieco a tutto ciò che il modello non include.

Emulazione e simulazione della rete Si eseguono i programmi veri: in una rete privata, come quella di Chutney usata in tutto il libro, o in un simulatore a eventi discreti come Shadow [JH12], che permette di far girare migliaia di relay e client su una sola macchina, con un tempo virtuale (sezione 19.9). È il metodo tipico per le prestazioni, perché permette di provare una modifica del protocollo su una rete realistica prima di distribuirla.

Misura della rete reale Solo così si sa che cosa succede davvero, ma è anche il metodo più rischioso per gli utenti, e va progettato con cura (sezione 29.5).

Una buona ricerca combina spesso più metodi: un'analisi che suggerisce un'ipotesi, una simulazione che la verifica su larga scala, una misura sulla rete reale che ne controlla il realismo.

Approfondimento: Misurare senza vedere

PrivCount, di Jansen e Johnson, mostra come conciliare misura e privacy [JJ16]. I relay che partecipano contano gli eventi di interesse, ma ogni contatore è mascherato con valori casuali condivisi con altri partecipanti e con del rumore aggiunto secondo i principi della *privacy differenziale*; solo la somma finale, a fine periodo, è visibile, e anche quella con un rumore che impedisce di dedurre il comportamento di un singolo utente. Con questo sistema gli autori stimarono, su una media di intervalli di dieci minuti misurati nel 2016, circa 710 000 indirizzi IP distinti di client che si collegavano alla rete, dei quali circa 550 000 classificati come attivi; non si tratta né di un conteggio esatto di persone né di connessioni simultanee in un singolo istante. Il traffico associato alle porte web costituiva il 91% dei byte di dati rilevati dagli exit partecipanti. Queste stime descrivono lo studio del 2016, non la rete attuale. È la stessa filosofia degli arrotondamenti delle statistiche dei relay (sezione 24.2), portata alle sue conseguenze.

29.4 Laboratorio: tempo alla prima compromissione

Laboratorio 19: Mesi di circuiti in una simulazione

Obiettivo. Stimare, sul consenso reale, quanti client un avversario con una data quota di banda riesce a deanonimizzare nel tempo, con e senza guard persistenti.
Prerequisiti. Laboratori 7 e 15; nessuna rete necessaria.
Tempo stimato. 1–2 ore.

Il modulo `labs/it/codice/torlab/simulazione.py` riusa i pesi del Laboratorio 7. L'avversario aggiunge alla rete un guard e un exit che valgono ciascuno una quota q della banda pesata della propria posizione. Simuliamo 2000 client per 180 giorni. Ogni giorno ogni client costruisce 20 circuiti. Il modello sostituisce un guard persistente secondo una cadenza approssimata per un guard *confermato*: nel guard-spec e nel codice (`entrynodes.c`) un guard confermato è rimosso dal campione `GUARD_CONFIRMED_MIN_LIFETIME` (60 giorni) dopo la conferma, con `CONFIRMED_ON_DATE` retrodatato fino a `GUARD_LIFETIME`/10 (12 giorni), per una cadenza simulata tra 48 e 60 giorni dall'inizializzazione. Il valore `GUARD_LIFETIME` (120 giorni) governa invece la scadenza dei guard non confermati nel campione. Nella rete reale, campione e lista dei guard confermati, raggiungibilità e sostituzioni rendono la rotazione più complessa: l'esempio non riproduce l'intero algoritmo. Una versione preliminare che applicava 120 giorni anche ai guard confermati sottostimava il rischio in questo modello. Per confronto, simuliamo

anche client che scelgono un guard nuovo per ogni circuito. Un client è compromesso il primo giorno in cui un suo circuito ha guard ed exit dell'avversario.

Il modello è volutamente semplice: ignora il relay intermedio, le famiglie, le reti /16, i guard di riserva e i fallimenti. Per restare veloce, non estrae i relay con `random.choices`, che ricalcolerebbe ogni volta i pesi di migliaia di relay, ma con una ricerca binaria sui pesi cumulati: milioni di estrazioni sono possibili senza ricalcolare ogni volta l'intera distribuzione; il tempo effettivo dipende dalla macchina.

Il consenso pubblico non è incluso nel repository: si scarica una volta sola con `make -C labs/it data`, che lo salva in `labs/it/dati/collector/` controllandone l'intestazione, non le firme. La simulazione è deterministica: a parità di modello, consenso, quota e seme, il risultato è sempre questo.

```
$ PYTHONPATH=labs/it/codice python3 -m torlab.simulazione labs/it/dati/collector/
    consensus-microdesc-2026-09-23-15 0.05
avversario: 5% della banda di guard e 5% della banda di exit; 2000 client, ...
guard persistente    1g: 2.7%  7g: 4.5%  30g:  4.5%  90g:  9.3%  180g:  18.2%  mediana
    : None
guard per circuito   1g: 5.3%  7g: 29.2% 30g: 77.1%  90g: 99.1%  180g: 100.0%  mediana
    : 14
```

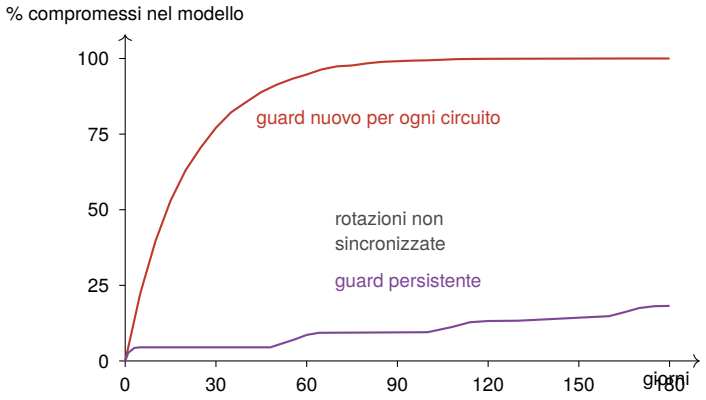


Figura 29.1: Laboratorio 19, simulazione semplificata: 2 000 client, 20 circuiti al giorno per 180 giorni, sulla fotografia del consenso del 23 settembre 2026; avversario con il 5% della banda pesata dei guard e il 5% degli exit. Il grafico mostra la quota cumulativa di client che, nel modello, hanno usato almeno un circuito con entrambi gli estremi ostili. Le rotazioni dei guard persistenti avvengono in tempi diversi per ogni client (intervalli simulati di 48–60 giorni), non in tre giorni comuni a tutti. I dati illustrano il modello, non compromissioni osservate nella rete pubblica.

La fig. 29.1 mostra le due curve. Senza guard persistenti, metà dei client è compromessa in due settimane e quasi tutti in tre mesi: ogni circuito è una nuova estrazione, e le occasioni si sommano. Con i guard, invece, il destino di un client si decide quasi tutto al momento della scelta del guard. Circa il 4,5% dei client, quelli che scelgono il guard ostile, viene compromesso presto; gli altri non possono essere compromessi attraverso questa specifica condizione del modello finché il guard non ruota, e durante le rotazioni individuali distribuite nel tempo (ogni 48–60 giorni circa per client, secondo il modello) nuove estrazioni aggiungono altri client, portando la frazione al 18% a sei mesi. Con una quota dell'1% si scende a circa il 4% a sei mesi, contro il 32% senza guard.

L'esempio illustra, su una singola istantanea storica della rete, il ruolo dei guard persistenti: nel modello non riducono necessariamente il rischio marginale per circuito, ma modificano

la distribuzione dell'esposizione fra i client. Per quelli con un guard onesto il rischio di questa specifica compromissione è nullo finché quel guard viene mantenuto; per chi ne sceglie uno ostile è elevato. È anche il motivo per cui la durata dei guard è un compromesso: una rotazione più lenta allunga la protezione di chi ha un guard onesto, ma prolunga l'esposizione di chi lo ha ostile. E un avversario paziente, che resti nella rete per mesi, vede arrivare nuovi client a ogni rotazione.

29.5 Fare ricerca in modo sicuro

Il capitolo 25 ha presentato le linee guida del Tor Research Safety Board [Tor]; qui aggiungiamo il punto di vista di chi progetta un esperimento. Una domanda utile da porsi, prima ancora di scegliere il metodo, è: *la mia domanda richiede davvero la rete reale?* Molte domande sulle prestazioni o sugli attacchi trovano risposta in Shadow o in una rete privata, con il vantaggio di poter ripetere l'esperimento e di controllare ogni variabile. Quando la rete reale serve, i principi sono quelli già visti: agire solo sul proprio traffico, raccogliere il minimo, aggregare prima di pubblicare, confrontarsi con il Board e con il Tor Project prima di iniziare.

La storia recente offre esempi in entrambe le direzioni. Il capitolo 27 ha raccontato un esperimento, o un attacco, condotto sugli utenti reali con un segnale leggibile dai relay riceventi; un osservatore esterno avrebbe dovuto prima decifrare le connessioni TLS fra i nodi. PrivCount mostra invece che si può misurare la rete in profondità senza esporre nessuno. La differenza non sta negli strumenti, ma nel progetto dell'esperimento.

Riepilogo

- La ricerca su Tor si pubblica a PETS e nelle grandi conferenze di sicurezza; il Tor Project offre dati, strumenti e un Research Safety Board.
- L'entropia misura l'anonimato di una singola osservazione; il grado di anonimato la normalizza tra 0 e 1. Nel tempo servono misure come la probabilità di compromissione entro un periodo.
- Analisi, simulazione del comportamento, emulazione o simulazione della rete e misura reale rispondono a domande diverse, con rischi diversi.
- Sul consenso reale, i guard persistenti riducono drasticamente la frazione di client compromessi nel tempo, concentrando il rischio su pochi.

Esercizi

Esercizio 29.1 $\star$ Un avversario osserva un sistema con quattro utenti e assegna probabilità $1/2$, $1/4$, $1/8$, $1/8$. Calcola l'entropia, il grado di anonimato e il numero equivalente di utenti ugualmente probabili.

Esercizio 29.2 $\star$ Per ciascuna domanda, scegli il metodo più adatto e giustifica la scelta: (a) quanto aumenta la latenza se i circuiti diventano di quattro salti? (b) quanti utenti usano Tor da un certo paese? (c) un nuovo algoritmo di scelta dei guard riduce il tempo alla prima compromissione?

Esercizio 29.3 $\star\star$ Chiama `simula` del Laboratorio 19 con `vita_guard` pari a 30 e a 180 giorni (invece dei 60 predefiniti), e confronta la frazione di client compromessi a 180 giorni. Come si spiega il risultato?

Esercizio 29.4 ⋆⋆ Ripeti il Laboratorio 19 con un avversario che ha il 10% della banda di exit e nessun guard, e con uno che ha il 10% della banda di guard e nessun exit. Che cosa ottiene ciascuno, e perché?

Esercizio 29.5 ⋆⋆⋆ Estendi il simulatore con tre elementi che il modello ignora: i circuiti che un client riusa per dieci minuti invece di costruirne di nuovi, il vincolo sulle reti /16 tra guard ed exit, e un avversario che controlla l'intero sistema autonomo di una grande rete. Quali cambiano di più il risultato?

Per approfondire

- C. Díaz, S. Seys, J. Claessens, B. Preneel, *Towards Measuring Anonymity* [Día+02].
- R. Jansen, A. Johnson, *Safely Measuring Tor* [JJ16].
- Il portale di ricerca del Tor Project, `https://research.torproject.org/`.
- La *Anonymity Bibliography*, `https://www.freehaven.net/anonbib/`.

Capitolo 30

Arti: Tor in Rust

Aggiornato al 23 settembre 2026 — Arti 2.6.0 (1 settembre 2026); tor 0.4.9.13; rete di laboratorio

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- spiegare perché il Tor Project sta riscrivendo tor in Rust e a che punto è il lavoro;
- orientarti nell'architettura di Arti e ritrovarvi i concetti studiati nella Parte II;
- leggere la tabella dei sottoprotocolli supportati e capire che cosa manca;
- configurare Arti come client della rete privata del libro (Laboratorio 20).

Tutto il libro si è basato su *tor*, l'implementazione in C nata nel 2002. Ma il futuro del client Tor si chiama **Arti**, una riscrittura completa in Rust. Il Tor Project ha dichiarato che Arti è il client preferito per lo sviluppo di nuove funzioni, e che ogni nuova funzione lato client avrà un'implementazione in Arti. Questo capitolo lo presenta con gli occhi di chi ha appena scritto, nei laboratori, un piccolo client Tor in Python: molte delle scelte di Arti sono le stesse che abbiamo dovuto fare anche noi, prese sul serio.

30.1 Perché riscrivere tor

Il documento introduttivo del progetto elenca quattro ragioni:

Sicurezza Rust riduce molte classi di errori di memoria nel codice sicuro; codice `unsafe`, dipendenze e difetti logici richiedono comunque verifiche. Secondo la stima del Tor Project, almeno metà delle vulnerabilità di sicurezza registrate per tor sarebbero state impossibili in Rust, e molte altre molto improbabili.

Velocità di sviluppo Le garanzie del linguaggio permettono di scrivere codice nuovo con più fiducia e meno tempo.

Flessibilità tor è nato come proxy SOCKS, e le funzioni di integrazione sono state aggiunte dopo. Arti è progettato fin dall'inizio come libreria modulare, da incorporare in altre applicazioni.

Pulizia Vent'anni di sviluppo hanno lasciato in tor relazioni intricate tra parti del codice. Una riscrittura permette di separare meglio le responsabilità.

Le tappe. La prima versione pubblica, 0.0.1, è dell'ottobre 2021; la 1.0.0, del 1º settembre 2022, ha dichiarato Arti stabile come client. Nell'agosto 2023, con la 1.1.7, il supporto per il lato client degli onion service è stato incluso per default nella compilazione, pur restando da attivare. La 2.0.0 è del febbraio 2026. La versione di riferimento per questo capitolo è la 2.6.0, del 1º settembre 2026, che rende sempre attivi il controllo della congestione (sezione 11.3) e la cifratura CGO (sezione 10.4).

Lo stato. Secondo la documentazione del progetto, Arti è un client completo, adatto all'uso generale, che supporta bridge e pluggable transports e sa sia raggiungere sia pubblicare onion service. Non può ancora funzionare come relay, bridge o directory authority: è il lavoro in corso, e buona parte delle ultime versioni riguarda proprio lo sviluppo del lato relay e delle autorità.

30.2 L'architettura

Arti è diviso in decine di *crate*, le unità di compilazione di Rust (oltre settanta nella versione 2.6.0). Una convenzione di nomi le distingue: iniziano con `arti` quelle rivolte all'utente, con `tor-` quelle specifiche del protocollo Tor, e hanno nomi generici quelle utili anche fuori da Tor. Un principio guida l'intera struttura: il codice che legge e scrive sulla rete deve essere il meno possibile, e la maggior parte del codice deve «fingere che la rete non esista», per essere facile da provare.

La fig. 30.1 mostra le crate principali accanto ai moduli di `torlab` che svolgono lo stesso compito. La corrispondenza non è casuale: entrambi seguono la struttura del protocollo, dal basso verso l'alto.

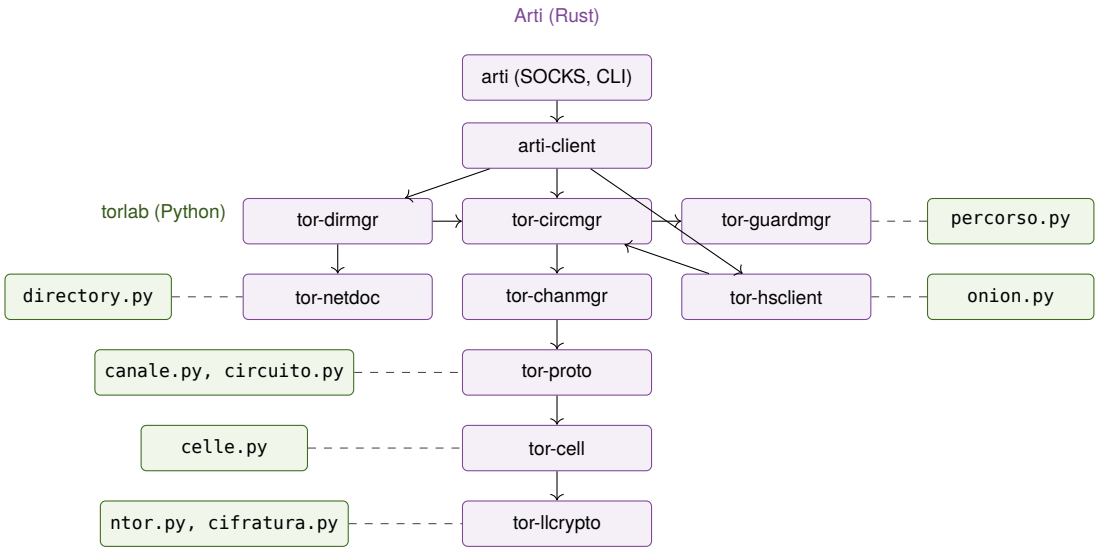


Figura 30.1: Schema funzionale delle principali crate Arti e moduli didattici di `torlab`. Le frecce continue sono relazioni architetturali semplificate, non una trascrizione delle dipendenze Cargo; i tratteggi segnalano analogie di funzione, non dipendenze fra progetti né equivalenze complete. `onion.py`, per esempio, decifra descrittori ma non implementa un client onion service completo; crate come `tor-bytes` e `tor-hsservice` sono omesse.

`tor-llcrypto`, `tor-bytes` primitive crittografiche e lettura e scrittura di dati binari: il fondamento su cui `ntor.py` e `cifratura.py` si appoggiavano alla libreria `cryptography` e al modulo `struct` di Python.

`tor-cell` le celle e i messaggi relay: il corrispondente di `celle.py` (capitolo 8).

`tor-proto` canali, circuiti, cifratura a strati, stream: ciò che abbiamo costruito con `canale.py`, `circuito.py` e `stream.py`.

`tor-chanmgr`, `tor-circmgr`, `tor-guardmgr` la gestione di molti canali e circuiti, la selezione dei percorsi e i guard: ciò che nel nostro client didattico mancava quasi del tutto, e che in un client reale è la parte più complessa.

`tor-netdoc`, `tor-dirmgr` lettura dei documenti di directory e loro aggiornamento: il corrispondente di `directory.py`.

`tor-hsclient`, `tor-hsservice` i due lati degli onion service, di cui `onion.py` implementava solo la decifratura del descrittore.

`arti-client`, `arti` la libreria di alto livello, da incorporare in altre applicazioni, e il programma a riga di comando che offre un proxy SOCKS.

30.2.1 Un confronto: scrivere una cella

Nel Laboratorio 2 abbiamo scritto una funzione che codifica una cella. La funzione corrispondente di Arti, `write_cell` in `crates/tor-cell/src/chancell/codec.rs`, segue gli stessi passi: scrive il CircID, poi il comando; se il comando è di una cella a lunghezza variabile, riserva due byte per la lunghezza, codifica il corpo e torna indietro a scrivere la lunghezza vera; altrimenti codifica il corpo e controlla che non superi i 509 byte (`CELL_DATA_LEN`). Due differenze sono istruttive:

- il CircID è scritto *sempre* su 4 byte. Arti non implementa le versioni del protocollo di collegamento precedenti alla 4, e quindi non ha bisogno del caso speciale dei CircID di 2 byte (capitolo 8); solo la cella **`VERSIONS`**, che precede la negoziazione, è trattata a parte;
- i controlli di lunghezza non sollevano un errore del chiamante, ma un errore *interno*: se una cella è troppo lunga, il difetto è nel codice di Arti, non nei dati ricevuti. È un modo di distinguere, nel tipo stesso dell'errore, i bug dagli input ostili.

La scelta di non supportare nulla di obsoleto è dichiarata come principio: Arti mira alla compatibilità con i protocolli *attualmente raccomandati*, e non ha implementato né intende implementare versioni obsolete di alcunché.

30.3 Che cosa supporta

Il modo più preciso di descrivere che cosa un'implementazione di Tor sa fare sono i **sottoprotocolli** ([tor-spec § Subprotocol-based versioning]): ogni relay e ogni client dichiara, per ciascuna famiglia di funzioni (`Link`, `Relay`, `FlowCtrl`, `HSIntro` e così via), quali versioni supporta. Nella documentazione di compatibilità, Arti elenca come supportate lato client, tra le altre:

- `Link=4` e `Link=5`, cioè i CircID di 4 byte e il padding di canale moderno (capitolo 7);
- `Relay=4` (ntor-v3, capitolo 9), `Relay=5` (negoziazione dei sottoprotocolli) e `Relay=6` (cifratura CGO);
- `FlowCtrl=1` (SENDME autenticati) e `FlowCtrl=2` (controllo della congestione);
- `Desc=4`, cioè le famiglie basate su chiave che abbiamo incontrato nella sezione 22.5.

Attenzione: Un refuso nella tabella di Arti

Il documento di compatibilità di Arti numera questi due sottoprotocolli come `FlowCtrl=0` e `FlowCtrl=1`. La specifica, però, assegna il valore 1 ai SENDME autenticati e il 2 al controllo della congestione [tor-spec § Subprotocol-based versioning], e nel consenso pubblico i relay dichiarano infatti `FlowCtrl=1-2`. È uno sfasamento di uno nella documentazione, senza conseguenze sul codice, ma un buon promemoria: quando due fonti non concordano, decide la specifica, e il consenso permette di controllare chi ha ragione.

Due funzioni di tor mancano ancora nel client Arti. `Conflux=1`, il multipercorso: la crate `tor-proto` lo supporta, ma Arti non costruisce ancora circuiti Conflux. E `Padding=2`, le macchine di padding per nascondere la costruzione dei circuiti degli onion service (sezione 26.4): `tor-proto` ha un'infrastruttura di padding basata sulla libreria *maybenot*, ma le macchine non sono ancora implementate. Lato relay, nessun sottoprotocollo è ancora spuntato: sono in gran parte scritti, ma vengono segnati come supportati solo quando sono disponibili su un relay funzionante.

30.4 Laboratorio: Arti nella rete privata

Laboratorio 20: Un secondo client, in Rust

Obiettivo. Compilare Arti, configurarlo come client della rete privata del libro e usarlo come proxy SOCKS al posto di `test008c`.
Prerequisiti. La rete di laboratorio avviata; un ambiente Rust (`cargo`).
Tempo stimato. 1–2 ore, di cui buona parte per la compilazione.

Passo 1: compilare. Dalla radice della repository del libro, scarica la versione di riferimento Arti 2.6.0 in `labs/it/vendor/arti` e compila soltanto il programma `arti`:

```
$ mkdir -p labs/it/vendor
$ git clone https://gitlab.torproject.org/tpo/core/arti.git labs/it/vendor/arti
$ git -C labs/it/vendor/arti checkout arti-v2.6.0
$ (cd labs/it/vendor/arti && cargo build --release -p arti --locked)
$ labs/it/vendor/arti/target/release/arti --version
Arti 2.6.0
```

La prima compilazione richiede alcuni minuti, perché coinvolge l'intero albero delle crate e delle loro dipendenze.

Attenzione: Versione didattica e aggiornamenti di sicurezza

Il tag `arti-v2.6.0` è fissato per rendere riproducibile il laboratorio su una rete privata isolata; non è una raccomandazione a usare questa versione per proteggere comunicazioni reali. Già Arti 2.7.0, pubblicato il 1º ottobre 2026, ha incluso correzioni di sicurezza di gravità da bassa ad alta, e il Tor Project ha invitato a effettuare l'aggiornamento, in particolare per gli onion service [The26]. Per l'uso sulla rete pubblica consultare sempre gli avvisi ufficiali e scegliere una release di sicurezza aggiornata. Un'eventuale sostituzione della versione nel laboratorio richiede di ricontrollare configurazione e risultato.

Passo 2: configurare. Arti non legge il `torrc`: si configura con un file TOML. Per usare una rete diversa da quella pubblica bisogna dirgli quali sono le autorità e da quali cache scaricare la

prima volta i documenti di directory, esattamente ciò che nel `torrc` di un nodo Chutney fanno le righe `DirAuthority` (sezione 19.6). Lo script `labs/it/laboratorio/arti_config.py` legge dai file dei nodi le identità delle tre autorità, cioè impronta RSA, chiave ed25519 e identità v3 del certificato, insieme alle loro porte. Con questi dati scrive `labs/it/laboratorio/net/arti/arti.toml`. Il frammento seguente è illustrativo: le identità sono generate da Chutney e i puntini non sono TOML valido:

```
[tor_network]
fallback_caches = [
  { rsa_identity = "7B465200C885ED566C6CD4F46DBB0492076D203A",
    ed_identity = "JwhsA9VS01mRVnQseL9xsKZfstoBae7OcgL3jBvDzis",
    orports = ["127.0.0.1:5100"] },
  ...
]
[tor_network.authorities]
v3idents = ["390998046DEFF1E68AC346D7577319791B32C91A", ...]
```

Tre altre impostazioni adattano Arti a una rete in cui tutto è su `127.0.0.1`: `allow_local_addrs` permette di collegarsi a indirizzi locali, che un client della rete pubblica rifiuta; due prefissi di rete impossibili (33 bit per IPv4, 129 per IPv6) disattivano il vincolo sulle stesse reti, che altrimenti vieterebbe qualunque circuito; e `hsdir_interval` accorcia il periodo degli onion service come fa tor con `TestingTorNetwork`. Sono le stesse scelte della configurazione che Chutney genera per i propri nodi Arti.

Passo 3: avviare. Dalla radice del libro, dopo aver generato la configurazione con `python3 labs/it/laboratorio/arti_config.py`, avvia `labs/it/vendor/arti/target/release/arti proxy -p 9150 -c labs/it/laboratorio/net/arti/arti.toml`. I tempi del bootstrap dipendono dalla macchina; in una nostra esecuzione:

```
INFO tor_dirmgr::bootstrap: 1: Looking for a consensus.
INFO tor_dirmgr::bootstrap: 1: Downloading certificates for consensus
     (we are missing 3/3).
INFO tor_dirmgr::bootstrap: 1: Downloading microdescriptors (we are missing 8).
INFO tor_dirmgr: Marked consensus usable.
WARN tor_netdir: Requested 60 relays, but only 8 were usable, ...  role=Guard
INFO arti::subcommands::proxy: Sufficiently bootstrapped; proxy now functional.
INFO tor_guardmgr::guard: We have found that guard [scrubbed] is usable.
```

Tre dettagli sono istruttivi. Il bootstrap segue passo per passo la sezione 12.7: prima il consenso, poi i certificati delle tre autorità, infine gli otto microdescrittori. L'avviso di `tor_netdir` ci ricorda che la rete del libro è minuscola: il gestore dei guard vorrebbe campionare 60 candidati, e ne trova 8. E il guard compare nel log come `[scrubbed]`: Arti marca i dati sensibili con la crate `safelog` e non li scrive nei log, l'equivalente di `SafeLogging` di tor. In altri log di aggiornamento può comparire `Applying a consensus diff` (non mostrato nel listato): Arti può scaricare soltanto le differenze, quando disponibili e negoziate mediante `DirCache=2`; non è garantito per ogni consenso.

Passo 4: usare. Arti offre un proxy SOCKS sulla porta 9150, e `curl` lo usa come usava `test008c`:

```
$ curl -s -o /dev/null -w '%{http_code} in %{time_total}s\n' \
       --noproxy "" --socks5-hostname 127.0.0.1:9150 http://127.0.0.1:8080/
200 in 0.047224s
```

L'opzione `--noproxy` con argomento vuoto evita che `NO_PROXY` escluda l'indirizzo loopback dal proxy. Il circuito attraversa i relay in C della rete privata: un client in Rust e relay in C parlano lo stesso protocollo, byte per byte, che abbiamo studiato nella Parte II. Un'ultima osservazione, da confrontare con il Laboratorio 18: anche una richiesta SOCKS con il solo indirizzo IP (`--socks5`) funziona, e nel log di Arti, al livello predefinito, non compare un avviso come quello di tor; la documentazione della crate `arti-client` mette però in guardia, nelle sue interfacce, contro la risoluzione locale dei nomi, che potrebbe rivelare la destinazione al resolver DNS, a seconda della configurazione. Ricevere una richiesta SOCKS con un indirizzo IP non dimostra, da solo, una fuga DNS (sezione 28.3).

Il laboratorio fa parte di `make verifica-lab` quando Arti è compilato in `labs/it/vendor/arti`; altrimenti la prova viene saltata e segnalata.

Riepilogo

- Arti è la riscrittura di tor in Rust: più sicura per la memoria, modulare, pensata come libreria. È stabile come client dal 2022 e oggi è il client preferito per le nuove funzioni.
- L'architettura segue gli strati del protocollo, dalle primitive alle celle, ai canali e circuiti, alla gestione di directory, percorsi e guard.
- I sottoprotocolli descrivono con precisione che cosa supporta: il client ha ntor-v3, CGO, controllo della congestione e famiglie basate su chiave; mancano ancora Conflux e le macchine di padding; il lato relay è in costruzione.
- Arti si configura per una rete privata indicando autorità e cache di directory, come fa Chutney.

Esercizi

Esercizio 30.1 ⋆ Per ciascun capitolo della Parte II, indica la crate di Arti che ne implementa il contenuto principale.

Esercizio 30.2 ⋆ Quali vantaggi e quali svantaggi ha la scelta di Arti di non implementare nessuna versione obsoleta del protocollo? Chi potrebbe esserne escluso?

Esercizio 30.3 ⋆⋆ Leggi `decode_cell` in `crates/tor-cell/src/chancell/codec.rs` e confrontala con la funzione `decodifica` del Laboratorio 2: come viene riconosciuta una cella a lunghezza variabile, e che cosa succede se i dati ricevuti non bastano a completare una cella?

Esercizio 30.4 ⋆⋆ Ripeti il Laboratorio 11 (isolamento degli stream) usando Arti come proxy SOCKS: le credenziali SOCKS diverse impediscono la condivisione dei circuiti in Arti? Distingui l'isolamento dei flussi dalla garanzia, inesistente, di un circuito unico e immutabile per ogni coppia di credenziali; cerca come viene configurato.

Esercizio 30.5 ⋆⋆⋆ Scrivi un piccolo programma Rust che usi la crate `arti-client` per scaricare una pagina dal server della rete di laboratorio, con la configurazione del Laboratorio 20 caricata da file.

Per approfondire

- Il sito del progetto, `https://arti.torproject.org/`, e il repository, con il README, l'*Architectural overview* (`doc/dev/Architecture.md`) e la guida di compatibilità (`doc/Compatibility.md`).

- Il CHANGELOG di Arti, che a ogni versione mensile descrive le novità.
- La configurazione di Arti generata da Chutney (`lib/chutney/arti/config.py`), da cui prende spunto il Laboratorio 20.

Capitolo 31

Problemi aperti e progetti in corso

Aggiornato al 23 settembre 2026 — proposte torspec fino alla 369; tor 0.4.9.13; Arti 2.6.0

Obiettivi del capitolo

Al termine di questo capitolo saprai:

- seguire il processo con cui il protocollo di Tor cambia, e leggere lo stato di una proposta;
- spiegare perché la crittografia post-quantistica richiede di cambiare il formato stesso dei messaggi;
- descrivere Conflux, Walking Onions e gli altri progetti che affrontano prestazioni e scalabilità;
- individuare i modi concreti in cui contribuire al progetto.

Nessun capitolo di questo libro descrive qualcosa di definitivo. Il protocollo che abbiamo studiato nel 2026 è diverso da quello del 2016 (niente ntor-v3, niente controllo della congestione per circuito introdotto dalla Proposta 324, niente CGO), e sarà diverso da quello del 2036. Il controllo di flusso più antico, con finestre e messaggi `RELAY_SENDME`, era già presente nel 2016: non va confuso con il successivo algoritmo di controllo della congestione. Quest'ultimo capitolo guarda avanti, organizzato per problemi: per ciascuno, che cosa non funziona abbastanza bene, che cosa è stato proposto, a che punto è il lavoro. Tutto ciò che segue è, per definizione, provvisorio: lo stato delle proposte è quello indicato nel repository delle specifiche alla data del capitolo.

31.1 Come cambia il protocollo

Dal 2007 le modifiche al protocollo passano per le **proposte**, numerate come le RFC dell'IETF. Chi ha un'idea la discute sulla lista degli sviluppatori, la scrive nel formato previsto e, se c'è un consenso di massima che meriti attenzione, gli editori la aggiungono al repository con un numero. La proposta viene discussa e migliorata finché è abbastanza dettagliata da essere implementata; poi viene implementata, e il suo contenuto viene riportato nelle specifiche. Una proposta, a differenza di una RFC, può cambiare nel tempo mantenendo lo stesso numero; e non è mai il documento definitivo di una funzione implementata: quel ruolo spetta sempre alle specifiche (Proposta 001).

Lo stato di una proposta (fig. 31.1) è la prima cosa da guardare leggendola. Nel libro abbiamo incontrato proposte in ogni stato: ntor-v3 (Proposta 332) è *Closed*, cioè implementata e riportata nelle specifiche, mentre il controllo della congestione (Proposta 324) e Conflux (Proposta 329) sono *Finished*, implementati ma non ancora riversati; CGO (Proposta 359) è

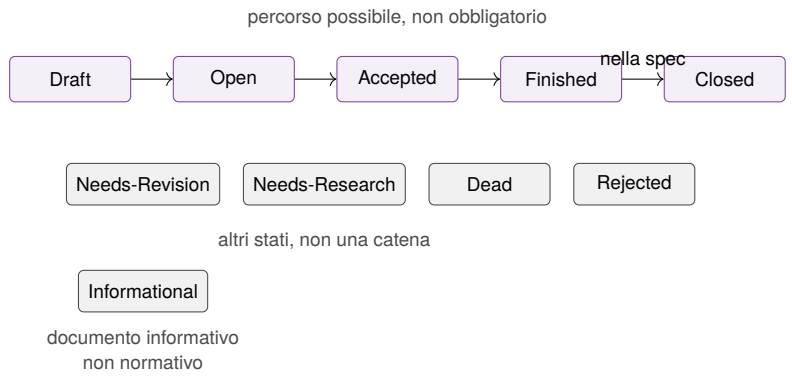


Figura 31.1: Stati delle proposte Tor secondo la Proposal 001 (schema semplificato e non esaustivo). Il percorso *Draft–Open–Accepted–Finished–Closed* è indicativo, non obbligatorio; *Finished* significa implementata e *Closed* anche integrata nelle specifiche. *Needs-Revision* e *Needs-Research* individuano blocchi diversi, *Dead* è potenzialmente riapribile, *Rejected* indica la rinuncia alla proposta descritta e *Informational* è un documento informativo che non diventa di per sé specifica. Altri stati, come *Meta*, *Reserve* e *Obsolete*, non sono rappresentati.

Open, pur essendo già nel codice di tor e di Arti; il piano di rimozione di TAP (Proposta 350) è *Accepted*; Walking Onions (Proposta 300) è *Informational*.

31.2 Il computer quantistico

Il problema. Tutte le chiavi pubbliche di Tor, X25519 negli handshake, Ed25519 nelle firme, RSA dove sopravvive, sarebbero rotte da un computer quantistico sufficientemente grande (capitolo 5). Nessuno sa quando, e se, un computer del genere esisterà. Ma per un sistema di anonimato il rischio inizia prima: un avversario può registrare oggi il traffico e decifrarlo domani.

Due scenari. La Proposta 355, del 2025, distingue due situazioni. In un ambiente di *transizione* basta resistere a un computer quantistico *futuro*: il traffico registrato oggi non deve essere decifrabile domani, mentre le firme tradizionali vanno ancora bene, perché un attacco attivo richiederebbe il computer quantistico adesso. In un ambiente di *nuova generazione* bisogna resistere a un computer quantistico *presente*, capace di falsificare le firme e di attaccare attivamente l'handshake.

Il costo. Nel 2024 il NIST ha standardizzato ML-KEM, un meccanismo post-quantistico per concordare chiavi, e le firme ML-DSA e SLH-DSA [NIS24]. La proposta ne confronta le dimensioni con quelle attuali:

Algoritmo	Chiave pubblica	Testo cifrato o firma
X25519	32 byte	32 byte
ML-KEM-768	1 184 byte	1 088 byte
Ed25519	32 byte	64 byte
ML-DSA-65	1 952 byte	3 309 byte

Nel capitolo 8 abbiamo visto che un messaggio relay porta al più 498 byte. Una sola chiave pubblica ML-KEM-768 ne richiederebbe tre celle, prima ancora di aggiungere il resto

dell'handshake. Ecco perché la crittografia post-quantistica non è solo una questione di algoritmi: richiede di cambiare il rapporto tra messaggi e celle. Nel formato relay corrente un messaggio così grande non può essere inviato come tale: le tre celle sono soltanto un limite inferiore di capacità, ipotizzando la frammentazione prevista da un nuovo formato.

Lo stato. Le conclusioni della proposta sono prudenti. Per la transizione, un handshake che aggiunge un meccanismo post-quantistico a ntor, come quelli già proposti anni fa, può bastare. Per la nuova generazione, la variante preferita usa due meccanismi di scambio di chiavi post-quantistici per ottenere anche la forward secrecy, se il costo è sostenibile. E ogni variante che aggiunga una chiave post-quantistica ai microdescrittori annullerebbe in parte il risparmio ottenuto rimuovendo le chiavi TAP (Proposta 350) e introducendo le famiglie con chiave (sezione 22.5). Per decidere servono dati che ancora mancano: quanto spesso si costruiscono circuiti da zero, quanta banda e quanta CPU dei relay vanno agli handshake. È un problema di ricerca aperto, e una buona occasione per chi vuole contribuire con misure (capitolo 29).

31.3 Messaggi più grandi, e più piccoli

La Proposta 340 (*Open*) affronta proprio il vincolo di un messaggio per cella. Propone di separare i messaggi relay dalle celle: un messaggio lungo può essere spezzato su più celle, e più messaggi brevi possono stare nella stessa cella. Le motivazioni sono due: i messaggi più lunghi di 498 byte, che servono agli handshake post-quantistici, al trasporto di UDP e a Walking Onions; e i messaggi molto brevi, come `SENDME`, `XON` e `XOFF`, che oggi occupano una cella intera e che si potrebbero nascondere nello spazio di riempimento di altri messaggi, invisibili a un osservatore. Il nuovo formato è pensato insieme alla nuova cifratura dei relay, di cui abbiamo visto la forma in CGO (sezione 10.4): per non moltiplicare i formati incompatibili, conviene cambiarli insieme.

31.4 Prestazioni: più percorsi

Il controllo della congestione (sezione 11.3) ha reso ogni circuito più efficiente. Il passo successivo è **Conflux** (Proposta 329, *Finished*): un client costruisce due circuiti verso lo stesso exit, con guard e intermedi diversi, li collega e distribuisce il traffico tra i due in base al tempo di andata e ritorno misurato su ciascuno. L'idea viene da un sistema di ricerca di AlSabah, Bauer, Elahi e Goldberg; la versione di Tor la adatta usando le informazioni del controllo della congestione e l'esperienza del TCP multipercorso. Abbiamo visto circuiti Conflux in azione nella rete di laboratorio (sezione 19.7.3) e con Stem (capitolo 18). La proposta raccomanda che le due gambe non condividano relay, a parte l'exit, così che non abbiano colli di bottiglia in comune; è una raccomandazione, non un obbligo, e nella rete di laboratorio, che ha pochi relay, si vede spesso violata. tor li usa dalla versione 0.4.8; Arti, come abbiamo visto nella sezione 30.3, non ancora. I punti di misura di OnionPerf con e senza Conflux permettono di confrontarne gli effetti sulla rete reale (sezione 24.3).

31.5 Scalabilità: Walking Onions

Ogni client di Tor conosce *tutti* i relay: scarica il consenso e i microdescrittori, e il traffico di directory cresce con il numero di relay. Microdescrittori e differenze tra consensi hanno ridotto il costo, ma non ne hanno cambiato la forma. E il costo è concentrato all'avvio: un client nuovo deve scaricare un intero elenco prima di costruire il primo circuito (Proposta 300).

Le soluzioni ovvie hanno difetti noti. Far conoscere a ogni client solo una parte della rete lo espone ad attacchi *epistemici*: se i client hanno viste diverse, le differenze possono essere usate per distinguerli. Far scegliere a ogni relay il successivo, come in alcuni sistemi decentralizzati, espone alla *cattura del percorso* da parte di relay ostili. Walking Onions propone un'altra strada: le autorità pubblicano la rete in una struttura firmata in cui ogni relay è associato a un intervallo di numeri; il client sceglie un numero a caso, e il relay precedente nel circuito gli fornisce, insieme all'estensione, la prova firmata di chi occupa quel numero. Il client verifica la firma, ma non ha mai bisogno dell'elenco completo. La proposta è informativa, e una sua versione dettagliata (Proposta 323) cerca di colmarne le lacune; le proposte più recenti, compresa quella post-quantistica, ne tengono conto.

31.6 Altri fronti aperti

UDP Tor trasporta solo TCP. Videochiamate, giochi, e il protocollo QUIC su cui si basa HTTP/3 ne restano esclusi, o costringono a scelte pericolose. La Proposta 348 (*Open*) analizza le possibilità, dai tunnel verso un exit al trasporto nativo dei datagrammi, e i rischi: nuove impronte delle applicazioni, raccolta di metadati tra pari, esaurimento delle porte.

Fingerprinting La sezione 26.4 si chiudeva con un problema aperto: nessuna difesa generale contro il website fingerprinting ha ancora un costo accettabile. Le macchine di padding, in tor per i circuiti degli onion service e in preparazione in Arti con la libreria *maybenot*, sono l'infrastruttura su cui sperimentare difese future.

Correlazione Il problema di fondo del capitolo 26 resta aperto: una rete a bassa latenza può conservare schemi temporali e di volume correlabili, pur modificati dal buffering e dal traffico concorrente. Le difese note (padding costante, ritardi) costano troppo; la ricerca continua su compromessi migliori e su selezioni dei percorsi che tengano conto della topologia di Internet.

DoS La prova di lavoro protegge gli onion service (capitolo 15); la sua regolazione automatica è oggetto di una proposta di aggiornamento (Proposta 362).

Arti come relay Quando Arti potrà funzionare da relay e da autorità, il codice in C potrà un giorno essere dismesso: è il progetto di più lungo respiro del Tor Project (capitolo 30).

31.7 Come contribuire

Chi è arrivato fin qui ha le competenze per contribuire in molti modi, non solo scrivendo codice.

- **Offrire banda**: un relay intermedio, un bridge o, con le cautele del capitolo 25, un exit (capitolo 22); oppure un proxy Snowflake, che si può eseguire anche da un browser.
- **Leggere e discutere le proposte**: molte sono *Open* proprio perché servono commenti, analisi e misure. La lista degli sviluppatori è pubblica.
- **Scrivere codice**: tor e Arti sono sviluppati sul GitLab del Tor Project, con problemi etichettati per chi inizia. Arti, in particolare, è il luogo in cui nascono le nuove funzioni.
- **Fare ricerca**: con Shadow, Chutney e i dati di Metrics, seguendo i principi del Research Safety Board (capitolo 29).

- **Tradurre e documentare**: Tor Browser, il manuale e il sito sono tradotti da volontari, e la documentazione in italiano, come dimostra questo libro, ha ancora molto spazio.
- **Usare Tor**: una comunità più ampia può contribuire all'insieme di anonimato, ma il solo numero degli utenti non garantisce indistinguibilità, soprattutto se le loro attività sono distinguibili per tempi, comportamenti o metadati.

Riepilogo

- Il protocollo cambia attraverso proposte numerate, che passano da *Draft* a *Closed* e vengono poi riportate nelle specifiche.
- La crittografia post-quantistica è necessaria anche prima che esista un computer quantistico, e richiede messaggi più grandi di una cella: è il motivo della proposta 340 sui messaggi frammentati e impacchettati.
- Conflux divide il traffico tra due circuiti; Walking Onions permetterebbe ai client di non conoscere tutta la rete.
- Restano aperti UDP, difese contro fingerprinting e correlazione, regolazione delle difese DoS, e la transizione verso Arti anche lato relay.

Esercizi

Esercizio 31.1 ⋆ Scegli tre proposte citate nel libro e indica per ciascuna lo stato attuale e che cosa significa: la funzione è già nella rete? È descritta nelle specifiche?

Esercizio 31.2 ⋆ Quante celle servono per trasportare una chiave pubblica ML-KEM-768 e un testo cifrato ML-KEM-768, se ogni messaggio relay porta al più 498 byte e non si possono unire messaggi diversi nella stessa cella? E con la Proposta 340? Distingui il limite inferiore teorico di capacità (senza overhead di frammentazione) dal numero effettivo di celle, che dipenderebbe dall'incapsulamento previsto dalla proposta.

Esercizio 31.3 ⋆⋆ Spiega con un esempio un attacco epistemico: se ogni client conoscesse una parte diversa della rete, come potrebbe un avversario sfruttarlo?

Esercizio 31.4 ⋆⋆ Con Stem, nella rete di laboratorio, individua le coppie di circuiti Conflux (stesso `CONFLUX_ID`) e controlla due cose: che le due gambe terminino sullo stesso exit e che non condividano altri relay, come raccomanda la Proposta 329. Che cosa osservi? Come lo spieghi?

Esercizio 31.5 ⋆⋆⋆ Scegli un problema aperto di questo capitolo e scrivi una breve proposta nel formato del Tor Project: motivazione, progetto, analisi di sicurezza, compatibilità, piano di implementazione.

Per approfondire

- L'indice delle proposte nel repository `torspec`, ordinato per stato, e il documento sul processo (Proposta 001).
- N. Mathewson, *Options for postquantum circuit extension handshakes* (Proposta 355).

- *Walking Onions: Scaling and Saving Bandwidth* (Proposta 300) e il suo seguito (Proposta 323).
- Il blog del Tor Project, dove le novità del protocollo vengono annunciate e spiegate.

Appendice A

Glossario italiano–inglese

Questo glossario raccoglie i termini tecnici usati nel libro, con il corrispondente inglese delle specifiche e il capitolo in cui sono introdotti. Nel libro si è tradotto quando esiste un termine italiano consolidato e si è mantenuto l'inglese negli altri casi; per i termini inglesi usati così come sono, la seconda colonna ripete il termine.

Termine	Inglese	Significato	Cap.
analisi del traffico	traffic analysis	ricavare informazioni da tempi, volumi e indirizzi delle comunicazioni, senza leggerne il contenuto	1
anonimato	anonymity	non essere identificabili all'interno di un insieme di soggetti, l'insieme di anonimato	1
attacco Sybil	Sybil attack	presentarsi alla rete con molte identità, per esempio molti relay, per essere scelti più spesso	26
autorità di directory	directory authority	uno dei pochi server che votano e firmano il consenso	12
bridge	bridge	relay non elencato nel consenso, usato come primo salto da chi è sottoposto a censura	16
canale	channel	connessione TLS tra due nodi adiacenti, su cui viaggiano le celle di molti circuiti	7
cella	cell	unità sul canale: 514 byte per le celle fisse con link protocol moderno (512 in quello precedente); lunghezza variabile per alcuni comandi	8
chiave accecata	blinded key	chiave derivata per ciascun periodo temporale (normalmente di 24 ore) dalla chiave d'identità di un onion service, che impedisce agli HSDir di ricavare direttamente l'indirizzo del servizio	14
chiave di famiglia	family key	chiave condivisa dai relay di uno stesso operatore, che ne certifica l'appartenenza alla famiglia	22
CircID	circuit ID	identificativo di un circuito su un singolo canale; cambia a ogni salto	8
circuito	circuit	percorso cifrato a strati attraverso più relay, costruito un salto alla volta	9
consenso	consensus	documento firmato dalle autorità che elenca i relay, i loro flag e i pesi	12

Termine	Inglese	Significato	Cap.
controllo della congestione	congestion control	regolazione della quantità di dati in transito su un circuito in base al tempo di andata e ritorno	11
controllo di flusso	flow control	meccanismo che impedisce a un estremo di inviare più dati di quanti l'altro possa ricevere (finestre, SENDME)	11
correlazione del traffico	traffic correlation	riconoscere che due flussi osservati in punti diversi appartengono alla stessa comunicazione	26
descrittore	descriptor	documento con cui un relay, o un onion service, pubblica chiavi, indirizzi e caratteristiche	12
exit	exit relay	ultimo relay di un circuito, che apre la connessione verso la destinazione	6
famiglia	family	insieme di relay gestiti dallo stesso operatore, che non devono comparire nello stesso circuito	13
flag	flag	etichetta assegnata dalle autorità a un relay nel consenso (`Guard`, `Exit`, `Stable`…)	12
forward secrecy	forward secrecy	proprietà per cui la compromissione di una chiave a lungo termine non permette di decifrare il traffico passato	5
guard	guard, entry guard	primo relay dei circuiti di un client, scelto una volta e usato per mesi	13
handshake	handshake	scambio di messaggi con cui due parti concordano chiavi (ntor, ntor-v3)	9
HSDir	hidden service directory	relay che conserva e distribuisce i descrittori degli onion service	14
intermedio	middle relay	relay in posizione centrale di un circuito, che non conosce né il client né la destinazione	6
isolamento degli stream	stream isolation	assegnazione di stream diversi a circuiti diversi, per esempio in base alle credenziali SOCKS	18
link protocol	link protocol	versione del protocollo usato su un canale, negoziata con la cella **`VERSIONS`**	7
messaggio relay	relay message	contenuto di una cella **`RELAY`**, destinato a un salto del circuito (`BEGIN`, `DATA`…)	8
microdescrittore	microdescriptor	versione ridotta del descrittore, con le sole informazioni che servono al client	12
modello di minaccia	threat model	descrizione esplicita di contro chi e contro che cosa un sistema vuole proteggere	4
non collegabilità	unlinkability	impossibilità, per un osservatore, di stabilire se due azioni appartengono allo stesso soggetto	1
onion service	onion service (un tempo hidden service)	servizio raggiungibile attraverso Tor, con indirizzo `.onion` v3 che codifica chiave pubblica Ed25519, checksum e versione	14
padding	padding	dati di riempimento, nelle celle o sul canale, che nascondono lunghezze e tempi	7
pesi di banda	bandwidth weights	coefficienti del consenso che bilanciano la scelta dei relay nelle diverse posizioni	13
pluggable transport	pluggable transport (PT)	programma che trasforma il traffico di Tor perché non sia riconoscibile o bloccabile (obfs4, Snowflake, WebTunnel)	16

Termine	Inglese	Significato	Cap.
politica di uscita	exit policy	regole con cui un exit indica quali destinazioni accetta	22
prova di lavoro	proof of work (PoW)	problema computazionale che un client risolve per ottenere priorità presso un onion service sotto attacco	15
proposta	proposal	documento numerato con cui si discute una modifica al protocollo	31
pseudonimo	pseudonym	identificativo diverso dal nome reale, che permette di collegare più azioni senza rivelare l'identità	1
punto di introduzione	introduction point	relay presso cui un onion service accetta le richieste di contatto	14
punto di rendezvous	rendezvous point	relay scelto dal client dove il suo circuito e quello dell'onion service si uniscono	14
relay	relay, onion router	nodo della rete Tor che inoltra celle	6
SENDME	SENDME	messaggio con cui un estremo del circuito conferma di aver ricevuto dati, riaprendo la finestra	11
sottoprotocollo	subprotocol	famiglia di funzioni con versioni proprie, dichiarate da relay e client (`Link`, `Relay`…)	30
stream	stream	flusso di dati di un'applicazione, per esempio una connessione TCP, trasportato su un circuito	11
vanguard	vanguard	relay intermedi fissati per un certo tempo, che proteggono il guard di un onion service	13
website fingerprinting	website fingerprinting	riconoscere la pagina visitata dalla sola sequenza di dimensioni e tempi del traffico cifrato	26

Appendice B

Complementi di crittografia

Aggiornato al 23 settembre 2026 — dir-spec e tor-spec (torspec commit `8e1f245`); libreria `cryptography`; rete di laboratorio

Il capitolo 5 ha presentato gli strumenti crittografici di Tor con il minimo di teoria. Questa appendice approfondisce tre punti che nel testo sono stati solo accennati, sempre con esempi che si possono eseguire: le chiavi X25519 e i loro bit fissati, la conversione tra due rappresentazioni birazionalmente equivalenti della curva 25519, verificata su un certificato vero, e la malleabilità dei cifrari a flusso, da cui dipende la marcatura della sezione 10.3.

B.1 Una curva, due forme

X25519 ed Ed25519 si basano su due modelli *birazionalmente equivalenti* di curva ellittica, sullo stesso campo degli interi modulo $p = 2^{255} - 19$. La RFC 7748 descrive la corrispondenza tra le due forme:

Forma di Montgomery $v^2 = u^3 + 486662\,u^2 + u$. È quella di X25519: una chiave pubblica è la sola coordinata u di un punto, 32 byte in ordine little-endian. La forma si presta a un algoritmo di moltiplicazione (la «scala di Montgomery») semplice, veloce e con tempo di esecuzione indipendente dalla chiave.

Forma di Edwards ritorta $-x^2 + y^2 = 1 + d\,x^2y^2$, con $d = -121665/121666$. È quella di Ed25519, adatta alle firme: una chiave pubblica è la coordinata y, con il segno di x nel bit più alto dell'ultimo byte.

Le due forme sono legate da una trasformazione esplicita, che il paragrafo B.2 usa.

B.1.1 I bit fissati delle chiavi X25519

Una chiave privata X25519 è un numero di 32 byte, ma prima di usarlo la libreria ne fissa alcuni bit: azzera i tre bit meno significativi, azzera il bit più alto e porta a 1 il penultimo. Azzerare i tre bit bassi rende la chiave un multiplo di 8, il *cofattore* della curva, così che il risultato non riveli nulla su eventuali componenti di ordine piccolo del punto ricevuto; fissare il bit alto rende costante il numero di passi della scala di Montgomery. Si può verificare con la libreria usata nei laboratori:

```
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
from cryptography.hazmat.primitives import serialization as s
import os
def pubblica(k):
```

```
    return X25519PrivateKey.from_private_bytes(k).public_key().public_bytes(
        s.Encoding.Raw, s.PublicFormat.Raw)
a = bytearray(os.urandom(32)); b = bytearray(a)
b[0] ^= 0x07                    # cambiano solo i tre bit bassi
print(pubblica(bytes(a)) == pubblica(bytes(b)))      # True
```

Due chiavi private che differiscono solo nei bit fissati producono la stessa chiave pubblica.

B.1.2 Il punto all'infinito

Nell'handshake ntor il relay e il client controllano che nessuna delle operazioni EXP() produca il punto all'infinito: la specifica osserva che, per curve25519, questo controllo sostituisce adeguatamente la verifica che Y appartenga al gruppo [tor-spec § The "ntor" handshake]. Un avversario che inviasse come chiave pubblica un punto di ordine piccolo otterrebbe altrimenti un segreto condiviso prevedibile, per esempio tutto zero. La libreria `cryptography` esegue il controllo da sola: `exchange` con una chiave pubblica di 32 byte nulli solleva un'eccezione (`Error computing shared key`) invece di restituire zero.

B.2 Da Montgomery a Edwards: il cross-certificato ntor

Ogni relay pubblica nel descrittore la propria chiave ntor, una chiave X25519, e dimostra di possederne la chiave privata con il campo `ntor-onion-key-crosscert`: un certificato Ed25519, di tipo `0a`, che certifica l'identità Ed25519 principale del relay ed è *firmato con la chiave ntor* [dir-spec § ntor-onion-key-crosscert]. Ma la chiave ntor è una chiave di Montgomery, e una firma Ed25519 richiede una chiave di Edwards. Serve la conversione descritta nella specifica [dir-spec § Converting a curve25519 public key to an ed25519 public key]:

$$y = \frac{u - 1}{u + 1} \bmod p.$$

La coordinata u determina y, ma non il segno di x: a ogni u corrispondono due chiavi di Edwards, simmetriche. Il segno non è ricavabile dalla chiave pubblica di Montgomery, e per questo il descrittore lo riporta esplicitamente, come primo argomento del campo (0 per positivo, 1 per negativo); il relay lo calcola dalla propria chiave privata.

Il modulo `labs/it/codice/torlab/conversione.py` implementa la conversione in poche righe. Il frammento seguente illustra la formula: il codice eseguibile aggiunge controlli sulla lunghezza della chiave, sul bit di segno e sul denominatore della divisione modulare:

```
P = 2**255 - 19
def montgomery_a_edwards(u_bytes, segno):
    u = int.from_bytes(u_bytes, "little") & ((1 << 255) - 1)
    y = (u - 1) * pow(u + 1, P - 2, P) % P      # divisione: inverso modulo p
    compressa = bytearray(y.to_bytes(32, "little"))
    compressa[31] |= (segno & 1) << 7          # segno di x nel bit alto
    return bytes(compressa)
```

e la usa per verificare il cross-certificato con il decodificatore di certificati del Laboratorio 1. Applicato ai descrittori della rete di laboratorio (chiave abbreviata per leggibilità; le chiavi cambiano a ogni nuova rete):

```
$ PYTHONPATH=labs/it/codice python3 -m torlab.conversione labs/it/laboratorio/net/
    nodes/000a/cached-descriptors test004r
test004r: ntor-onion-key DaxPE+Grnjex2Hmv+5qutl5dIX4La2jF6MIL7IK6+3E
  chiave Edwards (segno 0): c369d08f8083002a15fe22f65dca14bf...
  crosscert valido; identità certificata: kjPIgDB2OFsGJzuwsT+KMS/El9qrlE2/4P5WvIvmzC8
```

Nell'esempio storico la firma risulta valida e l'identità certificata coincide con la chiave `master-key-ed25519` indicata nel descrittore. Lo script controlla esplicitamente questa corrispondenza e rifiuta un certificato valido riferito a un'identità differente: verificare la sola firma del cross-certificato non basterebbe a collegarlo al relay descritto. Il file locale `fingerprint-ed25519` può essere confrontato separatamente, ma non viene letto dallo script. Con il segno sbagliato la verifica fallisce: è uno dei test automatici del modulo. La divisione modulo p si calcola con il piccolo teorema di Fermat, come moltiplicazione per $(u+1)^{p-2}$.

B.3 La malleabilità dei cifrari a flusso

AES in modalità contatore (sezione 5.2) produce un flusso di byte che viene combinato con il messaggio con uno XOR. Chi modifica il testo cifrato modifica *esattamente* gli stessi bit del testo in chiaro, senza conoscere la chiave: basta che conosca, o indovini, il contenuto originale. Un esempio in Python:

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os
k, n = os.urandom(16), bytes(16)
cifra = lambda d: Cipher(algorithms.AES(k), modes.CTR(n)).encryptor().update(d)
c = bytearray(cifra(b"BEGIN www.example.org:443"))
delta = bytes(x ^ y for x, y in zip(b"www.example.org", b"www.example.net"))
for i, d in enumerate(delta):
    c[6 + i] ^= d                # modifica mirata, senza la chiave
print(cifra(bytes(c)))           # b'BEGIN www.example.net:443'
```

In Tor la cifratura a strati tor1 usa proprio AES-CTR, e l'integrità è verificata solo all'estremo del circuito, con un digest di 4 byte. Un relay ostile può quindi alterare una cella in modo prevedibile e lasciare che un complice più avanti riconosca l'alterazione: è la marcatura discussa nella sezione 10.3, e una delle ragioni per cui CGO (sezione 10.4) usa una costruzione in cui qualunque modifica rende l'intera cella, e le successive, irrecuperabili.

Esercizi

Esercizio B.1 ⋆ Quanti valori distinti può assumere una chiave privata X25519 dopo che i suoi bit sono stati fissati? Confronta il risultato con 2^{256}.

Esercizio B.2 ⋆⋆ Ricava e implementa la trasformazione inversa, dalla coordinata y di una chiave pubblica Ed25519 alla coordinata u di Montgomery. Conserva il bit di segno di x della chiave originaria e usa `montgomery_a_edwards` di `conversione.py` per verificare che le due conversioni restituiscano la chiave Ed25519 iniziale (per $y \neq 1$).

Esercizio B.3 ⋆⋆⋆ Nel formato tor1, il digest di una cella è di 4 byte e il campo `recognized` di 2 byte nulli. Con che probabilità una cella alterata a caso viene accettata come valida dall'ultimo salto? Perché, nonostante questo, la marcatura resta possibile?

Appendice C

Guida alle specifiche torspec

Aggiornato al 23 settembre 2026 — torspec commit `8e1f245`

Le specifiche del Tor Project sono la fonte primaria di quasi ogni capitolo di questo libro. Sono complete, ma non sono un testo didattico: nate come file di testo separati, scritti e modificati nel corso di vent'anni, sono state riunite in un unico sito solo di recente, e le stesse convenzioni avvertono che la loro uniformità è un lavoro in corso. Questa appendice è una mappa per chi vuole leggerle dopo aver letto il libro.

C.1 Dove si trovano

Le specifiche sono pubblicate su `https://spec.torproject.org/` e mantenute nel repository `torspec` del GitLab del Tor Project, insieme alle proposte. Nel repository, i testi sono file Markdown nella cartella `spec/`, uno per argomento o una cartella per ciascun documento più ampio; la pagina `permalinks.md` elenca gli indirizzi stabili di ciascun documento (per esempio `https://spec.torproject.org/tor-spec`), che conviene usare nelle citazioni. Nel libro le specifiche sono citate nella forma [documento § sezione], con il titolo della sezione così come compare nel sito, e le proposte come «Proposta NNN».

C.2 Le convenzioni

Il documento introduttivo sulle convenzioni stabilisce alcune regole che valgono, salvo eccezioni, per tutti i testi:

- le parole `MUST`, `SHOULD`, `MAY` e le loro negazioni hanno il significato della RFC 2119: un requisito, una raccomandazione forte, una possibilità;
- le lunghezze sono in byte di 8 bit, e i prefissi binari («kibibyte») indicano multipli di 1024;
- gli interi di più byte sono codificati in ordine big-endian, «di rete»: 4660 su due byte è `12 34`.

Due eccezioni importanti all'ultima regola le abbiamo incontrate nel libro: le chiavi e i punti di curve25519 ed Ed25519 sono little-endian (appendice B), e alcuni formati di directory sono testuali, con i dati binari in base64.

C.3 Mappa dei documenti

Documento	Contenuto	Capitoli
`tor-spec`	il protocollo principale: canali, celle, circuiti, handshake, cifratura dei relay, stream, controllo di flusso e della congestione	7–11
`cert-spec`	i certificati Ed25519 di Tor	7
`dir-spec`	autorità, voti, consenso, descrittori, microdescrittori, extra-info	12, 24
`path-spec`	selezione dei percorsi e vincoli	13
`guard-spec`	algoritmo dei guard	13
`vanguards-spec`	vanguard e vanguard leggeri	13
`rend-spec`	onion service v3: chiavi, descrittori, introduzione, rendezvous	14, 21
`hspow-spec`	prova di lavoro per gli onion service	15
`dos-spec`	difese contro la negazione del servizio	15
`pt-spec`	interfaccia tra tor e i pluggable transports	16
`padding-spec`	padding di canale e di circuito	7, 26
`control-spec`	protocollo della ControlPort	18
`socks-extensions`	estensioni di Tor al protocollo SOCKS	11, 28
`param-spec`	parametri di rete del consenso e loro valori predefiniti	11, 15
`address-spec`	nomi speciali, come gli indirizzi `.onion`	14
`srv-spec`	valore casuale condiviso delle autorità	14

C.4 Come leggere una sezione

Qualche consiglio, maturato scrivendo il libro:

1. **Partire dal formato.** Le sezioni sui messaggi binari descrivono i campi in tabelle con nome e lunghezza: conviene ricostruirle prima su carta, come nelle figure dei formati del libro, e poi leggere le regole.
2. **Distinguere attuale e obsoleto.** Molti documenti conservano parti storiche (TAP, link protocol 1–3, onion service v2). Le parti obsolete sono spesso raccolte in sezioni o file dedicati, come `obsolete-channels.md` o la cartella `historical/`, ma non sempre: nel dubbio, controllare nel codice se il ramo esiste ancora.
3. **Confrontare con il codice.** Le specifiche descrivono l'intento; il codice di tor e di Arti mostra il comportamento. Più volte, nel libro, la differenza è stata istruttiva: il padding di canale che non compare sui circuiti di un solo salto (capitolo 7), il motivo di chiusura osservato sull'exit (sezione 22.4).
4. **Controllare lo stato delle proposte.** Una proposta può essere già implementata ma non ancora riportata nella specifica, o viceversa discussa ma non implementata (sezione 31.1).
5. **Verificare sul vivo.** Una rete privata come quella del libro permette di controllare in pochi minuti se un campo si comporta come descritto: è il metodo che ha guidato tutti i laboratori.

C.5 Arti e le specifiche

Il codice di Arti cita spesso la specifica in commenti e documentazione, e segnala i punti in cui la specifica è ambigua o in cui il comportamento di tor differisce. La tabella dei sottoprotocolli supportati (sezione 30.3) è un buon indice di quali parti della specifica sono implementate da entrambe le implementazioni, e quindi di quali parti descrivono il protocollo «vivo».

Appendice D

Soluzioni degli esercizi

Capitolo 1: Anonimato, sorveglianza e metadati

Soluzione 1.1. Con HTTPS il fornitore vede l'indirizzo IP del sito, il nome del sito (dalla risoluzione DNS e, spesso, dal campo SNI di TLS), gli orari, la durata e i volumi del traffico. Con una VPN vede solo la connessione verso il server della VPN, che però vede tutto ciò che prima vedeva il fornitore. Con Tor vede solo una connessione verso un relay Tor (o un bridge), con i suoi volumi e tempi; nessun singolo punto della rete Tor vede insieme l'utente e il sito.

Soluzione 1.2. Account con nome di fantasia: pseudonimia. Messaggio non firmato in una bacheca: anonimato (se nient'altro identifica l'autore). Lettera firmata con le iniziali: pseudonimia debole, spesso identificazione. Donazione in contanti: anonimato.

Soluzione 1.3. L'insieme di anonimato si restringe: dai 50 utenti Tor a quelli che parlano italiano e sono svegli alle 3 di notte, forse pochissimi. Tor nasconde l'indirizzo IP, ma non il comportamento: lingua, orari, stile di scrittura e informazioni personali possono identificare l'utente anche quando la rete non lo fa.

Soluzione 1.4. Il gestore di una VPN può associare l'utente alle destinazioni raggiunte, pur senza leggere il contenuto protetto da HTTPS. Con Tor la conoscenza è distribuita: normalmente il guard conosce l'origine e l'exit la destinazione, mentre nessun singolo relay conosce entrambe. La selezione del percorso evita, di norma, relay della stessa famiglia riconosciuta e applica vincoli di diversità degli indirizzi di rete, ma non può garantire che gli operatori siano davvero indipendenti. Un osservatore presente a entrambi gli estremi può tentare la correlazione dei flussi [path-spec § Path selection and constraints].

Capitolo 2: Le radici

Soluzione 2.1. A caso, 1 su 100. Se 99 messaggi del lotto sono dell'osservatore, questi sa riconoscere i propri in uscita, e l'unico rimanente è quello della vittima: l'anonimato scompare. È l'attacco detto $n - 1$, che le mix reali contrastano con lotti più complessi, messaggi fittizi e controlli sugli ingressi.

Soluzione 2.2. Serve un *indirizzo di ritorno anonimo* alla Chaum: l'utente prepara in anticipo una «cipolla di ritorno» cifrata a strati per una catena di mix, che contiene il proprio indirizzo nello strato più interno e una chiave per cifrare la risposta. Chi risponde allega il messaggio alla cipolla; ogni mix toglie uno strato, e nessuna, da sola, conosce l'indirizzo finale. Non serve alcuna tabella centrale.

Soluzione 2.3. A ogni passo la richiesta prosegue verso un altro membro con probabilità $3/4$ e va al sito con probabilità $1/4$: il numero di membri attraversati segue una distribuzione geometrica con media $1/(1/4) = 4$ (contando il primo inoltro). Un membro non può sapere se chi gli passa la richiesta ne è l'autore, perché con la stessa regola anche un inoltro intermedio arriva da un membro qualsiasi della folla.

Capitolo 3: Dall'onion routing del NRL al Tor Project

Soluzione 3.1. Con tre salti, un nodo centrale ostile conosceva subito le due reti agli estremi, perché i nodi di ingresso e di uscita erano nelle reti delle organizzazioni; con quattro, chi comprometteva il nodo vicino a un estremo sapeva quale altro nodo attaccare per ricostruire il circuito. Con cinque serviva un attacco in più. Tor si accontenta di tre salti perché i relay non stanno nelle reti degli utenti, e perché ogni salto in più aumenta la latenza senza ridurre molto il rischio principale, la correlazione tra primo e ultimo salto, contro cui agiscono invece i guard.

Soluzione 3.2. Con un'unica cipolla, cifrata con le chiavi pubbliche a lungo termine dei nodi, chi ottiene in futuro quelle chiavi può decifrare le cipolle registrate: niente forward secrecy. Con la costruzione telescopica ogni salto usa uno scambio effimero, cancellato dopo l'uso. Inoltre il client costruisce il circuito un salto alla volta e sa subito quale nodo non risponde, potendo sceglierne un altro, mentre con la cipolla unica il fallimento si scopre solo alla fine.

Soluzione 3.3. Esercizio di argomentazione. Elementi verificabili: codice e specifiche pubbliche, compilazioni riproducibili, analisi indipendenti della letteratura scientifica, rete gestita da operatori indipendenti. Elementi che richiedono fiducia: che le versioni distribuite corrispondano al codice (in parte verificabile), che le directory authority non colludano, che non esistano vulnerabilità sconosciute. La sicurezza non dipende dalle intenzioni di chi finanzia, ma dalla verificabilità e dalla distribuzione della fiducia.

Capitolo 4: Il modello di minaccia

Soluzione 4.1. (a) Sì: il gestore vede solo una connessione verso un relay Tor. (b) Sì: il sito vede l'indirizzo dell'exit. (c) No, per l'identità: l'utente si identifica da sé con l'account; Tor nasconde solo da dove si collega. (d) Probabilmente no: un avversario che osserva entrambi gli estremi, utente e giornale, può tentare la correlazione dei tempi, ed è proprio il caso escluso dal modello.

Soluzione 4.2. Se un sistema è difficile, lo usano solo poche persone molto motivate, spesso proprio quelle con più bisogno di protezione: il solo fatto di usarlo le distingue, e l'insieme di anonimato è piccolo. Un sistema facile, usato da milioni di persone per ragioni ordinarie, nasconde anche chi ne ha un bisogno particolare.

Soluzione 4.3. Esercizio aperto. Avversari possibili: il governo o il datore di lavoro della fonte, che osserva la rete locale; il servizio di posta o di messaggistica usato; chi controlla le reti di uno dei due paesi. Tor protegge dal fatto che la rete locale veda con chi comunica la giornalista, e dal servizio che veda il suo indirizzo IP; restano fuori il contenuto dei messaggi (serve cifratura da estremo a estremo), i metadati visibili al servizio (account, orari), la compromissione dei dispositivi e la correlazione da parte di chi osserva entrambi gli estremi. Un onion service, come SecureDrop, riduce molti di questi rischi.

Capitolo 5: Crittografia essenziale

Soluzione 5.1. $C_1 \oplus C_2 = P_1 \oplus P_2$, perché il flusso di chiavi si cancella. Conoscendo P_1: $P_2 = C_1 \oplus C_2 \oplus P_1$.

Soluzione 5.2. L'uscita di 16 byte coincide con i primi 16 byte di quella di 32: SHAKE produce un unico flusso che si legge per la lunghezza desiderata. Come KDF si può quindi chiedere un flusso lungo e ritagliarlo; ma chiedere lunghezze diverse per scopi diversi *non* separa gli usi, per cui servono etichette di dominio distinte.

Soluzione 5.3. I due segreti coincidono. Con una chiave pubblica di soli zeri, che rappresenta un punto di ordine piccolo, il risultato dello scambio è zero qualunque sia la chiave privata: un segreto noto a tutti. La libreria `cryptography` rifiuta di restituirlo; ntor chiede esplicitamente di controllare questo caso (sezione 9.6), altrimenti un avversario potrebbe forzare un segreto prevedibile.

Soluzione 5.4. Senza separazione, un valore calcolato in un contesto potrebbe essere accettato in un altro: per esempio un MAC calcolato per autenticare un messaggio di handshake potrebbe essere presentato come chiave o come prova in una fase diversa, o in un protocollo diverso che usa la stessa funzione e le stesse chiavi. Le etichette rendono gli input dei vari usi disgiunti per costruzione.

Capitolo 6: Architettura della rete

Soluzione 6.1. Guard: l'indirizzo IP dell'utente e i tempi e i volumi del traffico, non la destinazione. Middle: né l'utente né la destinazione. Exit: la destinazione e il contenuto non cifrato dall'applicazione, non l'utente. HSDir: nulla di un utente che visita un sito web normale. Directory authority: normalmente nulla della visita. Può vedere l'indirizzo IP di un client che le chieda documenti di directory *direttamente*, cosa che capita soprattutto in avvio; se invece il client li scarica da una fallback directory o da una cache, è quel server a vedere la richiesta, non l'autorità.

Soluzione 6.2. `Link=3-5`: canali con link protocol fino al 5 (capitolo 7); `Relay=2-6`: ntor, ntor-v3 (`Relay=4`) e CGO (`Relay=6`); `FlowCtrl=1-2`: SENDME autenticati e controllo della congestione. Come exit con CGO sì, purché abbia anche il flag `Exit` e si negozi insieme il controllo della congestione (sezione 9.7).

Soluzione 6.3. 100 KB di dati richiedono circa $102400/498 \approx 206$ celle DATA, cioè circa $206 \times 514 \approx 106$ KB sul collegamento, più poche celle per la richiesta, l'apertura dello stream e i SENDME (circa sette, uno ogni 31 celle), e l'intestazione dei record TLS: nell'insieme un 5–7 % oltre ai dati utili, esclusi gli handshake iniziali.

Capitolo 7: Canali e link protocol

Soluzione 7.1. Un client apre sempre lui le connessioni: verso il guard, verso i relay da cui scarica informazioni di directory. Nessuno ha motivo di contattarlo, e una porta in ascolto lo renderebbe anzi individuabile. Un relay apre canali come iniziatore soprattutto quando riceve un `EXTEND2` e deve raggiungere il salto successivo del circuito, e quando, come qualsiasi nodo, crea circuiti propri, per esempio per scaricare documenti di directory o per verificare la propria raggiungibilità.

Soluzione 7.2. `0x000772F7` $= 488183$ ore, cioè 9 settembre 2025, ore 23:00 UTC. Con i secondi, 4 byte senza segno si esaurirebbero nel 2106; con le ore il campo non si esaurisce prima di centinaia di migliaia di anni, e per certificati che durano giorni o settimane la precisione dell'ora è più che sufficiente. Attenzione a una nota della specifica: `cert-spec` afferma che «4 byte bastano fino al 10136», ma è un errore che la Proposta 338 segnala espressamente, nato dall'aver trattato il contatore come se contasse i *minuti* (2^{32} minuti sono infatti circa 8166 anni).

Soluzione 7.3. $38 = 32 + 2 + 2 \times 2$: due metodi. Con N_Methods uguale a 2 servono $32 + 2 + 4 = 38$ byte: una cella di 36 byte dichiara più metodi di quanti ne contenga, è malformata e va rifiutata. (La specifica chiede di ignorare i byte *in più*, non di tollerare quelli mancanti.)

Soluzione 7.4. Fallisce il confronto tra la chiave certificata dal tipo 5 e l'impronta SHA-256 del certificato TLS presentato dal server dell'avversario, che è diverso da quello del relay vero. Se l'avversario presentasse lo stesso certificato TLS, non riuscirebbe a completare l'handshake TLS, che richiede la chiave privata corrispondente al certificato.

Soluzione 7.5. `695661c0` (1º gennaio 2026, 12:00:00 UTC) · `04 04 c6336407` (l'indirizzo del client, 198.51.100.7) · `01` · `04 04 cb007105` (l'indirizzo del relay, 203.0.113.5), seguiti da zeri fino a 509 byte.

Soluzione 7.6. Primo: il guard vedrebbe la stessa identità anche quando l'utente cambia rete (casa, lavoro, telefono), e potrebbe collegare tra loro tutte le sue sessioni e i suoi indirizzi IP. Secondo: l'identità verrebbe presentata anche ai relay da cui il client scarica informazioni di directory, e relay complici, o un avversario che li osserva, potrebbero collegare le attività dello stesso utente su relay diversi. In entrambi i casi un identificatore persistente distrugge la non collegabilità che Tor vuole garantire.

Soluzione 7.7. Traccia: si decodifica il tipo 2 con una libreria X.509 (per esempio `cryptography.x509`), se ne estrae la chiave pubblica e si verifica che il certificato sia autofirmato e che l'esponente sia 65537; poi si interpreta il tipo 7 secondo il formato del cross-certificato, si controlla che la chiave Ed25519 certificata coincida con l'identità già verificata e si verifica la firma RSA (PKCS#1 v1.5 senza OID dell'hash) sul SHA-256 del prefisso più i campi. L'impronta del relay è lo SHA-1 della codifica DER (PKCS#1) della chiave pubblica RSA.

Soluzione 7.8. No. Il traffico del client `test008c` viaggia sui *suoi* canali verso il suo guard, non sul nostro. Il nostro canale non è mai stato usato per un circuito, quindi `channelpadding_decide_to_pad_channel` restituisce «non aggiungere padding». Per osservare il padding bisogna far transitare sul proprio canale un circuito di più salti, come faremo nel capitolo 10.

Soluzione 7.9. Il valore di TLSSECRETS è derivato dai segreti della sessione TLS tra A e l'intermediario, e la firma di A lo copre. B lo ricalcola dai segreti della *sua* sessione con l'intermediario, che sono diversi: la verifica fallisce anche se tutte le celle sono state inoltrate fedelmente, e con esse SLOG e CLOG coincidono. Senza TLSSECRETS (e SCERT) l'intermediario potrebbe presentarsi a B come A, inoltrando la firma di A, e B crederebbe di avere un canale autenticato con A.

Capitolo 8: Le celle

Soluzione 8.1. **`CREATE2`** è una cella fissa: 512 byte con il link protocol 3 (CircID di 2 byte) e 514 con il 5 (CircID di 4 byte). La cella **`VERSIONS`** usa sempre un CircID di 2 byte: $2+1+2+4 = 9$ byte.

Soluzione 8.2. 0 (**PADDING**): fissa. 7 (**VERSIONS**): variabile. 9 (**RELAY_EARLY**): fissa. 12 (**PADDING_NEGOTIATE**): fissa. 129 (**CERTS**): variabile. 200: sarebbe variabile per la regola ≥ 128, ma non è un comando definito e quindi non è ammesso; chi lo riceve può scartare la cella e chiudere il canale.

Soluzione 8.3. Solo `0x92345678`. Il valore 0 è riservato alle celle che non appartengono a un circuito; con il link protocol ≥ 4 chi ha aperto la connessione, qui il client, deve usare CircID con il bit più significativo a 1, e `0x12345678` ha MSB a 0.

Soluzione 8.4. Prima cella: CircID `00000000`, comando `0x81` = 129 (**CERTS**), quindi variabile; Length `0003`, corpo `aa bb cc`. Seconda cella: CircID `0x80000007`, comando `0x04` (**DESTROY**), fissa, quindi con 509 byte di corpo; il primo byte del corpo, `00`, indica il motivo NONE. (Il corpo `aabbcc` non è un CERTS valido: l'esercizio riguarda solo l'incorniciamento delle celle.)

Soluzione 8.5. `02 0000 0005 00000000 012c`: comando DATA (2), Recognized 0, StreamID 5, Digest provvisoriamente a zero, Length $300 =$ `0x012c`. Seguono $509 - 11 - 300 = 198$ byte di padding, di cui i primi 4 a zero e i restanti 194 casuali.

Soluzione 8.6. Per una cella ancora cifrata il campo Recognized vale zero con probabilità 2^{-16}: in media $10^6/65536 \approx 15$ celle. Recognized da solo non basta perché questi falsi positivi esistono; solo il confronto con il digest progressivo, che un nodo non destinatario non può far coincidere se non per caso (2^{-32} ulteriori), stabilisce che la cella è destinata al relay.

Soluzione 8.7. Un avversario controlla un guard e un exit. Quando l'exit vuole marcare un circuito, lo chiude con un motivo raro, per esempio `NOSUCHSERVICE`; se il middle e il guard propagassero il motivo verso il client, il guard vedrebbe arrivare quel valore insolito su uno dei suoi circuiti nello stesso istante, collegando così il client (noto al guard) alla destinazione (nota all'exit). Se tutti inviano e propagano solo NONE, il byte non trasporta informazione. Resta comunque la correlazione temporale della chiusura, che è un canale molto più rumoroso.

Soluzione 8.8. Formato tradizionale: $\lceil 1048576/498 \rceil = 2106$ celle, cioè $2106 \times 514 = 1\,082\,484$ byte sul canale, con un overhead del 3,2 %. CGO: $\lceil 1048576/488 \rceil = 2149$ celle, $1\,104\,586$ byte, overhead del 5,3 %. Nel conto non entrano i SENDME né i messaggi di apertura dello stream.

Soluzione 8.9. Traccia: con Hypothesis si genera una lista di tuple (CircID, comando, corpo) scegliendo il comando tra i valori di `Comando` e limitando il corpo a 509 byte per le celle fisse; si concatena l'output di `codifica`, si divide in frammenti di lunghezza casuale e li si passa uno alla volta a un buffer da cui si estrae con `decodifica` finché restituisce una cella. Le celle fisse vanno confrontate con il corpo esteso con zeri fino a 509 byte.

Soluzione 8.10. In Arti 2.6.0 il codec di canale è la struttura `ChannelCodec` nel file `crates/tor-cell/src/chancell/codec.rs`, creata con la versione del link protocol (`ChannelCodec::new`). Le due funzioni sono `write_cell`, che serializza una cella in un buffer, e `decode_cell`, che la estrae. Come la nostra `decodifica`, `decode_cell` gestisce la cella incompleta senza errore: restituisce `Ok(None)` finché i byte non bastano, e consuma dal buffer solo i byte di una cella completa. La differenza principale è che Arti lavora su un buffer mutabile (`BytesMut`) invece di restituire i byte avanzati, ed è generico sul tipo dei messaggi di canale (`M: ChanMsg`), il che permette di restringere l'insieme dei messaggi accettati. Un'altra differenza interessante: `decode_cell` legge il comando sempre all'offset 4 e assume celle fisse di 514 byte, cioè presuppone un CircID di 4 byte. Arti infatti offre solo i link protocol 4 e 5 (costante `LINK_PROTOCOLS` in `crates/tor-proto/src/channel/handshake.rs`), e la cella **VERSIONS** iniziale, con il suo CircID di 2 byte, viene gestita a parte durante l'handshake.

Capitolo 9: Costruzione dei circuiti

Soluzione 9.1. ntor: $20 + 32 + 32 = 84$ byte. ntor-v3 con 3 byte di messaggio: $32 + 32 + 32 + 3 + 32 = 131$ byte. HDATA può occupare al massimo $509 - 4 = 505$ byte, quindi CM al massimo $505 - 128 = 377$ byte.

Soluzione 9.2. Il segreto di **`CREATE_FAST`** è $X \| Y$, trasmesso in chiaro dentro il canale. Per il secondo salto X e Y transiterebbero nel circuito attraverso il primo relay, che conosce le chiavi del proprio strato e potrebbe leggerli: ricaverebbe le chiavi del secondo salto e potrebbe decifrare tutto ciò che il client scambia con esso e, per estensione, rompere la separazione tra i salti.

Soluzione 9.3. Con AES-CTR la cifratura è lo XOR con un flusso di chiavi che dipende solo da chiave e contatore. Con la stessa chiave nei due versi, la cella n inviata dal client e la cella n inviata dal relay sarebbero cifrate con lo stesso flusso: lo XOR dei due testi cifrati, osservabile da un relay intermedio, sarebbe uguale allo XOR dei due testi in chiaro, da cui è spesso possibile ricavare entrambi.

Soluzione 9.4. `test004r` ha ORPort 5104 = `0x13f0`: `03` (tre specificatori) · `00 06 7f000001 13f0`. Il messaggio completo è: $1 + (2 + 6) + (2 + 20) + (2 + 32) = 65$ byte di specificatori, più $2 + 2 = 4$ di HTYPE e HLEN, più $32 \cdot 3 + 1 + 32 = 129$ di HDATA, in tutto 198 byte, ben entro i 498 disponibili in un messaggio relay.

Soluzione 9.5. Senza EXP(B, x) il relay non è autenticato: il relay intermedio che inoltra l'handshake potrebbe rispondere lui stesso con un proprio Y e un $AUTH$ calcolato con il proprio y, fingendosi il salto successivo e leggendo poi tutto il traffico destinato ad esso (attacco *man-in-the-middle*). Senza EXP(Y, x) si perde la forward secrecy: chi ottenesse in futuro la chiave onion b potrebbe ricalcolare EXP(X, b) dagli handshake registrati e quindi tutte le chiavi di sessione.

Soluzione 9.6. `VER` lega l'handshake al suo contesto (per tor, `"circuit extend"`): entra in tutte le derivazioni, quindi un handshake calcolato per un contesto non produce chiavi né $AUTH$ validi in un altro. Senza `VER`, un avversario potrebbe prendere un messaggio di handshake scambiato nel protocollo estraneo e riproporlo a un relay Tor (o viceversa), sfruttando il fatto che le due parti accetterebbero gli stessi calcoli; si parla di attacchi *cross-protocol*.

Soluzione 9.7. Nella rete di laboratorio il relay `test003r` (sottoprotocolli `Relay=2-6`) rifiuta con **`DESTROY`**, motivo `PROTOCOL`, la sola richiesta `[02 06]`, e accetta la richiesta combinata con `CC_FIELD_REQUEST`, rispondendo con `CC_FIELD_RESPONSE`. Nel codice di tor un salto può usare CGO solo se usa anche il controllo della congestione (`enable_cgo = supports_cgo && use_congestion_control` in `src/core/or/extendinfo.c`). La Proposta 359 spiega che il nuovo formato dei messaggi, con lo StreamID solo per alcuni comandi, non è compatibile con i SENDME a livello di stream del controllo di flusso precedente: serve il controllo di flusso della Proposta 324 (`FlowCtrl=2`).

Soluzione 9.8. Esercizio di laboratorio: `ntor_ref.py` contiene le funzioni di client e server; fissando le chiavi (invece di generarle a caso) si ottengono valori riproducibili da confrontare con `ntor.py`.

Soluzione 9.9. Traccia: `server_part1` riceve HDATA, controlla `ID` e B, calcola $X_b = \text{EXP}(X, b)$, deriva le chiavi di fase 1, verifica il MAC e decifra CM; `server_part2` genera (y, Y), calcola *key_seed*, *verify*, la chiave per SM e $AUTH$. Con y fissato ai valori della proposta, la risposta deve coincidere con il vettore `server_handshake`.

Capitolo 10: La cipolla

Soluzione 10.1. Il client aggiorna il digest in avanti del secondo salto, poi cifra con Kf_2 e infine con Kf_1. Il terzo salto non vede nulla: la cella viene riconosciuta dal secondo e non prosegue.

Soluzione 10.2. Ogni cella consuma 509 byte del flusso: dopo 1000 celle il contatore è alla posizione $509 \times 1000 = 509\,000$ byte, cioè oltre il blocco AES numero $31\,812$ (blocchi di 16 byte). Riusare la chiave per un altro circuito, con il contatore che riparte da zero, farebbe riusare lo stesso flusso di chiavi: lo XOR di due celle cifrate con lo stesso flusso rivelerebbe lo XOR dei testi in chiaro. Per questo ogni circuito ha chiavi nuove, derivate da un handshake con valori effimeri.

Soluzione 10.3. Il digest del relay includerebbe una cella che il client non ha contato per quel salto: da quel momento gli stati del digest di client e relay sarebbero diversi e nessuna cella successiva verrebbe più riconosciuta, quindi il circuito verrebbe distrutto. Il problema si presenta con probabilità 2^{-16} per ogni cella in transito non destinata al relay: in media dopo $65\,536$ celle di questo tipo, circa 33 MB di traffico.

Soluzione 10.4. Conviene alterare byte il cui contenuto in chiaro è noto o prevedibile, così che l'exit possa riconoscere un'alterazione precisa: per esempio il comando relay (`02` per `DATA`), lo StreamID o i 4 byte a zero all'inizio del padding. L'exit complice, prima del controllo del digest, verifica se quei byte hanno il valore alterato atteso. Un exit onesto vede soltanto una cella con digest errato e distrugge il circuito.

Soluzione 10.5. Con CGO la cella alterata, una volta tolto lo strato dell'exit, produce un tag e un contenuto pseudocasuali: non c'è alcun marchio da leggere, perché la modifica si propaga in modo imprevedibile a tutta la cella. Poiché il tag entra nell'elaborazione della cella successiva, anche tutte le celle seguenti del circuito diventano indecifrabili e il circuito è da buttare. L'avversario può solo distruggere circuiti, cosa che può fare in ogni caso.

Soluzione 10.6. Intervalli: 4,1; 3,0; 5,7; 8,7; 6,8 secondi (media 5,7). Per il massimo di due uniformi su $[a, b]$ il valore atteso è $a + \frac{2}{3}(b-a) = 1{,}5 + \frac{2}{3} \cdot 8 \approx 6{,}8$ secondi. Con soli cinque campioni lo scarto è del tutto compatibile. Si noti anche che il timer riparte ogni volta che sul canale passa qualsiasi cella, non solo il padding.

Soluzione 10.7. Nella rete di laboratorio il middle non risponde a `DROP`: il messaggio viene riconosciuto e scartato in silenzio. La prova indiretta è che il circuito resta utilizzabile: dopo due `DROP` il client riesce ancora a estenderlo verso l'exit. Se il middle non avesse riconosciuto le celle, essendo l'ultimo salto avrebbe distrutto il circuito; se client e middle avessero contato le celle in modo diverso, i digest progressivi si sarebbero desincronizzati e l'estensione successiva sarebbe fallita.

Soluzione 10.8. Nella prova sulla rete di laboratorio, un bit alterato in una cella con `EXTEND2` diretta al guard (unico salto del circuito) fa fallire il controllo del digest: il guard, che è l'estremo del circuito, risponde con **`DESTROY`**, motivo 1 (`PROTOCOL`). Con un circuito più lungo, un'alterazione applicata dal client a una cella destinata al terzo salto verrebbe inoltrata dai primi due e rilevata solo dal terzo.

Capitolo 11: Stream, SENDME e controllo della congestione

Soluzione 11.1. Il nome va in minuscolo: `example.com:443` seguito da uno zero, cioè `6578616d706c652e636f6d3a343433 00`, poi FLAGS con il bit 1 (IPv6 accettato): `00000001`.

Soluzione 11.2. Indirizzo IPv4 `c0000201` = 192.0.2.1, TTL `0000012c` = 300 secondi. Il client può ignorarlo; se lo conserva, non deve riusarlo su altri circuiti, perché l'exit potrebbe aver mentito o aver restituito un indirizzo unico per riconoscerlo.

Soluzione 11.3. Senza optimistic data il client attende `CONNECTED` prima di inviare la richiesta: un RTT in più, circa 600 ms, su un tempo totale di almeno due RTT. Se la connessione fallisce i dati in coda vanno scartati perché non esiste una destinazione a cui consegnarli, e non devono finire su un'altra connessione.

Soluzione 11.4. Uno stream può avere in volo al massimo 500 celle, cioè circa $500 \times 498 \approx 249$ KB per RTT: con 400 ms, circa 620 KB/s. Un circuito, anche con molti stream, è limitato a 1000 celle per RTT, circa 1,2 MB/s. Con il controllo della congestione la finestra parte da sole 124 celle (circa 62 KB), ma cresce finché l'RTT non indica congestione, senza un limite fisso.

Soluzione 11.5. Un client apre molti stream verso un sito che invia molti dati (o un proprio server), legge pochissimo e invia a ripetizione `SENDME` falsi: l'exit continua a inviare, ma le celle si accumulano nelle code del guard, del middle e del client stesso, consumando memoria nei relay. Con i `SENDME` autenticati il client deve dimostrare di aver ricevuto la cella che fa scattare il `SENDME`. Se il contenuto delle celle fosse prevedibile (un file noto scaricato dall'avversario stesso), il client potrebbe provare a calcolare il digest senza ricevere nulla. Per questo la specifica raccomanda di introdurre byte casuali in almeno una cella `DATA` per incremento della finestra: il padding casuale lo rende possibile *quando resta spazio* nel corpo, mentre una cella con tutti i 498 byte occupati da dati non ha padding. Non basta quindi presumere che ogni cella contenga byte imprevedibili.

Soluzione 11.6. Traccia: si invia `RESOLVE` con uno StreamID nuovo e il nome seguito da zero; si interpreta `RESOLVED` come sequenza di (tipo, lunghezza, valore, TTL). Per un nome inesistente l'exit risponde con una voce di tipo errore (`0xF0` o `0xF1`), il cui contenuto va ignorato.

Soluzione 11.7. L'exit risponde con `END` di motivo 4 (`EXITPOLICY`), seguito dall'indirizzo di destinazione (4 byte per IPv4) e da un TTL di 4 byte. Il client usa l'informazione per non riprovare con lo stesso exit per quella destinazione.

Soluzione 11.8. Si misura il tempo che il `SENDME` impiega per arrivare all'exit, più il tempo necessario perché l'exit, sbloccato, invii nuove celle e queste arrivino al client: un'approssimazione dell'RTT del circuito vista dal client, quando l'exit sta aspettando il `SENDME`. Con un secondo download in parallelo le code si allungano e i valori crescono: è proprio il segnale che il controllo della congestione usa per ridurre la finestra.

Capitolo 12: Il sistema delle directory

Soluzione 12.1. L'intervallo di voto è di un'ora (da `valid-after` a `fresh-until`). Il client sceglie un istante a caso tra le $11{:}00 + \frac{3}{4} \cdot 60' = 11{:}45$ e $11{:}45 + \frac{7}{8} \cdot 75' \approx 12{:}50$. Se alle 13:00 non ha ancora un consenso nuovo, quello che possiede scade formalmente, ma il client continua a usarlo come *reasonably live* (scaduto da meno di 24 ore, `REASONABLY_LIVE_TIME` in `networkstatus.c`) e riesce ancora a costruire circuiti, mentre insiste per scaricarne uno più recente. Solo circa 24 ore dopo la scadenza smette di usarlo.

Soluzione 12.2. Probabilmente `Running` e `Valid`; `Fast` (500 KB/s supera i 100 KB/s); forse `V2Dir` se fa da directory cache. Non `Stable`, `Guard` né `HSDir`, che richiedono giorni o settimane di attività. Non `Exit`, perché la policy permette solo la porta 443 e il flag richiede l'uscita sia sulla 80 sia sulla 443.

Soluzione 12.3. La cache invia al client bersaglio un consenso di due ore prima, ancora valido, in cui mancano i relay aggiunti di recente: il client sceglierà percorsi tra un insieme leggermente diverso da quello degli altri, e il suo traffico potrà essere riconosciuto. L'attacco è limitato dalla validità di tre ore del consenso, dal fatto che il consenso deve comunque essere firmato dalle autorità (la cache non può inventarne uno) e dal fatto che i client scelgono le cache a caso. Chiedere i microdescrittori per impronta, anziché per identità del relay, impedisce alla cache di inviare al singolo client una versione diversa della chiave di un relay: l'impronta è fissata nel consenso firmato.

Soluzione 12.4. Con tre misure si usa la mediana: 5200. Una sola misura esagerata non sposta il risultato. Se la terza autorità fosse l'unica a misurare il relay, non ci sarebbero tre misure: il consenso userebbe la banda dichiarata o votata, con `Unmeasured=1` e un limite superiore, quindi la misura gonfiata non avrebbe effetto.

Soluzione 12.5. Il voto contiene righe che il consenso non ha o ha in forma diversa: per esempio le misure `Measured` delle autorità di banda, le righe `m` con i metodi di consenso supportati, i flag assegnati da quella sola autorità, le soglie usate (`flag-thresholds`) e gli impegni per la casualità condivisa. Nel consenso restano solo i valori combinati.

Soluzione 12.6. Il testo va dalla riga `onion-key` che apre il microdescrittore fino al ritorno a capo che precede il microdescrittore successivo (o la fine del documento), comprese le righe della chiave RSA in formato PEM. L'impronta SHA-256, in base64 senza `=` finali, deve coincidere con la riga `m`.

Soluzione 12.7. Con un'autorità ferma, le altre due continuano a votare e il consenso ha due firme: bastano, perché ne servono $\lfloor 3/2 \rfloor + 1 = 2$. Con due autorità ferme la sola rimasta non raggiunge la maggioranza: non viene prodotto un consenso nuovo. Il client, però, continua a usare l'ultimo come *reasonably live* — fino a circa 24 ore dopo la scadenza — e per un bel po' costruisce ancora circuiti: nella rete di laboratorio il consenso vale 40 secondi, ma la finestra *reasonably-live* resta di 24 ore, quindi l'interruzione non si osserva subito.

Soluzione 12.8. Il consenso `ns` contiene di norma firme SHA-1 (la specifica ammette solo quelle per questa variante). La riga `r` contiene in più l'impronta del descrittore completo del relay, e ogni voce ha anche la riga `p` con il riassunto della exit policy.

Capitolo 13: Selezione del percorso

Soluzione 13.1. L'exit è il relay più vincolato: deve accettare la destinazione richiesta. Scegliendolo per primo si garantisce che esista; scegliendo prima il guard si potrebbe escludere, per i vincoli di famiglia o di rete, proprio l'unico exit adatto, o costringere a ripetere la scelta. C'è anche una ragione statistica: le esclusioni si applicano alle posizioni scelte dopo, e conviene che ricadano su quelle con più candidati.

Soluzione 13.2. Come guard: $20000 \times 0/10000 = 0$ (`Wgd=0`). Come middle: $20000 \times 0 = 0$ (`Wmd=0`). Come exit: $20000 \times 10000/10000 = 20000$ (`Wed=10000`). Con i pesi di quel consenso il relay viene usato solo come exit.

Soluzione 13.3. Senza guard la probabilità supera il 50 % quando $1 - (1 - c^2)^C > 0{,}5$, cioè $C > \ln 0{,}5 / \ln(1 - c^2)$: circa 6932 circuiti per $c = 1\,\%$ e 17 circuiti per $c = 20\,\%$. Con il guard fisso la probabilità resta intorno a c: 1 % e 20 %. Con un avversario molto forte i guard non bastano, ma la differenza resta enorme.

Soluzione 13.4. Deve fornire banda reale: la probabilità dipende dalla banda misurata dalle autorità, non da quella dichiarata, e l'uscita è la posizione che attira più controlli (`BadExit`).

Il limite di due relay per indirizzo lo costringe a usare molti indirizzi; se dichiara la famiglia, i suoi relay non compaiono mai due volte nello stesso circuito; se non la dichiara e viene scoperto, le autorità possono escluderli. I pesi, infine, distribuiscono l'uso degli exit solo in uscita, dove la sua banda conta davvero.

Soluzione 13.5. Senza limite, un avversario che controlla la rete locale del client potrebbe bloccare la connessione a ogni guard onesto: il client continuerebbe ad aggiungere guard al campione finché non ne trova uno raggiungibile, cioè, prima o poi, uno dell'avversario. Con un campione limitato il client esaurisce i tentativi e smette di aggiungere guard, restando senza connessione piuttosto che finire su un guard ostile.

Soluzione 13.6. Esercizio di laboratorio. I microdescrittori si trovano su CollecTor nella stessa area dei consensi; conviene costruire, per ogni relay, l'insieme delle famiglie (identificativi `family-ids` o liste reciproche `family`) e rifiutare un'estrazione se interseca quello di un relay già scelto.

Soluzione 13.7. Esercizio di ricerca. In genere le famiglie più grandi concentrano una quota di probabilità di uscita molto maggiore dei singoli relay più grandi: il rischio reale dipende dagli operatori, non dai relay. È una delle ragioni dell'attenzione della comunità per la diversità degli operatori e per le famiglie non dichiarate (capitolo 27).

Capitolo 14: Onion services v3

Soluzione 14.1. $32+2+1 = 35$ byte, cioè 280 bit; il base32 codifica 5 bit per carattere: $280/5 = 56$ caratteri, senza caratteri di riempimento. Un errore casuale produce, con probabilità circa 2^{-16}, un checksum che per caso coincide; inoltre il byte di versione deve restare `0x03`, il che rende ancora più improbabile che un errore passi inosservato.

Soluzione 14.2. Alle 09:00 del 1° gennaio 2026 sono trascorsi $29\,454\,300$ minuti dall'epoca Unix; tolti i 720 minuti di sfasamento restano $29\,453\,580$, e $\lfloor 29453580/1440 \rfloor = 20453$. Il periodo $20\,453$ è iniziato alle 12:00 UTC del 31 dicembre 2025, perché alle 09:00 non è ancora mezzogiorno.

Soluzione 14.3. Deve conoscere la chiave accecata del periodo (quindi l'indirizzo del servizio), l'SRV del periodo e le posizioni sull'anello dei relay che controlla, per generare identità di relay che cadano subito dopo gli indici del servizio. L'SRV si conosce solo quando le autorità lo pubblicano, circa mezza giornata prima dell'inizio del periodo in cui viene usato: l'avversario non può preparare i relay giusti con giorni di anticipo, e i nuovi relay devono comunque ottenere il flag `HSDir`, che richiede almeno 96 ore di attività.

Soluzione 14.4. L'HSDir vede una chiave accecata, non l'indirizzo, e riceve richieste da circuiti anonimi. Il punto di introduzione vede una chiave di autenticazione e un circuito che arriva dal servizio, e conta le richieste di introduzione. Il punto di rendezvous vede un cookie, due circuiti e il traffico cifrato che scorre tra loro. Il punto di introduzione è il più interessante per stimare la popolarità: vede ogni tentativo di connessione, anche se non sa a quale indirizzo corrisponda (lo potrebbe scoprire, però, chi conosce già l'indirizzo e scarica il descrittore).

Soluzione 14.5. Su un circuito di un salto l'HSDir è il primo relay, e vede l'indirizzo IP del client: potrebbe collegare *chi* cerca *quale* servizio (almeno a livello di chiave accecata, e quindi dell'indirizzo, se lo conosce). Rifiutando le richieste non anonime, l'HSDir riceve solo richieste che passano da altri relay.

Soluzione 14.6. Esercizio di laboratorio: con più HSDir di 2×4 il servizio non carica più il descrittore ovunque; scaricandolo da ogni HSDir con un circuito di tre salti, lo si deve trovare

esattamente sugli HSDir calcolati per il periodo corrente o per quello successivo, dato che il servizio pubblica entrambi.

Soluzione 14.7. Il valore base64 contiene un numero di specificatori seguito dagli specificatori nel formato di `EXTEND2` (sezione 9.2.2): per esempio il tipo `00` con indirizzo e ORPort, il tipo `02` con l'impronta RSA e il tipo `03` con l'identità Ed25519. Dall'ORPort (5100 + numero del nodo) si risale al relay. La riga `auth-key` contiene un certificato di tipo 9, firmato dalla chiave di firma del descrittore, che certifica la chiave di autenticazione del servizio presso quel punto di introduzione.

Capitolo 15: Difese contro DoS

Soluzione 15.1. Nel primo minuto: i 60 gettoni del secchio pieno più $2 \times 60 = 120$ ricaricati, circa 180 circuiti (qualcuno in meno, perché il client viene marcato quando il secchio arriva a zero). In un'ora, a regime: circa $60 + 2 \times 3600 = 7260$. Un client normale ne crea poche decine.

Soluzione 15.2. Solo il primo relay vede l'indirizzo IP reale del client: più avanti, l'«indirizzo» del canale è quello del relay precedente. Un middle che limitasse i circuiti per indirizzo limiterebbe in realtà i circuiti di un guard, che trasporta il traffico di migliaia di utenti: colpirebbe tutti, non l'attaccante. Per questo i limiti riguardano solo le connessioni dei client, non autenticate.

Soluzione 15.3. Il client invia un messaggio `INTRODUCE1` su un circuito che può riusare. Il servizio deve decifrarlo, eseguire l'handshake hs-ntor e costruire un nuovo circuito di almeno tre salti verso il punto di rendezvous, con altrettanti handshake ntor per i relay coinvolti, più le celle di creazione ed estensione in ogni salto. Una sola cella del client genera così diverse operazioni a chiave pubblica e decine di celle nella rete: l'amplificazione è di uno o due ordini di grandezza.

Soluzione 15.4. Con k bit a zero si possono chiedere solo sforzi che sono potenze di due (1, 2, 4, ...): tra uno sforzo e il successivo il costo raddoppia. Con il test lineare si può chiedere qualsiasi intero E. Un controllo a retroazione che aggiusta lo sforzo per piccoli passi, come quello del servizio, ha bisogno di una scala fine: con i raddoppi oscillerebbe tra valori troppo bassi e troppo alti.

Soluzione 15.5. No. Lo sforzo E fa parte della sfida su cui si calcolano sia la soluzione Equi-X sia l'hash R: dichiarando $E = 5000$ la sfida cambia, e la soluzione trovata per $E = 50$ non è più valida. Il client avrebbe dovuto scegliere $E = 5000$ prima di cercare.

Soluzione 15.6. Esercizio di laboratorio. Le celle si accumulano nella coda del circuito sul relay intermedio finché il totale supera `MaxMemInQueues`; a quel punto il gestore della memoria chiude i circuiti con i dati in coda più vecchi, tra cui il nostro, e registra nel log quanta memoria ha liberato.

Soluzione 15.7. Esercizio di programmazione. Ci si aspetta che le richieste a sforzo alto abbiano tempi di attesa brevi e stabili, e che quelle a sforzo zero vengano servite solo quando la coda è quasi vuota, o scartate durante gli svuotamenti: è l'effetto voluto, che protegge gli utenti disposti a pagare con un po' di calcolo.

Capitolo 16: Censura, bridge e pluggable transports

Soluzione 16.1. Un bridge svolge il ruolo del guard, primo salto del circuito, ma non compare nel consenso e il client ne conosce l'indirizzo da una bridge line ottenuta in privato.

Un client con `UseBridges` usa i bridge al posto dei guard: aggiungere un guard pubblico significherebbe collegarsi direttamente a un indirizzo pubblicato, cioè proprio ciò che il censore blocca o sorveglia.

Soluzione 16.2. «Non assomigliare a niente»: obfs4 (il censore dovrebbe bloccare tutto il traffico non riconoscibile). «Assomigliare a qualcosa di permesso»: meek (bloccare un grande servizio di distribuzione di contenuti), Snowflake (bloccare WebRTC e il broker, e molti proxy di volontari), WebTunnel (bloccare HTTPS verso siti sconosciuti o i siti che ospitano i bridge), Conjure (bloccare ampi intervalli di indirizzi presso i fornitori che collaborano).

Soluzione 16.3. Un censore potrebbe notare connessioni lunghe verso indirizzi senza un nome di dominio associato, con byte ad alta entropia fin dal primo pacchetto, dimensioni e tempi dei pacchetti con distribuzioni tipiche (per esempio raffiche legate alle celle di Tor), e rapporti tra traffico in ingresso e in uscita simili alla navigazione web. `iat-mode` aggiunge ritardi e frammentazioni casuali che rendono meno riconoscibili dimensioni e tempi, a costo di prestazioni.

Soluzione 16.4. Il censore blocca i bridge che riceve, e un bridge bloccato impedisce di salire di livello: le sue credenziali restano ai livelli bassi, senza possibilità di invitare. Per aggirare il meccanismo potrebbe astenersi dal bloccare i bridge ricevuti finché non ha raggiunto livelli alti, e poi bloccarli tutti insieme: ma dovrebbe mantenere per mesi molte identità pazienti, e intanto quei bridge servirebbero gli utenti; inoltre ogni credenziale di livello alto dà accesso a pochi bridge.

Soluzione 16.5. Entrambe hanno entropia vicina a 8 bit per byte, perché anche TLS, dopo l'handshake, trasporta dati cifrati. La differenza sta in pochi byte: le intestazioni dei record TLS (`16 03 01`, `17 03 03` seguiti dalla lunghezza) e l'handshake in chiaro all'inizio della connessione, che un DPI riconosce senza bisogno di misure statistiche.

Soluzione 16.6. Nella cattura di riferimento il Laboratorio 1 scade dopo il tempo massimo (`TimeoutError`): nessun handshake Tor viene stabilito e non arriva una risposta utile prima della scadenza. Con un `cert` sbagliato, `obfs4proxy` lato client non completa l'handshake e il bridge può chiudere la connessione dopo un ritardo; nei log compaiono al più messaggi di errore, secondo il livello di registrazione. **Il timeout di un singolo client non garantisce silenzio perpetuo, né rende impossibile il rilevamento statistico o il blocco di un bridge già noto.**

Capitolo 17: Installazione e torrc

Soluzione 17.1. tor segnala l'opzione sconosciuta con un messaggio del tipo `Unknown option '...'. Failing.` e termina senza avviarsi. `--dump-config short` sul `torrc` del client mostra solo le opzioni diverse dai valori predefiniti: per la rete di laboratorio, tra le altre, `TestingTorNetwork`, le `DirAuthority` e le porte.

Soluzione 17.2. Se posta e forum condividono un circuito, l'exit vede nello stesso intervallo di tempo, dallo stesso circuito, il collegamento al server di posta dell'utente (spesso con il suo nome) e quello al forum pseudonimo, e può collegare lo pseudonimo alla persona. Soluzioni: due porte SOCKS distinte, una per applicazione (le porte diverse sono isolate per impostazione predefinita, salvo assegnare loro lo stesso `SessionGroup`), oppure credenziali SOCKS diverse per le due applicazioni con `IsolateSOCKSAuth`, o ancora `IsolateDestAddr` sulla porta usata da entrambe. **Isolare i circuiti non garantisce exit differenti**: due circuiti indipendenti possono selezionare lo stesso relay d'uscita. Se la posta identifica l'utente a

livello applicativo, anche l'isolamento dei circuiti non nasconde quell'identità al server di posta; occorre separare anche gli account e il comportamento.

Soluzione 17.3. Un avversario che osserva le reti italiane vedrebbe sia l'utente entrare nella rete Tor sia l'exit italiano collegarsi al sito italiano: entrambi gli estremi nello stesso territorio, cioè proprio il caso di correlazione escluso dal modello di minaccia. Inoltre gli exit italiani sono pochi, e restringere la scelta rende l'utente riconoscibile e più facile da colpire con relay malevoli.

Capitolo 18: Control port e Stem

Soluzione 18.1. `552` è un errore permanente in risposta a un comando: la chiave `pippo` non esiste. Un `650` non risponde a un comando: è un evento asincrono, inviato perché il client lo ha chiesto con `SETEVENTS`, e può arrivare in qualsiasi momento, anche mentre si attende la risposta a un altro comando.

Soluzione 18.2. Traccia: il metodo `create_ephemeral_hidden_service` del controllore, con la corrispondenza tra la porta virtuale 80 e la porta locale 8080 e l'opzione `await_publication`, restituisce l'indirizzo del servizio; con `curl --socks5-hostname 127.0.0.1:9008 http://<indirizzo>.onion/` lo si raggiunge. Le chiavi di un servizio effimero sono tenute in memoria da tor e, per impostazione predefinita, la chiave privata può essere restituita al controllore, che deve proteggerla come un segreto e può salvarla solo se intende riusare quell'identità. Alla chiusura della connessione di controllo il servizio scompare, salvo che sia stato creato con l'opzione `detach` (o proprietà equivalente). Un onion service temporaneo non rende automaticamente temporanei i log o gli account dell'applicazione che vi risponde.

Soluzione 18.3. Traccia: registrare una funzione per `EventType.BW`, che riceve ogni secondo i byte letti e scritti (`evento.read`, `evento.written`), e stamparli; durante il download il valore letto cresce fino al limite imposto dal controllo della congestione e dalle risorse della macchina.

Soluzione 18.4. Con `__LeaveStreamsUnattached` tor non assegna automaticamente i nuovi stream ai circuiti; `EXTENDCIRCUIT 0 test004r,test005r,test006e` richiede il circuito e restituisce il relativo identificativo, ma bisogna **attenderne lo stato `BUILT`** prima di usarlo. All'arrivo dell'evento `STREAM` con stato `NEW` si chiama `ATTACHSTREAM` con lo stream e il circuito; se il controller non gestisce lo stream, la connessione può restare sospesa. È un esercizio confinato alla rete privata: scegliere manualmente i relay aggira le protezioni della selezione pesata e dei guard, e può rendere l'utente riconoscibile o favorire relay controllati da un avversario.

Capitolo 19: Una rete Tor privata

Soluzione 19.1. `test006e` è il nodo numero 6: ORPort 5106, ControlPort 8006. `test009h` non ha ORPort: un onion service non è un relay e non accetta connessioni da altri nodi; si collega lui stesso ai punti di introduzione e di rendezvous tramite circuiti in uscita.

Soluzione 19.2. L'opzione `TestingDirAuthVoteGuard *` nel torrc delle autorità. Nella rete reale il flag Guard richiede stabilità e banda misurate nel tempo (capitolo 12). In laboratorio, quindi, qualsiasi relay può essere scelto come guard: le statistiche sulla scelta del primo salto non rappresentano la rete reale.

Soluzione 19.3. Con `--socks5` è `curl` a risolvere `localhost` localmente e a comunicare a Tor un indirizzo IP; con `--socks5-hostname` passa il nome, che Tor fa risolvere sul lato d'uscita

(quando la destinazione e la policy lo permettono). **Questo specifico test con `localhost` non dimostra di per sé una fuga DNS**: il nome è normalmente risolto dal file `/etc/hosts`, senza interrogare un resolver esterno, e il risultato può cambiare tra IPv4 e IPv6. Il rischio reale riguarda invece *domini non locali* risolti preventivamente dall'applicazione mediante il DNS del sistema: tali interrogazioni possono rivelare le destinazioni fuori da Tor.

Soluzione 19.4. Ogni 20 secondi, come stabilito da `V3AuthVotingInterval`. In un giorno: $86400/20 = 4320$ consensi in laboratorio contro 24 nella rete reale, dove l'intervallo è di un'ora.

Soluzione 19.5. In una rete di test che richiede una maggioranza di tre autorità, fermarne una lascia la possibilità di ottenere due firme, ma **non garantisce** che tutti i voti e le pubblicazioni si completino immediatamente: dipendono dal calendario dei voti e dalla disponibilità delle autorità. Fermandone due, la sola rimasta non raggiunge la maggioranza e non si può formare un nuovo consenso accettabile. I client, però, continuano a usare l'ultimo come *reasonably live* fino a circa 24 ore dopo la scadenza (capitolo 12), quindi costruiscono ancora circuiti a lungo: nella rete di laboratorio il consenso vale 40 secondi, ma la finestra *reasonably-live* resta di 24 ore.

Soluzione 19.6. Traccia: `c.get_network_status(impronta)` restituisce una voce del consenso con i campi `address` e `or_port`; basta chiamarla per ogni impronta in `circ.path` (il primo elemento di ogni coppia).

Soluzione 19.7. Il file ha la stessa struttura di quello della rete del libro, con `getN(5)`, `getN(10)` e `getN(4)`. Il tempo di bootstrap dipende dalla macchina; con più nodi cresce soprattutto la fase in cui i relay devono comparire nel consenso e i client scaricare i microdescrittori.

Soluzione 19.8. Esercizio di esplorazione: i messaggi da cercare riguardano la scelta del percorso, l'invio di **`CREATE2`**/`EXTEND2` e la ricezione delle risposte, fino al completamento del circuito. Il livello *info* non mostra ogni cella; per un dettaglio maggiore si può attivare il log *debug* nel torrc del client.

Soluzione 19.9. No: dopo l'handshake TLS le celle viaggiano cifrate e `tcpdump` vede solo record TLS. Restano osservabili gli indirizzi, le porte, le dimensioni dei record e i tempi: proprio le informazioni su cui si basano gli attacchi di analisi del traffico del capitolo 26.

Capitolo 20: Osservare il traffico

Soluzione 20.1. TLS non allinea i record ai confini delle celle: tor scrive le celle in un buffer e la libreria TLS può raggrupparle in record il cui testo in chiaro ha un massimo ordinario di 2^{14} byte, a cui TLS aggiunge intestazioni e protezione crittografica. TCP può inoltre spezzare o riunire i dati dei record in segmenti diversi. La dimensione fissa di 514 byte riguarda *una cella Tor dopo il protocollo di collegamento v4*, non la dimensione dei record TLS né quella dei segmenti TCP; il parametro `sendme_inc` del controllo della congestione non va dedotto dalle dimensioni dei record.

Soluzione 20.2. Traccia: dopo **`CREATED2`**, le celle **`RELAY_EARLY`** e **`RELAY`** del circuito si decifrano togliendo, nell'ordine, gli strati con le chiavi dei salti di `circuito.py` (attenzione: i flussi AES-CTR vanno fatti avanzare nello stesso ordine in cui sono stati usati). Durante la costruzione si vedono `EXTEND2` ed `EXTENDED2`, poi eventuali `BEGIN`, `CONNECTED`, `DATA`, `SENDME`.

Soluzione 20.3. Su una connessione client–guard *abilitata al padding di collegamento* possono apparire record anche nei periodi di inattività: Tor campiona un timer di padding tipicamente compreso fra 1,5 e 9,5 secondi, secondo i parametri del consenso; il traffico effettivo reimposta il timer. Non tutti i canali ricevono padding: il protocollo riguarda soprattutto il collegamento

applicativo del client al guard o bridge, e il client può negoziare una riduzione o la disattivazione. Inoltre keepalive e attività di directory possono produrre traffico indipendente dal padding. Non è quindi garantito né un numero fisso di record né il silenzio su ogni canale senza circuiti applicativi [padding-spec § Connection-level padding].

Soluzione 20.4. Esercizio di programmazione: un analizzatore Lua registrato come *heuristic dissector* sui dati TLS decifrati può leggere i primi 2 byte (prima di **`VERSIONS`**) o 4 byte di CircID, il byte di comando e, per le celle variabili, la lunghezza; va tenuto uno stato per connessione e per verso, come in `traffico.py`.

Capitolo 21: Pubblicare onion service

Soluzione 21.1. $32^6 \approx 1{,}07 \times 10^9$ tentativi, circa 107 secondi a 10 milioni al secondo; $32^8 \approx 1{,}1 \times 10^{12}$, circa $1{,}1 \times 10^5$ secondi, cioè poco più di 30 ore. Ogni carattere in più moltiplica il tempo per 32.

Soluzione 21.2. Rivela il software e la versione del server (informazioni utili per cercare vulnerabilità) e un indirizzo che potrebbe svelare l'origine dell'onion service *se fosse realmente associato al server*. Nell'esercizio `192.0.2.10` è però **solo un indirizzo di documentazione** della rete IANA `192.0.2.0/24` (TEST-NET-1): non identifica un server pubblico reale. Il rischio illustrato riguarda l'eventuale divulgazione di un vero indirizzo dell'host. Si riduce eliminando dalle risposte le informazioni superflue e vincolando il server a localhost o a un socket Unix, con controlli applicativi e di rete.

Soluzione 21.3. Si rimuove la credenziale di Alice dal servizio e si riavvia il processo tor, seguendo le istruzioni ufficiali sulla revoca: **non dare per scontato che il solo segnale HUP basti**. La nuova configurazione deve caricare almeno una chiave valida di un *altro* client, altrimenti l'autorizzazione alla scoperta risulta **disattivata** e chi conosce l'indirizzo può accedere al servizio (sezione 21.3). Tor 0.4.9.13 considera abilitata l'autorizzazione solo quando la lista dei client contiene almeno una chiave valida. Se Alice era l'unica autorizzata, prima di eliminarla occorre predisporre e verificare un'altra credenziale oppure sospendere il servizio; **lasciare vuota la cartella non costituisce una revoca sicura**. I descrittori già scaricati e le connessioni già stabilite possono conservare validità per un periodo: per una revoca tempestiva è necessaria una difesa anche a livello applicativo.

Soluzione 21.4. Si usa `HiddenServicePort 80 unix:/percorso/socket` e si configura il server web per ascoltare su quel socket. Oltre a non aprire porte TCP, il socket si protegge con i permessi del file system, non è raggiungibile da altri processi di rete e rende impossibile, anche per errore, esporre il servizio su un'interfaccia pubblica.

Capitolo 22: Gestire un relay o un bridge

Soluzione 22.1. Sì: la prima regola che corrisponde è `accept *:80`, e la successiva non viene mai considerata. Basta invertire l'ordine: `reject 192.0.2.0/24:*`, `accept *:80`, `reject *:*`.

Soluzione 22.2. Il riassunto contiene solo porte, non indirizzi, ed è un'approssimazione: una porta è elencata come accettata se l'exit la accetta per la «maggior parte» degli indirizzi, che nella dir-spec è un criterio tecnico preciso, non un semplice $>50\,\%$: la porta è accettata se l'exit non ne blocca più di 2^{25} indirizzi IPv4 (un /7) o 2^{112} IPv6 (un /16), ignorando gli indirizzi non pubblici. Ricevuta `END` con motivo `EXITPOLICY`, il client annota, per il TTL indicato, che quell'exit rifiuta quell'indirizzo e porta, e ritenta lo stream su un altro circuito con un altro exit.

Soluzione 22.3. Con `MyFamily` la famiglia vale solo per le dichiarazioni reciproche: i cinque relay vecchi restano una famiglia tra loro, ma il sesto no, perché nessuno lo elenca; i client possono quindi usarlo nello stesso circuito di uno degli altri. Con la chiave di famiglia basta installare la chiave sul nuovo relay: il certificato che firma lo lega alla famiglia senza toccare gli altri.

Soluzione 22.4. Nei log compaiono in sequenza «in-progress connect failed …(Connection refused)» e «Didn't recognize errno 115». Se il motivo fosse calcolato sull'errore restituito da `getsockopt(SO_ERROR)`, il client riceverebbe `CONNECTREFUSED`; allo stesso modo sarebbero riportati correttamente `TIMEOUT` (`ETIMEDOUT`), `NOROUTE` (`ENETUNREACH`, `EHOSTUNREACH`) e `CONNRESET`, che oggi, per le connessioni che falliscono dopo `EINPROGRESS`, diventano tutti `MISC`. Per il client la conseguenza è modesta: cambia il messaggio d'errore, non il comportamento.

Soluzione 22.5. A 10 MB/s il relay trasferisce circa 864 GB al giorno, contati come il massimo tra traffico inviato e ricevuto: 1 TB si esaurisce in poco più di un giorno. Un relay che resta spento per gran parte del mese perde i flag `Stable` e `Guard` e fa cadere i circuiti in corso quando si spegne; uno che funziona tutto il mese con banda minore è molto più utile, e per questo conviene regolare `RelayBandwidthRate` in modo che il tetto basti per l'intero periodo.

Capitolo 23: Tor Browser, Tails e Whonix

Soluzione 23.1. Il nuovo circuito cambia soltanto il circuito del sito della scheda attiva, senza toccare cookie, cronologia o gli altri siti: il sito può ancora riconoscere l'utente dallo stato che ha salvato. La nuova identità chiude tutto, cancella ogni stato e fa usare circuiti nuovi ovunque. Il guard non cambia perché cambiarlo spesso aumenterebbe la probabilità di sceglierne prima o poi uno ostile (sezione 13.3); il guard, d'altra parte, non vede i siti visitati.

Soluzione 23.2. Guadagna meno tracciatori caricati, ma Tor Browser già isola i tracciatori per sito e ne limita gli identificatori. Perde uniformità: la presenza dell'estensione e le sue liste possono essere rilevate dai siti, e distinguono l'utente dalla massa degli utenti di Tor Browser; inoltre l'estensione è codice in più con accesso a tutte le pagine.

Soluzione 23.3. Con credenziali uguali per le due schede dello stesso sito, tor assegna i loro stream allo stesso circuito; la terza scheda, con credenziali diverse, ottiene un circuito diverso. Con il comando `GETINFO stream-status` si vede l'identificativo del circuito di ciascuno stream.

Soluzione 23.4. (a) Tor Browser basta, eventualmente con un bridge. (b) Tails: si avvia da chiavetta, non lascia tracce sul computer di terzi e conserva solo ciò che si mette nella memoria persistente cifrata, fermo restando il rischio di hardware manomesso. (c) Whonix: tutte le applicazioni della Workstation passano per tor per costruzione, e il lavoro dura nel tempo senza bisogno di amnesia.

Soluzione 23.5. Esercizio aperto. Esempi: dimensioni dello schermo e della finestra, fuso orario, lingua, caratteri disponibili, rendering di un'immagine su canvas. In un browser comune alcune di queste portano da pochi bit (fuso orario) a molti (caratteri, canvas). Tor Browser arrotonda le dimensioni con il letterboxing, dichiara UTC e una lingua comune, limita i caratteri e uniforma o blocca le interfacce più rivelatrici; restano differenze legate al sistema operativo, al livello di sicurezza e alla dimensione dello schermo arrotondata.

Capitolo 24: Misurare la rete

Soluzione 24.1. Le richieste si dimezzerebbero e, con il divisore fisso a 10, anche la stima: i grafici mostrerebbero un calo improvviso e simultaneo in tutti i paesi, in coincidenza con la diffusione della nuova versione di tor. Un calo globale e graduale che segue l'adozione di una versione è il segno di un cambiamento del software, non degli utenti; il metodo andrebbe corretto cambiando il divisore.

Soluzione 24.2. $20\,000\,000/(0{,}6 \cdot 10) \approx 3\,333\,333$ utenti. Con una frazione dell'8% la stima viene scartata, perché inferiore al 10%.

Soluzione 24.3. Traccia: la quota si ottiene filtrando i relay con entrambi i flag e dividendo la loro banda per il totale; per la probabilità si usano le probabilità di `probabilita(consenso, "exit")` del Laboratorio 7, sommando quelle dei relay le cui reti /16 sono tra le cinque più frequenti.

Soluzione 24.4. Esercizio aperto. In genere gli utenti diretti crollano nel giorno del blocco, mentre quelli dei bridge, e in particolare di Snowflake o WebTunnel, crescono nei giorni successivi, man mano che gli utenti passano ai transport; l'indicatore di Metrics segnala il giorno del calo.

Soluzione 24.5. Senza arrotondamento, un avversario che sa di essere l'unico utente di un certo paese su un relay (o che conosce il numero esatto di altri utenti) potrebbe confermare dall'extra-info se un'altra persona di quel paese si è collegata: una differenza di uno nei conteggi. L'arrotondamento nasconde le piccole differenze. Un avversario che controlla molti relay, invece, non ha bisogno delle statistiche pubblicate: vede direttamente le connessioni che arrivano ai propri relay.

Capitolo 25: Aspetti legali ed etici

Soluzione 25.1. (a) Un exit che generasse di propria iniziativa richieste verso siti, per esempio per «mescolare» il traffico, darebbe origine a trasmissioni. (b) Un exit che dirottasse le connessioni verso un indirizzo diverso da quello chiesto nella cella `BEGIN` selezionerebbe il destinatario. (c) Un exit che inserisse contenuti nelle pagine HTTP, o filtrasse contenuti in base al loro significato, selezionerebbe o modificherebbe le informazioni. Sono, non a caso, comportamenti per cui le autorità assegnano il flag `BadExit`.

Soluzione 25.2. Un coltello da cucina è lecito, un'aggressione no. Un utente che legge un giornale censurato con Tor usa lecitamente uno strumento lecito; lo stesso utente che accede abusivamente a un sistema commette un reato, con o senza Tor. L'operatore di relay offre capacità di trasmissione senza scegliere i contenuti: la sua posizione è quella del trasportatore, non dell'autore, finché non partecipa alle attività illecite.

Soluzione 25.3. Si spiega che l'indirizzo è un exit della rete Tor, verificabile con gli strumenti pubblici del Tor Project; che l'operatore non conosce né registra gli utenti e non può identificarli; che il sito può, se lo ritiene, trattare in modo diverso le connessioni dagli exit, il cui elenco è pubblico. In concreto l'operatore può aggiungere alla politica di uscita un rifiuto per l'indirizzo del sito che si lamenta, se ritiene che il beneficio superi il costo.

Soluzione 25.4. La proposta osserva il traffico di utenti reali senza il loro consenso, raccoglie dati non pubblicabili (domini visitati) e ha un beneficio modesto: viola più linee guida e, in Italia, può porre problemi penali e di protezione dei dati. Alternative: usare dati aggregati già pubblici (Metrics), misurare con un proprio client quali siti di notizie sono raggiungibili

da Tor, o proporre al Board uno schema di conteggio con aggregazione e rumore, come quelli usati per le statistiche dei relay.

Soluzione 25.5. Esercizio di ricerca: vanno cercate le decisioni nelle banche dati ufficiali (per l'Italia, le raccolte della Cassazione e dei tribunali; per l'Unione, il sito della Corte di giustizia), non su riassunti di terzi. Per ciascuna si indicano il giudice, la data, la norma discussa e l'esito, distinguendo le decisioni su relay da quelle su utenti di Tor.

Capitolo 26: Correlazione, fingerprinting, Sybil

Soluzione 26.1. Correlazione: entrambi gli estremi (relay o reti); difese: guard, vincoli sui percorsi, pesi misurati. Website fingerprinting: il solo collegamento client–guard; difese: padding (limitato), ricerca in corso. Scoperta del guard: un relay intermedio e la possibilità di far costruire circuiti; difese: guard e vanguard. Sybil: molti relay; difese: banda misurata, famiglie e /16, sorveglianza. Attacchi di instradamento: un AS o un IXP; difese: esterne a Tor (monitoraggio BGP).

Soluzione 26.2. Su 100 000 visite, circa 0,95 allarmi corretti e circa $0{,}001 \cdot 99\,999 \approx 100$ falsi: la frazione corretta è $0{,}95/(0{,}95 + 100) \approx 0{,}9\%$. Quasi tutti gli allarmi sono sbagliati.

Soluzione 26.3. Con intervalli di 2 secondi e durata di 5 restano due o tre punti per serie, e la correlazione diventa casuale; con lo stesso ritmo per tutti i client (stesso seme) le serie diventano quasi identiche e l'abbinamento è indistinguibile. La modifica più dannosa è l'uniformità dei ritmi: è il principio del traffico a velocità costante di PipeNet (sezione 2.4).

Soluzione 26.4. A ritmo costante le serie d'ingresso non variano più, e la correlazione di Pearson non è nemmeno definita (varianza nulla) o è dominata dal rumore della rete: l'attacco fallisce. Il costo è inviare sempre la banda massima, anche in silenzio: nel laboratorio, da poche decine a centinaia di KB/s per client, cioè molte volte il traffico utile.

Soluzione 26.5. Con un guard fisso, l'avversario vince solo se quel guard è suo (probabilità circa 5%), e allora vede circa il 5% dei circuiti come exit: dopo sei mesi di uso intenso la probabilità di almeno una compromissione tende al 5% (nessuna altrimenti). Con un guard casuale per circuito, ogni circuito ha probabilità circa $0{,}05 \cdot 0{,}05 = 0{,}0025$, e dopo qualche migliaio di circuiti la compromissione è quasi certa. È l'argomento a favore dei guard; la classe `Avversario` di `percorso.py` permette di verificarlo per simulazione.

Capitolo 27: Casi reali contro la rete

Soluzione 27.1. Il contenuto di una cella che va verso il client viene cifrato con uno strato in più da ogni relay che attraversa (capitolo 10): il guard vede un contenuto cifrato con le chiavi dell'intermedio e del proprio salto, e non può leggere ciò che l'HSDir vi ha scritto. Il comando della cella, invece, sta nell'intestazione che ogni relay legge e riscrive uguale.

Soluzione 27.2. Con tre guard, la probabilità che almeno uno sia ostile è circa tre volte più alta, e ogni guard ostile vede una parte del traffico del client. Con un guard solo la probabilità è minore e, se il guard è onesto, il client è protetto a lungo. Lo svantaggio: se il guard unico è ostile, vede tutto il traffico del client; e se il guard è lento o irraggiungibile le prestazioni ne risentono.

Soluzione 27.3. Il laboratorio usa già celle `DROP`, non richieste di estensione: il conteggio in `command.c` riguarda il comando della cella, prima della decifratura, e quindi non dipende

dal contenuto. Per la direzione opposta, il codice registra un avviso e chiude il circuito con motivo `PROTOCOL`, sia che il circuito sia del client sia che sia di un relay.

Soluzione 27.4. Esempi: relay che entrano nel consenso nello stesso minuto; stessa versione di tor insolita e stesso sistema operativo nei descrittori; indirizzi nella stessa rete o presso lo stesso fornitore, con porte e nomi simili, contatto vuoto. Falsi positivi: grandi operatori onesti che aggiornano tutti i relay insieme, fornitori molto usati, installazioni automatiche con la stessa configurazione. Nessun criterio basta da solo: servono più segnali e, se possibile, un contatto con l'operatore.

Soluzione 27.5. In chiaro, per un relay intermedio: CircID (cambia a ogni salto, quindi non passa), comando della cella (**`RELAY`** o **`RELAY_EARLY`**: il canale del 2014, oggi chiuso nella direzione verso il client e limitato a 8 in quella opposta), la cella **`DESTROY`** e il suo motivo, i tempi e il numero delle celle (sempre disponibili: canale di conferma passiva, nessuna difesa completa). Il contenuto è cifrato a strati, ma con tor1 un relay può modificarlo in modo che un altro relay riconosca la modifica (marcatura, sezione 10.3); CGO (sezione 10.4) rende le modifiche irrecuperabili per tutti.

Capitolo 28: Errori operativi

Soluzione 28.1. Traffico fuori da Tor, documenti aperti in rete, fuga DNS: applicazione (Tor Browser, Tails o Whonix, `SafeSocks`, apertura dei file senza rete). Metadati, accesso ad account, identità mescolate: contenuto (strumenti di pulizia, sessioni separate). Stile, orari, uso di Tor in un insieme piccolo: comportamento (abitudini, bridge, attesa prima di pubblicare).

Soluzione 28.2. Se l'applicazione interroga un resolver esterno prima di usare Tor, il resolver può conoscere il dominio cercato e associarlo all'indirizzo di rete dell'utente, anche se la successiva connessione al sito attraversa il circuito. Se la query DNS viaggia in chiaro, anche il fornitore di connettività e chi osserva quel collegamento possono leggerne il nome; con DNS cifrato, invece, un osservatore del solo collegamento non può normalmente leggerlo, pur vedendo il contatto con il resolver. Un nome già in cache o un indirizzo IP inserito direttamente non implica alcuna query DNS e non dimostra una fuga: il Laboratorio 18 osserva il comportamento SOCKS, non tutto il traffico dell'applicazione.

Soluzione 28.3. Con Stem ci si iscrive a `EventType.STATUS_CLIENT`; la richiesta con il solo indirizzo produce un evento di gravità `WARN` con azione `DANGEROUS_SOCKS` e argomenti `PROTOCOL` e `ADDRESS`. Un'interfaccia grafica potrebbe mostrare un avviso che nomina l'applicazione e la destinazione, invitando a correggerne la configurazione.

Soluzione 28.4. Con obfs4 l'osservatore avrebbe visto una connessione cifrata senza struttura riconoscibile verso un indirizzo qualunque, e non avrebbe potuto dire con certezza che si trattava di Tor. Restano però altri indizi: il volume e i tempi di quella connessione, la sua durata insolita, la presenza di altre connessioni simili, e naturalmente il contenuto dei messaggi e il movente, che restringevano l'insieme a chi doveva sostenere quell'esame.

Soluzione 28.5. Esercizio aperto. Una buona risposta segue la struttura del capitolo 4: avversari e loro capacità, ciò che ciascuno può osservare a ogni livello della fig. 28.1, strumenti scelti e motivazioni, e un elenco esplicito dei rischi non coperti (per esempio la correlazione da parte di un avversario che controlla entrambi gli estremi, o il sequestro dei dispositivi).

Capitolo 29: La ricerca accademica su Tor

Soluzione 29.1. $H = \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + 2 \cdot \frac{1}{8} \cdot 3 = 1{,}75$ bit; $H_M = \log_2 4 = 2$ bit; $d = 0{,}875$. Il numero equivalente di utenti è $2^{1{,}75} \approx 3{,}4$.

Soluzione 29.2. (a) Simulazione della rete con Shadow o, in piccolo, emulazione con Chutney: serve il comportamento vero di tor, e nessun utente deve essere coinvolto. (b) Misura della rete reale con dati aggregati e pubblici, come fa Tor Metrics (sezione 24.2). (c) Simulazione del comportamento, alla TorPS, su consensi reali di un lungo periodo, eventualmente seguita da un'analisi teorica.

Soluzione 29.3. Con `vita_guard` di 30 giorni (durata effettiva 18–30 per la retrodatazione) ci sono circa sette o otto estrazioni del guard in 180 giorni, e la simulazione dà 32,8%; con i 60 giorni predefiniti (durata 48–60), circa tre estrazioni e 18,2%; con 180 giorni, una o due estrazioni e 8,6%. In prima approssimazione, con k estrazioni la frazione compromessa tende a $1 - (1 - q)^k$ con $q = 5\%$: $k \approx 7,5$ dà circa il 32%, coerente con il caso a 30 giorni. Più il guard dura, meno estrazioni, meno client esposti; ma chi ha un guard ostile resta esposto più a lungo.

Soluzione 29.4. Nessuno dei due compromette alcun client secondo il criterio del laboratorio, che richiede entrambi gli estremi. Ciascuno però ottiene qualcosa di diverso: l'exit vede destinazioni e traffico non cifrato senza sapere chi sono i client; il guard vede gli indirizzi dei client che lo hanno scelto, senza sapere dove vanno, e può tentare il website fingerprinting (sezione 26.4).

Soluzione 29.5. Esercizio di ricerca. Il riuso dei circuiti riduce il numero di estrazioni per giorno e rallenta la curva senza guard; il vincolo /16 ha un effetto piccolo, salvo avversari concentrati in una rete; un avversario a livello di sistema autonomo cambia il modello, perché osserva il traffico senza gestire relay, e richiede dati sulla topologia di Internet come in [Joh+13].

Capitolo 30: Arti

Soluzione 30.1. Cap. 5: `tor-llcrypto`; cap. 7: `tor-proto` (canali) e `tor-chanmgr`; cap. 8: `tor-cell`; capp. 9–11: `tor-proto` (circuiti, cifratura, stream e controllo di flusso); cap. 12: `tor-netdoc`, `tor-dirmgr`, `tor-dirclient`; cap. 13: `tor-circmgr`, `tor-guardmgr`, `tor-relay-selection`; cap. 14: `tor-hsclient`, `tor-hsservice`, `tor-hscrypto`; cap. 15: `equix` e `hashx` per la prova di lavoro; cap. 16: `tor-ptmgr`.

Soluzione 30.2. Vantaggi: meno codice, meno superficie d'attacco, nessun caso speciale (come i CircID di 2 byte), nessuna crittografia debole da mantenere. Svantaggi: Arti non può parlare con relay molto vecchi; ma la rete stessa esclude già le versioni non più raccomandate, quindi l'esclusione riguarda soprattutto reti private o installazioni non aggiornate.

Soluzione 30.3. `decode_cell` legge il comando al quinto byte e ne deduce se la cella è variabile; in quel caso la lunghezza è nei due byte successivi e la cella è lunga 7+ lunghezza, altrimenti 514 byte. Se i dati disponibili sono meno di quelli necessari, la funzione non segnala un errore ma restituisce `Ok(None)`: la cella è solo incompleta, e il chiamante riproverà quando arriveranno altri byte. È lo stesso comportamento che serve a un decodificatore di flusso come quello del Laboratorio 2.

Soluzione 30.4. Sì: il proxy SOCKS di Arti isola per default gli stream con credenziali SOCKS diverse (e isola anche i listener diversi). La verifica si fa osservando i circuiti dal

lato dei relay (per esempio con la ControlPort dell'exit) o dai log di Arti a livello di debug. Per controllare l'isolamento in modo esplicito, usando Arti come libreria, si ricorre a `TorClient::isolated_client` o agli isolation token di `StreamPrefs`; Arti 2.6.0 non offre ancora la configurazione per-porta equivalente a quella di tor in C (nel sorgente, `cfg.rs`, è annotata come sviluppo futuro).

Soluzione 30.5. Esercizio di programmazione: si crea una `TorClientConfig` a partire dal file TOML del laboratorio, si avvia il client con `TorClient::create_bootstrapped` e si apre uno stream con `connect(("127.0.0.1", 8080))`, su cui si scrive una richiesta HTTP. Serve abilitare nella configurazione l'accesso agli indirizzi locali, come nel file generato dal laboratorio.

Capitolo 31: Problemi aperti

Soluzione 31.1. Esempi: Proposta 332 (ntor-v3) è *Closed*: implementata e riportata nelle specifiche. Proposta 324 (controllo della congestione) è *Finished*: implementata, in attesa che il testo confluisca del tutto nelle specifiche. Proposta 359 (CGO) è *Open*: già nel codice, ma ancora in discussione come proposta. Lo stato si legge nell'intestazione di ogni proposta e nell'indice per stato del repository.

Soluzione 31.2. Con il formato tradizionale, come semplice conto dimensionale, la chiave pubblica ML-KEM-768 da 1184 byte richiede $\lceil 1184/498 \rceil = 3$ celle e il testo cifrato da 1088 byte altre $\lceil 1088/498 \rceil = 3$: sei celle in tutto. Il protocollo attuale, però, non consente nemmeno di frammentare così un singolo messaggio relay. La Proposta 340 introduce il framing che serve — frammentare un messaggio su più celle e impacchettarne più d'uno nella stessa cella — con un corpo di 493 byte per relay cell (non 498: il nuovo formato spende 16 byte per il tag di autenticazione, $509 - 16 = 493$, come nel capitolo 8), e con 490 byte utili nel primo frammento di un messaggio, dove sta anche l'intestazione. Mantiene però, per ora, un massimo di 493 byte per i body dei messaggi ordinari: perché un `EXTEND2` post-quantistico possa superarlo servirà anche una nuova versione del sottoprotocollo `Relay` che alzi quel limite. Anche ipotizzando tale estensione, la chiave pubblica occupa comunque 3 celle e il testo cifrato altre 3; e poiché i due valori viaggiano in direzioni *opposte* — il client manda la chiave pubblica al relay, il relay risponde con il testo cifrato incapsulato — non possono essere impacchettati insieme. Il totale resta di sei celle.

Soluzione 31.3. Se ogni client scaricasse un sottoinsieme diverso di relay, un avversario che conosce (o influenza) il sottoinsieme di un utente potrebbe riconoscerne i circuiti: un circuito che usa relay noti solo a quell'utente rivela chi lo ha costruito. Anche senza conoscerlo in anticipo, osservare quali relay un client usa nel tempo restringe l'insieme dei client possibili.

Soluzione 31.4. Si filtrano i circuiti con `purpose CONFLUX_LINKED` e si raggruppano per `CONFLUX_ID` (capitolo 18); per ogni coppia si confrontano i percorsi. L'exit è sempre lo stesso, come previsto. Le altre posizioni, invece, risultano spesso condivise: per esempio le due gambe `[test007e, test003r, test006e]` e `[test003r, test007e, test006e]` usano gli stessi tre relay in ordine diverso. Non è un difetto di tor: la rete di laboratorio ha pochi relay idonei, e la Proposta 329 chiede di non condividerli con un `SHOULD NOT`, non con un divieto. Sulla rete pubblica, con migliaia di relay, la condivisione è molto più rara.

Soluzione 31.5. Esercizio aperto. Il formato di riferimento è descritto in Proposta 001: intestazione con numero, titolo, autore, data e stato, poi motivazione, progetto, analisi di sicurezza e di compatibilità, piano di implementazione e di distribuzione.

Appendice B: Complementi di crittografia

Soluzione B.1. Su 256 bit, cinque sono fissati (i tre bassi, il più alto e il penultimo): restano 2^{251} valori distinti, contro 2^{256}. La riduzione non indebolisce la chiave: la sicurezza della curva è comunque di circa 128 bit.

Soluzione B.2. Da $y = (u-1)/(u+1)$ si ricava $u = (1+y)/(1-y) \bmod p$. In Python: si legge y dai 32 byte little-endian azzerando il bit del segno, si calcola `(1 + y) * pow(1 - y, P - 2, P) % P` e si codifica il risultato su 32 byte little-endian. Applicando `montgomery_a_edwards` con il segno originale si ritrova la chiave di Edwards di partenza.

Soluzione B.3. Una cella alterata a caso supera il controllo con probabilità circa $2^{-16} \cdot 2^{-32} = 2^{-48}$ (due byte di `recognized` nulli e quattro di digest corretti). Ma la marcatura non cerca di far accettare la cella: il relay complice che la vede passare, o l'estremo che la rifiuta, riconosce l'alterazione dall'effetto, per esempio dalla chiusura del circuito o da un contenuto prevedibile, e questo basta a collegare i due punti del circuito.

Bibliografia

[03] *Decreto legislativo 30 giugno 2003, n. 196, Codice in materia di protezione dei dati personali*. Testo coordinato pubblicato dal Garante per la protezione dei dati personali. 30 Giu. 2003. URL: https://www.garanteprivacy.it/ (visitato il giorno 23/09/2026).

[17] *Legge 20 novembre 2017, n. 167, art. 24* (*termine di conservazione dei dati di traffico per finalità di accertamento e repressione dei reati*). GU Serie Generale n. 277 del 27.11.2017. 20 Nov. 2017. URL: https://www.normattiva.it/uri-res/N2Ls?urn:nir:stato:legge:2017-11-20;167.

[22] *Regolamento (UE) 2022/2065 del Parlamento europeo e del Consiglio, del 19 ottobre 2022, relativo a un mercato unico dei servizi digitali* (*regolamento sui servizi digitali*). GU L 277 del 27.10.2022. 19 Ott. 2022. URL: https://eur-lex.europa.eu/eli/reg/2022/2065/oj.

[Bau+07] Kevin Bauer, Damon McCoy, Dirk Grunwald, Tadayoshi Kohno e Douglas Sicker. «Low-Resource Routing Attacks Against Tor». In: *ACM Workshop on Privacy in the Electronic Society* (*WPES*). 2007.

[Boc+24] Cecylia Bocovich, Arlo Breault, David Fifield, Serene e Xiaokang Wang. «Snowflake, a censorship circumvention system using temporary WebRTC proxies». In: *Proceedings of the 33rd USENIX Security Symposium*. 2024.

[BPW13] Alex Biryukov, Ivan Pustogarov e Ralf-Philipp Weinmann. «Trawling for Tor Hidden Services: Detection, Measurement, Deanonymization». In: *IEEE Symposium on Security and Privacy*. 2013.

[Cha81] David L. Chaum. «Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms». In: *Communications of the ACM* 24.2 (1981), pp. 84–88.

[Cha88] David Chaum. «The Dining Cryptographers Problem: Unconditional Sender and Recipient Untraceability». In: *Journal of Cryptology* 1.1 (1988), pp. 65–75.

[Cor22] Corte di giustizia dell'Unione europea. *SpaceNet e Telekom Deutschland, cause riunite C-793/19 e C-794/19*. ECLI:EU:C:2022:702. 20 Set. 2022. URL: https://eur-lex.europa.eu/legal-content/IT/TXT/?uri=CELEX:62019CJ0793 (visitato il giorno 03/11/2026).

[DDM03] George Danezis, Roger Dingledine e Nick Mathewson. «Mixminion: Design of a Type III Anonymous Remailer Protocol». In: *Proceedings of the 2003 IEEE Symposium on Security and Privacy*. 2003, pp. 2–15.

[Día+02] Claudia Díaz, Stefaan Seys, Joris Claessens e Bart Preneel. «Towards Measuring Anonymity». In: *Privacy Enhancing Technologies* (*PET 2002*). LNCS. 2002.

[Din14] Roger Dingledine. *Tor security advisory: «relay early» traffic confirmation attack*. 30 Lug. 2014. URL: https://blog.torproject.org/tor-security-advisory-relay-early-traffic-confirmation-attack/ (visitato il giorno 23/09/2026).

[DM06] Roger Dingledine e Nick Mathewson. «Anonymity Loves Company: Usability and the Network Effect». In: *Proceedings of the Fifth Workshop on the Economics of Information Security* (*WEIS 2006*). 2006.

[DMS04] Roger Dingledine, Nick Mathewson e Paul Syverson. «Tor: The Second-Generation Onion Router». In: *Proceedings of the 13th USENIX Security Symposium*. 2004, pp. 303–320.

[EDG09] Nathan S. Evans, Roger Dingledine e Christian Grothoff. «A Practical Congestion Attack on Tor Using Long Paths». In: *Proceedings of the 18th USENIX Security Symposium*. 2009.

[Ele] Electronic Frontier Foundation. *Legal FAQ for Tor Relay Operators*. URL: `https://community.torproject.org/relay/community-resources/eff-tor-legal-faq/` (visitato il giorno 23/09/2026).

[Fif+15] David Fifield, Chang Lan, Rod Hynes, Percy Wegmann e Vern Paxson. «Blocking-resistant communication through domain fronting». In: *Proceedings on Privacy Enhancing Technologies* 2015.2 (2015), pp. 46–64.

[Fro+19] Sergey Frolov, Jack Wampler, Sze Chuen Tan, J. Alex Halderman, Nikita Borisov e Eric Wustrow. «Conjure: Summoning Proxies from Unused Address Space». In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security* (*CCS*). 2019.

[Gar21] Garante per la protezione dei dati personali. *Segnalazione sulla disciplina della conservazione, a fini di giustizia, dei dati di traffico telefonico e telematico*. Doc-Web n. 9685978. 22 Lug. 2021. URL: `https://www.garanteprivacy.it/web/guest/home/docweb/-/docweb-display/docweb/9685978` (visitato il giorno 03/11/2026).

[GRS96] David M. Goldschlag, Michael G. Reed e Paul F. Syverson. «Hiding Routing Information». In: *Information Hiding, First International Workshop*. Vol. 1174. LNCS. Springer, 1996, pp. 137–150.

[GSU13] Ian Goldberg, Douglas Stebila e Berkant Ustaoglu. «Anonymity and One-Way Authentication in Key Exchange Protocols». In: *Designs, Codes and Cryptography* 67.2 (2013), pp. 245–269.

[Jan+14] Rob Jansen, Florian Tschorsch, Aaron Johnson e Björn Scheuermann. «The Sniper Attack: Anonymously Deanonymizing and Disabling the Tor Network». In: *Proceedings of the Network and Distributed System Security Symposium* (*NDSS*). 2014.

[JH12] Rob Jansen e Nicholas Hopper. «Shadow: Running Tor in a Box for Accurate and Efficient Experimentation». In: *Proceedings of the Network and Distributed System Security Symposium* (*NDSS*). 2012.

[JJ16] Rob Jansen e Aaron Johnson. «Safely Measuring Tor». In: *ACM Conference on Computer and Communications Security* (*CCS*). 2016.

[Joh+13] Aaron Johnson, Chris Wacek, Rob Jansen, Micah Sherr e Paul Syverson. «Users Get Routed: Traffic Correlation on Tor by Realistic Adversaries». In: *ACM Conference on Computer and Communications Security* (*CCS*). 2013.

[Jua+14] Marc Juarez, Sadia Afroz, Gunes Acar, Claudia Diaz e Rachel Greenstadt. «A Critical Evaluation of Website Fingerprinting Attacks». In: *ACM Conference on Computer and Communications Security* (*CCS*). 2014.

[Jus14] Just Security. *Video clip of former director of NSA and CIA: «We kill people based on metadata»*. dal Johns Hopkins Foreign Affairs Symposium, 1 aprile 2014. 2014. URL: `https://www.justsecurity.org/10318/video-clip-director-nsa-cia-we-kill-people-based-metadata/` (visitato il giorno 24/09/2026).

[MD05] Steven J. Murdoch e George Danezis. «Low-Cost Traffic Analysis of Tor». In: *IEEE Symposium on Security and Privacy*. 2005.

[NBH18] Milad Nasr, Alireza Bahramali e Amir Houmansadr. «DeepCorr: Strong Flow Correlation Attacks on Tor Using Deep Learning». In: *ACM Conference on Computer and Communications Security* (*CCS*). 2018.

[NIS24] NIST. *FIPS 203* (*ML-KEM*), *FIPS 204* (*ML-DSA*), *FIPS 205* (*SLH-DSA*). i tre standard post-quantistici, pubblicati insieme. 13 Ago. 2024. URL: `https://csrc.nist.gov/pubs/fips/203/final` (visitato il giorno 24/09/2026).

[OON] OONI. *Specification: Tor* (*ts-023*) *and Tor using Snowflake* (*ts-030*). URL: `https://github.com/ooni/spec/tree/master/nettests` (visitato il giorno 23/09/2026).

[ØS06] Lasse Øverlier e Paul Syverson. «Locating Hidden Servers». In: *Proceedings of the 2006 IEEE Symposium on Security and Privacy*. 2006, pp. 100–114.

[Pan+11] Andriy Panchenko, Lukas Niessen, Andreas Zinnen e Thomas Engel. «Website Fingerprinting in Onion Routing Based Anonymization Networks». In: *ACM Workshop on Privacy in the Electronic Society* (*WPES*). 2011.

[Per+18] Mike Perry, Erinn Clark, Steven Murdoch e Georg Koppen. *The Design and Implementation of the Tor Browser*. Bozza. 15 Giu. 2018. URL: `https://2019.www.torproject.org/projects/torbrowser/design/` (visitato il giorno 23/09/2026).

[PH10] Andreas Pfitzmann e Marit Hansen. *A terminology for talking about privacy by data minimization: Anonymity, Unlinkability, Undetectability, Unobservability, Pseudonymity, and Identity Management*. Versione 0.34. 2010.

[RR98] Michael K. Reiter e Aviel D. Rubin. «Crowds: Anonymity for Web Transactions». In: *ACM Transactions on Information and System Security* 1.1 (1998), pp. 66–92.

[San13] Runa A. Sandvik. *Harvard Student Receives F For Tor Failure While Sending «Anonymous» Bomb Threat*. 18 Dic. 2013. URL: `https://www.forbes.com/sites/runasandvik/2013/12/18/harvard-student-receives-f-for-tor-failure-while-sending-anonymous-bomb-threat/` (visitato il giorno 24/09/2026).

[Sir+18] Payap Sirinam, Mohsen Imani, Marc Juarez e Matthew Wright. «Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning». In: *ACM Conference on Computer and Communications Security* (*CCS*). 2018.

[Sun+15] Yixin Sun, Anne Edmundson, Laurent Vanbever, Oscar Li, Jennifer Rexford, Mung Chiang e Prateek Mittal. «RAPTOR: Routing Attacks on Privacy in Tor». In: *USENIX Security Symposium*. 2015.

[Syv11] Paul Syverson. «A Peel of Onion». In: *Proceedings of the 27th Annual Computer Security Applications Conference* (*ACSAC*). 2011, pp. 123–137.

[Taia] Tails. *How Tails works*. URL: `https://tails.net/about/` (visitato il giorno 23/09/2026).

[Taib] Tails. *Warnings: Tails is safe but not magic!* URL: `https://tails.net/doc/about/warnings/` (visitato il giorno 23/09/2026).

[TG23] Lindsey Tulloch e Ian Goldberg. «Lox: Protecting the Social Graph in Bridge Distribution». In: *Proceedings on Privacy Enhancing Technologies* 2023.1 (2023), pp. 494–509.

[Thea] The Tor Project. *Checking for DNS leaks with SOCKS and Tor*. URL: `https://support.torproject.org/little-t-tor/troubleshooting/check-for-leaks/` (visitato il giorno 04/11/2026).

[Theb] The Tor Project. *History*. https://www.torproject.org/about/history/. Consultato il 23 settembre 2026.

[Thec] The Tor Project. *Onionbalance: documentazione e guida per gli onion service v3*. https://onionservices.torproject.org/apps/base/onionbalance/. Consultato il 23 settembre 2026.

[Thed] The Tor Project. *Reproducible Metrics*. URL: https://metrics.torproject.org/reproducible-metrics.html (visitato il giorno 23/09/2026).

[Thee] The Tor Project. *Tor Browser User Manual*. URL: https://tb-manual.torproject.org/ (visitato il giorno 23/09/2026).

[Thef] The Tor Project. *Tor Specifications*. https://spec.torproject.org/. Consultato il 23 settembre 2026.

[Theg] The Tor Project. *Tor Support: Staying anonymous*. URL: https://support.torproject.org/faq/staying-anonymous/ (visitato il giorno 23/09/2026).

[Theh] The Tor Project. *Tor Support: Using Bridges; Unblocking Tor*. https://support.torproject.org/little-t-tor/circumvention/using-bridges/. Consultato il 23 settembre 2026.

[The22] The Tor Project. *Malicious relays and the health of the Tor network*. 25 Apr. 2022. URL: https://blog.torproject.org/malicious-relays-health-tor-network/ (visitato il giorno 23/09/2026).

[The23a] The Tor Project. *Introducing Proof-of-Work Defense for Onion Services*. 23 Ago. 2023. URL: https://blog.torproject.org/introducing-proof-of-work-defense-for-onion-services/ (visitato il giorno 23/09/2026).

[The23b] The Tor Project. *Tor is slow right now. Here is what is happening*. 7 Feb. 2023. URL: https://blog.torproject.org/tor-network-ddos-attack/ (visitato il giorno 23/09/2026).

[The24a] The Tor Project. *Hiding in plain sight: Introducing WebTunnel*. https://blog.torproject.org/introducing-webtunnel-evading-censorship-by-hiding-in-plain-sight/. 12 marzo 2024. 2024.

[The24b] The Tor Project. *Making new connections: from BridgeDB to Rdsys*. 2024. URL: https://blog.torproject.org/making-connections-from-bridgedb-to-rdsys/ (visitato il giorno 24/09/2026).

[The24c] The Tor Project. *Uniting for Internet Freedom: Tor Project & Tails Join Forces*. 26 Set. 2024. URL: https://blog.torproject.org/tor-tails-join-forces/ (visitato il giorno 24/09/2026).

[The26] The Tor Project. *Arti 2.7.0 released*. 1 Ott. 2026. URL: https://blog.torproject.org/arti_2_7_0_released/ (visitato il giorno 04/11/2026).

[Tor] Tor Research Safety Board. *Tor Research Safety Board*. URL: https://research.torproject.org/safetyboard/ (visitato il giorno 23/09/2026).

[Tor25] Tor Project Forum. *Feedback from China [April 2025]: increased GFW censorship; obfs4/meek/snowflake unusable, webtunnel connects but is quickly blocked*. 2025. URL: https://forum.torproject.org/t/feedback-from-china-april-2025-increased-gfw-censorship-obfs4-meek-snowflake-unusable-webtunnel-connects-but-is-quickly-blocked/18233 (visitato il giorno 24/09/2026).

[WBU13] WBUR News. *Student Charged For Bomb Threats At Harvard*. 17 Dic. 2013. URL: https://www.wbur.org/news/2013/12/17/harvard-bomb-hoax-student-charges (visitato il giorno 23/09/2026).

[Who] Whonix. *onion-grater*. URL: `https://www.whonix.org/wiki/Onion-grater` (visitato il giorno 23/09/2026).

[WL14] Philipp Winter e Stefan Lindskog. «Spoiled Onions: Exposing Malicious Tor Exit Relays». In: *arXiv:1401.4917*. poi confluito, con altri autori, in *Privacy Enhancing Technologies Symposium* (PETS 2014). 2014.

Indice analitico

AUTH_CHALLENGE, 36, 39, 42, 44, 52, 56, 161
AUTHENTICATE, 36, 40, 44, 45, 52
AUTHORIZE, 52

CERTS, 29, 36–40, 42, 44, 52, 56, 161, 259
CREATE, 52
CREATE2, 29, 51, 52, 58, 60, 63–65, 67, 71, 72, 258, 268
CREATE_FAST, 23, 52, 66, 72, 128, 260
CREATED, 52
CREATED2, 52, 58, 63–65, 71, 268
CREATED_FAST, 52

DESTROY, 52, 58, 60, 61, 65, 71, 77, 84, 85, 88, 89, 128, 213, 259–261, 273

iniziatore, 33

NETINFO, 36, 40–44, 50, 52, 56, 161

PADDING, 41, 44, 50, 52, 71, 81, 82, 259
PADDING_NEGOTIATE, 52, 259
Proposte
001, 239, 243, 275
214, 51
216, 66, 70
220, 34
224, 112
236, 106
271, 106
289, 54, 79, 86, 89, 90
291, 106
300, 240, 241, 244
302, 208
323, 242, 244
324, 55, 68, 87, 88, 90, 239, 260, 275
327, 126
329, 55, 148, 160, 239, 241, 243, 275
332, 67, 70, 72, 239, 275
333, 107
338, 41, 258
339, 55
340, 56, 241, 243, 275
346, 30
348, 242
350, 66, 240, 241
355, 240, 243
359, 56, 72, 78, 79, 239, 260, 275
362, 242

RELAY, 42, 52, 53, 58, 79, 167, 169, 211, 246, 268, 273
RELAY_EARLY, 42, 52, 53, 58, 60, 61, 65, 79, 81, 82, 211–215, 259, 268, 273
RELAY_SENDME, 239
rispondente, 33

torrc
__LeaveStreamsUnattached, 149, 152
AccountingMax, 178, 213
AssumeReachable, 157, 158
AutomapHostsOnResolve, 144
BridgeDistribution, 132, 182, 183
ClientOnionAuthDir, 173, 175
ContactInfo, 178
ControlPort, 144, 147
CookieAuthentication, 144
DirAuthority, 157, 266
DirPortFrontPage, 199
DNSPort, 85, 144
EntryNodes, 144
ExcludeNodes, 144
ExitNodes, 144
ExitPolicy, 183
ExitPolicyRejectPrivate, 179
ExitPolicyRejectPrivate 0, 179
ExitRelay 0, 178
FamilyId, 183
HashedControlPassword, 144
HiddenServiceDir, 171, 172, 175, 176
HiddenServiceNonAnonymousMode, 175

HiddenServicePort, 118, 172, 176
HiddenServicePoWDefensesEnabled, 214
HiddenServiceSingleHopMode, 175
HSLayer2Nodes, 108
HSLayer3Nodes, 108
IsolateSOCKSAuth, 186, 207
KeepalivePeriod, 34
Log, 169
MaxCircuitDirtiness, 144
MaxMemInQueues, 124, 129, 130, 265
MyFamily, 181–183, 270
OfflineMasterKey, 183
OfflineMasterKey 1, 178
ReducedExitPolicy, 179, 183
RelayBandwidthBurst, 178
RelayBandwidthRate, 124, 178, 270
SafeLogging, 145, 168, 199, 235
SafeSocks, 219, 220, 273
SessionGroup, 143, 266
SigningKeyLifetime, 39
SocksPort, 143, 145
StrictNodes, 144
TestingDirAuthVoteGuard, 157, 267
TestingLinkCertLifetime, 39
TestingTorNetwork, 157, 162, 235, 266
TestSocks, 218–220
TransPort, 144
UseBridges, 266
V3AuthDistDelay, 96
V3AuthNIntervalsValid, 96
V3AuthVoteDelay, 96
V3AuthVotingInterval, 96, 157, 268

VERSIONS, 36, 42, 43, 49, 50, 52, 56, 58, 60, 160, 161, 168, 233, 246, 258, 259, 269
VPADDING, 36, 43, 52